

Discrete mathematics, programming and statistics

Jon Eivind Vatne
Institutt for datateknologi, elektroteknologi og realfag
Fakultet for ingeniør- og naturvitenskap
Høgskulen på Vestlandet
e-mail: jev@hvl.no

July 2, 2020

Introduction

When trying to understand structures we see around us, in nature and in technology, mathematics is the fundamental language we use to discover and express patterns. In these notes, we will investigate some *discrete* phenomena, like counting problems, the computer's understanding of numbers, and the logical structure of testing for truth. We will also look at continuous and discrete phenomena in statistics and in probability theory. In parallel, you will learn about programming in other courses. We will use programming to support our understanding of mathematics and statistics, and mathematics to support our understanding of programming. Programming examples will be given both using Java or MATLAB, and using *pseudocode*. The latter will demand that you unravel the structure of the code, and translate it into for instance Java, to get a working programme.

Exercises are mostly collected at the end of each chapter. If programming exercises are too difficult when you first read them, don't worry! The recommended approach is to read the text of the exercise, and return to it when the necessary programming techniques are covered in the programming course.

These notes are a work in progress. Do not hesitate to contact the author with suggestions, errors you find, or questions you might have!

Contents

1	Boolean algebra	11
1.1	Negation, conjunction and disjunction	11
1.2	Implication and equivalence	14
1.3	Boolean expressions in Java	17
1.4	Exercises	18
1.5	Appendix: standard identities	20
2	Sets	21
2.1	Union, intersection and complement	22
2.2	De Morgan's laws	26
2.3	Sets and Boolean algebra	28
2.4	Relations	29
2.5	Elementary counting	31
2.6	Exercises	33
3	Integers	39
3.1	Addition and multiplication - a review	40
	Addition	40
	Multiplication	41
3.2	Binary numbers	42
3.3	Integers and integer types in Java	44
	The number of bits	46
	Binary logarithm	48
	Binary representation of negative numbers	50
3.4	Division	51
	Modulo	52
	Modulo in Java	53
3.5	Bitwise operations	54
3.6	Exercises	55

4	Floating points	59
4.1	Real numbers	59
	Binary representation	61
4.2	What are floating points?	61
4.3	Float types in Java	62
	Special values	64
4.4	Casting between floats and integers	65
4.5	Exercises	65
5	Sequences and series	67
5.1	Sequences	68
	Arithmetic sequences	70
	Geometric sequences	70
5.2	Series	72
	Arithmetic series	73
	Geometric series	74
5.3	Exercises	76
6	Functions	79
6.1	Functions between sets	79
	Functions and remainders	82
6.2	Methods in Java	87
6.3	Exercises	91
7	Loops	93
7.1	Looping in an example	93
7.2	Types of loops in Java	95
7.3	Summing series	98
7.4	Exercises	102
8	Proofs and reasoning	105
8.1	Propositional logic	105
8.2	Predicate logic	106
8.3	Rules of deduction	108
8.4	Mathematical induction	110
8.5	Checking an algorithm	114
8.6	Exercises	115
9	Eigenvectors	117
9.1	Definition of eigenvectors	117
9.1.1	The characteristic polynomial	118

9.2	Diagonalization	121
9.3	The power method	127
9.4	Extended example: PageRank	130
9.5	Exercises	132
10	Homogeneous coordinates	133
10.1	Definition of homogeneous coordinates	133
10.2	Homogeneous coordinates in the plane	136
10.2.1	Rotation and translation	137
10.2.2	Projections	139
10.3	Homogeneous coordinates in three-space	143
10.3.1	Rotations and translations	143
10.3.2	Projections	145
10.4	Exercises	148
11	Functions of two variables	149
11.1	Partial derivatives and applications	151
11.2	Partial derivatives in image processing	158
11.3	Least squares	161
11.3.1	Partial derivatives approach	161
11.3.2	Linear algebra approach	163
11.4	Exercises	165
12	Statistical analysis of text	167
12.1	Counting in texts	167
12.2	Permutations of letters and the Caesar Cipher	172
12.3	Comparing frequencies	175
12.4	Sampling text	177
12.5	Exercises	181
13	A very short introduction to MATLAB	183
13.1	Getting started with MATLAB	183
13.2	Simple plotting in MATLAB	185
13.3	Sound in MATLAB	188
13.4	User-defined functions	190
13.4.1	Anonymous functions and function handles	191
14	Sampling from discrete distributions	193
14.1	Expectation and variance	193
	Appendix: Properties of expectance and variance	196
14.2	Biased coin flips - random walks	198

14.3	Drawing randomly from a finite set	201
14.4	The Poisson distribution	209
14.5	Exercises	212
15	Graphs	215
15.1	Definition of a graph	215
15.2	Examples of graphs	218
15.2.1	Trees	218
15.2.2	Regular graphs	220
15.3	Special cycles	222
15.4	Directed graphs, weighted graphs	225
15.5	Exercises	230
16	Probability	235
16.1	Stochastic variables	235
16.2	Sets, counting and probabilities	239
16.3	Conditional probabilities	242
16.4	Correlation and independence	244
16.5	Continuous distributions and integration	249
16.6	Exercises	252
17	The normal distribution	255
17.1	The normal distributions	255
17.2	Confidence intervals	257
17.3	Hypothesis testing	263
17.4	Student's t-distribution	268
17.5	The central limit theorem and normal approximations	270
17.6	Exercises	274
18	Regression	279
18.1	Regression in different guises	279
	Quadratic approximations	282
	Sinusoidal approximation	285
	Two-dimensional linear approximations	288
18.2	χ^2 -statistics and quality of fits	290
18.3	Exercises	294
19	RSA cryptography	299
19.1	Modulo a prime	299
19.2	Modulo pq	303
19.3	The RSA algorithm	306

19.3.1 Finding large primes	308
19.4 Implementation of the RSA algorithm	311
19.5 Exercises	314
20 Generating functions	317
20.1 Power series	317
20.2 Sequences as coefficients	320
20.3 Exercises	325
21 Principal component analysis	327
21.1 Introduction to principal component analysis	327
21.2 Mathematical preliminaries	330
21.3 Extended example - meteorological data	334
Index	340
A Solutions to exercises	343
A.1 Hints	343
A.2 Answers	344
A.3 Solutions	353

Chapter 1

Boolean algebra

In this chapter, we will introduce basic logical operators. These operators are fundamental to mathematics (to clarify reasoning) and to computers (bitwise operations are based on the logical operators), to analyze the logics of ordinary language, and to the structure of electrical circuits.

1.1 Negation, conjunction and disjunction

The basic truth-values are **true** and **false**. The first is the abstract notion associated to being true, correct or right. The second is similarly associated to being false, incorrect or wrong. A *Boolean* variable is a variable that can be either of these values. For instance, a Boolean variable x can represent the truth-value of the statement “Today is Thursday”. Is its value true or false?

By combining several Boolean variables, we can form *Boolean expressions* built up from the variables and *logical operators*. There are several logical operators. The simplest ones are *negation*, with symbol $\neg$, *conjunction*, with symbol $\wedge$, and *disjunction*, with symbol $\vee$. These are also called **NOT**, **AND** and **OR**, respectively, which may make their meaning clearer. In Java, these operators are denoted by `!`, `&&` and `||`, respectively. Negation is written in front of the expression to be negated, conjunction and disjunction is written between the two expressions that are combined.

In this chapter, we will use the value 0 for **false** and 1 for **true**. Boolean variables will typically be denoted x, y, z (lowercase letters near the end of the alphabet). To give an example of how to understand a statement in this setting, let again x be the truth-value of “Today is Thursday”. Introduce a second statement “The sun is shining”, whose truth-value is called y . Then $\neg x$ represents the truth-value of the statement “Today isn’t Thursday” and $x \wedge y$ represents the truth-value of the state-

ment “Today is Thursday, and the sun is shining”.

It should be clear, for instance, that if $x = 1$ (*i.e.* “Today is Thursday” is true), then $\neg x = 0$ (*i.e.* “Today isn’t Thursday” is false), that if $x = 0$ and $y = 1$ (*i.e.* “Today is Thursday” is false and “The sun is shining” is true), then $x \wedge y = 0$ (*i.e.* “Today is Thursday and the sun is shining” is false, because it isn’t Thursday). It should also be clear that $x = 0$ and $y = 1$ gives that $x \wedge y = 0$ *no matter* what statements x and y represent. This observation tells us that it is meaningful to study the Boolean expressions in their own right, without attaching particular statements to the variables involved.

Using the symbols we have for negation and conjunction, the underlying logical structure of the reasoning in the last paragraph can be written

$$\begin{aligned}\neg 1 &= 0 \\ 0 \wedge 1 &= 0\end{aligned}$$

Instead of analyzing many statements to explain the symbols, we can write tables of the truth-values involved, and *define* the logical operators by their values in a given table.

Definition 1.1.1 (Definition of conjunction). The logical operator of *conjunction*, with symbol $\wedge$, is defined by the following table:

$$\begin{aligned}0 \wedge 0 &= 0 \\ 0 \wedge 1 &= 0 \\ 1 \wedge 0 &= 0 \\ 1 \wedge 1 &= 1\end{aligned}\tag{1.1}$$

In words, the expression $x \wedge y$ is true if both x and y are true, and false otherwise (*i.e.* if at least one of x , y is false). $x \wedge y$ is pronounced “x and y”.

Definition 1.1.2 (Definition of disjunction). The logical operator of *disjunction*, with symbol $\vee$, is defined by the following table:

$$\begin{aligned}0 \vee 0 &= 0 \\ 0 \vee 1 &= 1 \\ 1 \vee 0 &= 1 \\ 1 \vee 1 &= 1\end{aligned}\tag{1.2}$$

In words, the expression $x \vee y$ is true if one or both of x and y are true, and false if they are both false. $x \vee y$ is pronounced “x or y”.

Definition 1.1.3 (Definition of negation). The logical operator of *negation*, with symbol $\neg$, is defined by the following table:

$$\begin{aligned}\neg 0 &= 1 \\ \neg 1 &= 0\end{aligned}\tag{1.3}$$

$\neg x$ is pronounced “not x ”.

It is also common to write the defining tables by declaring Boolean variables, and applying the operators to these variables. For instance, a variant of the description of disjunction is

Definition 1.1.4 (Disjunction - variant). Let x and y be two Boolean variables. Then the disjunction $x \vee y$ is defined by the following truth table:

x	y	$x \vee y$
0	0	0
0	1	1
1	0	1
1	1	1

Exercise 1.1.5. Let again x be the truth-value of “Today is Thursday”. and y the truth-value of “The sun is shining”. For each row of the above table, check that the truth-value of $x \vee y$ seems correct.

To construct more complex statements, several variables and operators can be combined. The truth table of the compound statement can then be found by repeated application of the defining tables. There are fixed rules for the order operators are applied in. We use parentheses to override these rules (and sometimes for extra clarity). For instance, $\neg x \wedge y$ means that x is negated, and that this negated value is conjoined with y , whereas $\neg(x \wedge y)$ means that x and y are conjoined, and this conjoined value is negated. For emphasis, we might write $(\neg x) \wedge y$, but since negation has precedence over conjunction, its value is the same as $\neg x \wedge y$. We will give two examples now; more come later when we have introduced more operators, and when this theory is applied.

Example 1.1.6. The truth table for $(\neg x \vee y) \wedge (x \vee \neg y)$ can be found by building up the necessary parts as follows. Read the table from left to right, and make sure that all expressions needed in a given column have their truth-values computed to the left.

x	y	$\neg x$	$\neg x \vee y$	$\neg y$	$x \vee \neg y$	$(\neg x \vee y) \wedge (x \vee \neg y)$
0	0	1	1	1	1	1
0	1	1	1	0	0	0
1	0	0	0	1	1	0
1	1	0	1	0	1	1

In words, the statement is true if both x and y are false, true if both are true, and false if one is true and the other is false.

Example 1.1.7. The truth table for $z \vee (x \wedge y)$ is similarly computed in this way:

x	y	z	$x \wedge y$	$z \vee (x \wedge y)$
0	0	0	0	0
0	0	1	0	1
0	1	0	0	0
0	1	1	0	1
1	0	0	0	0
1	0	1	0	1
1	1	0	1	1
1	1	1	1	1

Exercise 1.1.8. Determine the truth tables for $x \vee (y \vee z)$ and $(x \vee y) \vee z$. Why are they equal? Introduce example statements for x, y, z and check that your result makes sense.

1.2 Implication and equivalence

In this section, we will define the two important derived operators *implication* (material conditional) and *equivalence* (biconditional), and for completeness we will also formally include *exclusive or*. We will denote implication by $\rightarrow$, equivalence by $\leftrightarrow$ and exclusive or by $\oplus$. We will derive them from the basic operations in the previous section, as well as giving direct truth-value tables. Like conjunction and disjunction, these operators are placed between the expressions they combine.

We start with implication.

Definition 1.2.1 (Definition of implication). Define the *implication*, with symbol $\rightarrow$, by the truth table

$$\begin{aligned}
 0 \rightarrow 0 &= 1 \\
 0 \rightarrow 1 &= 1 \\
 1 \rightarrow 0 &= 0 \\
 1 \rightarrow 1 &= 1
 \end{aligned} \tag{1.4}$$

In words, $x \rightarrow y$ is false if x is true and y is false, and true otherwise. $x \rightarrow y$ is pronounced “ x implies y ”.

This can be written using the simpler expressions as

$$x \rightarrow y = \neg x \vee y$$

Exercise 1.2.2. Compute the truth table of $\neg x \vee y$, and check that this is the same as the defining table for $x \rightarrow y$.

Example 1.2.3. In ordinary language, implication is often written using if-then. For instance, “If I forget my umbrella, then it will rain” is an implication. If we write x = “I forget my umbrella” and y = “It will rain”, this should be clear. The truth table tells us that the implication is false only if x is true and y is false. This makes sense if you think about it: There is no statement about what happens if I don’t bring my umbrella, so as long as x is false the implication can’t be false.

Definition 1.2.4 (Definition of equivalence). Define the *equivalence*, with symbol $\leftrightarrow$, by the truth table

$$\begin{aligned} 0 \leftrightarrow 0 &= 1 \\ 0 \leftrightarrow 1 &= 0 \\ 1 \leftrightarrow 0 &= 0 \\ 1 \leftrightarrow 1 &= 1 \end{aligned} \tag{1.5}$$

In words, $x \leftrightarrow y$ is true if the truth-values of x and y are equal, false if they are different. $x \leftrightarrow y$ is pronounced “ x is equivalent to y ”.

This can be written using the simpler expressions as

$$x \leftrightarrow y = (\neg x \vee y) \wedge (x \vee \neg y).$$

It can also be written

$$x \leftrightarrow y = (x \rightarrow y) \wedge (y \rightarrow x).$$

Exercise 1.2.5. By computing the truth table of $(x \rightarrow y) \wedge (y \rightarrow x)$, check that this is the same as equivalence. Note that the other description, $x \leftrightarrow y = (\neg x \vee y) \wedge (x \vee \neg y)$, was checked in Example 1.1.6.

Example 1.2.6. The statement “I forget my umbrella exactly when it will rain” is an equivalence. In this statement, we claim both that if I forget my umbrella, it will rain, and that if I don’t forget my umbrella, it will not rain. A common way to formulate equivalences is by saying if and only if, as in $x \geq 2$ if and only if $2x + 3 \geq 7$ (a true statement). This is so common in mathematics that some authors abbreviate if and only if to *iff*.

Definition 1.2.7 (Definition of exclusive or). Define *exclusive or*, with symbol $\oplus$, by the truth-value

$$\begin{aligned} 0 \oplus 0 &= 0 \\ 0 \oplus 1 &= 1 \\ 1 \oplus 0 &= 1 \\ 1 \oplus 1 &= 0 \end{aligned} \tag{1.6}$$

Exercise 1.2.8. Compute the truth table of $\neg(x \leftrightarrow y)$, and conclude that

$$x \oplus y = \neg(x \leftrightarrow y)$$

Example 1.2.9. Exclusive or is typically formulated using either - or. For instance, “Either he orders a glass of red wine or a glass of white wine”. Even if the related statement “He orders a glass of red wine or a glass of white wine” is formulated without explicitly using exclusive or, the intention is usually that he only orders one glass of wine. Beware that strict logical statements are less common in ordinary text (or speech).

There are various notations for these operators in various contexts. In Java, equivalence of Boolean expressions is checked with the symbol `==`, two equality signs. In these notes, this will appear as a long equality sign:

`x == y`

Recall that a single equality sign is reserved for assignment in Java. Exclusive or is sometimes written *XOR*. Java has the notation `^` for this operator, but this is not completely standard. It is more common to negate equality, although this might be problematic if one of the arguments doesn’t have a proper Boolean value (null in the wrapper class).

Example 1.2.10. Find the truth table for the compound expression

$$(x \rightarrow y) \vee (x \wedge y).$$

We proceed in the same fashion as in our previous computations of truth tables.

x	y	$x \rightarrow y$	$x \wedge y$	$(x \rightarrow y) \vee (x \wedge y)$
0	0	1	0	1
0	1	1	0	1
1	0	0	0	0
1	1	1	1	1

Exercise 1.2.11. Find the truth tables for the following expressions.

(a). $x \leftrightarrow (y \vee x)$

(b). $x \leftrightarrow (y \rightarrow z)$

(c). $(\neg x \rightarrow \neg y) \leftrightarrow (y \rightarrow x)$

The truth table in part c) is an example of a *tautology*, an expression that is true no matter what the truth-values of its constituent expressions are. These expressions are important because they form the basis for rules of deduction, see Section 8.3.

1.3 Boolean expressions in Java

The reader is encouraged to read this section now, but if the programming skills that are necessary to get the code examples running are lacking: Don't worry! We will return to this later on. For now, just read the code, and convince yourself that there is something Boolean going on.

Boolean expressions are extremely important in programming, where they are often used to determine whether a block of code is executed. You will see this many times, both in this course and in other computer science courses. Boolean expressions come in several guises, and explicitly declared Boolean variables are only one of them.

Example 1.3.1. Look at the following code snippet:

```
boolean test1 = true;
boolean test2 = false;
if test1{
    System.out.println("This code is executed.");
}
if test2{
    System.out.println("This code isn't executed.");
}
```

Here the explicitly declared Boolean variables `test1` and `test2` are used by `if`. Since `test1` is true, the first `if` checks out, and the code between `{` and `}` is executed. Since `test2` is false, the second `if` fails, and the code is not executed.

Boolean expressions that are not explicitly declared as Boolean variables are more common. These expressions can for instance appear by checking whether one numerical value is greater than another.

Example 1.3.2. The first two `ifs` check out, and are executed, the third fails:

```
int smallNumber = 13;
if (smallNumber > 5){
    System.out.println("Greater than 5");
}
if (smallNumber > 10){
    System.out.println("Greater than 10");
}
if (smallNumber > 15){
    System.out.println("Greater than 15");
}
```

If you want to check two conditions, it is possible to use a *nested if*-statement, but it is usually better to use a logical operator. Recall that *AND* is written `&&` in Java.

Exercise 1.3.3. What code blocks are executed, and which are not?

```
int smallNumber = 13;
if ((smallNumber > 5) && (smallNumber < 10)){
    System.out.println("Between 5 and 10");
}
if ((smallNumber > 10) && (smallNumber < 15)){
    System.out.println("Between 10 and 15");
}
if ((smallNumber > 15) && (smallNumber < 20)){
    System.out.println("Between 15 and 20");
}
```

What would happen if the first line was changed to

```
int smallNumber = 10;
// the rest unchanged
```

1.4 Exercises

Exercise 1.4.1. Compute truth tables for the following expressions.

(a). $x \wedge (y \wedge z)$ and $(x \wedge y) \wedge z$.

(b). $x \wedge (y \vee z)$ and $(x \wedge y) \vee z$.

(c). $x \vee (y \wedge z)$ and $(x \vee y) \wedge z$.

Exercise 1.4.2. Compute truth tables for the following expressions.

(a). $((x \rightarrow y) \wedge x) \rightarrow y$

(b). $((x \rightarrow y) \wedge \neg y) \rightarrow \neg x$

(c). $((x \rightarrow y) \wedge (y \rightarrow z)) \rightarrow (x \rightarrow z)$.

Exercise 1.4.3. This code block

```
boolean[] truthValues = {false, true};
for (boolean x : truthValues){
    for (boolean y : truthValues){
        System.out.printf("%b and %b is %b\n", x, y, x && y);
    }
}
```

produces a truth table for $x \wedge y$. Modify it to make truth tables for each statement.

(a). $x \vee y$

(b). $x \rightarrow y$

(c). $z \rightarrow (x \vee y)$

1.5 Appendix: standard identities

In this appendix, we list a number of identities about the logical operators. Everything can be checked by computing truth tables. Some of the identities have already appeared, others are new. Some are obvious, others less so. x, y, z represent Boolean variables, and equalities mean that two expressions always have the same truth-values (this is otherwise often represented by $\Leftrightarrow$). 0 represents **false** and 1 represents **true**.

$$\begin{aligned}
 x \vee (y \vee z) &= (x \vee y) \vee z \\
 x \wedge (y \wedge z) &= (x \wedge y) \wedge z \\
 x \vee y &= y \vee x \\
 x \wedge y &= y \wedge x \\
 x \vee (x \wedge y) &= x \\
 x \wedge (x \vee y) &= x \\
 x \vee (y \wedge z) &= (x \vee y) \wedge (x \vee z) \\
 x \wedge (y \vee z) &= (x \wedge y) \vee (x \wedge z) \\
 x \vee \neg x &= 1 \\
 x \wedge \neg x &= 0 \\
 x \vee 1 &= 1 \\
 x \wedge 1 &= x \\
 x \vee 0 &= x \\
 x \wedge 0 &= 0 \\
 \neg \neg x &= x \\
 (\neg x) \vee (\neg y) &= \neg(x \wedge y) \\
 (\neg x) \wedge (\neg y) &= \neg(x \vee y)
 \end{aligned} \tag{1.7}$$

Chapter 2

Sets

In this section we will study basic operations on sets, and relate them to the Boolean operations from the previous chapter. We will not discuss what a set really *is*, just take for granted that the reader has an intuitive feeling for it, and introduce notation for dealing with sets.

We write a set using curly braces, as in

$$S = \{1, 4, 9, 16, 25\}$$

Thus, S is the set consisting of the five numbers displayed, and these five numbers are called the *elements* of S . We also say, for instance, that 9 is an element of S , and write this symbolically as

$$9 \in S$$

The symbol $\in$ is a variant of the Greek letter *epsilon*, but we read it as “is an element of”. We write $10 \notin S$ to say that 10 is not an element of S .

When there are more elements in a set than can be listed conveniently, we use either ellipsis (...) or a rule for defining the numbers. For instance the set of even positive numbers less than or equal to 100 can be written

$$\{2, 4, 6, \dots, 98, 100\}$$

or

$$\{n \mid 0 < n \leq 100 \text{ and } n \text{ is even}\}$$

Here the symbol $\mid$ is read “such that”. The two descriptions are equivalent, and define the same set.

The order elements are listed in is not part of the definition, so that

$$S = \{1, 4, 9, 16, 25\} = \{25, 16, 9, 4, 1\} = \{9, 4, 25, 1, 16\}$$

Also, the elements of sets can be any sorts of objects, so there is also a set $\{\text{pig}, \text{cat}, \text{leprechaun}\}$ and a set $\{a, b, c, \dots, z\}$ (the latter set is called the English alphabet).

The symbol $\emptyset$ refers to the special set that does not contain any element at all; it is aptly called *the empty set*.

There are also infinite sets, like the set of all integers, or the set of all real numbers. We can write ($\mathbb{Z}$ is short for German “Zahlen”)

$$\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$$

for the set of all integers, where the dots now mean that we proceed infinitely far in both directions. We also write

$$\mathbb{R} = \text{the set of all real numbers.}$$

In Java (and other programming languages), these infinite sets are approximated by finite sets of integer type and floating point type, respectively. We will see much more about this later (see Chapter 3 for integers and Chapter 4 for floating points).

2.1 Union, intersection and complement

If we are given two sets contained in a larger set, we can ask how they relate to each other. Consider for instance the following two sets of integers:

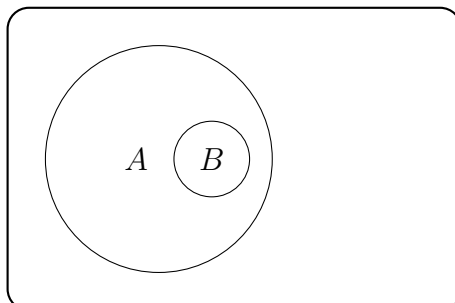
$$\begin{aligned} A &= \{2, 6, 7, 13\} \\ B &= \{6, 13\} \end{aligned}$$

In this case, every element of B is also an element of A .

Definition 2.1.1 (Inclusion). If every element in a set B is also an element of a set A , we say that B is contained in A , or is a subset of A . Symbolically, this is written

$$B \subset A \text{ or } A \supset B.$$

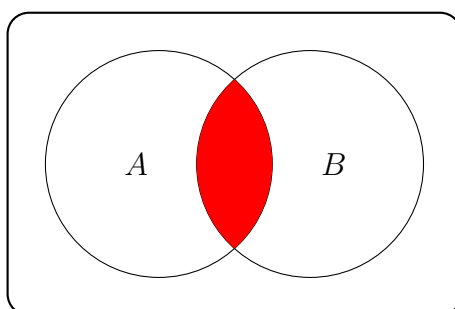
It is often convenient to use a diagram showing the relation between two sets, called a *Venn* diagram. The large, unnamed set, of which both sets are subsets (in our case, the integers), is represented by a large rectangle. The sets we are interested in are portrayed as parts of this, typically as circles. Inclusion is expressed as in Figure 2.1.

Figure 2.1: B is a subset of A 

Definition 2.1.2 (Intersection). The next property we will discuss is the *intersection* of two sets A and B . This is the set consisting of all elements that are both in A and in B , and is written

$$A \cap B = \{x \mid x \in A \text{ and } x \in B\}$$

The corresponding Venn diagram is displayed in Figure 2.2.

Figure 2.2: Intersection of A and B 

Example 2.1.3. Let two sets of integers be defined by

$$\begin{aligned} A &= \{2, 6, 7, 13\} \\ B &= \{5, 6, 13, 14, 15\} \end{aligned}$$

By inspection, we see that 6 and 13 are in both sets, and no other elements are in both, so

$$A \cap B = \{6, 13\}.$$

Exercise 2.1.4. What is the intersection of the two sets

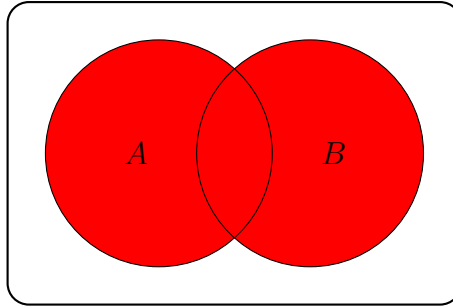
$$\begin{aligned} A &= \{-4, 3, 9\} \\ B &= \{3, 9, 13, 14, 15\} \end{aligned}$$

Definition 2.1.5 (Union). We can also form the *union* of two sets A and B , consisting of the elements that are elements in one or both of A and B . The symbol for this is $\cup$, which we should think of as the letter U (short for union).

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

See Figure 2.3 for the Venn diagram.

Figure 2.3: Union of A and B



Example 2.1.6. Let two sets of integers be defined by

$$\begin{aligned} A &= \{2, 6, 7, 13\} \\ B &= \{5, 6, 13, 14, 15\} \end{aligned}$$

By inspection, we see that the union is

$$A \cup B = \{2, 5, 6, 7, 13, 14, 15\}.$$

Exercise 2.1.7. What is the union of the two sets

$$\begin{aligned} A &= \{-4, 3, 9\} \\ B &= \{3, 9, 13, 14, 15\} \end{aligned}$$

Definition 2.1.8 (Disjoint). Two sets A and B are called disjoint if they have no elements in common, that is if the intersection is empty. See Figure 2.4. This can be written $A \cap B = \emptyset$.

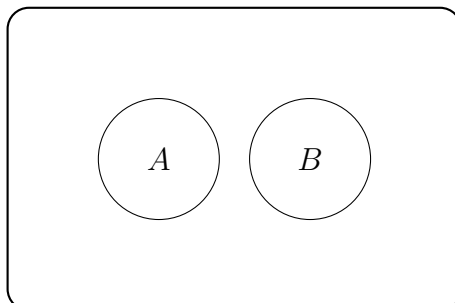
Example 2.1.9. The two sets

$$\begin{aligned} A &= \{2, 6, 7, 13\} \\ B &= \{5, 6, 13, 14, 15\} \end{aligned}$$

are not disjoint, since for instance 6 is a common element. The two sets

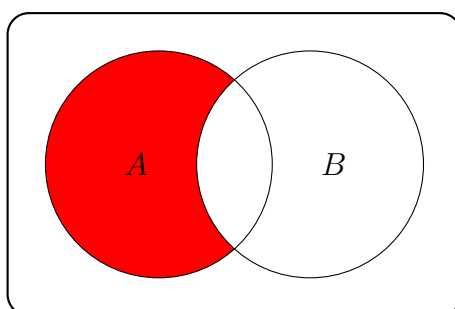
$$\begin{aligned} C &= \{2, 6, 7, 13\} \\ D &= \{15, 16, 23, 34, 45\} \end{aligned}$$

are disjoint.

Figure 2.4: A and B are disjoint

Definition 2.1.10 (Difference). The *difference* between the set A and the set B , written $A \setminus B$, consist of those elements in A that are not in B . $A \setminus B$ is read “A minus B”.

See Figure 2.5 for the Venn diagram.

Figure 2.5: The difference $A \setminus B$ 

Example 2.1.11. Consider the two sets

$$\begin{aligned} A &= \{2, 6, 7, 13\} \\ B &= \{5, 6, 13, 14, 15\} \end{aligned}$$

Then $A \setminus B = \{2, 7\}$ whereas $B \setminus A = \{5, 14, 15\}$.

Exercise 2.1.12. Find $A \setminus B$ and $B \setminus A$ when

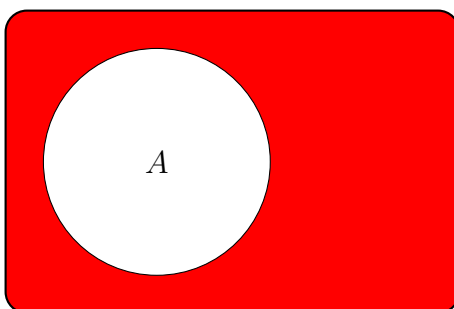
$$\begin{aligned} A &= \{-4, 3, 9\} \\ B &= \{3, 9, 13, 14, 15\} \end{aligned}$$

Definition 2.1.13 (Complement). Finally, we also include a notion related to only one set, namely the *complement* of a set A . This is the set of elements in the large set that are not in A , and is written $\overline{A}$.

$$\overline{A} = \{x \mid x \notin A\}$$

The Venn diagram is displayed in Figure 2.6.

Figure 2.6: The complement of A



Example 2.1.14. If the set A is the set of even integers,

$$A = \{\dots, -4, -2, 0, 2, 4, \dots\}$$

the complement of A is the set of odd integers

$$\overline{A} = \{\dots, -3, -1, 1, 3, 5, \dots\}$$

2.2 De Morgan's laws

The properties of unions and intersections are interchanged by negation. This statement is precisely formulated in de Morgan's laws.

Theorem 2.2.1 (de Morgan's laws - sets). *For two sets A and B contained in a common large set, we have the following two equalities.*

$$\overline{A \cup B} = \overline{A} \cap \overline{B} \tag{2.1}$$

$$\overline{A \cap B} = \overline{A} \cup \overline{B} \tag{2.2}$$

Example 2.2.2. In a group of people in their sixties, consider the sets A = the set of women and B = the set of retired people. Consider first Equation 2.1. On the

left, we have the set of people that are not women or retired. On the right, we have those that both are not women and not retired. Both groups coincide with the group of men that are still working.

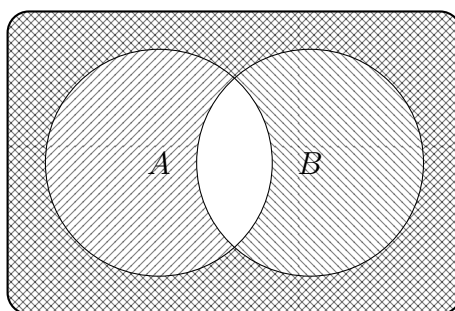
Then look at Equation 2.2. On the left, we have the people that are not both women and retired. On the right, we have those that are not women or not retired. In both cases, this includes all men (retired or not) and all that are still working (women or men).

Proof. We will prove Equation 2.1 in detail, and draw a corresponding Venn diagram. For Equation 2.2, we will only draw the Venn diagram, and leave the details to Exercise 2.2.3.

To prove that two sets are equal, we can prove that the first is contained in the second, and that the second is contained in the first. We begin by proving that $\overline{A \cup B} \subset \overline{A} \cap \overline{B}$. Assume $x \in \overline{A \cup B}$. Since $A \cup B$ consists of all elements that are in one or both of A and B , the complement consists of all elements that are outside both A and B , so x is outside both sets. Since $\overline{A}$ is the set of elements not in A , $x \in \overline{A}$. Similarly, since $\overline{B}$ is the set of elements not in B , $x \in \overline{B}$. Since x is in both these sets, it is also in the intersection, so $x \in \overline{A} \cap \overline{B}$.

We now prove that $\overline{A} \cap \overline{B} \subset \overline{A \cup B}$. Assume that $x \in \overline{A} \cap \overline{B}$. Then $x \in \overline{A}$, so x is not in A . Similarly, since $x \in \overline{B}$, x is not in B . Therefore, x cannot be in the set $A \cup B$ of elements that are in one or both these sets, so it must be in the complement: $x \in \overline{A \cup B}$. See the Venn diagram in Figure 2.7, where $\overline{A}$ is drawn with lines going down to the right, $\overline{B}$ similarly with lines going up to the right. Thus, the set in Equation 2.1 is the crossed set.

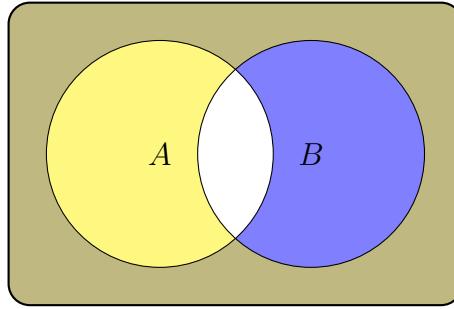
Figure 2.7: Venn diagram for $\overline{A \cup B}$



□

Exercise 2.2.3. Prove Equation 2.2 in a similar way. See Figure 2.8 for the Venn diagram. Try to identify what the colours signify. (Partial answer: Everything that is coloured form the set in Equation 2.2).

Figure 2.8: Venn diagram for $\overline{A \cap B}$



2.3 Sets and Boolean algebra

The relation between operators on sets and logical operators is basically simple, but it can be difficult to keep track of everything in compound statements. Consider sets as being contained in a large set as before, and attach to a set A the Boolean expression $x \in A$. If A is the set of even integers, this expression is true for x an even integer, and false otherwise, that is if x is an odd integer. Then reading the definitions of intersection, union and complement tells us that there is a table of corresponding symbols:

Sets	Logic
$A \cap B$	$x \in A \wedge x \in B$
$A \cup B$	$x \in A \vee x \in B$
$\overline{A}$	$\neg(x \in A)$

See Exercise 2.6.6 for equivalence and implication.

This correspondence allows us to go back and forth between logic and sets; for now let us only formulate the statement of de Morgan's laws in logic.

Theorem 2.3.1 (de Morgan's laws - logic). *The following two identities always hold for Boolean expressions.*

$$\neg(x \vee y) = (\neg x) \wedge (\neg y) \quad (2.3)$$

$$\neg(x \wedge y) = (\neg x) \vee (\neg y) \quad (2.4)$$

Proof. We will give a direct proof of Equation 2.3, and leave the proof of Equation 2.4 to Exercise 2.3.2. We will prove the theorem in a boring way, simply by computing the truth tables for the two statements and check that they are equal.

x	y	$\neg x$	$\neg y$	$x \vee y$	$\neg(x \vee y)$	$(\neg x) \wedge (\neg y)$
0	0	1	1	0	1	1
0	1	1	0	1	0	0
1	0	0	1	1	0	0
1	1	0	0	1	0	0

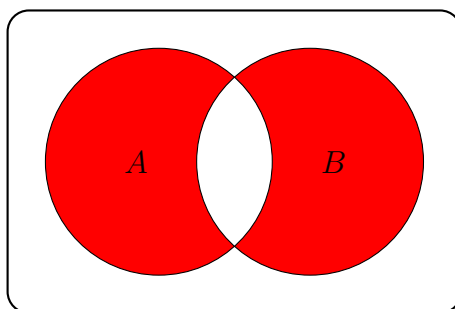
Since the last two columns are equal, the two expressions have the same truth-values for all constituent Boolean expressions, and the theorem is proved. $\square$

Exercise 2.3.2. Prove Equation 2.4 in a similar way.

Example 2.3.3. Consider the statements x = “The computer is fast” and y = “The computer is new”. Equation 2.3 says that if the computer is not new or fast, it is both not new and not fast. Equation 2.4 says that if the computer is not both new and fast, it is not new or not fast.

The set-theoretical version of $\oplus$ is called the *symmetric difference*. The symmetric difference of A and B is the set of elements that are in one, but not both, of A and B . See Figure 2.9. Note that the symmetric difference of A and B is equal to the union of $A \setminus B$ and $B \setminus A$.

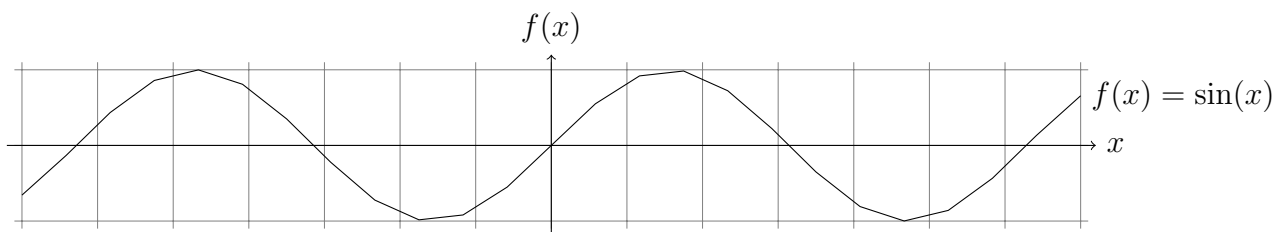
Figure 2.9: Symmetric difference of A and B



2.4 Relations

A *relation* on a set A is a collection of pairs of elements of A . If the pair (x, y) is in a relation called R , we will often write $x \sim_R y$ (read “ x is R -related to y ”). If the name of the relation is unimportant, we will simply write $x \sim y$. In order to get anything meaningful, we need to impose various conditions.

Example 2.4.1. The graph of the sine function consists of all pairs of real numbers (x, y) such that $y = \sin x$. This is a set of pairs of real numbers, so it is a relation. Formally, $x \sim y$ if and only if $y = \sin x$.



We say that a relation is a *function* from A to A if, for each element $x \in A$, there is exactly one pair (x, y) in the relation. y is then the function value of x .

We will consider functions in Chapter 6.

We proceed to consider some properties of relations that are particularly important in computer science and mathematics.

Definition 2.4.2. Let A be a set with a relation R .

- i) R is called *reflexive* if $x \sim_R x$ for all $x \in A$.
- ii) R is called *symmetric* if $x \sim_R y$ implies $y \sim_R x$ for all $x, y \in A$.
- iii) R is called *transitive* if $x \sim_R y$ and $y \sim_R z$ implies $x \sim_R z$ for all $x, y, z \in A$.

If all three properties hold, R is called an equivalence relation.

Example 2.4.3. For real numbers, define a relation by saying that x and y are related if $x \leq y$. This relation is reflexive since $x \leq x$, and transitive since $x \leq y$ and $y \leq z$ implies $x \leq z$. It is not symmetric. To check this, note that it is enough to find a single counter-example. So since $0 \leq 1$ but $1 \leq 0$ is false, this relation is not symmetric.

Example 2.4.4. Define a relation on integers by saying that x and y are related if $x - y$ is divisible by 2. This is the same as saying that x and y are related if both are even or if both are odd. This relation is in fact an equivalence relation.

Exercise 2.4.5. Show that the relation from Example 2.4.4 is an equivalence relation by checking the three conditions.

Definition 2.4.6. Let A be a set with a relation R .

- iv) R is called *anti-symmetric* if $x \sim_R y$ and $y \sim_R x$ implies that $x = y$ for all $x, y \in A$.

v) R is called *total*¹ if $x \sim_R y$ or $y \sim_R x$ is true for all $x, y \in A$.

A relation satisfying i), iii) and iv) is called a *partial order*. If also v) holds, R is called a *total order*. Linear order is another common term used for a total order.

Example 2.4.7. Define a relation on pairs of real numbers by saying that $(x_1, y_1) \sim (x_2, y_2)$ if $x_1 \leq x_2$ and $y_1 \leq y_2$. For instance,

$$(1, 17) \sim (2, 22) \text{ but } (2, 3) \text{ is not related to } (4, 1).$$

This relation is reflexive ($(x, y) \sim (x, y)$ since $x \leq x$ and $y \leq y$). It is transitive (if $(x_1, y_1) \sim (x_2, y_2)$ and $(x_2, y_2) \sim (x_3, y_3)$ then $x_1 \leq x_2 \leq x_3$ and $y_1 \leq y_2 \leq y_3$ so that $(x_1, y_1) \sim (x_3, y_3)$). It is anti-symmetric (if $(x_1, y_1) \sim (x_2, y_2)$ and $(x_2, y_2) \sim (x_1, y_1)$ then $x_1 \leq x_2 \leq x_1$, so $x_1 = x_2$, and similarly for the y s, so $(x_1, y_1) = (x_2, y_2)$). Thus, it is a partial order. It is not, however, total, since for instance $(2, 3)$ and $(4, 1)$ are incomparable.

Exercise 2.4.8. Show that the relation coming from $\leq$ in Example 2.4.3 is in fact a total order.

2.5 Elementary counting

One of the first things to consider about a set is the number of elements. This is sometimes called the *cardinality* of the set. We write the number of elements in a set using absolute value signs, as in

$$\begin{aligned} S &= \{0, 2, 4, 7\} \\ |S| &= 4 \end{aligned}$$

Let us list a few connections between the set operations we have introduced and the number of elements in the relevant sets. The formulas below are correct as long as they are meaningful, which essentially means that the sets are finite.

1. If $A \subset B$ then $|A| \leq |B|$
2. $|A \cup B| \leq |A| + |B|$
3. $|A \cap B| \leq |A|$ and $|A \cap B| \leq |B|$
4. If $A \subset B$ then $|B \setminus A| = |B| - |A|$
5. If the *universe* has n elements, then $|\overline{A}| = n - |A|$

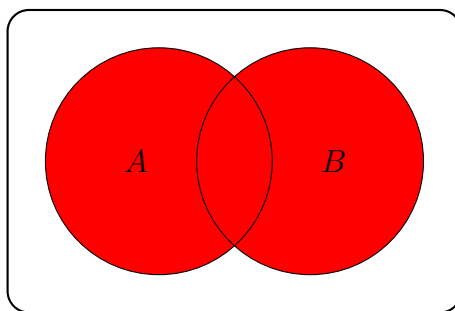
¹Standard only for orders

6. If A and B are disjoint, $|A \cup B| = |A| + |B|$
7. If $A \subset B$ then $|B| = |B \setminus A| + |A|$
8. $|A \cup B| = |A| + |B| - |A \cap B|$

To see why this is true requires only reading the relevant definitions, maybe with a short extra argument. For instance, if $A \subset B$ then B contains all the elements of A , and possibly some more. Therefore $|A| \leq |B|$.

The last of the listed properties is perhaps the least intuitive. Consider the Venn diagram for the union again. If we want to count the number of elements in $A \cup B$,

Figure 2.10: Union of A and B again



we can start by counting the number of elements in A and in B . Then, however, we have counted the elements in the intersection twice. Subtracting the number of elements in $A \cap B$ ensures that each element in the union is counted exactly once.

Example 2.5.1. Consider the two sets

$$A = \{1, 3, 5, 7, 9, 11\} \text{ and } B = \{1, 2, 3, 4, 5\}$$

Then $|A| = 6$, $|B| = 5$, so $|A| + |B| = 11$. But the three elements 1, 3, 5 have now been counted twice, so that the number of elements in the union is $|A \cup B| = 5 + 6 - 3 = 8$.

Example 2.5.2. We have introduced a relation as a set of pairs. How many pairs are there?

If the set $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, the set of pairs is the set $\{(0, 0), (0, 1), \dots, (9, 9)\}$. For each element in A , we can form a pair using this element as the first part of the pair, and any element for the second. This can be repeated for each element of A , so if $|A| = n$ then there are $n \cdot n = n^2$ pairs directly from the definition of multiplication used in elementary school!

Similarly, if we consider lists of elements in A of length 3 (a pair is a list of length 2), we will have n^3 possibilities. For lists of length m we will have n^m possibilities.

Let us look at some concrete examples of counting such lists.

Example 2.5.3. If the set $A = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$, the lists of length m correspond to integers that can be written with m digits. There are 10^m such integers.

If the set A is the English alphabet, the lists of length m correspond to words (not necessarily real, meaningful words) with m letters. There are 26^m such words.

If the set $A = \{0, 1\}$, the lists of length m are sequences of zeroes and ones of length m . There are 2^m such lists.

2.6 Exercises

Exercise 2.6.1. Let P be the set of all subsets of a set A . Define a relation on P by saying that two subsets X and Y in A are related if $X \subset Y$. Show that this is a partial order, but not a total order (if A has at least two elements).

Exercise 2.6.2. Let P be the set of all subsets of a set A with n elements. We are going to prove the number of elements in P is 2^n .

(a). List all the elements of P in case n is two or three.

(b). To any subset of A , associate a string of zeroes and ones of length n by putting a 1 in place i of the string if the i th element is in the subset, a 0 otherwise. If $A = \{a, b, c, d, e, f\}$, what is the string associated to the subset $\{a, e, f\}$?

(c). Reversing the above operation, what is the subset associated to the string 001101?

(d). Explain why there are exactly 2^n elements in P .

Exercise 2.6.3. Draw two Venn diagrams, each representing the three sets A , B and C . In the first, make it clear that $A \cap B \cap C \neq \emptyset$, in the second $A \cap B \cap C = \emptyset$, but $A \cap B \neq \emptyset$, $A \cap C \neq \emptyset$ and $B \cap C \neq \emptyset$.

Exercise 2.6.4. If A, B and C are subsets of a common set, and $A \subset B$, then

$$A \cap C \subset B \cap C$$

Draw a Venn diagram showing this situation. Prove the statement.

Exercise 2.6.5. The *Cartesian product* of two sets A and B is the set of all pairs (a, b) where $a \in A$ and $b \in B$. This is sometimes written $A \times B$. For instance, the Cartesian product of $\mathbb{R}$ and $\mathbb{R}$ is the plane $\mathbb{R}^2$.

(a). If there are n elements in A , and m elements in B , how many elements are there in $A \times B$?

- (b). If $A' \subset A$ and $B' \subset B$, explain why $A' \times B' \subset A \times B$.
- (c). Find $\mathbb{Z} \times \mathbb{Z}$ as a subset of $\mathbb{R}^2$.
- (d). Explain why $A \times (B \times C) = (A \times B) \times C$. Hint: Consider the set of triples (a, b, c) .

Exercise 2.6.6. To get set-theoretic versions of equivalence and implication, we note that $x \in A \leftrightarrow x \in B$ is true if x is in both A and B , or outside both A and B . Therefore the set corresponding to $x \in A \leftrightarrow x \in B$ is

$$(A \cap B) \cup (\overline{A} \cap \overline{B}).$$

- (a). Similarly, show that $x \in A \rightarrow x \in B$ corresponds to the set $(A \cap B) \cup \overline{A}$.

Recall that a statement is a tautology if it is always true.

- (b). Show that $x \in A \rightarrow x \in B$ is a tautology if and only if $A \subset B$.
- (c). What corresponds to $x \in A \leftrightarrow x \in B$ being a tautology?

Exercise 2.6.7. In this exercise, we define a couple of sets of integers, and let Java compute the set-theoretical operations. We first fix our universe, which will consist of the 11 elements

$$universe = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

A and B will be some more or less random subsets. The initialization of what we have discussed so far can be done as follows:

```
int [] universe = {0,1,2,3,4,5,6,7,8,9,10};
int [] setA = {0,1,2,3,4,5};
int [] setB = {2,3,5,7};
```

To check whether an element, say 3, is in the union of A and B , we can use the code block

```
boolean isInUnion = false;
for (int numberA : setA) {
    if (3 == numberA) {
        isInUnion = true;
    }
}
for (int numberB : setB) {
    if (3 == numberB) {
        isInUnion = true;
    }
}
```

```

}
if (isInUnion) {
    System.out.printf("%d is in the union\n", 3);
}

```

To check for all elements, we loop through the entire universe:

```

for (int number : universe) {
    boolean isInUnion = false;
    for (int numberA : setA) {
        if (number == numberA) {
            isInUnion = true;
        }
    }
    for (int numberB : setB) {
        if (number == numberB) {
            isInUnion = true;
        }
    }
    if (isInUnion) {
        System.out.printf("%d is in the union\n", number);
    }
}

```

Modify this programme to find the intersection. Also, find the complement of A .

For the more ambitious, modify the code so that fewer equalities are checked. Use for instance **break**.

Exercise 2.6.8. In this exercise, we will use Java to check whether a relation is a partial order. First, we define a set and a relation on it. This time, we use **char**.

```

char[] setA = {'a', 'b', 'c', 'd', 'e', 'f'};
char[][] relation = {{ 'a', 'a' }, { 'b', 'b' }, { 'c', 'c' }, { 'd', 'd' },
    { 'e', 'e' }, { 'f', 'f' }, { 'a', 'd' }, { 'a', 'f' }, { 'c', 'd' },
    { 'c', 'e' }, { 'c', 'f' }, { 'd', 'f' } };

```

We need to check reflexivity, anti-symmetry and transitivity.

To check for reflexivity, we create a pair (c, c) for each element c of A , and checks if it is in the relation. We then count how many times this happens, and if it happens as many times as there are elements in A , the relation is reflexive.

```

int reflexivePairs = 0;
for (char c : setA) {
    char[] cPair = new char[2];

```

```

    cPair[0] = c;
    cPair[1] = c;
    for (char[] pair : relation) {
        if (pair[0] == cPair[0] && pair[1] == cPair[1]) {
            reflexivePairs++;
        }
    }
}
if (reflexivePairs == setA.length) {
    System.out.println("The relation is reflexive");
}

```

Note that we checked the equality of pairs by checking both entries.

To check for anti-symmetry, we search for pairs (x, y) and (y, x) . If both are in the relation, we check whether $x = y$. Unless we find a counter-example, this condition holds. Therefore, we initialize

```
boolean isAntiSymmetric = true;
```

and change the value if we find a counter-example.

```

boolean isAntiSymmetric = true;
for (char[] pair : relation) {
    for (char[] newPair : relation) {
        if (newPair[0] == pair[1] && newPair[1] == pair[0]
            && pair[0] != pair[1]) {
            isAntiSymmetric = false;
        }
    }
}
if (isAntiSymmetric) {
    System.out.println("The relation is antisymmetric");
}

```

The main task left to you is to check for transitivity. As for antisymmetry, the condition holds unless we find a counter-example. We need to check, for all two pairs (x, y) and (y, z) , whether (x, z) is also in the relation.

Finally, write a statement if all three conditions are fulfilled.

Exercise 2.6.9. A *contradiction* is in Boolean algebra a statement that is always false. For instance, $x \wedge \neg x$ is a contradiction.

(a). Is $(x \wedge y) \vee (\neg x \wedge \neg y)$ a contradiction?

(b). Is $(x \wedge y) \wedge (\neg x \wedge \neg y)$ a contradiction?

(c). Is $\neg((x \rightarrow y) \wedge x) \rightarrow y$ a contradiction?

We now look at the set-theoretic variant of contradictions.

(d). Translate $x \wedge \neg x$ into a set-theoretic statement.

(e). Explain how the empty set $\emptyset$ corresponds to the notion of contradiction.

(f). Find the set-theoretic version of the statement in a), and check if it corresponds to the empty set.

(g). Find the set-theoretic version of the statement in b), and check if it corresponds to the empty set.

Exercise 2.6.10. We will count the number of true and false statements in a Java generated truth table, as in Exercise 1.4.3. The expressions we will be considering have three variables x, y, z . In the beginning of this exercise, consider the expression $x \wedge (y \rightarrow z)$.

(a). Declare an array of Booleans, that is of type `boolean[]`, to hold the values of the truth table. How long must the array be? How long would it have to be if there were 12 variables instead of just 3?

(b). Find a loop structure to generate the table as in Exercise 1.4.3. Inside the loop, fill the table with values instead of printing the values.

(c). Iterate through the table to find the number of times `true` appears. Write a statement about the number of true and false values.

(d). Write an additional statement if the expression is a contradiction or a tautology.

(e). Modify the programme to test the expressions $(x \wedge z) \wedge (y \wedge \neg z)$ and $(x \vee z) \vee (y \vee \neg z)$. Does it work as planned?

Chapter 3

Integers

The first numbers we learn about as children are integers. These are also the numbers that are most easily handled by the computer. We write

$$\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$$

The *positive integers* are

$$\{1, 2, 3, 4, 5, \dots\}.$$

The *non-negative*:

$$\{0, 1, 2, 3, 4, 5, \dots\}.$$

The *negative*:

$$\{-1, -2, -3, -4, -5, \dots\}.$$

The *non-positive*:

$$\{0, -1, -2, -3, -4, -5, \dots\}.$$

Using the notation for conditions, we can write for instance the non-negative integers as

$$\{n \in \mathbb{Z} \mid n \geq 0\}. \quad (3.1)$$

Exercise 3.0.1. Write the positive, negative and non-positive integers using the notation for conditions, as in Equation 3.1.

We usually write numbers in base 10, using the digits

$$0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9$$

and position to represent integers. For instance

$$240036$$

To make this work, we have to make sure that the two numbers have the same number of digits, by padding the shorter number with zeroes if necessary. Also note that the answer might contain one digit more than the numbers we start with, so we need to reserve enough space for this extra digit.

Adding more than two numbers can be achieved by adding the first two, then adding the third to the answer, and so on. It is also possible to use a direct approach, starting by summing the ones of all the numbers (for instance when summing the scores in the game Yatzy).

Multiplication

Multiplying a single digit with a number is quite similar to adding two numbers, except that we need to carry higher numbers than 1:

$$\begin{array}{r} \\ \\ \\ \hline 6 5 8 \\ 3 5 8 \end{array}$$

Here $6 \cdot 8 = 48$, so we need to carry the 4, and so on.

Exercise 3.1.1. Write down the precise steps for multiplication of a single digit with an arbitrary integer, similar to the description of addition.

What if both numbers have several digits? We can use what we've seen already, together with the position system. Calculating $73 \cdot 278$ is done by thinking about it as $73 = 7 \cdot 10 + 3$ and using

$$73 \cdot 278 = (7 \cdot 10 + 3)278 = 7 \cdot 10 \cdot 278 + 3 \cdot 278 = 7 \cdot 2780 + 3 \cdot 278.$$

If there were more digits, we would get one multiplication for each digit, and more numbers to add in the end. It is customary to write the multiplication in a table, for instance as

$$\begin{array}{r|l} 7 & 3 & . & 2 & 7 & 8 \\ \hline & & & 8 & 3 & 4 \\ & 1 & 9 & 4 & 6 & 0^* \\ \hline 2 & 0 & 2 & 9 & 4 & \end{array} \quad \begin{array}{l} 3 \cdot 278 \\ 7 \cdot 2780 \end{array} \quad (3.2)$$

(The zero labelled with $*$ is often omitted).

Note that in performing addition and multiplication of integers in base 10, we have only used our knowledge of the result of the operations on single digits, and the position system.

3.2 Binary numbers

We usually write numbers in base 10, so that the position tells us which power of 10 to use. We could write numbers in any base (possibly at least 2) in the same way. For instance, in base 3, the number 212 would mean $2 \cdot 3^2 + 1 \cdot 3^1 + 2 \cdot 3^0 = 23$ in ordinary numbers. To avoid confusion, if we use different bases, we write the base as a subscript:

$$212_3 = 23_{10}$$

We will usually not write base 10 explicitly.

Binary numbers are numbers written in base 2. To represent numbers in binary, we need only the two digits 0 and 1, and the position system. Thus

$$1001101_2 = 1 \cdot 2^6 + 0 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 64 + 8 + 4 + 1 = 77 \quad (3.3)$$

Exercise 3.2.1. What is 110110_2 ?

To get started with addition and multiplication, we need to know the addition table and the multiplication table. In binary, these tables are much easier than in base 10:

$$\begin{array}{c|c|c} + & 0 & 1 \\ \hline 0 & 0 & 1 \\ 1 & 1 & 10 \end{array} \quad \text{and} \quad \begin{array}{c|c|c} \cdot & 0 & 1 \\ \hline 0 & 0 & 0 \\ 1 & 0 & 1 \end{array} \quad (3.4)$$

Exercise 3.2.2. Write down the addition and multiplication tables for arithmetic in base 3, using the digits 0, 1 and 2.

We add numbers using the position system as we did in base 10.

Example 3.2.3. For instance, adding 11011 and 1001 results in

$$\begin{array}{rcccccc}
 & 1 & & 1 & & 1 & & 1 \\
 & & 1 & & 1 & & 0 & & 1 & & 1 \\
 + & & & 1 & & 0 & & 0 & & 1 \\
 \hline
 1 & 0 & 0 & 0 & 1 & 0 & 0
 \end{array}$$

Exercise 3.2.4. Add 11011001_2 and 101101_2 .

Multiplication is also done in the same way, but since the digits 0 and 1 are very easy to multiply by, it seems easier.

Example 3.2.5. Multiply 101_2 and 1101_2 .

$$\begin{array}{r}
 1 \ 0 \ 1 \ . \ 1 \ 1 \ 0 \ 1 \\
 \hline
 1 \ 1 \ 0 \ 1 \\
 0 \ 0 \ 0 \ 0 \\
 1 \ 1 \ 0 \ 1 \\
 \hline
 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 1
 \end{array}$$

Exercise 3.2.6. Was this unclear? Go through the calculation, keeping track of what is carried.

Exercise 3.2.7. Multiply 1011_2 and 1010_2 .

How do we find the binary representation of a number? The basic idea is to perform division repeatedly, and keep track of the remainders.

Theorem 3.2.8. *Let m and n be positive integers. Then there exist uniquely defined integers q and r such that*

$$m = q \cdot n + r \text{ and } 0 \leq r < n$$

q is called the quotient and r is called the remainder when m is divided by n .

Proof. The sequence $n, 2n, 3n, \dots$ is increasing to infinity, and sooner or later it will pass m . Let q be the greatest integer such that $nq \leq m$. Then $m - nq < n$, otherwise $n(q+1) \leq m$ as well. Define $r = m - nq$. Then q and r satisfy the conditions in the theorem.

To prove that they are unique, consider two potentially different expressions

$$m = qn + r = q'n + r'.$$

Here all numbers are integers, and $0 \leq r < n$ and $0 \leq r' < n$. Then we have

$$0 = m - m = qn + r - (q'n + r')$$

Then $n(q - q') = r' - r$, so $r' - r$ is divisible by n . Since both the numbers are smaller than n , they must be equal. Then $n(q - q') = 0$, so $q - q' = 0$ and these numbers are also equal. $\square$

To find the binary representation of a number, we repeatedly divide quotients by 2, and then read the remainders backwards.

Example 3.2.9. Write 2093 in binary.

The divisions are

$$\begin{array}{rcl}
 2093 & = & 1046 \cdot 2 + 1 \\
 1046 & = & 523 \cdot 2 + 0 \\
 523 & = & 261 \cdot 2 + 1 \\
 261 & = & 130 \cdot 2 + 1 \\
 130 & = & 65 \cdot 2 + 0 \\
 65 & = & 32 \cdot 2 + 1 \\
 32 & = & 16 \cdot 2 + 0 \\
 16 & = & 8 \cdot 2 + 0 \\
 8 & = & 4 \cdot 2 + 0 \\
 4 & = & 2 \cdot 2 + 0 \\
 2 & = & 1 \cdot 2 + 0 \\
 1 & = & 0 \cdot 2 + 1
 \end{array} \tag{3.5}$$

Thus we have

$$2093 = 100000101101_2$$

Exercise 3.2.10. Write 3027 in binary.

The reason this works is that division by 2 removes the last digit in the binary representation, in the same way that division by 10 removes the last digit in ordinary numbers (moving the decimal point). The procedure works in any base. For instance, we can see that this works for 2093 in base 10:

$$\begin{array}{rcl}
 2093 & = & 209 \cdot 10 + 3 \\
 209 & = & 20 \cdot 10 + 9 \\
 20 & = & 2 \cdot 10 + 0 \\
 2 & = & 0 \cdot 10 + 2
 \end{array}$$

As expected, the digits in 2093 are found by reading the remainders backwards.

3.3 Integers and integer types in Java

Computers are well suited to simple arithmetic tasks. However, there are some pitfalls, mostly because of limitations of various data types. In this section, we will introduce the approach Java takes to integers, and see what problems might arise.

One of the integer types is called “integer”, and abbreviated

`int`

If we write

```
System.out.printf("%d+ %d = %d", 23, 45, 23 + 45);
```

Java will perform the operation using the integer type. Here the letter “d” stands for “decimal integer”, that is for integers written in base 10. We can also declare integer variables to get the same effect:

```
int sum1 = 23;
int sum2 = 45;
System.out.printf("%d+ %d = %d", sum1, sum2, sum1 + sum2);
```

To find a problem, let us change the above code slightly.

```
int sum1=2146469892;
int sum2=426543645;
System.out.printf("%14d%n + %11d%n = %11d%n", sum1, sum2, sum1 + sum2);
```

Running this code results in

```
2146469892
+ 426543645
= -1721953759
```

This is obviously wrong! The problem is that Java only allocates a fixed number of bits to each integer. If the answer to a calculation requires more bits than allocated, the answer will be wrong. For `int`, Java allocates 32 bits, and this type can represent any integer from $-2,147,483,648$ to $2,147,483,647$ (there are exactly $2^{32} = 4294967296$ numbers in this set). We can see this more clearly by executing

```
System.out.println(2147483647);
System.out.println(2147483647+1);
```

The result is

```
2147483647
-2147483648
```

For now, we will just remark that the first of the 32 bits represents sign, so that the 2^{31} numbers

$$0, 1, \dots, 2147483647$$

have first bit 0, and the 2^{31} numbers

$$-2147483648, -2147483547, \dots, -1$$

have first bit 1. 0 is here counted among the positive numbers; this is the reason that there is one more negative number than there are positive numbers.

In the same way, Java allocates 16 bits for `short` and 64 bits to `long`. The biggest `long` is

$$9,223,372,036,854,775,807 = 2^{63} - 1.$$

You might think that 19 digits are enough for most purposes, but for instance in cryptography, this is not enough. Modern RSA-codes typically use primes with about 300 digits.

The number of bits

For a given problem, we can consider what integer type to use. If we use an integer type with n bits, we will have 2^n different numbers represented in this system. For instance, these are the $2^4 = 16$ possibilities with 4 bits:

```
0000 0100 1000 1100
0001 0101 1001 1101
0010 0110 1010 1110
0011 0111 1011 1111
```

Here is a table for the first powers of two, telling us how many numbers we can represent with a low number of bits:

n	2^n	n	2^n
0	1	11	2048
1	2	12	4096
2	4	13	8192
3	8	14	16384
4	16	15	32768
5	32	16	65536
6	64	17	131072
7	128	18	262144
8	256	19	524288
9	512	20	1048576
10	1024	$\vdots$	$\vdots$

Java usually represents the numbers -2^{n-1} to $2^{n-1} - 1$ with n bits. It is also possible to represent only non-negative numbers, in which case the numbers represented are 0 to $2^n - 1$. The character type `char` is formally an example of this. Here is the list of basic integer types in Java:

Table of integer types and their values		
name	values	bits
<code>byte</code>	-128 to 127	8
<code>short</code>	-32,768 to 32,767	16
<code>int</code>	-2,147,483,648 to 2,147,483,647	32
<code>long</code>	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	64
<code>char</code>	0 to 65,535	16

When two numbers of the same type are added, the answer gets the same type. But what happens when Java encounters different types? Different programming languages handle this differently. Java will change the type of one number so that the two summands have the same type, and the answer gets this type. The type with more bits is chosen. We can also explicitly change the type. This is called *casting*. If Java does it for us, we call the casting “implicit”. If we do it ourselves, we say it is “explicit”.

```
int anInteger = 12;
long aLong = (long) anInteger;
```

If you want to declare a **long** by assigning a numerical value, write 'L' after the numerical value:

```
long anotherLong = 23512L;
```

Consider this example programme, showing what happens when two numbers, declared as **int**, are converted and added in various ways.

```
int i1 = 1147538874;
int i2 = 2004213452;
int i3 = i1 + i2;
long i4 = i1 + i2;           /* (1) */
long i5 = i1;
i5 = i5 + i2;                /* (2) */
int i6 = (int) i5;
short i7 = (short) i6;
short i8 = (short) i5;      /* (3) */
long i9 = (long) i1 + (long) i2; /* (4) */
System.out.printf("i1 is %d\n", i1);
System.out.printf("i2 is %d\n", i2);
System.out.printf("i3 is %d\n", i3);
System.out.printf("i4 is %d\n", i4);
System.out.printf("i5 is %d\n", i5);
System.out.printf("i6 is %d\n", i6);
System.out.printf("i7 is %d\n", i7);
System.out.printf("i8 is %d\n", i8);
System.out.printf("i9 is %d\n", i9);
```

The two **ints** are too large to have their sum represented as an **int**, so **i3** is not what you would expect. In note (1), the addition is performed as **int**, and the (incorrect) answer is cast to **long**. In note (2), one of the summands is a **long**, so the addition is performed correctly. When going from a longer to a shorter type, information is lost. There is, however, some consistency, so in note (3) we get the same answer by

converting from a `long` to a `short` as we get when going via an `int`. Finally, in note (4), we explicitly convert to `long` before performing the addition, thus making sure that the computation will be performed on `longs`.

The output from the programme is

```
i1 is 1147538874
i2 is 2004213452
i3 is -1143214970
i4 is -1143214970
i5 is 3151752326
i6 is -1143214970
i7 is -4986
i8 is -4986
i9 is 3151752326
```

Note that converting from a shorter to a longer type can be done implicitly, but if we convert from a longer type to a shorter, we have to do it explicitly. In this way Java prevents us from accidentally losing information. Writing

```
int i1 = 14;
short i2 = i1; // Error
```

leads to a compilation error.

Binary logarithm

Recall that a^b represents the number a multiplied with itself b times. If $a > 0$, we can also ask for this value when b is rational, say $b = m/n$. Then

$$a^{m/n} = \sqrt[n]{a^m}$$

For real values of b , we define a^b by taking the limit over nearby rational numbers.

The inverse operation is called $\log_a$, the logarithm in base a . It can be defined by saying that

$$\log_a(a^b) = b.$$

Thus $\log_a(x)$ is defined for $x > 0$, and not defined for $x \leq 0$. All real numbers are values of the logarithm. We will be interested in this primarily for $a = 2$, and $\log_2$ is referred to as the *binary logarithm*. An example is

$$\log_2(8) = 3 \text{ since } 2^3 = 8$$

Logarithms aren't very easy to compute by hand, so we usually resort to computers or calculators for the numerical values.

If we need to represent n different values, how many bits must be allocated? We will need at least $\log_2(n)$ bits! For instance, if we need the numbers -1000 to 1000 (there are 2001 different values), we need at least

$$\log_2(2001) \approx 10.97$$

bits. Thus 11 bits will suffice, but 10 won't.

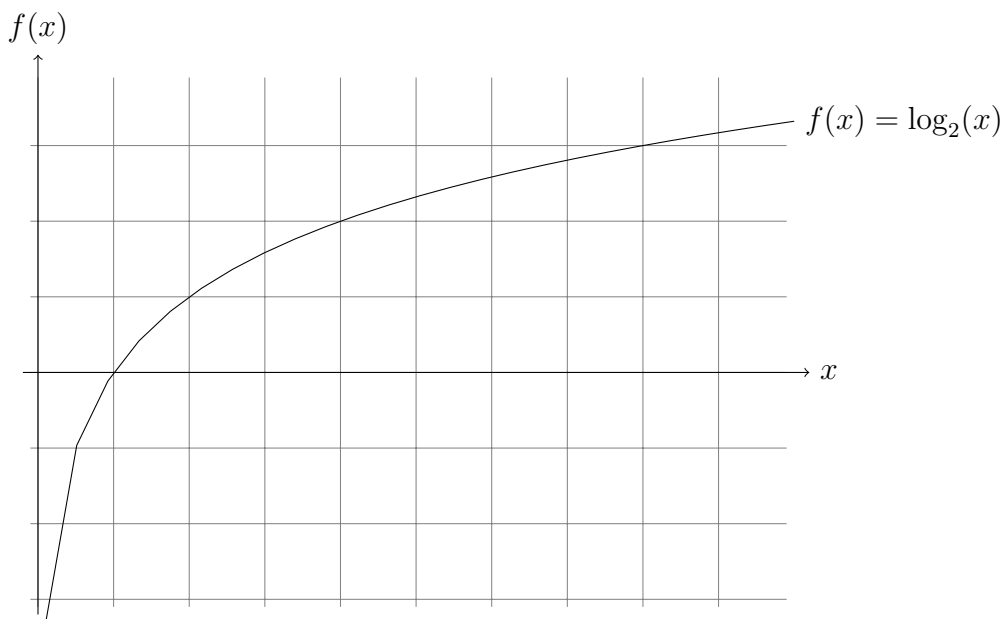
Example 3.3.1. There is a simple approximation that is often good enough for applications in computer science. Since the number of bits is an integer, we only have to find the smallest integer greater than or equal to $\log_2(n)$, and this can be found by counting how many times we can divide n by 2 before the answer drops below 1. For instance, to find $\log_2(71)$, divide by 2 repeatedly and get (approximately)

$$71 \quad 35,5 \quad 18 \quad 9 \quad 4,5 \quad 2,25 \quad 1,2 \quad 0,6 < 1.$$

We divided by 2 seven times, so $7 \approx \log_2(71)$. In fact, 6 is closer to the correct value, but we want to get the closest integer greater than $\log_2(71)$.

Alternatively, you can multiply repeatedly by 2, starting from 1, and see how many times you have to multiply before you pass 71.

Exercise 3.3.2. Use multiplication to approximate $\log_2(71)$.



Binary representation of negative numbers

Negative numbers of type `byte`, `short`, `int` or `long` are all represented in the same way. We will explain this representation using only four bits (lazy?), where we can write everything explicitly. The numbers we can represent are then the numbers from -8 to 7 . For the positive numbers, there are no surprises:

Positive numbers	
binary	decimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7

To get the negative numbers, Java uses a trick called two's complement. Take the corresponding positive number, switch 0s and 1s, and add $1 = 0001$. -8 doesn't have an associated positive, and is represented by `1000`.

For instance, to find -3 , we start with `0011`. We then switch 0s and 1s to get `1100`. Adding 1 gives `1101`. The table is as follows

Negative numbers	
binary	decimal
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

This technique for representing negative numbers allows us to compute with positive and negative numbers in the same way. If we need to carry something beyond the bits allocated, it is simply discarded.

For instance, adding 4 and -3 is done as follows:

$$\begin{array}{r|rrrr}
 1 & 1 & & & \\
 & 0 & 1 & 0 & 0 \\
 + & 1 & 1 & 0 & 1 \\
 \hline
 1 & 0 & 0 & 0 & 1
 \end{array}$$

Since we only have four bits, the first 1 is discarded, and we're left with 0001, representing 1.

Exercise 3.3.3. Add -4 and 3 in the same way.

Adding 7 and 1 results in

$$\begin{array}{r|rrrr}
 & 1 & 1 & 1 & \\
 & 0 & 1 & 1 & 1 \\
 + & 0 & 0 & 0 & 1 \\
 \hline
 & 1 & 0 & 0 & 0
 \end{array}$$

representing -8 . This explains not only why we get the wrong answer, but also what the wrong answer is.

3.4 Division

Java always gives an integer as answer when dividing two integers (division by zero is not allowed, and will lead to compilation error). For instance, $7/3$ returns 2, so the remaining $1/3$ is discarded. It is, however, possible to get the remainder by using a different command. We can write division with remainder as

$$\frac{7}{3} = 2 + \frac{1}{3}$$

but to stay within the realm of integers, it is convenient to multiply by the denominator 3, and instead consider

$$7 = 2 \cdot 3 + 1$$

Then the two numbers 2 and 1 can be referred to as the *quotient* and *remainder* respectively. See also Theorem 3.2.8. Java finds these two numbers in this way:

```
System.out.printf("7/3 = %d\n", 7/3); //quotient
System.out.printf("7 %% 3 = %d\n", 7 % 3); //remainder
```

Exercise 3.4.1. Check how Java deals with division involving one negative and one positive number.

Modulo

The notion of *congruence* provides a way to keep track of remainders of division when other arithmetic operations are performed. The term *modulo* refers to the integer that we divide by.

Definition 3.4.2. Two integers m and n are congruent modulo an integer $r > 0$ if both have the same remainder when divided by r . In this case we write

$$m = n \bmod r$$

Equivalently, $m = n \bmod r$ if there exists an integer x such that $m = n + x \cdot r$.

Example 3.4.3. Some congruent numbers:

$$\begin{aligned} 112 &= 32 \bmod 10 \\ 50 &= 1 \bmod 7 \\ -7 &= 0 \bmod 7 \\ 4229 &= 2279 \bmod 325 \end{aligned}$$

Some numbers that are not congruent:

$$\begin{aligned} 27 &\neq 43 \bmod 10 \\ -4 &\neq 11 \bmod 8 \\ 1 &\neq 0 \bmod 2 \\ 4228 &\neq 2279 \bmod 325 \end{aligned}$$

Exercise 3.4.4. Determine, for each pair of integers, whether they are congruent modulo 6.

- (a). 13 and 45
- (b). -2 and 28
- (c). 123 and 21
- (d). -123 and -21

Some basic properties of congruence modulo r are collected in the next theorem.

Theorem 3.4.5. Let $r > 0$, k , m , n, m' and n' be integers, and assume

$$m = m' \bmod r \text{ and } n = n' \bmod r$$

Then the following are true:

- a) $m = 0 \bmod r$ if and only if m is divisible by r .

$$b) \ m + n = m' + n' \bmod r.$$

$$c) \ m \cdot n = m' \cdot n' \bmod r.$$

$$d) \ km = km' \bmod r.$$

In words, the standard arithmetic operations can be performed before or after the modulo operation, with the same result.

Proof. For part a), $m = 0 \bmod r$ if and only if m has remainder 0 when divided by r if and only if m is divisible by r .

Parts b) and c) are proved by first finding integers x and y such that

$$m = m' + x \cdot r \text{ and } n = n' + y \cdot r.$$

Then

$$m + n = m' + x \cdot r + n' + y \cdot r = m' + n' + (x + y) \cdot r,$$

proving b). Similarly,

$$m \cdot n = (m' + x \cdot r)(n' + y \cdot r) = m' \cdot n' + (m' \cdot y + x \cdot n' + x \cdot y)r$$

proving c).

Finally, d) follows in the same way from

$$km = k(m' + x \cdot r) = km' + (kx) \cdot r.$$

□

Modulo in Java

Remainders are found by using the percentage sign `%` in Java. Unfortunately, this doesn't work exactly as a mathematician would want. For instance, two numbers are congruent modulo 2 if and only if both are even or both are odd. In Java, we cannot check this as simply as this:

```
boolean equalMod2 = (7%2 == (-7)%2);
System.out.println(equalMod2);
```

Even though both 7 and -7 are odd, this code block will return `false`. The reason is that `%` can give negative as well as positive remainders. It is therefore usually best to check whether the difference is divisible by 2 instead, as in this code block:

```
boolean equalMod2 = (7-(-7))%2 == 0;
System.out.println(equalMod2);
```

This returns `true`, and will ensure that the mathematical notion of congruence works with the implementation in Java.

Now that we know both how division works and how remainders are found, we can divide without losing information. Also, the conventions for dealing with negative numbers in Java ensure that the following works both for negative and positive numbers.

```
int m = 23;
int n = 8;
System.out.printf("%d = %d.%d+%d", m, m/n, n, m%n);
```

Exercise 3.4.6. Check how this works in Java for negative numbers.

3.5 Bitwise operations

In order to study the bitwise operations, it is convenient to consider the binary representation of integers, and to get Java to display it. In this section, we will only deal with `int`. The following code converts an `int` to the `String` containing its binary representation.

```
int anInt = 2345;
String binaryInt = String.format("%32s",
    Integer.toBinaryString(anInt)).replace(' ', '0');
```

Exercise 3.5.1. Use this conversion to a `String` to check your computations in Exercises 3.2.4 and 3.2.7.

Here is the list of bitwise operations. Notice that the symbols for “and” and “or” are similar to the boolean operations we’ve seen earlier.

operation	symbol
negation	<code>~</code>
and	<code>&</code>
or	<code> </code>
xor	<code>^</code>

These operations are performed on each bit of the binary representation of an integer. For instance, since Java represents negative integers by negating each bit, and adding 1, we can check that

```
System.out.println((~23 + 1) == -23);
```

returns `true`.

Example 3.5.3. To see what the bitwise operations do to a pair of integers, consider 43 and 3245. Then

returns 41 and 3247. In binary, using the above method for writing binary strings, we get

This makes more sense than just seeing the result displayed as a decimal integer.

Exercise 3.5.4. Check that the symbol $\wedge$ works as “exclusive or”, by displaying $43 \wedge 3245$ in binary. The number you’ll get is 3206.

3.6 Exercises

$$m \sim n \text{ if and only if } m = n \bmod r.$$

Prove that this is an equivalence relation.

Exercise 3.6.2. It is not allowed to use modulo 0, %0, in Java (because of division by zero). Check how Java treats modulo 1.

Exercise 3.6.3. For each pair of numbers, convert them to binary, add them and multiply them in binary, and convert the answer back to decimal to check your answer.

- (a). 7 and 8
- (b). 45 and 5
- (c). 256 and 13

Exercise 3.6.4. Hexadecimal numbers, numbers in base 16, are often used in computer science. In this exercise, we consider some aspects of these numbers.

The digits are

0 1 2 3 4 5 6 7 8 9 A B C D E F

C , for instance, represents 12, and $2C$ represents $2 \cdot 16 + 12 = 44$.

- (a). Find the decimal representation of the hexadecimal number $4AB2$.
- (b). Add the two hexadecimal numbers $1A$ and $B2E$. If necessary, write down an addition table.
- (c). Multiply the two numbers $1A$ and $B2E$. If necessary, write down a multiplication table.
- (d). Using the same procedure as in Equation 3.5, find the hexadecimal representation of 15224.
- (e). Transforming between hexadecimal numbers and binary numbers can be performed “digit by digit” in the hexadecimal number, because four binary digits correspond exactly to one hexadecimal digit. Write down the correspondence table, and use it to write $4E24C$ in binary and 10011100110 in hexadecimal.

Exercise 3.6.5. Subtraction is typically performed by borrowing instead of carrying. We can think of this as carrying -1 , as in the following computation of $103 - 88$:

$$\begin{array}{r} -1 \quad -1 \\ 1 \quad 0 \quad 3 \\ - \quad 8 \quad 8 \\ \hline 1 \quad 5 \end{array}$$

If we try to do this for $88 - 103$, we will end up with a problem:

$$\begin{array}{r} 8 \quad 8 \\ - \quad 1 \quad 0 \quad 3 \\ \hline ? \quad 8 \quad 5 \end{array}$$

One of the good things about two’s complement is that this problem doesn’t occur, so subtraction of binary integers with a fixed number of bits always works. For instance, computing $2 - 5$ with four bits (keeping in mind that $10_2 - 1_2 = 1_2$):

$$\begin{array}{r} -1 \quad -1 \quad -1 \\ 0 \quad 0 \quad 1 \quad 0 \\ - \quad 0 \quad 1 \quad 0 \quad 1 \\ \hline ? \quad 1 \quad 1 \quad 0 \quad 1 \end{array}$$

The answer contains a question mark, but since we only keep four bits, it is discarded, and we are left with the (correct) answer.

Check in examples that $a + (-b)$, computed using addition, is the same as $a - b$ computed using subtraction, for binary numbers with four bits. Can you prove that this is always true?

Exercise 3.6.6. Using the representation with four bits, perform these multiplications. For each multiplication, try to predict the answer before computing it.

(a). $2 \cdot 3$

(b). $(-2) \cdot (-3)$

(c). $2 \cdot 4$

(d). $2 \cdot 5$

(e). $(-3) \cdot 2$

(f). $(-6) \cdot 7$

Exercise 3.6.7. The most important logarithm (possibly apart from in computer science), is the so-called *natural* logarithm. It is the logarithm with base

$$e = 2.718281828\dots$$

In this exercise, we check some of its main properties from the properties of exponentiation. All the results presented remain valid for any logarithm with base > 1 . The natural logarithm is usually written $\ln$, so that $\ln x = \log_e x$.

(a). Show that $\ln(1) = 0$ and $\ln(e) = 1$.

(b). Show that $\ln(ab) = \ln(a) + \ln(b)$. Hint: $e^{ab} = e^a e^b$.

(c). Show that $\ln(a^b) = b \ln(a)$. Hint: $(e^a)^b = e^{ab}$.

(d). Show that $\lim_{x \rightarrow 0^+} \ln(x) = -\infty$. Hint: $\lim_{x \rightarrow -\infty} e^x = 0$

(e). Show that $\lim_{x \rightarrow \infty} \ln(x) = \infty$. Hint: $\lim_{x \rightarrow \infty} e^x = \infty$

Exercise 3.6.8. Bitwise operations can be used to set flags. This means that the states of some system are described using several switches, that could be turned on or off. We will model this using bitwise operations using the integer type `byte`, which has 8 bits. To get the binary representation as a binary `String`, use

```

byte aByte = 57;
int theInt = 0;
if (aByte < 0){
    theInt = 256 + aByte;
} else{
    theInt = aByte;
}
String binaryByte = String.format("%8s",
    Integer.toBinaryString(theInt)).replace(' ', '0');

```

The +256 is a trick to alleviate the problems from casting to `int`. The bit in the 16-place is set (the switch is turned on) if its value is 1.

(a). Explain why the bit in the 4-place is set by this code (`stateByte` represents the current value for the states). Print the binary strings for different `stateBytes` so that you see both what happens if this bit was set and if it wasn't set before running the code.

```

// stateByte is already defined
stateByte = (byte) (stateByte | 4);

```

The bitwise operations are performed as `int`, so we need to cast to `byte` in the end.

(b). Explain why the bit in the 4-place is turned off by this code. Again, give examples both with the 4-bit set and not set before running this code block.

```

// stateByte is already defined
stateByte = (byte) (stateByte & ~5);

```

(c). What does exclusive or (^) with 0 do? What does exclusive or with 4 do? What does exclusive or with -1 do?

(d). Given two integers `intX` and `intY`, what is the value of each of them after running this code:

```

intX = intX ^ intY;
intY = intX ^ intY;
intX = intX ^ intY;

```

If it is difficult to see what happens, run a couple of examples.

Chapter 4

Floating points

There are, in any given interval $[a, b]$ with $a < b$ on the real line, infinitely many points. The computer has only finitely many states, so it is impossible for it to model real numbers accurately. An approximation, which has so many upsides that it has become the standard, is using *floating points*.

4.1 Real numbers

The integers can be added, subtracted and multiplied with each other. If you want to divide, you also have to include fractions like $\frac{2}{3}$. We can formally define the set of fractions, called the *rational numbers* as

$$\mathbb{Q} = \left\{ \frac{m}{n} \mid m, n \in \mathbb{Z}, n \neq 0 \text{ and } m \text{ and } n \text{ have no common factor} \right\}$$

The letter $\mathbb{Q}$ stands for “quotient”. The integers are contained in this set as $\frac{m}{1}$.

In the rational numbers, we can add, subtract, multiply and divide. We can also solve many equations, for instance some quadratic equations. Recall that the solutions of

$$ax^2 + bx + c = 0$$

are

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

To give an example, the equation

$$2x^2 + 5x - 3 = 0$$

has solutions

$$x = \frac{-5 \pm \sqrt{5^2 - 4 \cdot 2 \cdot (-3)}}{2 \cdot 2} = \frac{-5 \pm \sqrt{49}}{4} = \frac{5 \pm 7}{4} = \begin{cases} -3 \\ \frac{1}{2} \end{cases}$$

However, the equation $x^2 = 2$ cannot be solved using only rationals, since $\pm\sqrt{2}$ aren't rational. Also, numbers like π (the circumference divided by the diameter of a circle) or e (the base of the natural exponential function) aren't rational. To include these numbers, we expand the rational numbers to the real numbers. To give a precise definition of real numbers is difficult, so we'll do with an informal definition.

Definition 4.1.1. The *real numbers* are informally defined as the set of all (finite) limits of rational numbers. We can also view them as numbers with (possibly) infinite decimal expansions. The symbol for the real numbers is $\mathbb{R}$.

For instance, the numbers

$$\begin{aligned} 1 &= 1.000000\dots \\ \pi &= 3.141596\dots \\ \sqrt{2} &= 1.414214\dots \\ \frac{1}{9} &= 0.111111\dots \\ e &= 2.718282\dots \end{aligned}$$

are real numbers (in some cases, the last digit displayed might seem wrong because of rounding). Real numbers can also be added, subtracted, multiplied and divided. They can be compared to each other using $<$ or $\leq$. When we draw a line, we think of the points on the line as corresponding to the real numbers, with $a < b$ if a is to the left of b .

It is also common to consider intervals of real numbers, consisting of all numbers lying between two reals. There are different notations for intervals depending on whether the endpoints are included or not. So assume that $a < b$, and define

$$\begin{aligned} [a, b] &= \{x \mid a \leq x \text{ and } x \leq b\} && \text{closed interval} \\ (a, b) &= \{x \mid a < x \text{ and } x < b\} && \text{open interval} \\ [a, b) &= \{x \mid a \leq x \text{ and } x < b\} && \text{half-open interval} \\ (a, b] &= \{x \mid a < x \text{ and } x \leq b\} && \text{half-open interval} \end{aligned}$$

Infinity is not a real number, but we sometimes write $(2, \infty)$ to signify all real numbers greater than 2, for instance.

There are infinitely many real numbers in any interval. For instance, in the interval $(1, 1.1)$ we can find the infinitely many numbers

$$\begin{aligned} &1.05 \\ &1.005 \\ &1.0005 \\ &1.00005 \\ &\vdots \end{aligned}$$

Exercise 4.1.2.

- (a). Write the interval of all numbers greater than -1 and smaller than or equal to π .
- (b). Write the interval of all numbers greater than or equal to $\sqrt{7}$.
- (c). Write the interval of all numbers smaller than 3 .

Binary representation

Recall that the exponential notation a^b makes sense for $a > 0$ and negative b , so that for instance

$$10^{-2} = \frac{1}{10^2} = \frac{1}{100} = 0.01$$

This is the basis for representing reals using the position system. For instance,

$$32.786$$

means

$$3 \cdot 10^1 + 2 \cdot 10^0 + 7 \cdot 10^{-1} + 8 \cdot 10^{-2} + 6 \cdot 10^{-3}$$

The same meaning can be attached to any base, but we will only consider binary numbers (base 2). The procedure should be clear, for instance

$$10.011_2 = 1 \cdot 2^1 + 0 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} + 1 \cdot 2^{-3} = 2 + \frac{1}{4} + \frac{1}{8} = 2.375$$

Any number with a finite representation as a binary real number, will also have a finite representation as a decimal number (since 2^{-n} has), but the converse is not true. For instance, $1/5 = 0.2$ is represented in binary as

$$0.00110011 \dots$$

(the pattern 0011 is repeated indefinitely).

Exercise 4.1.3. Convert 101.01011 to decimal. Convert 3.75 to binary.

4.2 What are floating points?

Since a computer is a finite machine, real numbers cannot be accurately modelled by the computer. Instead, the computer uses an approximation called floating points. Like for integers, the computer uses a fixed number of bits to represent an approximation to real numbers. Recall the standard scientific notation for numbers:

$$3204.47 = 3.20447 \cdot 10^3$$

The convention is to start with a digit from 1 to 9, and then a point, a certain number of digits, and finally 10 to some power. This convention is convenient for instance when dealing with uncertainty of measurements (significant figures). In Java, we can declare numbers using scientific notation (in fact, slightly more is true) like

```
float aFloat = 2.5e-12f;
```

Here the “e” is for exponential, and “f” is for **float**. The mathematical meaning is $aFloat = 2.5 \cdot 10^{-12}$.

In binary, this is slightly simpler, because when the digit 0 is excluded, we’re only left with the digit 1, so the binary equivalent will contain representations like

$$1.xxx \cdot 2^{xxx}$$

For instance, the representation of $18.75 = 10010.11_2$ would be

$$1.001011 \cdot 2^4 = 1.001011 \cdot 2^{100}$$

We will need to write both the exponent in binary, and the *mantissa* (the number in front). There are thus two numbers that we need to fix the number of allocated bits for; since there is always a 1 in front of the point, we don’t really have to store this. However, we need to store the sign, since floating points also model negative numbers.

Definition 4.2.1. The set of floating points with k bits mantissa and n bits exponent is the set of all real numbers of the form

$$\pm 1.(k \text{ bit number}) \cdot 2^{n \text{ bit number}}$$

A few of these possibilities are reserved for special values, and thus don’t have the value expected from the description above. These special values include *NaN* (“Not a Number”), $\pm\infty$ and the signed zeroes ± 0 .

There is another hidden difficulty here, coming from the desire to express values very close to zero more accurately, but we will not go into details (search the web for “subnormal numbers”).

4.3 Float types in Java

Java uses two floating point types as standard, called **float** and **double**. **float** is a 32 bit type, where the mantissa has 23 bits, the exponent 8 bits, and the last bit is for sign. The 8 bit exponent would, if we use the same standards for representing

integers as before, give the numbers from -128 to 127 , but since there are some reserved values, in fact only the values from -126 to 127 are available. Because we want high precision near 0 , the smallest expressible positive number is in fact $2^{-149} \approx 1.4 \cdot 10^{-45}$, whereas the largest is $(2 - 2^{-23}) \cdot 2^{127} \approx 3.4 \cdot 10^{38}$.

Since floating points are finite approximations to real numbers, there are gaps between them. For numbers close to zero, the gaps are small, for large numbers the gaps are larger. An additional way to say how good a floating point type is, is to find the first gap containing two integers, or equivalently, the first integer that cannot be represented. For `float` this is $2^{24} + 1 = 16,777,217$. The problem can be seen by running this code block (a number is considered as a `float` if suffixed by the letter “f”):

```
float big = 16777216f;
System.out.println((big)==(big +1));
```

Similar data for `double` is found in this table

Type	mantissa	exponent	Max	Min	Smallest integer not represented
<code>float</code>	23 bits	8 bits	$(2 - 2^{-23}) \cdot 2^{127}$	2^{-149}	$2^{24} + 1 = 16,777,217$
<code>double</code>	52 bits	11 bits	$(2 - 2^{-52}) \cdot 2^{1023}$	2^{-1074}	$2^{53} + 1 = 9007199254740993$

How large are these numbers? For comparison, the mass of an electron is about $9.2 \cdot 10^{-31}$ kg, whereas the mass of the Milky Way is estimated at about $1.4 \cdot 10^{42}$ kg. So using `float` is not good enough for the Milky Way, but works for the electron. `double` works for both, as can be seen from the next exercise:

Exercise 4.3.1. Compute the values of the smallest and the largest `double` as a decimal number.

Rounding errors

Since floating points are only approximations to real numbers, computations involving them will invariably produce incorrect results. To bypass this problem, we need to control the size of the error. We also need to understand how the errors occur. A simple example will illustrate the problem. Other problems will be addressed later on, as we develop more familiarity with programming.

Example 4.3.2 (Testing for equality). Rounding errors will produce results that are slightly off what you might expect. Here is a simple example.

```
System.out.println((0.1+0.1+0.1+0.1+0.1+0.1+0.1+0.1+0.1+0.1)==1);
```

This returns `false`, since there are rounding errors on the left hand side of the equation.

Example 4.3.3 (Testing for approximate equality). Instead of testing for equality, define a threshold, and check whether the error is smaller than this number. For instance

```
double threshold = 0.00001;
double sum = 0.1+0.1+0.1+0.1+0.1+0.1+0.1+0.1+0.1+0.1;
boolean equal = ((sum - 1) < threshold && (1-sum) < threshold);
System.out.println(equal);
```

returns **true**. If you want smoother code, look up **Math.abs**.

Special values

We will now produce the special values *NaN*, $\pm\infty$ and the signed zeroes, to see how they might appear from mathematical expressions. Running

```
System.out.println(1.0f/0.0f);
System.out.println(-1.0f/0.0f);
System.out.println(Math.sqrt(-0.0f));
System.out.println(2.0f-2.0f);
System.out.println(0.0f/0.0f);
```

results in

```
Infinity
-Infinity
-0.0
0.0
NaN
```

Note also that

```
System.out.println((-0.0f)==(0.0f));
```

returns **true**. Even stranger,

```
float d = 0.0f/0.0f;
System.out.println(d==d);
```

returns **false**. This is because “0/0” is represented as *NaN*, and this value is not supposed to be used for anything.

The operations leading to $\pm\infty$ and *NaN* are mathematically unsound, and if we try to perform these operations on integers, we will get compilation errors.

Exercise 4.3.4. See how Java handles 0/0, 1/0 and -1/0 for integers (use **int**). Also check how Java handles $\sqrt{-1}$ (use **Math.sqrt**).

4.4 Casting between floats and integers

It is relatively straightforward to convert an integer into the corresponding floating point, except that if the integer is very large, it might be in a gap between floating points with the given precision. Converting from floats to integers involves taking the integer part, so that any (positive) float is rounded down to the nearest integer below it (discard whatever comes after the point).

Example 4.4.1. Here are two simple castings, showing the structure for conversions.

```
double aDouble = 23.8;
int anInt = (int) aDouble;
double anotherDouble = (double) anInt;
System.out.println(aDouble);
System.out.println(anInt);
System.out.println(anotherDouble);
```

Running this code results in

```
23.8
23
23.0
```

Because of rounding errors, some unexpected results will occur.

Example 4.4.2. Multiplying first by 0.01 and then by 100 ought to return the original number. But

```
double f = 29*0.01;
System.out.println((int)(f*100));
```

outputs 28, because of rounding errors in the `double`.

Exercise 4.4.3. Convert the float 124.234f to long.

4.5 Exercises

Exercise 4.5.1. The associative law for multiplication states that $(ab)c = a(bc)$. Run this code:

```
float a=0.25e-12f;
float b=4e12f;
float c=5e35f;
System.out.println((a*b)*c);
System.out.println(a*(b*c));
```

What happens? Change all the `floats` into `doubles`, and try again. What has changed? Can you create a similar problem with `doubles`?

Exercise 4.5.2. In this exercise, we will use Java to solve quadratic equations. The formula for solving the quadratic equation

$$ax^2 + bx + c = 0$$

is

$$x_0 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \quad x_1 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}.$$

Choose values for a, b, c as `floats`, and write a programme that computes these two solutions. Note that the square root is found in Java as `Math.sqrt`.

Then add a test to the programme, checking whether the numbers you have calculated actually solves the original problem. Try out your programme on these four sets of coefficients.

a	b	c
1	2	-3
1	2	1
1	1	1
1	-20000	1

What happens? Why?

If you get a “possible loss of precision” error, explicitly cast to `float`.

Exercise 4.5.3. One can use `floats` in loops, which is practical if we want for instance points on a graph. Try to use a `for` loop of the form

`for (double x = startValue, x < stopValue, x = x + increment) { ... }`

to compute points on the graph of $f(x) = \sin x$ in the interval $[-7, 7]$. To make sure that the point are close enough to get a nice graph from plotting them, compute the sine for each of the points $-7, -6.99$ and so forth.

Then try to do the same computation using an `int` as index in the loop.

Exercise 4.5.4. In this exercise, we will compute as many powers of two that we can, using the primitive numerical types in Java. Start by defining `int power = 2`, and multiply repeatedly by 2. Check (how?) if the computation is correct using `int`, and if it isn’t, convert to `long`. When this also fails, convert to `float`. When this fails, convert to `double`. Finally, write a statement about the largest powers handled by the different types.

Can you modify the programme to work with powers of 3?

Exercise 4.5.5. Create multiplication and addition tables for $\pm\infty$, `NaN`, -0 and $+0$. To get the values, use for instance the wrapper class `Float`, as in

`Float minusInfinity = new Float(Float.NEGATIVE_INFINITY);`

Chapter 5

Sequences and series

In this chapter, we will introduce two types of sequences (numbers in some fixed order) and series (sums of sequences) that are especially simple to understand and to implement on the computer.

For starters, let us consider an example telling something about what a series is.

Example 5.0.1. Finding the maximum number in a sequence of numbers can be done in the following way. Assume there are N different numbers, stored in a list called `numbers`.

```
max = numbers[0]
for (i = 1 to N-1)
    if numbers[i] > max
        max = numbers[i]
    end if
end for
print max
```

How many times will `max` be assigned a value on average, if we assume that the numbers are randomly chosen (with a uniform distribution)?

If there are just two numbers, `max` is first set equal to the first number. The probability that the second number is larger than the first, is exactly one half. If the second is larger, `max` is assigned a value twice, if it isn't, just once. Thus the average number of assignments is

$$1 \cdot \frac{1}{2} + 2 \cdot \frac{1}{2} = \frac{1}{2} + 1 = \frac{3}{2}.$$

If there are three numbers, the average number of assignments *before* we come to the last number is $\frac{3}{2}$, for the same reason as before. The probability that the last

number is larger than both the previous numbers is $\frac{1}{3}$. In this case, there is an extra assignment. Thus the average number of assignments is

$$\frac{3}{2} + 1 \cdot \frac{1}{3} = \frac{11}{6}.$$

We can continue in this way (see Section 8.4), and always obtain the average for a list of n numbers by adding $\frac{1}{n}$ to the average for a list of $n - 1$ numbers. We are therefore looking at the sum

$$1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{N}$$

These sums are called *harmonic numbers*.

What happens when the length of the list becomes very large? Mathematically, $N \rightarrow \infty$. Obviously, the sum becomes larger and larger, but it is not quite clear whether there is a bound for the sum or not, since the terms that we add become smaller and smaller. In fact

$$\lim_{N \rightarrow \infty} \left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{N} \right) = \infty$$

so the sum is unbounded. See Exercise 5.3.3.

In this example, smaller and smaller numbers are added, but still the sum goes to infinity! This is a central phenomenon in the theory of series.

5.1 Sequences

Definition 5.1.1. A *sequence* is a collection of numbers with a given order.

A set of numbers hasn't got a fixed order, so as sets $\{2, 3\} = \{3, 2\}$. The sequences are different, however, since the order is important.

Example 5.1.2. Here are some typical examples of sequences.

$$\begin{aligned} (1, 2, 3, 4, 5, \dots) \\ (2, \pi, 14) \\ (-1, 1, -1, 1, -1, \dots) \\ (1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \frac{1}{5}, \dots) \\ (3, 1, 4, 1, 5, 9, \dots) \end{aligned}$$

A sequence can contain finitely or infinitely many numbers, there can be repetitions, there can be patterns, or the sequence can seem more arbitrary.

An unspecified sequence will often be referred to as $a = (a_0, a_1, a_2, \dots, a_n)$, or similarly if there are infinitely many numbers. Often, we will declare a sequence by giving a formula for the n th number, like $a_n = \sin(n)$.

There are some basic questions that apply to infinite sequences. What happens when we go very far out in the sequence? Will the elements tend to some number? Will the elements grow unbounded towards infinity? What else can happen?

Definition 5.1.3 (Limit and convergence). Let $a = (a_0, a_1, a_2, \dots)$ be a sequence. A number L is called the *limit* of a if the numbers a_n are arbitrarily close to L for all sufficiently large n . More formally, for any positive number ϵ , we demand that there exists an integer N such that

$$|a_n - L| < \epsilon \text{ for all } n > N.$$

In this case, we also say that the sequence *converges* (to L), and write

$$\lim_{n \rightarrow \infty} a_n = L \text{ or more simply } a_n \rightarrow L.$$

There need not exist any such L . If not, we say that the sequence *diverges*, or that the limit doesn't exist. In the special case that the sequence diverges, but grows unboundedly, we say that it *diverges to infinity*. Formally, we demand that for any number R there exists a number N such that

$$a_n > R \text{ for all } n > N.$$

A sequence can similarly diverge to $-\infty$.

Example 5.1.4. Here are some examples of convergent sequences, with their limit.

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{1}{n^2} &= 0 \\ \lim_{n \rightarrow \infty} n \cdot \sin\left(\frac{1}{n}\right) &= 1 \\ \lim_{n \rightarrow \infty} \frac{3n^2 - 4n + 3}{7n^2 + 5n + 10} &= \frac{3}{7} \end{aligned}$$

Do you understand why these limits are as written? If you understand the second, I'm impressed!

Example 5.1.5. Here are some examples of divergent sequences.

$$\begin{aligned} \lim_{n \rightarrow \infty} n^2 &= \infty \\ \lim_{n \rightarrow \infty} (-1)^n &\text{ does not exist} \\ \lim_{n \rightarrow \infty} \sin(n) &\text{ does not exist} \\ \lim_{n \rightarrow \infty} 3 - 2^n &= -\infty \end{aligned}$$

Again, do you understand why these sequences are divergent?

Arithmetic sequences

Definition 5.1.6. An *arithmetic* sequence is a sequence with numbers of the form

$$a_n = s + n \cdot t \text{ where } s \text{ and } t \text{ are some numbers.}$$

This can also be written

$$a = (s, s + t, s + 2t, s + 3t, \dots)$$

In words, we start with some number s , and each new number in the sequence is a fixed number t plus the previous number.

Example 5.1.7. These sequences are arithmetic.

$$\begin{aligned} &(1, 2, 3, 4, 5, 6, 7, \dots) \\ &(-2.2, 0.2, 2.6, 5, \dots) \\ &(1, -3, -7, -11, \dots) \end{aligned}$$

Exercise 5.1.8. What is the hundredth number in each of the sequences in the previous example?

Exercise 5.1.9. What are s and t in the arithmetic sequence

$$2, 8, 14, 20, \dots?$$

Exercise 5.1.10. Write down the first five terms of the arithmetic sequence with $s = 2.3$ and $t = -1.7$.

Note that no arithmetic sequence converges (except in the trivial case $t = 0$), but will diverge to ∞ (if $t > 0$) or $-\infty$ (if $t < 0$).

Geometric sequences

Definition 5.1.11. A *geometric* sequence is a sequence with numbers of the form

$$a_n = r \cdot k^n \text{ where } r \text{ and } k \text{ are some numbers.}$$

This can also be written

$$a = (r, r \cdot k, r \cdot k^2, r \cdot k^3, \dots)$$

In words, we start with some number, and each new number in the sequence is a fixed number times the previous number.

Example 5.1.12. These sequences are geometric.

$$\begin{aligned}(1, 2, 4, 8, 16, \dots) \\ (1, -1, 1, -1, 1, \dots) \\ (2, \frac{2}{3}, \frac{2}{9}, \frac{2}{27}, \dots)\end{aligned}$$

Exercise 5.1.13. What is the hundredth number in each of the sequences in the previous example?

Exercise 5.1.14. What is r and k in the geometric sequence

$$4, 12, 36, \dots?$$

Exercise 5.1.15. Write down the first five terms of the geometric sequence with $r = 0.5$ and $k = -3$.

Theorem 5.1.16. A geometric sequence $a_n = rk^n$ (with $r \neq 0$) converges if and only if $-1 < k \leq 1$.

- If $-1 < k < 1$ it converges to 0.
- If $k = 1$ it is constant, and hence converges to r .
- If $k > 1$ it diverges to ∞ .
- If $k \leq -1$ it diverges, but not to $\pm\infty$.

Proof. We will prove the theorem using logarithms, to get some experience handling logarithms. See Exercise 3.6.7. Assume first that all the numbers involved are positive. Then

$$\ln(rk^n) = \ln(r) + \ln(k^n) = \ln(r) + n \ln(k)$$

This is an arithmetic sequence, so if $\ln(k) > 0$ it diverges to ∞ . This corresponds to $k > 1$, and the original sequence diverges to $\lim_{x \rightarrow \infty} e^x = \infty$. If $\ln(k) < 0$ it diverges to $-\infty$. This corresponds to $0 < k < 1$, and the original sequence converges to $\lim_{x \rightarrow -\infty} e^x = 0$.

If k is negative, the sign changes at each step, but otherwise the sequence behaves as $|rk^n|$, which converges to 0 if $|k| < 1$ and diverges if $|k| > 1$. Note that, for $k < -1$, the sequence doesn't diverge to ∞ or $-\infty$ because of the sign change.

Finally, for $k = -1$, the sequence becomes $(r, -r, r, -r, \dots)$, which is divergent, and for $k = 1$ it becomes $(r, r, r, r, \dots)$, which obviously converges to r .

□

5.2 Series

A *series* is the sum of a sequence. We often write it using a Greek capital sigma $\sum$. As long as we're interested in finite series (sums of finite sequences), the meaning of summing is clear (ordinary addition). If there are infinitely many numbers to be added, we have to take a limit. In Chapter 7, we will see how to sum series using Java.

Definition 5.2.1. The *sigma-notation* for the sum of a finite sequence $a = (a_0, a_1, \dots, a_n)$ is

$$\sum_{i=0}^n a_i = a_0 + a_1 + \dots + a_n.$$

Here i is a dummy variable keeping track of the index in the sequence.

For infinite series, we take the limit, if it exists, of the finite partial sums:

$$\sum_{i=0}^{\infty} a_i = \lim_{n \rightarrow \infty} \sum_{i=0}^n a_i.$$

If the limit exists, we say that the series *converges*, otherwise it *diverges*. We also write

$$\sum_{i=0}^{\infty} a_i = a_0 + a_1 + a_2 + \dots$$

In some cases, it is convenient to start summing from a different index.

Example 5.2.2. Here are some finite sums, written with the sigma-notation.

$$\begin{aligned} \sum_{i=0}^3 2i + 1 &= 2 \cdot 0 + 1 + 2 \cdot 1 + 1 + 2 \cdot 2 + 1 + 2 \cdot 3 + 1 = 1 + 3 + 5 + 7 = 16 \\ \sum_{i=1}^4 \frac{1}{n} &= \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} = \frac{25}{12} \\ \sum_{i=-1}^1 \sin(i) &= \sin(-1) + \sin(0) + \sin(1) = 0 \end{aligned}$$

Exercise 5.2.3. Write down and compute these sums.

$$\begin{aligned} &\sum_{i=0}^4 2i - 2 \\ &\sum_{n=1}^3 n^3 \\ &\sum_{n=1}^{10} n \end{aligned}$$

Example 5.2.4. Here are some quite nontrivial examples of series, and what they converge to.

$$\begin{aligned} 1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \dots &= \sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6} \\ 1 - \frac{1}{2} + \frac{1}{3} - \frac{1}{4} + \dots &= \sum_{n=0}^{\infty} \frac{(-1)^n}{n+1} = \ln 2 \\ 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots &= \sum_{i=0}^{\infty} \frac{(-1)^i}{2i+1} = \frac{\pi}{4} \end{aligned}$$

When dealing with a finite sum, we are interested in the value of the sum. When dealing with infinite sums, we first need to understand whether the sum exists, that is if there is a value to look for. If the sum exists, we are interested in its value, exactly or approximately. The rest of this section is dedicated to atypical examples, where everything can be checked explicitly. An important condition for the sum to exist, is that the terms tend to zero.

Theorem 5.2.5. *If*

$$\sum_{i=0}^{\infty} a_i$$

exists, then $\lim_{i \rightarrow \infty} a_i = 0$. If $\lim_{i \rightarrow \infty} a_i \neq 0$, or doesn't exist, the series diverges.

If the sum diverges, we don't know whether the terms tend to zero. If the terms tend to zero, we don't know whether the series converges.

Proof. If $\sum_{i=0}^{\infty} a_i = S$, then the partial sums tend to S . This means that, for any $\epsilon > 0$, there exists N such that for $n > N$

$$\left| \sum_{i=0}^n a_i - S \right| < \epsilon$$

and also

$$\left| S - \sum_{i=0}^{n-1} a_i \right| < \epsilon$$

But

$$a_n = \sum_{i=0}^n a_i - \sum_{i=0}^{n-1} a_i = \sum_{i=0}^n a_i - S + S - \sum_{i=0}^{n-1} a_i$$

Taking absolute values, we find

$$|a_n| = \left| \sum_{i=0}^n a_i - S + S - \sum_{i=0}^{n-1} a_i \right| \leq \left| \sum_{i=0}^n a_i - S \right| + \left| S - \sum_{i=0}^{n-1} a_i \right| < 2\epsilon$$

So all terms are small if we go far enough, and the limit must be zero. $\square$

Arithmetic series

First we will add a finite arithmetic series.

Theorem 5.2.6. *Let s and t be some numbers, and define, for $i \geq 0$*

$$a_i = s + i \cdot t$$

Then

$$\sum_{i=0}^n a_i = \frac{(n+1)(2s+nt)}{2}$$

Proof. A trick that proves this theorem is to sum the series twice, and turn the second summing around like so:

$$\begin{array}{cccccccc} s & + & s+t & + & \cdots & + & s+(n-1)t & + & s+nt \\ + & s+nt & + & s+(n-1)t & + & \cdots & + & s+t & + & s \end{array}$$

Adding the numbers above and below, we find that each sum is $2s + nt$. This happens for each of the $n+1$ terms, so the displayed sum is equal to

$$(n+1)(2s+nt)$$

But we summed the series twice, so this answer must be divided by two. We end up with the formula in the theorem. $\square$

Exercise 5.2.7. Sum the finite arithmetic series

$$8 + 11 + 14 + \cdots + 71$$

Infinite arithmetic series are never convergent (except in the trivial case $s = t = 0$), since the terms don't tend to zero (see Theorem 5.2.5).

Geometric series

We will now sum the finite geometric series.

Theorem 5.2.8. *Let r and k be some numbers, and define, for $i \geq 0$*

$$a_i = r \cdot k^i$$

Then, if $k \neq 1$,

$$\sum_{i=0}^n a_i = \frac{r - rk^{n+1}}{1 - k}$$

If $k = 1$ the sum is $(n+1)r$.

Proof. A trick to show this theorem, is to multiply the series by $1 - k$. Thus we first write the series (times one), and then subtract the series multiplied by $-k$, which increases the exponent by one.

$$\begin{array}{cccccccc} r & + & rk & + & rk^2 & + & \cdots & + & rk^n \\ & - & rk & - & rk^2 & - & \cdots & - & rk^n & - & rk^{n+1} \end{array}$$

Now all terms, except r and $-rk^{n+1}$, appear twice, once with $+$ and once with $-$. Thus we're left only with $r - rk^{n+1}$. We started by multiplying with $1 - k$, so to get the answer, we must divide by this number. This produces the formula in the theorem. $\square$

Exercise 5.2.9. Sum the finite geometric series

$$2 + 6 + 18 + \cdots + 13122$$

To sum the infinite geometric series, we use what we've already seen for the limits of the numbers.

Theorem 5.2.10. Let $r \neq 0$ and k be some numbers, and define, for $i \geq 0$

$$a_i = r \cdot k^i$$

Then the series

$$\sum_{i=0}^{\infty} a_i$$

converges if and only if $|k| < 1$, and the sum is given by

$$\sum_{i=0}^{\infty} a_i = \frac{r}{1 - k}$$

Proof. We know that rk^n tends to 0 if and only if $|k| < 1$, so if $|k| \geq 1$, the series must necessarily diverge. If $|k| < 1$, we can compute the limit of the partial sums as

$$\sum_{i=0}^{\infty} a_i = \lim_{n \rightarrow \infty} \frac{r - rk^{n+1}}{1 - k} = \frac{r}{1 - k}$$

$\square$

Exercise 5.2.11. Sum the infinite geometric sequence

$$17 + \frac{17}{7} + \frac{17}{49} + \cdots$$

5.3 Exercises

Exercise 5.3.1. Write down the first five terms $a_0, \dots, a_4$ of the sequence defined by $a_n = (2n - 1)/4$.

Exercise 5.3.2. Find a formula for a_n when the sequence starts

$$-1, 1, 3, 5, 7, 9, 11, \dots$$

Exercise 5.3.3. We will again consider the harmonic series

$$1 + \frac{1}{2} + \frac{1}{3} + \dots$$

We will prove that the series diverges to ∞ by thinking in binary, and performing some approximations. More precisely, we will round each term $\frac{1}{n}$ down to the nearest $\frac{1}{2^m}$ smaller than it, and sum those terms instead. At each step, since we're always rounding down, we will get a lower bound for the sum. Finally, we will see divergence because the lower bound diverges to ∞ .

(a). Write explicitly the series up to the term $\frac{1}{16}$, and round each term down to the closest $\frac{1}{2^m}$. What is the lower bound for $1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{16}$ coming from this?

(b). Use the binary logarithm to find a formula for m such that $\frac{1}{n}$ rounds down to $\frac{1}{2^m}$.

(c). How many terms round down to the same $\frac{1}{2^m}$?

(d). Use the above to find explicitly a lower bound for the sum

$$1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{2^m}.$$

(e). Conclude from d) that the lower bound diverges to infinity, and therefore the original sum also diverges to infinity.

Exercise 5.3.4. Formulate a precise definition for divergence to $-\infty$.

Exercise 5.3.5. Summing the finite geometric series is performed in Theorem 5.2.8. Using that, for $n > m$,

$$\sum_{i=m+1}^n a_i = \sum_{i=0}^n a_i - \sum_{i=0}^m a_i$$

find a formula for

$$\sum_{i=m+1}^n rk^i$$

Check your formula by applying Theorem 5.2.8 directly, using rk^{m+1} instead of r .

Exercise 5.3.6. Theorem 5.2.10 produces a formula for the infinite geometric series. Find a formula for

$$\sum_{i=m}^{\infty} rk^i$$

when $|k| < 1$ in two ways. One is to subtract a convenient finite geometric series. The other is to consider this series as a full geometric series with first term rk^m .

Chapter 6

Functions

In this chapter, we will consider the notion of functions in mathematics and programming. On an abstract level, the term *function* can be defined such that it is applicable in both these contexts. In practice, it is often convenient to think about the two phenomena slightly differently.

6.1 Functions between sets

It is not difficult to say what a function *is*.

Definition 6.1.1. Given two sets A and B , a *function* from A to B is a rule that to each and every element in A associates one and only one element of B . If we call the function f , we often write

$$f : A \rightarrow B \text{ or } A \xrightarrow{f} B$$

to signify that f is a function from A to B . Then, for $a \in A$, $f(a) \in B$ is the element associated to a . Sometimes A is called the *domain* and B the *codomain* of f .

Example 6.1.2. Let A be the set of all triangles, and let B be the set of real numbers. Define a function from A to B by assigning to a triangle the smallest angle, measured in degrees. This is a function.

Example 6.1.3. Let A be the set of pixels on a computer screen, and let B be the set of colours. Then we can define a function from A to B by assigning to a pixel its colour.

Example 6.1.4. Let r and k be real numbers. Define a function $f : \mathbb{Z} \rightarrow \mathbb{R}$ by declaring

$$f(n) = \begin{cases} 1 & n < 0 \\ r & n = 0 \\ r + rk + \cdots + rk^n & n > 0 \end{cases}$$

This is a function.

Example 6.1.5. We can't define a function from $\mathbb{R}$ to $\mathbb{R}$ by associating to a the solution to $x^2 = a$. The reason is that the equation can have zero, one or two solutions, depending on a . Thus we can't choose one and only one element for each a .

To understand more about functions, we need more notions.

Definition 6.1.6. Let $f : A \rightarrow B$ be a function.

- a) We say that f is *injective* if $f(a) = f(a')$ implies that $a = a'$. In words, different elements are sent to different elements.
- b) We say that f is *surjective* if for any $b \in B$ there exists an element $a \in A$ such that $f(a) = b$. In words, each element of B is "hit by" an element of A .
- c) We say that f is *bijective* if for any $b \in B$ there exists exactly one element $a \in A$ such that $f(a) = b$. This is the same as saying that f is both injective and surjective.

Definition 6.1.7. The *image* of a function $f : A \rightarrow B$ is the set of elements $b \in B$ such that there exist an element $a \in A$ with $f(a) = b$.

Definition 6.1.8. Let A, B and C be sets, $f : A \rightarrow B$ and $g : B \rightarrow C$ functions. Note that the domain of g is equal to the codomain of f . Then, for any $a \in A$, $f(a) \in B$, and we can apply g to this element. By definition

$$g \circ f(a) : A \rightarrow C$$

is the function that to $a \in A$ associates $g(f(a)) \in C$. $g \circ f$ is called the *composition* of f and g .

To see how these notions work, we will prove a couple of results. More results in the same spirit can be found in the exercises.

Theorem 6.1.9. f, g, h are functions, A, B, C, D are sets, so that

$$f : A \rightarrow B, g : B \rightarrow C, h : C \rightarrow D$$

- a) If f and g are both injective, $g \circ f$ is injective.
- b) If f and g are both surjective, $g \circ f$ is surjective.
- c) If f and g are both bijective, $g \circ f$ is bijective.

d) The two double compositions $h \circ (g \circ f)$ and $(h \circ g) \circ f$ are equal as functions from A to D .

Proof. To prove a , we need to show that if $g \circ f(a) = g \circ f(a')$ then $a = a'$. Recall that $g \circ f(a) = g(f(a))$. If $g(f(a)) = g(f(a'))$, since g is injective, $f(a) = f(a')$. Then, since f is also injective, $a = a'$. This proves that $g \circ f$ is injective.

For b), let $c \in C$ be any element. We need to find $a \in A$ such that $g \circ f(a) = c$. Since g is surjective, there exists $b \in B$ such that $g(b) = c$. Similarly, since f is surjective, there exists $a \in A$ such that $f(a) = b$. But then $g \circ f(a) = g(f(a)) = g(b) = c$, as required. Thus $g \circ f$ is surjective.

c) is just the case where both a) and b) apply.

To prove d), it is enough to realize that both the double compositions refer to the function that to $a \in A$ associates $h(g(f(a)))$. $\square$

Because of d), it is customary to drop parentheses and simply write $h \circ g \circ f$ for a double composition.

Exercise 6.1.10. Let $f : A \rightarrow B$ be a function, and let $B' \subset B$ be the image of f . Define a new function $g : A \rightarrow B'$ by $g(a) = f(a)$ for all $a \in A$. Show that g is surjective. Show that if f is injective, g is bijective.

Example 6.1.11. Define a function from pairs of real numbers to real numbers, $f : \mathbb{R}^2 \rightarrow \mathbb{R}$, by multiplying the two numbers. We write $f(a, b) = ab$. This is surjective since any number can be written as a product (in many ways, for instance as one times itself). It is not injective, since a number can be written as a product in many ways (for instance $12 = 3 \cdot 4 = 2 \cdot 6$).

Example 6.1.12. The function $f : \mathbb{R} \rightarrow \mathbb{R}$ defined by $f(x) = \sin x$ is neither injective nor surjective. For instance, $f(0) = 0 = f(\pi)$, and there is no real number such that the sine of this number is equal to 2.

The function $g(x) = \tan^{-1} x$ (inverse tangent), if you know about it, is injective but not surjective. Its image is the open interval $(-\frac{\pi}{2}, \frac{\pi}{2})$.

Exercise 6.1.13. Explain, or compute, why $f(x) = \tan x$ is surjective but not injective.

Exercise 6.1.14. Explain why the function $f(x) = x + 2$ is bijective from $\mathbb{R}$ to itself.

It is often important to associate many things at once, which can be understood by considering a function of several variables (as in the example with multiplication). We can also consider functions between sets of pairs, triples and so on. Formally,

this is best described using Cartesian products, introduced in Exercise 2.6.5. See Exercise 6.3.3 for details. In the main text, we will take a less formal approach, and only give a single example.

Example 6.1.15. Let A be the set of pairs (bus, busdriver), and let B be the set of triples of real numbers $\mathbb{R}^3$. Then there is a function that to each bus and each busdriver associates the amount of petrol consumed by the bus and the number of kilometers driven and the wages the busdriver earns while this busdriver drives this bus.

We can consider this as a function with two inputs and three outputs.

Functions and remainders

We will now produce a lot of similar functions by using multiplication and the modulo operation. The functions we define will be important mathematically, they will give examples of many of the properties introduced, and they are relatively easily implemented in Java.

For varying positive integers n , m and k , we will consider the sets

$$A = \{0, 1, 2, \dots, n-1\} \text{ and } B = \{0, 1, 2, \dots, m-1\}$$

and a function $f : A \rightarrow B$ defined by

$$f(a) = k \cdot a \bmod m.$$

Since Java uses negative remainders, we will restrict ourselves to consider positive integers only. Note also that with this choice of A , we can construct a table of values $f(a) = b$ in Java by letting a simply be the index in the array.

Example 6.1.16. Let both

$$A = \{0, 1, 2, 3\} \text{ and } B = \{0, 1, 2, 3\}$$

so that $n = m = 4$. Multiplication by 0 or 1 produces rather obvious results, so let f be multiplication by $k = 2$. We produce the following table:

$$\begin{aligned} f(0) &= 0 \cdot 2 = 0 \bmod 4 \\ f(1) &= 1 \cdot 2 = 2 \bmod 4 \\ f(2) &= 2 \cdot 2 = 0 \bmod 4 \\ f(3) &= 3 \cdot 2 = 2 \bmod 4 \end{aligned}$$

We see that multiplication by 2 is neither injective ($f(0) = f(2) = 0$) nor surjective (we can never get 3).

Finally, consider multiplication by $k = 3$. Then

$$\begin{aligned} f(0) &= 0 \cdot 3 = 0 \bmod 4 \\ f(1) &= 1 \cdot 3 = 3 \bmod 4 \\ f(2) &= 2 \cdot 3 = 2 \bmod 4 \\ f(3) &= 3 \cdot 3 = 1 \bmod 4 \end{aligned}$$

We see that this is both injective and surjective, so it is bijective.

Example 6.1.17. Let both

$$A = \{0, 1, 2, 3, 4\} \text{ and } B = \{0, 1, 2, 3, 4\}$$

so that $n = m = 5$. Multiplying by $k = 1, 2, 3$ or 4 will be bijective. For instance, with $k = 3$,

$$\begin{aligned} f(0) &= 0 \cdot 3 = 0 \bmod 5 \\ f(1) &= 1 \cdot 3 = 3 \bmod 5 \\ f(2) &= 2 \cdot 3 = 1 \bmod 5 \\ f(3) &= 3 \cdot 3 = 4 \bmod 5 \\ f(4) &= 4 \cdot 3 = 2 \bmod 5 \end{aligned}$$

Exercise 6.1.18. Check that multiplication by $k = 2$ is also bijective in this case.

The reason for the difference between multiplication, and taking remainders, modulo 4 and 5, is that 5 is prime. Multiplication by 2 is not bijective modulo 4, the reason is that 2 and 4 have a common factor.

Theorem 6.1.19. *Let*

$$A = B = \{0, 1, 2, \dots, n-1\}$$

and $f : A \rightarrow B$ be defined by

$$f(a) = k \cdot a \bmod n.$$

Then f is bijective if and only if k and n don't have any common factor. If they have a common factor, f is neither injective nor surjective.

Proof. We will prove that if f is not injective, k and n have a common factor, and that if n and k have a common factor, then f is not injective.

Assume first that f is not injective, so that $f(a) = f(a')$ but $a \neq a'$. This means that

$$\begin{aligned} ak &= a'k \pmod{n} \\ (a - a')k &= 0 \pmod{n} \end{aligned}$$

Thus $(a - a')k$ is divisible by n . But $|a - a'| < n$, so multiplying by k takes a number that isn't divisible by n to a number that is divisible by n . Thus k and n have a common factor.

Assume then that n and k have a common factor d . That means that $n = dn'$ and $k = dk'$ for some smaller integers n' and k' . But then

$$kn' = k'dn' = k'n = 0 \pmod{n}$$

and this is the same as $k \cdot 0$. Since $n' \neq 0$, f is not injective.

Finally, note that a function between two finite sets with the same number of elements is surjective if and only if it is injective (Exercise 6.3.2). This concludes the proof. $\square$

From the theorem, we see that if k and n have no common factor, there is a number a such that $ak = 1 \pmod{n}$. It is usually computed by a procedure involving the *Euclidean algorithm*, which you will learn about in Section 19.1.

Example 6.1.20. We will now consider two different sets,

$$A = \{0, 1, 2, \dots, 5\} \text{ and } B = \{0, 1, 2, \dots, 17\}$$

so that $n = 6$ and $m = 18$. First, let $f : A \rightarrow B$ be defined by multiplication by $k = 4$, and taking remainders modulo 18. We start with 4 since multiplication by a smaller number will not get us higher than 15, so nothing happens when we reduce modulo 18. We get the table

$$\begin{aligned} f(0) &= 0 \cdot 4 = 0 \pmod{18} \\ f(1) &= 1 \cdot 4 = 4 \pmod{18} \\ f(2) &= 2 \cdot 4 = 8 \pmod{18} \\ f(3) &= 3 \cdot 4 = 12 \pmod{18} \\ f(4) &= 4 \cdot 4 = 16 \pmod{18} \\ f(5) &= 5 \cdot 4 = 2 \pmod{18} \end{aligned}$$

This is an injective map, since there are no repetitions in the table. Similarly, mul-

multiplication by 5 turns out to be injective. Multiplication by 6 produces this table:

$$\begin{aligned}
 f(0) &= 0 \cdot 6 = 0 \bmod 18 \\
 f(1) &= 1 \cdot 6 = 6 \bmod 18 \\
 f(2) &= 2 \cdot 6 = 12 \bmod 18 \\
 f(3) &= 3 \cdot 6 = 0 \bmod 18 \\
 f(4) &= 4 \cdot 6 = 6 \bmod 18 \\
 f(5) &= 5 \cdot 6 = 12 \bmod 18
 \end{aligned}$$

It is not injective, there are three different values in the image, each appearing twice. Multiplication by 7 and 8 turns out to be injective. Multiplication by 9 gives

$$\begin{aligned}
 f(0) &= 0 \cdot 9 = 0 \bmod 18 \\
 f(1) &= 1 \cdot 9 = 9 \bmod 18 \\
 f(2) &= 2 \cdot 9 = 0 \bmod 18 \\
 f(3) &= 3 \cdot 9 = 9 \bmod 18 \\
 f(4) &= 4 \cdot 9 = 0 \bmod 18 \\
 f(5) &= 5 \cdot 9 = 9 \bmod 18
 \end{aligned}$$

so there are two values, each appearing three times.

Continuing in this way, we will eventually find that multiplication by 12 works as multiplication by 6, whereas multiplication by each of 10, 11, 13, 14, 15, 16, 17 is injective.

Since there are more elements in B than in A , a map $g : B \rightarrow A$ can never be injective. We can produce a table for multiplication by 3, for instance:

$$\begin{aligned}
g(0) &= 0 \cdot 3 = 0 \bmod 6 \\
g(1) &= 1 \cdot 3 = 3 \bmod 6 \\
g(2) &= 2 \cdot 3 = 0 \bmod 6 \\
g(3) &= 3 \cdot 3 = 3 \bmod 6 \\
g(4) &= 4 \cdot 3 = 0 \bmod 6 \\
g(5) &= 5 \cdot 3 = 3 \bmod 6 \\
g(6) &= 6 \cdot 3 = 0 \bmod 6 \\
g(7) &= 7 \cdot 3 = 3 \bmod 6 \\
g(8) &= 8 \cdot 3 = 0 \bmod 6 \\
g(9) &= 9 \cdot 3 = 3 \bmod 6 \\
g(10) &= 10 \cdot 3 = 0 \bmod 6 \\
g(11) &= 11 \cdot 3 = 3 \bmod 6 \\
g(12) &= 12 \cdot 3 = 0 \bmod 6 \\
g(13) &= 13 \cdot 3 = 3 \bmod 6 \\
g(14) &= 14 \cdot 3 = 0 \bmod 6 \\
g(15) &= 15 \cdot 3 = 3 \bmod 6 \\
g(16) &= 16 \cdot 3 = 0 \bmod 6 \\
g(17) &= 17 \cdot 3 = 3 \bmod 6
\end{aligned}$$

This map is not surjective; there are only two values in the image, each appearing three times. Continuing in this way, we will see that multiplication by 1 or 5 is surjective, and multiplication by 2 or 4 isn't.

By now, our patience should wear thin. It is time to leave these tedious computations to the computer.

Example 6.1.21. Here is some pseudocode for constructing the function f we have considered.

```

n = 6
m = 18
A = {0,1,2,..., n-1}
B = {0,1,2,...,m-1}
k = 9

for a in A
    f[a] = a*k mod m
end for

```

To check its properties, we can compute how many different values there are:

```

numberOfValues = 0
for b in B
    if <b in the table of f[]>
        numberOfValues++
    end if
end for
if numberOfValues = n
    <f is injective>
end if
if numberOfValues = m
    <f is surjective>
end if

```

See the exercises for an implementation in Java.

6.2 Methods in Java

The notion of a *method* in Java refers to a sort of function, typically with a number of inputs and a number of outputs. Unlike ordinary functions, methods can also perform tasks that are not directly connected to producing the output variables from the input variables. In fact, there are methods that have neither inputs nor outputs¹. Before we do anything formal, let us consider an example where there are both inputs and outputs.

Example 6.2.1. In this section, we consider a class called `Combinatorics` that contain several mathematical functions defined on integers. We are only interested in collecting functions for now, so the class can be declared as `abstract`, meaning that we can't instantiate objects, and the methods are `static` (belong to the class). The first function is called `computeFactorial`, takes one non-negative integer as input and gives one (positive) integer as output. This function multiplies the first n integers, so that if it is applied to 4 it produces $1 \cdot 2 \cdot 3 \cdot 4 = 24$. In mathematics, this is written using the exclamation mark $!$, so that $4! = 24$. There are conventions for $n = 0$ and $n = 1$, so the entire definition of the factorial function is

$$n! = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ 1 \cdot 2 \cdot 3 \cdot \dots \cdot n & \text{otherwise} \end{cases}$$

To define a method with the correct input and output types,

¹see Khalid Azim Mughal, Torill Hamre and Rolf W. Rasmussen: Java som første programmeringsspråk, tredje utgave. Cappelen, 2006., 4.3 p117 for several examples

```
static int computeFactorial(int n){
    //Compute the function ...
}
```

Since the input has to be non-negative, we should produce some sort of error if the input is negative. The details about producing this sort of behaviour is best left to the programming course; let us instead change the sign for negative inputs, and just write a message to the user.

```
if (n < 0 ){
    n = -n;
    System.out.println("Negative number. Sign changed");
}
```

If $n = 0$ or $n = 1$, we should return 1.

```
if (n == 0 || n == 1){
    return 1;
}
else {} // the computation for n>=2
```

We then declare a local integer variable, to keep track of the partial products as we multiply more and more numbers. In the end, this variable will keep the answer to our problem, so we call it **answer**. To make sure that multiplication works as we want it to, we initialize it with the value 1. Then we use this variable in a loop, and make sure that the index of the loop exactly covers the numbers we want to multiply.

```
int answer = 1;
for (int i = 2; i <= n; i++){
    answer *= i;
}
return answer;
```

To use this method, we call `Combinatorics.computeFactorial` from `main`. For instance,

```
// in main
int fiveFactorial;
fiveFactorial = Combinatorics.computeFactorial(5);
System.out.printf("5! = %d\n", fiveFactorial);
```

In addition to computing the value of the factorial function, this method checks whether the input is reasonable for the function, and prints a message if it isn't.

Exercise 6.2.2. Check that the programme works by running it on a small negative number, 0, and a small positive number. Also check it for $n = 17$. What do you think happens?

In our next example, we will use two inputs to produce one output.

Example 6.2.3. The *binomial coefficients* are defined for two integers satisfying

$$0 \leq k \leq n$$

by

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

using the factorial function that we have already introduced. We pronounce $\binom{n}{k}$ “n choose k”. For instance

$$\binom{5}{3} = \frac{5!}{3!(5-3)!} = \frac{120}{6 \cdot 2} = 10.$$

Despite being defined as a fraction, the binomial coefficients are always integers. In Java, we add a new method to the `Combinatorics`-class to compute binomial coefficients, call it `computeBinomialCoeff`. It takes two inputs, and returns one output:

```
static int computeBinomialCoeff(int n, int k){
    // compute ...
}
```

As for the factorial, we should check that the inputs satisfy the conditions we impose. Let us do this simply by returning `-1` if something is wrong, and write a message for the user.

```
if (! (0 <= k && k <= n)){
    System.out.println("Error. Wrong inputs");
    return -1;
}
else {} //compute ...
```

Since we already know how to compute factorials, we can simply call the previously defined method three times, and return the value given by the definition.

```
return computeFactorial(n)/(computeFactorial(k)*computeFactorial(n-k));
```

If we called `computeFactorial` from outside the `Combinatorics`-class, we would have to write `Combinatorics.computeFactorial(n)` and so on.

To use the method, call `Combinatorics.computeBinomialCoeff()` from main, as in

```
System.out.println(Combinatorics.computeBinomialCoeff(5,3));
```

Exercise 6.2.4. The binomial coefficients are read “n choose k” because they count the number of ways to choose k objects from a collection of n objects. For instance,

$$\binom{34}{7} = 5379616$$

is the number of ways to choose 7 numbers from $1, 2, \dots, 34$. This is the number of possibilities in the Norwegian lotto. Try this computation in Java. What do you think is the reason you get the wrong answer?

Let us finally look at an example that gives two integers as outputs. Note that there are better solutions than the one we present here, if you think in an object oriented way.

Example 6.2.5. We will include in the `Combinatorics`-class a method that finds the minimum and the maximum values in a table of integers. This is a function with two outputs and an unspecified number of inputs, both will be considered as arrays. We start with the input and output types of this method.

```
public static int [] findMaxMin (int [] anArray){
    // the method in here ...
}
```

To find the maximum, we initialize a variable to the first element in the array. For each element, we then compare the current maximum to the element. If the new element is larger, we update the maximum. In the end, this variable contains the maximum of the whole list. A similar computation produces the minimum. This way of finding the maximum has already been considered in Example 5.0.1. Since the output type is `int []`, we can collect the two local variables in an array as well, and initialize as follows:

```
int [] answer = new int [2];
answer[0] = anArray[0];
answer[1] = anArray[0];
```

The loop can then be written

```
for (int i = 1; i < anArray.length; i++){
    if (answer[0] < anArray[i]){
        answer[0] = anArray[i];
    }
    if (answer[1] > anArray[i]){
        answer[1] = anArray[i];
    }
}
```

Finally, we need to return the answer.

```
return answer;
```

Calling this function from `main` requires declaring an array to hold the output, as in

```
int [] test = {2,1,3,6,2,1,7};
int [] maxMinOfTest = new int [2];
maxMinOfTest = Combinatorics.findMaxMin(test);
```

Exercise 6.2.6. Check that the maximum and minimum of the array `test` are correctly computed by Java.

6.3 Exercises

Exercise 6.3.1. Let A , B and C be sets, and $f : A \rightarrow B$ and $g : B \rightarrow C$ be functions. Prove each statement:

- (a). If $g \circ f$ is injective, f is injective.
- (b). If $g \circ f$ is surjective, g is surjective.
- (c). If $g \circ f$ is bijective, and one of f and g is bijective, then the other one is bijective as well.

Produce examples showing

- (d). If $g \circ f$ is injective, g need not be injective.
- (e). If $g \circ f$ is surjective, f need not be surjective.
- (f). If $g \circ f$ is bijective, it is possible that neither f nor g is bijective.

Exercise 6.3.2. Let A and B be finite sets with the same number of elements. Prove that the following are equivalent for a function $f : A \rightarrow B$:

- a) f is injective.
- b) f is surjective.
- c) f is bijective.

Exercise 6.3.3. This exercise considers functions and Cartesian products, as introduced in Exercise 2.6.5. Let $f : A \rightarrow C$ and $g : B \rightarrow D$ be functions between sets.

Define $f \times g(a, b) = (f(a), g(b))$. Check that this defines a function $A \times B \rightarrow C \times D$.

- (a). If f and g are injective, prove that $f \times g$ is injective.
- (b). If f and g are surjective, prove that $f \times g$ is surjective.
- (c). If f and g are bijective, prove that $f \times g$ is bijective.

Exercise 6.3.4. Add another method to the `Combinatorics` class called `computeCatalan`. This method takes one input and gives one output. The mathematical formula for the n th *Catalan number* is

$$C_n = \frac{1}{n+1} \binom{2n}{n} \text{ for } n \geq 0.$$

The input should be n , the output C_n .

To check your programming, produce the first few Catalan numbers, and compare with "<http://oeis.org/A000108>" from the online encyclopedia of integer sequences.

Exercise 6.3.5. We will now implement the functions $f : A \rightarrow B$, where

$$A = \{0, 1, 2, \dots, n-1\} \text{ and } B = \{0, 1, 2, \dots, m-1\}$$

and the function f is defined by

$$f(a) = k \cdot a \bmod m.$$

See Example 6.1.21. If you want a more challenging exercise, look only at the Example, and produce a Java programme for these tasks. Otherwise, read the following hints.

To initialize A , first give a value to n , define A as an array of this length, and define $A[i] = i$ in a loop. Do the same for B .

Then declare the function f as an array of the same length as A , and define $f[i]$ as in the pseudocode in the example.

The most involved part of this exercise is to compute the number `numberOfValues` correctly. Do this!

Finally, write a statement if f is injective or surjective.

If you want to check your programme, use it for some of the values in the main text.

Chapter 7

Loops

Loops are the essential tool for using the computer to work with sequences and series. They are also useful for many other purposes, both in mathematics and elsewhere. In this course, and in the programming course, you have seen many loops already. Our goal in this chapter is to apply loops to some mathematical phenomena we have seen earlier.

7.1 Looping in an example

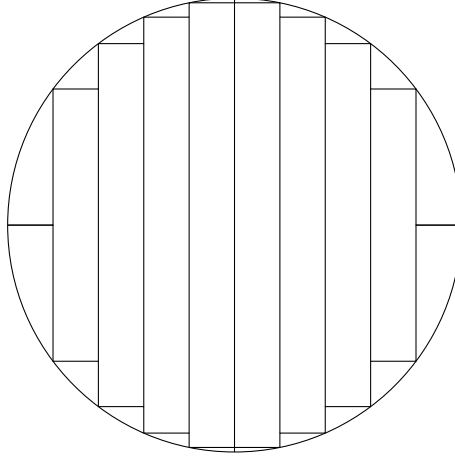
In this section, we show a way to produce an approximation to π by calculating the area of a circle. If you are primarily interested in the programming, you can skip to Equation 7.1.

Recall that the area of a circle is πr^2 , so if we choose a circle with radius $r = 1$, the area is exactly π . We can also position the circle in a coordinate system, so that the origin $(0, 0)$ is the centre of the circle. The equation for the circle is then $x^2 + y^2 = 1$ (Pythagoras). To see the approximation we will use, for each even integer $m = 2n$, divide the line segment $[-1, 1]$ into m equal pieces by marking the points $0, \pm\frac{1}{n}, \pm\frac{2}{n}$ and so on, until $\pm\frac{n-1}{n}$ and finally $\pm\frac{n}{n} = \pm 1$. For each line segment, draw a rectangle from the bottom to the top of the circle, stopping when the first corner comes into contact with the circle. See Figure 7.1.

The area of each rectangle can be computed as height times length of the base. They all have the same base length, $1/n$. The height of the rectangle with base the line segment $[\frac{i}{n}, \frac{i+1}{n}]$, for $i \geq 0$, is $2\sqrt{1 - (i + 1/n)^2}$. It can be calculated by noting that the two right corners have coordinates (x, y) where $x = \frac{i+1}{n}$ and y can be found from $x^2 + y^2 = 1$. So the area of each rectangle can be found as

$$\frac{1}{n} \cdot 2\sqrt{1 - (i + 1/n)^2} \text{ for } i = 0, 1, \dots, n - 1$$

Figure 7.1: Rectangles in a circle



Since the diagram is symmetric, the rectangles on the left of the circle have the same area as the rectangles on the right, so we can simply multiply by two to get the full answer. Thus we are left with computing

$$\pi > 2 \sum_{i=0}^{n-1} \frac{1}{n} \cdot 2\sqrt{1 - (i + 1/n)^2} = 4 \frac{1}{n} \sum_{i=0}^{n-1} \sqrt{1 - (i + 1/n)^2} \quad (7.1)$$

As we increase n , the sum will become larger, and approximate, but always be lower than, π . How would we compute this in Java? The following code involves a trick or two, that will be explained shortly.

```
int n = 5;
double approx = 0;
for (double i = 0; i < n; i++){
    approx += Math.sqrt(1 - (i+1)/n*(i+1)/n);
}
approx = approx * 4 / n;
System.out.println(approx);
```

Running this code results in about 2.6. As an approximation of π , this is not very good. To get better estimates, we need to increase n . Experimenting for a bit, we soon see that n has to be well over 1000 before the sum gets larger than 3.14, which is the (lower) estimate for π that everybody knows. Obviously, we should improve the way we have structured this computation if we want to get good approximations. In this case, it is not practical to set the integer n , that is the number of times the loop should be calculated, first.

Exercise 7.1.1. Run the code for approximations of π with 1000, 2000 and 10000 iterations. How many digits do you believe are correct?

A couple of remarks about the code. We have called the library function `Math.sqrt`. This produces $\sqrt{}$ as a `double`. Note also that the looping index i has been declared as a `double`. This is to make sure that the division $(i + 1)/n$ is considered as a `double`, and not as the result of integer division (which would be 0).

7.2 Types of loops in Java

We will consider the standard `for`-loop, the simplified `for`-loop, and the `while`-loop. You have seen all of these before.

A typical `for`-loop looks like this:

```
for (int i = 0; i < n; i++){
    //do something n times
}
```

This is useful when we for some reason know in advance how many times we want to do something. We did this for the computation of factorials (except for the starting point), since if we want to compute $n!$ we need to know in advance what n is. Take a look at Example 6.2.1 again, if you need a reminder. If we need to perform the same operation on each of the coordinates x, y, z in $\mathbb{R}^3$, this is also easily handled by a `for`-loop (with $n = 3$).

Example 7.2.1. Suppose another programmer has made a method for determining whether a number is prime or not:

```
public static boolean isPrime (int n){
    // Returns true if n is prime, false otherwise
}
```

Using this method, we can count the number of primes below an integer n using a `for`-loop like this:

```
int numberOfPrimes = 0;
for (int i = 0; i < n; i++){
    if (isPrime(i)){
        numberOfPrimes++;
    }
}
```

This is a typical example for counting something.

Example 7.2.2. It is also convenient to loop over other types than `int`. If we want to use the index in a computation, for instance dividing by it, it might be useful to declare it as a floating point.

```
for (float i = 0; i < 20; i++){
    System.out.println(i/20);
}
```

If `i` were declared as an `int`, the division would be considered an integer division, and all the results would be 0.

Since the letters of the English alphabet are numbered consecutively as characters, we can loop over the alphabet like this:

```
for (char i = 'a'; i <= 'z'; i++){
    if (i == 'a' || i == 'e' || i == 'i' || i == 'o' || i == 'u'){
        System.out.printf("%c is a vowel\n", i);
    }
}
```

The update and the stopping conditions can also be different than the standard. For instance, we could print a table of powers of two less than one million:

```
for (int i = 1; i < 1000000; i *= 2){
    System.out.println(i);
}
```

Exercise 7.2.3. Print the negative powers of 2 that are greater than 0.000001 using a `for` loop.

Example 7.2.4. To compute the integer approximation to the binary logarithm $\log_2(n)$ by the procedure from Example 3.3.1, we can use a `for`-loop like this

```
int n = 54;
int binLog = 0;
for (double i = n; i > 1; i /= 2){
    binLog++;
}
System.out.println(binLog);
```

When we are looping through an array, we can use a `for`-loop like this:

```
// anArray is some array of doubles, say
for (int i = 0; i < anArray.length; i++){
    //do something
}
```


Here the looping index has the same value as the array index. But it is often more interesting to use the actual objects stored in the array, and the simplified `for`-loop allows us to do that directly.

```
// anArray is some array of doubles, say
for (double x : anArray){
    //do something
}
```

Then we don't have access to the array index, but to the object itself.

Example 7.2.5. To compute the sine of each number in an array, we can write

```
// anArray is some array of doubles, say
for (int i = 0; i < anArray.length; i++){
    System.out.println(Math.sin(anArray[i]));
}
```

Using the simplified `for`-loop, this becomes

```
// anArray is some array of doubles, say
for (double x : anArray){
    System.out.println(Math.sin(x));
}
```

The second loop seems simpler, because the computation doesn't use the index, just the object.

`while`-loops are performed when we don't know, or don't care, how many times a loop must be executed. The boolean expression used to check whether we execute something, must be updated in the loop.

Example 7.2.6. You might remember that

$$\lim_{x \rightarrow 0} \frac{\sin x}{x} = 1.$$

Assuming that $x > 0$, how close to 0 must we choose x in order that $\frac{\sin x}{x} > 0.99999$? We can try with the values $x = 1$, $x = 0.1$, $x = 0.01$ and so on:

```
double x = 1.0;
while (Math.sin(x)/x < 0.99999){
    x /= 10;
}
System.out.println(x);
```

When the computation stops, we are close enough to zero to have a very good approximation to the limit.

Exercise 7.2.7. Modify the code in Example 7.2.6 to also tell you how many times the loop was executed.

This is a typical example of a mathematical application of a **while**-loop: We want to increase accuracy until some bound is reached.

Example 7.2.8. This is an example of a **while**-loop, where we want to know how many times a computation is performed. The $3x + 1$ -conjecture is a conjecture about an operation on positive integers. If an integer is even, the operation is to divide it by 2. If the integer is odd, the operation is to multiply it by 3 and add 1. Then the conjecture states that for any positive integer, we will finally come to 1 by iterating this procedure. We want to see how many times the operation is performed for a given integer. To get a feeling for the operation, we also print the value of the integer in the loop.

```
int n = 245123;
int timesPerformed = 0;
while (n != 1){
    timesPerformed++;
    if (n%2 == 0){ // n is even
        n /= 2;
    } else { // n is odd
        n = 3 * n + 1;
    }
    System.out.println(n);
}
System.out.println(timesPerformed);
```

For this input, the operation is performed 181 times.

Exercise 7.2.9. Using an outer loop for the integer n , find the highest possible `timesPerformed` for the procedure in Example 7.2.8 for an integer less than 1000. You probably don't want to print much during the computation, since printing is slow.

7.3 Summing series

For a finite series, the **for**-loop is the preferred method for getting the sum. The indexing of the loop can usually be the same as the indexing of summation.

Example 7.3.1. Sum the finite series

$$\sum_{i=0}^n \frac{1}{1 + \sqrt{i}}$$

for $n = 56$. We need to declare a variable to hold the answer, and we add the listed term in a loop:

```
int n = 56;
double answer = 0.0;
for (int i = 0 ; i <= n; i++){
    answer += 1/(1 + Math.sqrt(i));
}
System.out.println(answer);
```

We get about 11.4.

Exercise 7.3.2. Find the smallest n such that the sum in Example 7.3.1 is higher than 8.

Summing an infinite series is more difficult, since we don't know in general whether it is convergent or not, nor how many terms we need to make sure that we have the precision we want.

Example 7.3.3. The mathematical constant e can be computed as a series

$$e = \sum_{i=0}^{\infty} \frac{1}{i!} = \frac{1}{0!} + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \cdots$$

We can get many digits by summing many terms. How can we make sure that the first five digits, say, are correct? We can try various values for n in this **for**-loop:

```
int n = 5;
double e = 0;
for (int i = 0; i < 5; i++){
    e += 1.0/Combinatorics.computeFactorial(i);
}
```

if we still remember the **Combinatorics**-class. For various reasons, this is unsatisfactory. First, it is a procedure of trial and error, so we need to change the value of n to see whether there are significant changes (we could loop through some values of n), and determine ourselves if we think the approximation is good enough. Second, the **computeFactorial** method that we computed only works for small values of the input, because of overflow.

Let us give an *ad hoc* reason for how many terms we need. If we, from n on, change $1/(n+k)!$ by the larger number $1/(n!n^k)$, we get a convergent geometric series that we can compute the sum of:

$$\frac{1}{n!} + \frac{1}{(n+1)!} + \frac{1}{(n+2)!} + \cdots < \frac{1}{n!} + \frac{1}{n! \cdot n} + \frac{1}{n! \cdot n^2} + \cdots$$

Using the notion for geometric series, we have $r = \frac{1}{n!}$ and $k = 1/n$. The sum is, by Theorem 5.2.10

$$\frac{1}{n!} + \frac{1}{n! \cdot n} + \frac{1}{n! \cdot n} + \cdots = \frac{r}{1 - k} = \frac{1/n!}{1 - 1/n}$$

So if we stop before the n th term, the error we will do is smaller than this number. Let us in addition replace the computation of factorials to a computation involving doubles, and we get this **while**-loop. To get smoother code, we note that the first two terms $1/0! + 1/1! = 1 + 1 = 2$, so we skip to $n = 2$.

```
double e = 2.0;
double n = 2.0;
double nfactorial = 2.0;
double bound = 0.00001;
while ((1 / nfactorial) / (1 - 1 / n) > bound) {
    e += 1 / nfactorial;
    n++;
    nfactorial *= n;
}
System.out.println(e);
```

This returns 2.71827876984127, and the correct six first digits are 2.71828.

We will give another example, where the reason we can be sure of the accuracy of the approximation, is that we have both a lower and an upper bound for the value.

Example 7.3.4. Consider again the approximation of π from Section 7.1. We found in Equation 7.1

$$\pi > 2 \sum_{i=0}^{n-1} \frac{1}{n} \cdot 2\sqrt{1 - (i + 1/n)^2} = 4 \frac{1}{n} \sum_{i=0}^{n-1} \sqrt{1 - (i + 1/n)^2} \quad (7.2)$$

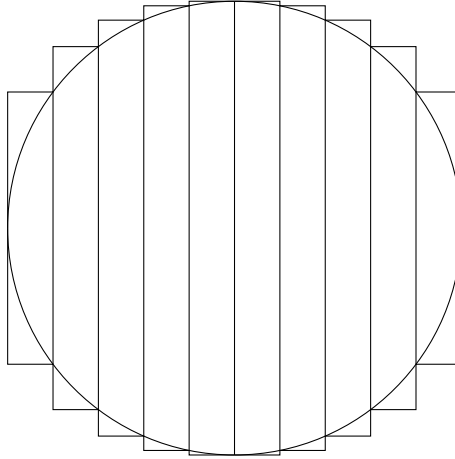
by considering rectangles inside a circle. But we could have found another approximation by instead considering a circle inside a collection of rectangles as in Figure 7.2.

It should be clear that the combined area of the rectangles is greater than the area of the circle (π). By arguing as for the smaller rectangles, one can find that

$$\pi < 4 \frac{1}{n} \sum_{i=0}^{n-1} \sqrt{1 - (i/n)^2}$$

Looping the same way as in Section 7.1 produces an upper bound.

Figure 7.2: A circle in a collection of rectangles



```

int n = 5;
double approx = 0;
for (double i = 0; i < n; i++){
    approx += Math.sqrt(1-i/n*i/n);
}
approx = approx * 4 / n;
System.out.println(approx);

```

Running this code returns about 3.4, which is indeed larger than π . But we can combine the two approximations to get a bound for the error as well! In very short pseudocode,

```

while upperBound - lowerBound > errorTolerance
    increase n
    repeat the computation of upperBound and Lowerbound
end
print (upperBound-lowerBound)/2

```

The ambitious reader is encouraged to stop reading and start programming now!

We need some sort of initialization to get started. One way to do this is by implementing the loop as a **do-while** loop, so that we now the computation is performed once before we check our condition. This is the approach we will use. Another approach is to initialize the values in such a way that we know that π is between the values. Since this relies on our previous understanding of π , it is not so well suited to other problems of the same nature. We also note that since we had to increase n a lot to get above 3.14 in Section 7.1, it is not reasonable to increase n slowly. Let us multiply it by 5 for each iteration.

```

double errorTolerance = 0.001;
double lowerBound;
double upperBound;
int n = 1;
do{
    n *= 5;
    lowerBound = 0;
    upperBound = 0;
    for (double i = 0; i < n; i++){
        lowerBound += 4*Math.sqrt(1-(i+1)/n*(i+1)/n)/n;
        upperBound += 4*Math.sqrt(1-i/n*i/n)/n;
    }
} while (upperBound - lowerBound > errorTolerance);
System.out.println((upperBound + lowerBound)/2);
System.out.printf("%f < pi < %f\n", lowerBound, upperBound);

```

The output is

```

3.14159205148815
3.141464 < pi < 3.141720

```

The first is an approximation of π , the second more accurately tells us explicitly a lower and an upper bound for the value of π .

7.4 Exercises

Exercise 7.4.1. Find the sum of the integers underlying the `chars` in the English alphabet.

Exercise 7.4.2. Find the smallest n such that

$$\sum_{i=1}^n i^2$$

is too large to be represented as an `int`.

Exercise 7.4.3. In the television show *Futurama*, two robots having serial numbers 2716057 and 3370318 are delighted to find that both of them have serial numbers that are expressible as a sum of two cubes. Another example is $35 = 8 + 27 = 2^3 + 3^3$. Find the expressions as sums of two cubes for the two serial numbers. Hint: Cubes can also be negative.

Exercise 7.4.4. The value of the natural logarithm at 2, $\ln 2$, can be computed by the infinite sum

$$\ln 2 = \sum_{n=1}^{\infty} \frac{(-1)^{n+1}}{n}$$

This is a standard result in connection with *Taylor series*. By experimenting with the number of terms, try to find a numerical approximation for $\ln 2$. Compare this with the result of calling `Math.log(2)`.

Exercise 7.4.5. In the theory of *Fourier series*, a standard exercise shows that

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

By experimenting with the number of terms, try to find a numerical approximation for π^2 . Compare this with the result of calling `Math.PI*Math.PI`. Also compare the number of terms needed to get good approximations with the number of terms needed in Exercise 7.4.4.

Chapter 8

Proofs and reasoning

We have seen many arguments showing that something is true. In this chapter, we will look more formally at the structure of a proof, and give some examples both in mathematics and programming. We will also have a new look at previous results and statements.

8.1 Propositional logic

When we first motivated the independent study of boolean algebra, we started from statements that could be true or false, and introduced variables that could be true or false. We now go back the statements, or *propositions*.

Definition 8.1.1. A *proposition* is a statement that is clear enough to allow us to attach a truth value to it.

This definition is in a sense utilitarian, we demand that a proposition is sufficiently clear, but is might not be clear if it is clear enough!

Example 8.1.2. Let p be the proposition “5 is a prime number”. This is quite clear, and the truth value is **true**.

Let q be the proposition “Today is Thursday”. Again, it is quite clear, but the truth value must be determined when the proposition is read.

It is customary to use letters like p, q, r for propositions. They can be combined in the same way as boolean expressions.

Example 8.1.3. Let again p be the proposition “5 is a prime number”, and let q be the proposition “5 is odd”. Then $p \wedge q$ is the statement “5 is an odd prime”. Again, this is true.

Note that propositions are more than their truth values. Formally, we can say that there is a function from the set of propositions to the set of truth values, that attach to a proposition its truth value. This is similar to what we said about statements in connection with boolean algebra, as in “ x is the truth value of the statement ...”.

Exercise 8.1.4. Find statements p, q and r so that the following proposition corresponds to $(p \vee q) \rightarrow r$: “If I’m wet or dirty, I put on new clothes”.

8.2 Predicate logic

Sometimes we want to study truth values of many similar statements at once. This is the subject of *predicate logic*.

Definition 8.2.1. Let x be in some set A , and let P be a function from A to the set of propositions, so that there is a proposition $P(x)$ attached to x . Then P is called a *predicate*.

Example 8.2.2. Let $x \in \mathbb{Z}$, $x \geq 1$. Let P be the predicate $P(x) = “x \text{ is a prime}”$. Then $P(5)$ is the proposition “5 is prime”, which is true, and $P(6)$ is the proposition “6 is prime”, which is false. The predicate P incorporates infinitely many propositions, some true, some false.

Example 8.2.3. Let A be a set (as usual, contained in some large, unspecified set). Let P be the predicate $P(x) = “x \in A”$. Then $P(x)$ is true for x inside A and false for x outside A .

Example 8.2.4. Consider the parabola with equation $y = x^2$. Consider the predicate, for $(x, y) \in \mathbb{R}^2$, $P(x, y) = “y = x^2”$. This is true for a point lying on the parabola, false for all other points.

We need new terminology for predicates, similar to the boolean operations we have considered earlier.

Definition 8.2.5. Let P be a predicate. The symbol $\forall$, read *for all*, and called the *universal quantifier*, is used as in

$$\forall x P(x)$$

to express that $P(x)$ is true for all x . Similarly, the symbol $\exists$, read *there exists*, and called the *existence quantifier*, is used as in

$$\exists x P(x)$$

to express that $P(x)$ is true for at least one x .

Note that both $\forall x P(x)$ and $\exists x P(x)$ are propositions.

Example 8.2.6. Let P be the predicate $P(x) = “x \text{ is a prime}”$ considered above. $\forall x P(x)$ is false, since there exists numbers that are not prime. For instance, $P(6)$ is false.

$\exists x P(x)$ means that there exist at least one prime number. This is true, since for instance $P(5)$ is true.

Example 8.2.7. Consider again the predicate $P(x) = “x \in A”$. $\exists x P(x)$ means that there is at least one element in A , so that $A \neq \emptyset$.

$\forall x P(x)$ means that all elements are in A . If A is a set of integers, this means that A is equal to the set of *all* integers.

The two quantifiers can also be combined, for instance with several variables, as in the next example.

Example 8.2.8. Let $f : A \rightarrow B$ be a function between two sets. Consider the following expression:

$$\forall b \in B \exists a \in A f(a) = b$$

If we unravel this statement, it says that for all elements $b \in B$, there exists an element $a \in A$ such that $f(a) = b$. So this is the definition for a function to be *surjective*, phrased in predicate language (see Definition 6.1.6).

Exercise 8.2.9. Let $f : A \rightarrow B$ be a function between two sets. What does

$$\exists b \in B \forall a \in A f(a) = b$$

mean? Hint: What is the image (see Definition 6.1.7)?

Since $\forall$ says that something must be true for all the inputs, it is similar to logical conjunction. In fact, if the set where P is defined is the set $\{x_0, x_1, x_2, \dots, x_n\}$ then

$$\forall x P(x) = P(x_0) \wedge P(x_1) \wedge P(x_2) \wedge \dots \wedge P(x_n) \quad (8.1)$$

Similarly, $\exists$ corresponds to disjunction, so

$$\exists x P(x) = P(x_0) \vee P(x_1) \vee P(x_2) \vee \dots \vee P(x_n) \quad (8.2)$$

Compare to Exercise 1.4.1.

Exercise 8.2.10.

(a). Explain why we need to check something for every x if we want to conclude that $\forall x P(x)$ is true, or that $\exists x P(x)$ is false.

(b). Explain why it is enough to find a single x if we want to conclude that $\forall x P(x)$ is false, or that $\exists x P(x)$ is true.

(c). Formulate de Morgan's laws for predicates, see Theorem 2.3.1.

8.3 Rules of deduction

The most important reason that mathematicians are interested in propositional logic and predicate logic, is that this formalism can be used to clarify mathematical proofs. To use it, we need to talk about truth, and what expressions we actually claim are true. For instance, $x \vee \neg x$ is always true, but we are usually interested in more complicated statements, and we want to know that something is true based on knowing that something simpler is true. The rules of deduction that we will talk about now, are the most basic ways we have to conclude that an expression is true based on some simpler expressions. We will formulate our results both in propositional and in predicate form.

Definition 8.3.1. There are three basic rules of deduction in propositional logic, called respectively *modus ponens*, *modus tollens* and *sylllogism*. We formulate them by setting up a scheme of two expressions that are known to be true, and then a conclusion that is necessarily true as a result.

Modus ponens:

$$\frac{p \rightarrow q \quad p}{q}$$

In words, if it is true that p implies q , and p is true, then q is true.

Modus tollens:

$$\frac{p \rightarrow q \quad \neg q}{\neg p}$$

In words: If it is true that p implies q , but q is false, then p is false.

Sylllogism:

$$\frac{p \rightarrow q \quad q \rightarrow r}{p \rightarrow r}$$

In words: If p implies q and q implies r , then p implies r .

Compare Exercise 1.4.2.

Example 8.3.2. Consider the three propositions

$$\begin{aligned} p &= \text{“Mona lives in Bergen”} \\ q &= \text{“Mona lives in Norway”} \\ r &= \text{“Mona lives in Europe”} \end{aligned}$$

Then it is true that $p \rightarrow q$ and $q \rightarrow r$. We don't know where Mona lives, so each of the three basic propositions can be true or false. In modus ponens, we assume

that Mona lives in Bergen, and conclude that she lives in Norway. In modus tollens, we assume that Mona doesn't live in Norway, and conclude that she doesn't live in Bergen. In the syllogism, we conclude that if Mona lives in Bergen, she lives in Europe (skipping the statement about living in Norway).

In predicate logic, we usually consider general implications, and use them when we know something about a particular element. Since the syllogism is really formulated in the same way for predicates as it is for propositions, we only include modus ponens and modus tollens.

Definition 8.3.3. The rule *modus ponens* is in predicate logic formulated as

$$\frac{\forall x P(x) \rightarrow Q(x) \quad P(a)}{Q(a)}$$

In words, if it is true that $P(x)$ implies $Q(x)$ for all x , and $P(a)$ is true for some element a , then $Q(a)$ is true.

Modus tollens:

$$\frac{\forall x P(x) \rightarrow Q(x) \quad \neg Q(a)}{\neg P(a)}$$

In words, if it is true that $P(x)$ implies $Q(x)$ for all x , and if $Q(a)$ is false for some element a , then $P(a)$ is false.

Example 8.3.4. In this example, all numbers are real numbers. Let $P(x)$ be the predicate that $x \geq 0$, and $Q(x)$ the predicate that x is the square of some number. For instance, $P(-3)$ is false and $Q(5)$ is true, since $-3 < 0$ and $5 = (\sqrt{5})^2$. Then it is true that $\forall x P(x) \rightarrow Q(x)$: Every non-negative number is the square of some number. An example of modus ponens:

$$\frac{\forall x P(x) \rightarrow Q(x) \quad P(5)}{Q(5)}$$

In words, we would say "Since $5 \geq 0$, 5 is a square". A similar example for modus tollens:

$$\frac{\forall x P(x) \rightarrow Q(x) \quad \neg Q(-3)}{\neg P(-3)}$$

In words, -3 is not non-negative since it is not a square.

Exercise 8.3.5. Give examples for modus ponens and modus tollens for the two predicates $P(n) = "n \text{ is even}"$ and $Q(n) = "n + 1 \text{ is odd}"$. Here $n \in \mathbb{Z}$. Note that n is a commonly used letter for an integer variable.

8.4 Mathematical induction

Mathematical induction is a procedure for proving the validity of a predicate P for positive integers. We will usually write n for the integer variable, so we want to prove $\forall n P(n)$ for some types of predicates.

Example 8.4.1 (Interest). You start a new bank account, and deposit 1000 in your favourite currency. The interest is 2%. What is the balance of your account after n years?

This is really a question of a sequence a_n , where $a_0 = 1000$ and at each subsequent step, we take the previous value (the balance last year) and add interest. The first few values are

$$\begin{aligned} a_0 &= 1000 \\ a_1 &= a_0 + 0.02 \cdot a_0 = 1020 \\ a_2 &= a_1 + 0.02 \cdot a_1 = 1040.4 \\ &\vdots \end{aligned}$$

We might guess that $a_n = 1000 \cdot 1.02^n$. This is really a predicate!

We can argue that this is in fact true by defining the predicate $P(n) = "a_n = 1000 \cdot 1.02^n"$, saying that $\forall n \geq 0 P(n) \rightarrow P(n+1)$ is true, and that $P(0) = "a_0 = 1000"$ is true.

To see that $\forall n \geq 0 P(n) \rightarrow P(n+1)$, note that $a_{n+1} = a_n + 0.02 \cdot a_n$. If it is true that $a_n = 1000 \cdot 1.02^n$, then

$$a_{n+1} = a_n + 0.02 \cdot a_n = (1 + 0.02) \cdot a_n = 1.02 \cdot 1000 \cdot 1.02^n = 1000 \cdot 1.02^{n+1}$$

This equality is exactly $P(n+1)$!

Then we can conclude from repeated application of modus ponens, where $Q(n) =$

$P(n+1)$, where the conditions for using modus ponens comes from the step before:

$$\begin{array}{c}
 \forall n \, P(n) \rightarrow Q(n) \\
 \frac{P(0)}{P(1)} \\
 \\
 \forall n \, P(n) \rightarrow Q(n) \\
 \frac{P(1)}{P(2)} \\
 \\
 \forall n \, P(n) \rightarrow Q(n) \\
 \frac{P(2)}{P(3)} \\
 \vdots
 \end{array}$$

The conclusion will be that $P(n)$ is true for all n , which is what we wanted.

Note that this really is an example of a geometric sequence.

Theorem 8.4.2. *Let $P = P(n)$ be a predicate defined for each integer $n \geq 0$. If $P(0)$ is true and it is true that $P(n) \rightarrow P(n+1)$ for all n , then $P(n)$ is true for all n .*

Proof. To give a complete formal proof of this theorem really needs a formalization of several properties of the integers, so we will not go into the details here. Let us just note that in order to prove that $P(k)$, for a given k , is true, we can use modus ponens k times as in

$$\begin{array}{c}
 \forall n \, P(n) \rightarrow P(n+1) \\
 \frac{P(0)}{P(1)} \\
 \\
 \forall n \, P(n) \rightarrow P(n+1) \\
 \frac{P(1)}{P(2)} \\
 \\
 \forall n \, P(n) \rightarrow P(n+1) \\
 \frac{P(2)}{P(3)} \\
 \vdots \\
 \forall n \, P(n) \rightarrow P(n+1) \\
 \frac{P(k-1)}{P(k)}
 \end{array}$$

□

Proving $P(0)$ is called the basis step of the induction, proving $P(n) \rightarrow P(n+1)$ for all $n \geq 0$ is called the induction step. When we use $P(n)$ to prove $P(n+1)$, we refer to $P(n)$ as the *induction hypothesis*. Note that we can assume $P(n)$ is true, since the only way $P(n) \rightarrow P(n+1)$ can be false is if $P(n)$ is true while $P(n+1)$ is false. It is often customary to suppress the name of the predicative, and use the formulation in words or mathematical symbols instead.

Example 8.4.3. Prove that $\sum_{i=0}^n i = \frac{n(n+1)}{2}$ for all $n \geq 0$.

We can refer to the equation, if we want to, as $P(n)$. The basis step, $P(0)$, says that $\sum_{i=0}^0 i = 0$ should be the same as $0 \cdot 1/2 = 0$. This is true, so $P(0)$ has been checked.

For the induction step, assume $P(n)$ for some $n \geq 0$:

$$\sum_{i=0}^n i = \frac{n(n+1)}{2}$$

We want to use this to prove $P(n+1)$:

$$\sum_{i=0}^{n+1} i = \frac{(n+1)(n+1+1)}{2}$$

The sum on the left has one more summand than the corresponding sum for $P(n)$, so using the induction hypothesis we see that

$$\sum_{i=0}^{n+1} i = \sum_{i=0}^n i + n+1 = \frac{n(n+1)}{2} + n+1$$

We can write $n+1 = 2 \cdot (n+1)/2$ to add fractions, so

$$\sum_{i=0}^{n+1} i = \frac{n(n+1)}{2} + n+1 = \frac{n(n+1) + 2(n+1)}{2} = \frac{n^2 + n + 2n + 2}{2} = \frac{n^2 + 3n + 2}{2}$$

Factoring the numerator (how?), we see that $n^2 + 3n + 2 = (n+1)(n+2)$, so that

$$\sum_{i=0}^{n+1} i = \frac{(n+1)(n+2)}{2}$$

as required. The formula has been proved by mathematical induction.

Note that this really is an example of an arithmetic sequence.

Sometimes we need to start at a number different from 0.

Example 8.4.4. Prove that $2^n < n!$ for $n \geq 4$.

The basis step is checking that $2^4 = 16$ is less than $4! = 2 \cdot 3 \cdot 4 = 24$, which is true. For the induction step, assume $2^n < n!$ for some $n \geq 4$. We want to prove that $2^{n+1} < (n+1)!$. Then

$$2^{n+1} = 2 \cdot 2^n < 2 \cdot n!$$

since we have assumed that $2^n < n!$. Furthermore

$$2 \cdot n! < (n+1)n!$$

since $2 < 4 \leq n$. Putting everything together, we see that

$$2^{n+1} = 2 \cdot 2^n < 2 \cdot n! < (n+1)n! = (n+1)!$$

We have proved the inequality for all $n \geq 4$ by mathematical induction.

Exercise 8.4.5. Prove that $3^n < n!$ for $n \geq 7$.

Example 8.4.6. Prove by induction that the sequence defined by $a_0 = 1$, $a_n = \sqrt{a_{n-1} + 2}$ for $n > 0$, is a) increasing and b) bounded by 2.

Increasing means that $a_{n+1} > a_n$ for all $n \geq 0$, and bounded by 2 means that $a_n \leq 2$ for all $n \geq 0$.

To prove that $a_{n+1} > a_n$, note first that (the basis step)

$$a_1 = \sqrt{a_0 + 2} = \sqrt{3} > 1 = a_0$$

Assume that the statement is true for some value of $n \geq 0$, so that $a_{n+1} > a_n$. We want to prove that $a_{n+2} > a_{n+1}$. But

$$a_{n+2} = \sqrt{a_{n+1} + 2} > \sqrt{a_n + 2}$$

since $a_{n+1} > a_n$. But the right hand side is now just the definition of a_{n+1} , so we get the inequality we need. Therefore the inequality is true for all $n \geq 0$.

To prove that the sequence is bounded by 2, note first that the basis step is true: $a_0 = 1 < 2$. For the induction step, assume that $a_n < 2$ for some $n \geq 0$. We want to prove that $a_{n+1} < 2$. We see that

$$a_{n+1} = \sqrt{a_n + 2} < \sqrt{2 + 2} = 2$$

since $a_n < 2$. Therefore the inequality is true for all $n \geq 0$.

Exercise 8.4.7. Prove that the sequence defined by $a_0 = 1$, $a_n = \sqrt{a_{n-1} + 1}$ for $n > 0$ is a) increasing and b) bounded by 2.

8.5 Checking an algorithm

In this Section, we will use mathematical induction to validate an algorithm we have already seen. Consider this as an example of the application of induction to software.

Problem description Given a table of numerical values, arranged as a `float[]`, find the maximum element. Give a precise mathematical proof that your procedure works.

We need to assume that the length of the table is at least one, since we haven't got a choice for the maximum of an empty set of numbers. Also, we call the table `arrayOfFloats`.

Algorithm for a solution Set a variable `max`, of type `float`, equal to the first element of `arrayOfFloats`, `arrayOfFloats[0]`.

Set a variable `i`, of type `int`, equal to 1.

Check if `i < arrayOfFloats.length`.

If it isn't, return `max` as the solution, and stop the computation.

If it is, check if `max` is greater than or equal to `arrayOfFloats[i]`.

If it is, increase `i` by 1 and repeat the steps from "Check if ..." onwards.

If it isn't, set `max` equal to `arrayOfFloats[i]`, increase `i` by 1 and repeat the steps from "Check if ..." onwards.

Proof that the algorithm returns the maximum The proof is by mathematical induction on the length of the array, `n = arrayOfFloats.length`.

By assumption, $n > 0$. If $n = 1$ there is only one number in the array, and this number is then necessarily the maximum. In this case the algorithm stops while $i = 1$, so there is no chance for `max` to be changed from the first value `arrayOfFloats[0]`, and the algorithm gives the correct answer.

Now assume that $n \geq 2$ and that the algorithm works for any array of length $n - 1$. An example of such an array is the array which is equal to `arrayOfFloats` in the first $n - 1$ places, but where the last element of `arrayOfFloats` is dropped. Call this array `shortenedArray`. Since the algorithm proceeds one index at a time, exactly the same steps are performed both for `arrayOfFloats` and for `shortenedArray` until `i < shortenedArray.length` fails. This happens for $i = n - 1$.

So before we start on what the algorithm does for $i = n - 1$, `max` holds the maximum of `shortened.Array` by the induction hypothesis. The only possibility if this is not the same as the maximum of `arrayOfFloats` is if the last element is the largest. If this isn't the case, `max >= arrayOfFloats[n-1]`, so we increase i by 1, see that `i < arrayOfFloats.length` fails (it now reads $n < n$), and so the correct `max` is returned. If, on the other hand, the last element is the largest, `max >= arrayOfFloats[n-1]` fails, so we set `max = arrayOfFloats[n-1]`, increase i by 1, see that `i < arrayOfFloats.length` fails and return the correct `max`.

Therefore the algorithm produces the correct maximum, proved by mathematical induction.

8.6 Exercises

Exercise 8.6.1. It is sometimes useful to be able to say that there exists exactly one of something. This is sometimes written $\exists!$, and is read “there exists unique”. Do not confuse this with the Java symbol for negation!

(a). Let, for $x \geq 1$ an integer, the predicate $P(x) = “x \text{ is an even prime}”$. Explain why this expression is true.

$$\exists!x P(x)$$

(b). What does this expression mean, given a function $f : A \rightarrow B$?

$$\forall b \in B \exists!a \in A f(a) = b$$

Compare to Definition 6.1.6.

(c). Explain, similar to Equations 8.1 and 8.2, how $\exists!$ compares to exclusive or.

Chapter 9

Eigenvectors

Eigenvalues and eigenvectors are invariants attached to linear transformations from a vector space to itself, or to square matrices (which are representations of such transformations). The basic idea is to take something complicated (action of a matrix, many-dimensional problems), and reduce it to something much easier (multiplication by scalars, several one-dimensional problems). In good cases, we can find a basis for our vector space such that the action of a given matrix becomes completely transparent.

9.1 Definition of eigenvectors

Let A be the matrix

$$A = \begin{pmatrix} 3 & 4 \\ 1 & 6 \end{pmatrix}$$

We might try to multiply it with various vectors. For instance,

$$\begin{aligned} A \begin{pmatrix} 0 \\ 1 \end{pmatrix} &= \begin{pmatrix} 4 \\ 6 \end{pmatrix} & A \begin{pmatrix} 2 \\ 2 \end{pmatrix} &= \begin{pmatrix} 14 \\ 14 \end{pmatrix} \\ A \begin{pmatrix} -1 \\ 1 \end{pmatrix} &= \begin{pmatrix} 1 \\ 5 \end{pmatrix} & A \begin{pmatrix} 2 \\ \frac{1}{2} \end{pmatrix} &= \begin{pmatrix} 8 \\ 5 \end{pmatrix} \end{aligned}$$

In one of these four cases, the product $A\vec{x}$ is in fact parallel with $\vec{x}$. This means that the result of multiplying the *matrix* and the vector, which is complicated, is the same as the result of multiplying a *scalar* with the vector, which is easy. This is an example of an eigenvector:

Definition 9.1.1. Let $A_{n \times n}$ be a square matrix. A vector $\vec{x} \neq \vec{0}$ is called an *eigenvector* for A if there exists a scalar λ such that

$$A\vec{x} = \lambda\vec{x}.$$

λ is the corresponding *eigenvalue*.

Example 9.1.2. $\vec{x} = [2, 2]^T$ is an eigenvector for the above matrix A , with corresponding eigenvalue 7. The vector $\vec{v} = [0, 1]^T$ is not an eigenvector for this matrix.

Finding eigenvectors is a complicated business, but checking whether a given vector is in fact an eigenvector is simple. We just multiply, and see if the result is parallel to the vector we start with. This also means that it is easy to check your computations.

Example 9.1.3. By multiplication,

$$\begin{pmatrix} 1 & 5 & 0 & 7 & 1 \\ -1 & -8 & -1 & 3 & 1 \\ 0 & 6 & 4 & 3 & 0 \\ -3 & -2 & 0 & 3 & 1 \\ 2 & 3 & 2 & 2 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 2 \\ 0 \\ 3 \end{pmatrix} = \begin{pmatrix} 4 \\ 0 \\ 8 \\ 0 \\ 12 \end{pmatrix} = 4 \begin{pmatrix} 1 \\ 0 \\ 2 \\ 0 \\ 3 \end{pmatrix}$$

Therefore we have an eigenvector with corresponding eigenvalue 4. If we try

$$\begin{pmatrix} 1 & 5 & 0 & 7 & 1 \\ -1 & -8 & -1 & 3 & 1 \\ 0 & 6 & 4 & 3 & 0 \\ -3 & -2 & 0 & 3 & 1 \\ 2 & 3 & 2 & 2 & 2 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 0 \\ 1 \\ 4 \end{pmatrix} = \begin{pmatrix} 28 \\ -19 \\ 21 \\ -5 \\ 23 \end{pmatrix}$$

we don't get an eigenvector (from the first coordinate, we get $\lambda = 14$, from the second $\lambda = -19/3$).

9.1.1 The characteristic polynomial

When we want to compute eigenvectors, we usually start by computing eigenvalues. Recall that a homogeneous system of linear equations, with a square coefficient matrix, has a unique solution if and only if the determinant is non-zero. Let us transform the defining equation for eigenvectors into such a system:

$$\begin{aligned} A\vec{x} &= \lambda\vec{x} \\ A\vec{x} &= \lambda I_{n+1}\vec{x} && \text{multiplication by } I_{n+1} \text{ changes nothing} \\ (A - \lambda I_{n+1})\vec{x} &= \vec{0} && \vec{x} \text{ is a common factor} \end{aligned}$$

Now we have a homogeneous system, with coefficient matrix $A - \lambda I_{n+1}$. If $\vec{x}$ is an eigenvector, this system has solutions beside the trivial solution $\vec{0}$. Thus the determinant of $A - \lambda I_{n+1}$ must be zero!

The argument can also be reversed, saying that if the determinant is zero, there are solutions $\vec{x}$, which then by definition are eigenvectors.

Definition 9.1.4. The *characteristic polynomial* is the following polynomial in the unknown λ :

$$\det(A - \lambda I_{n+1}) = \begin{vmatrix} a_{00} - \lambda & a_{01} & a_{02} & \cdots & a_{0n} \\ a_{10} & a_{11} - \lambda & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \ddots & \vdots \\ a_{n0} & a_{n1} & a_{n2} & \cdots & a_{nn} - \lambda \end{vmatrix}$$

The *characteristic equation* is the equation $\det(A - \lambda I_{n+1}) = 0$. Thus, the solutions to the characteristic equation are exactly the eigenvalues of A .

Example 9.1.5. Consider again the matrix

$$A = \begin{pmatrix} 3 & 4 \\ 1 & 6 \end{pmatrix}$$

The characteristic polynomial is

$$\begin{vmatrix} 3 - \lambda & 4 \\ 1 & 6 - \lambda \end{vmatrix} = (3 - \lambda)(6 - \lambda) - 4 \cdot 1 = \lambda^2 - 9\lambda + 14.$$

The characteristic equation is then

$$\lambda^2 - 9\lambda + 14 = 0$$

with solutions

$$\lambda = \frac{-(-9) \pm \sqrt{(-9)^2 - 4 \cdot 1 \cdot 14}}{2 \cdot 1} = \begin{cases} 7 \\ 2 \end{cases}$$

So in addition to the eigenvalue 7 that we saw earlier, we also have that 2 is an eigenvalue.

When the eigenvalues are found, we can use them to compute the eigenvectors by solving the homogeneous system of linear equations $(A - \lambda I_{n+1})\vec{x} = \vec{0}$. Since there are infinitely many solutions to this, we will always get at least one free parameter in the solution.

Example 9.1.6. Let us look at the two eigenvalues $\lambda = 2$ and $\lambda = 7$ separately.

For $\lambda = 7$ we get that $A - \lambda I_2 = A - 7I_2$ is the matrix

$$A - 7I_2 = \begin{pmatrix} 3 - 7 & 4 \\ 1 & 6 - 7 \end{pmatrix} = \begin{pmatrix} -4 & 4 \\ 1 & -1 \end{pmatrix}$$

Gauss-Jordan elimination yields

$$\begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix}$$

Since the second row is zero, we only get the equation $x_1 - x_2 = 0$. Therefore x_2 is a free parameter and $x_1 = x_2$. The vector form of the solution is

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_2 \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Thus any value of x_2 gives us an eigenvector. $x_2 = 2$ gives us the eigenvector $[2, 2]^T$, which we saw in the beginning.

Turning to the other choice, $\lambda = 7$, Gauss-Jordan elimination of the matrix

$$A - 2I_2 = \begin{pmatrix} 3 - 2 & 4 \\ 1 & 6 - 2 \end{pmatrix} = \begin{pmatrix} 1 & 4 \\ 1 & 4 \end{pmatrix}$$

yields

$$\begin{pmatrix} 1 & 4 \\ 0 & 0 \end{pmatrix}$$

The only equation we are left with is $x_1 + 4x_2 = 0$, x_2 is a free parameter, and the general solution is

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_2 \begin{pmatrix} -4 \\ 1 \end{pmatrix}$$

Again, any choice of x_2 gives an eigenvector.

We ought to exclude the choice $x_2 = 0$ in this example, since $\vec{0}$ is explicitly ruled out as an eigenvector.

Note that there aren't any other eigenvectors, the list we have seen is complete.

Example 9.1.7. In some sense, the ordinary situation is to have one free parameter in the solution set for each eigenvalue, and to have as many eigenvalues as the degree of the characteristic polynomial (that is n if the matrix has size $n \times n$). Let us look at three examples where something different happen. We will consider the three matrices

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \quad C = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$$

We can compute that A and B have the same characteristic polynomial, namely $(1 - \lambda)^2$, so the characteristic equation has only one solution: $\lambda = 1$. For A ,

$$A - 1 \cdot I_2 = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

There are two free parameters, x_1 and x_2 , and all vectors in the plane are eigenvectors for this matrix.

Turning to B , we get

$$B - 1 \cdot I_2 = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$$

Now x_1 is a free parameter, $x_2 = 0$, so the general solution is

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Thus B has only one line of eigenvectors.

For C , the characteristic equation is $\lambda^2 + 1 = 0$. There are no (real) solutions, so C has no (real) eigenvalues and no (real) eigenvectors. Geometrically, C corresponds to a rotation, which never rotates a vector into a parallel vector!

9.2 Diagonalization

If there are “enough” eigenvectors, we can choose a basis for our space consisting only of eigenvectors. This basis is then optimal for multiplication with the matrix we study, since it only acts by multiplying each coordinate by the corresponding eigenvalue. It is a *diagonal matrix*.

To get started, we state the following theorem:

Theorem 9.2.1. *Let A be a square matrix, and $\vec{x}_1, \vec{x}_2$ up to $\vec{x}_k$ eigenvectors with different eigenvalues $\lambda_1, \dots, \lambda_k$. Then the eigenvectors are linearly independent.*

So eigenvectors with different eigenvalues are linearly independent. There are two possibilities for how a matrix can fail to have enough eigenvectors. First, there could be *repeated* eigenvalues, where the set of eigenvectors has lower dimension than the multiplicity. This was the case for the matrix B in Example 9.1.7. Geometrically, this corresponds to some sort of shearing (or rather scaling composed with shearing). Second, there could be too few real solutions, like for the matrix C in Example 9.1.7. Geometrically, this corresponds to some sort of rotation (or maybe scaling composed with rotation). There are also combinations of these two cases, but then the geometry is less clear, and this never happens in dimensions less than 4.

We will now look at the best case, where we actually get as many eigenvectors as we want. First, we will look at an example, and then state the general procedure.

Example 9.2.2. In Example 9.1.6, we saw that the matrix

$$A = \begin{pmatrix} 3 & 4 \\ 1 & 6 \end{pmatrix}$$

has eigenvectors

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_2 \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

and

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x_2 \begin{pmatrix} -4 \\ 1 \end{pmatrix}$$

The eigenvalues are 7 and 2, respectively. We can choose a basis, consisting of one eigenvector for each choice of free parameter, as

$$\mathcal{B} = \left\{ \vec{u}_1 \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \vec{u}_2 \begin{pmatrix} -4 \\ 1 \end{pmatrix} \right\}$$

By definition, the matrix with these vectors as columns is the coordinate change matrix from coordinates in the basis $\mathcal{B}$ to coordinates in the standard basis $\mathcal{S}$:

$$M_{\mathcal{S} \leftarrow \mathcal{B}} = \begin{pmatrix} 1 & -4 \\ 1 & 1 \end{pmatrix}$$

Therefore (check the answer!)

$$M_{\mathcal{B} \leftarrow \mathcal{S}} = M_{\mathcal{S} \leftarrow \mathcal{B}}^{-1} = \frac{1}{5} \begin{pmatrix} 1 & 4 \\ -1 & 1 \end{pmatrix}$$

Now since $[\vec{u}_1]_{\mathcal{B}} = [1, 0]^T$ and $[\vec{u}_2]_{\mathcal{B}} = [0, 1]^T$, the matrix A should act as the diagonal matrix with 7 and 2 on the diagonal. But A is expressed in terms of the standard basis, so to get the action of A expressed in $\mathcal{B}$, we need to change coordinates from $\mathcal{B}$ to $\mathcal{S}$, then apply A , and change back to $\mathcal{B}$ again. The result is that

$$M_{\mathcal{B} \leftarrow \mathcal{S}} A M_{\mathcal{S} \leftarrow \mathcal{B}}$$

is the matrix using $\mathcal{B}$ that acts in the same way as the matrix A . Computing the matrix product indeed gives

$$M_{\mathcal{B} \leftarrow \mathcal{S}} A M_{\mathcal{S} \leftarrow \mathcal{B}} = \frac{1}{5} \begin{pmatrix} 1 & 4 \\ -1 & 1 \end{pmatrix} \begin{pmatrix} 3 & 4 \\ 1 & 6 \end{pmatrix} \begin{pmatrix} 1 & -4 \\ 1 & 1 \end{pmatrix} = \begin{pmatrix} 7 & 0 \\ 0 & 2 \end{pmatrix}$$

Definition 9.2.3. A *diagonalization* of a square matrix $A_{n \times n}$ with enough eigenvectors, can be computed as follows: Find a basis consisting of eigenvectors for A :

$$\mathcal{B} = \{\vec{u}_1, \vec{u}_2, \dots, \vec{u}_n\}$$

Then the coordinate change matrix

$$M_{\mathcal{S} \leftarrow \mathcal{B}}$$

is the matrix with columns given by the $\vec{u}_i$. The diagonalization of A is the matrix D such that

$$D = M_{\mathcal{B} \leftarrow \mathcal{S}} A M_{\mathcal{S} \leftarrow \mathcal{B}} = M_{\mathcal{S} \leftarrow \mathcal{B}}^{-1} A M_{\mathcal{S} \leftarrow \mathcal{B}}$$

It is the diagonal matrix with the eigenvalues of A along the diagonal (in the same order as the eigenvectors $\vec{u}_i$).

The process of diagonalization includes both finding the eigenvalues, and finding the basis of eigenvectors. Equivalently, we need both the matrices $M_{\mathcal{S} \leftarrow \mathcal{B}}$ and D . We have already seen one complete computation of the diagonalization, but since the procedure was spread out over several examples, let us take a look at one more example. Now we perform everything we need to do in the correct order.

Example 9.2.4. Diagonalize the matrix

$$A = \begin{pmatrix} 3 & 2 \\ 3 & 4 \end{pmatrix}$$

First, we compute the characteristic polynomial:

$$\begin{vmatrix} 3 - \lambda & 2 \\ 3 & 4 - \lambda \end{vmatrix} = (3 - \lambda)(4 - \lambda) - 3 \cdot 2 = \lambda^2 - 7\lambda + 6$$

Then we solve the characteristic equation $\lambda^2 - 7\lambda + 6 = 0$ to find the eigenvalues:

$$\lambda = \frac{-(-7) \pm \sqrt{(-7)^2 - 4 \cdot 1 \cdot 6}}{2 \cdot 1} = \begin{cases} 6 \\ 1 \end{cases}$$

For each solution, we compute the eigenvectors. We start with $\lambda = 6$. Then we need to perform Gauss-Jordan elimination on the matrix

$$\begin{pmatrix} 3 - 6 & 2 \\ 3 & 4 - 6 \end{pmatrix} = \begin{pmatrix} -3 & 2 \\ 3 & -2 \end{pmatrix}$$

The result is

$$\begin{pmatrix} 1 & -\frac{2}{3} \\ 0 & 0 \end{pmatrix}$$

The solutions are, on vector form,

$$\begin{pmatrix} x \\ y \end{pmatrix} = y \begin{pmatrix} \frac{2}{3} \\ 1 \end{pmatrix}$$

Setting $y = 1$ yields the eigenvector $[\frac{2}{3}, 1]^T$.

Moving on to $\lambda = 1$, we perform Gauss-Jordan elimination on

$$\begin{pmatrix} 3 - 1 & 2 \\ 3 & 4 - 1 \end{pmatrix} = \begin{pmatrix} 2 & 2 \\ 3 & 3 \end{pmatrix}$$

The result is

$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$$

The solutions are, on vector form,

$$\begin{pmatrix} x \\ y \end{pmatrix} = y \begin{pmatrix} -1 \\ 1 \end{pmatrix}$$

Setting $y = 1$ yields the eigenvector $[-1, 1]^T$. We use the basis $\mathcal{B}$ consisting of the two eigenvectors we have chosen. Then the two matrices are

$$M_{\mathcal{S} \leftarrow \mathcal{B}} = \begin{pmatrix} \frac{2}{3} & -1 \\ 1 & 1 \end{pmatrix} \quad D = \begin{pmatrix} 6 & 0 \\ 0 & 1 \end{pmatrix}$$

Example 9.2.5. Here is an example showing what might happen if the matrix has repeated eigenvalues. Most of the computations are left to the reader (or the reader's computer).

Consider

$$M = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -1 & 3 \\ 0 & 1 & 1 \end{pmatrix}$$

By computation, the characteristic equation is $-(\lambda - 2)^2(\lambda + 2) = 0$, so there are only two solutions, $\lambda = 2$ and $\lambda = -2$. Looking first at $\lambda = -2$, we get that

$$M - (-2)I_3 = \begin{pmatrix} 4 & 1 & -1 \\ 0 & 1 & 3 \\ 0 & 1 & 3 \end{pmatrix}$$

Gauss-Jordan elimination yields

$$\begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & 3 \\ 0 & 0 & 0 \end{pmatrix}$$

We see that x_3 is the free parameter, and we get the general solution

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = x_3 \begin{pmatrix} 1 \\ -3 \\ 1 \end{pmatrix}$$

The other eigenvalue, $\lambda = 2$, gives us

$$M - 2I_3 = \begin{pmatrix} 0 & 1 & -1 \\ 0 & -3 & 3 \\ 0 & 1 & -1 \end{pmatrix}$$

Gauss-Jordan elimination produces

$$\begin{pmatrix} 0 & 1 & -1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

There are two free parameters, x_1 and x_3 . The general solution is

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = x_1 \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} + x_3 \begin{pmatrix} 0 \\ 1 \\ 1 \end{pmatrix}$$

Thus we see that we can choose our basis, and hence our coordinate change matrix, as

$$M_{S \leftarrow \mathcal{B}} = \begin{pmatrix} 1 & 1 & 0 \\ -3 & 0 & 1 \\ 1 & 0 & 1 \end{pmatrix}$$

Finally, the diagonalization of M is then

$$D = M_{S \leftarrow \mathcal{B}}^{-1} M M_{S \leftarrow \mathcal{B}} = \begin{pmatrix} -2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}$$

There is one case where the diagonalization is guaranteed to exist, from a simple criterion.

Theorem 9.2.6. *Assume A is a symmetric $n \times n$ -matrix, $A^T = A$.*

Then A has n linearly independent eigenvectors (and all the eigenvalues are real). Furthermore, there exists choices for n eigenvectors $\vec{x}_1, \dots, \vec{x}_n$, such that

- $\vec{x}_i \bullet \vec{x}_i = 1$ for all i .
- $\vec{x}_i \bullet \vec{x}_j = 0$ for $i \neq j$.

Note that $\bullet$ signifies scalar product, so the two conditions say that all the vectors have length one, and different vectors are orthogonal to each other.

By choosing the vectors as above, the inverse of the coordinate change matrix is equal to the transpose of the coordinate change matrix (which is much easier to compute).

We will skip the proof, but give an example.

Example 9.2.7. Consider the matrix

$$A = \begin{pmatrix} 1 & 3 \\ 3 & 5 \end{pmatrix}$$

The two appearances of 3 shows that the matrix is symmetric. Solving the characteristic equation gives us the two eigenvalues

$$\lambda = 3 \pm \sqrt{13}$$

Gauss-Jordan-elimination of the two matrices

$$\begin{pmatrix} 1 - (3 \pm \sqrt{13}) & 3 \\ 3 & 5 - (3 \pm \sqrt{13}) \end{pmatrix}$$

results in (the choice $\lambda = 3 + \sqrt{13}$ being on the left)

$$\begin{pmatrix} 1 & -\frac{3}{2+\sqrt{13}} \\ 0 & 0 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} 1 & \frac{3}{\sqrt{13}-2} \\ 0 & 0 \end{pmatrix}$$

The vector forms of the corresponding eigenvectors is

$$\begin{pmatrix} x \\ y \end{pmatrix} = y \begin{pmatrix} \frac{3}{2+\sqrt{13}} \\ 1 \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} x \\ y \end{pmatrix} = y \begin{pmatrix} \frac{3}{2-\sqrt{13}} \\ 1 \end{pmatrix}$$

The first part of the claim was that the eigenvectors for different eigenvalues are orthogonal. We can check this by computing the scalar product:

$$y \begin{pmatrix} \frac{3}{2+\sqrt{13}} \\ 1 \end{pmatrix} \bullet y \begin{pmatrix} \frac{3}{2-\sqrt{13}} \\ 1 \end{pmatrix} = y^2 \left(\frac{3 \cdot 3}{(2\sqrt{13})(2 - \sqrt{13})} + 1 \cdot 1 \right) = y^2 \left(\frac{9}{9} - 1 \right) = 0$$

The second part of the theorem was that we can choose eigenvectors of unit length. Since the numbers now are so complicated that we get iterated square roots, we switch to decimal representation:

$$\left| y \begin{pmatrix} \frac{3}{2+\sqrt{13}} \\ 1 \end{pmatrix} \right| = y \sqrt{\left(\frac{3}{2+\sqrt{13}} \right)^2 + 1^2} \approx y \cdot 1.13$$

Choosing $y = 1/1.13 \approx 0.88$ then yields an eigenvector of unit length. Doing the same for the other eigenvalue, we finally come to the coordinate change matrix:

$$M_{S \leftarrow B} \begin{pmatrix} 0.4719 & -0.8817 \\ 0.8817 & 0.4719 \end{pmatrix}$$

The reader is urged to check that

$$M_{B \leftarrow S} = M_{S \leftarrow B}^{-1} = \begin{pmatrix} 0.4719 & -0.8817 \\ 0.8817 & 0.4719 \end{pmatrix}^T = \begin{pmatrix} 0.4719 & 0.8817 \\ -0.8817 & 0.4719 \end{pmatrix}$$

9.3 The power method

The *power method* is a simple procedure that often finds a single eigenvector. The basic idea is that if we start with a random vector $\vec{x}$, then $A\vec{x}$ might be closer to an eigenvector than $\vec{x}$ is. There are situations where the procedure fails.

To see the rationale behind the power method, assume first that we are in $\mathbb{R}^2$, and that some matrix A has the two eigenvalues $\lambda = 1$ and $\lambda = 0.5$. We assume also that the corresponding eigenvectors $\vec{v}$ and $\vec{u}$ are not known. Start with some randomly chosen vector $\vec{x} = a\vec{v} + b\vec{u}$. Then, with $\mathcal{B}$ the basis of eigenvectors,

$$[\vec{x}]_{\mathcal{B}} = \begin{pmatrix} a \\ b \end{pmatrix}$$

Since the basis consist of eigenvectors, we get that

$$A\vec{x} = A(a\vec{v} + b\vec{u}) = 1 \cdot a\vec{v} + 0.5 \cdot b\vec{u}.$$

In the basis $\mathcal{B}$,

$$[A\vec{x}]_{\mathcal{B}} = \begin{pmatrix} a \\ 0.5b \end{pmatrix}$$

We can now apply A again to get

$$[A^2\vec{x}]_{\mathcal{B}} = \begin{pmatrix} a \\ (0.5)^2b \end{pmatrix}$$

For any power of A ,

$$[A^n\vec{x}]_{\mathcal{B}} = \begin{pmatrix} a \\ (0.5)^nb \end{pmatrix}$$

The second coordinate becomes smaller and smaller, and the sequence of vectors $\{A^n\vec{x}\}$ comes closer and closer to $a\vec{v} + 0\vec{u} = a\vec{v}$, which is an eigenvector!

This pattern is not limited to $\mathbb{R}^2$, but can be seen in any dimension. The efficiency of the method is related to the size of the eigenvalues, which we will seldom know before we start. Also, to compare $\vec{x}$ and $A\vec{x}$, we should normalize at each step (that is, replace $\vec{x}$ with the normalized unit length vector $\vec{x}/|\vec{x}|$). This effect is illustrated in Figure 9.1 and Figure 9.2, where the same matrix is applied to the same starting vector, but only normalized in one. It is clear from both pictures that the direction of eigenvectors has been found, but by normalizing it is much easier to control the result numerically.

Here is some pseudo-code for the general power method:

Figure 9.1: Power method without normalization

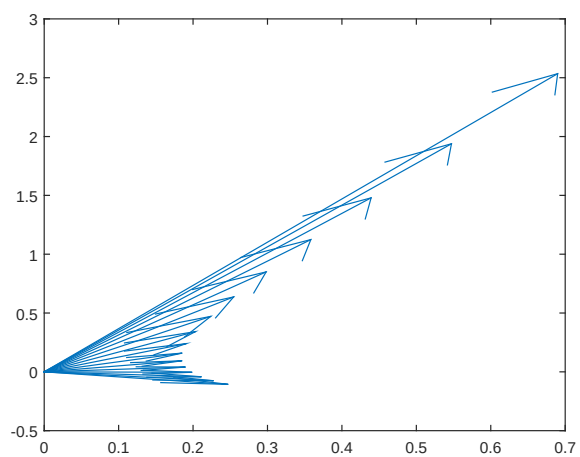
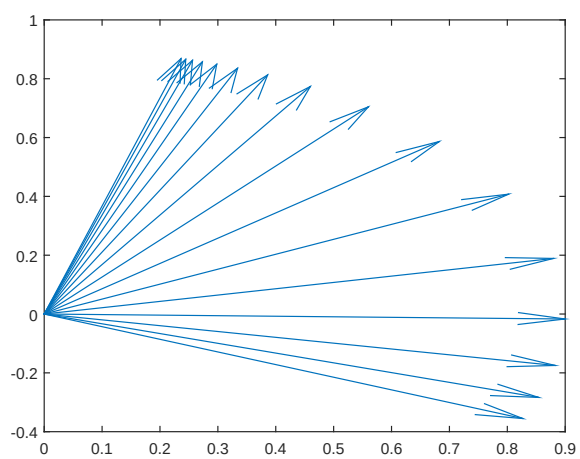


Figure 9.2: Power method with normalization




```

A = <some square matrix>
x = <random vector>
x = normalize(x)
Ax = normalize(A*x)
while (length(Ax-x) > errorTolerance)
    x = Ax
    Ax = normalize(A*x)
end while

```

When the loop terminates, $A\vec{x} \approx \lambda\vec{x}$, so we have found an approximate eigenvector.

Example 9.3.1. We consider the matrix

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 1 & -1 \\ 2 & 0 & 2 & 4 & 6 & 1 \\ 1 & 4 & 0 & -1 & 7 & 1 \\ 5 & 5 & 5 & 7 & 20 & 1 \\ 5 & 4 & 5 & 7 & 1 & 0 \\ 10 & 4 & 5 & 1 & 5 & 4 \end{pmatrix}$$

Applying the power method to some random starting vector, we get the sequence

$$\begin{pmatrix} 0.0225 \\ 0.2685 \\ 0.1164 \\ 0.6443 \\ 0.4974 \\ 0.5012 \end{pmatrix} \begin{pmatrix} 0.0181 \\ 0.3009 \\ 0.2105 \\ 0.8066 \\ 0.3216 \\ 0.3330 \end{pmatrix} \begin{pmatrix} 0.0257 \\ 0.3055 \\ 0.1541 \\ 0.7738 \\ 0.4272 \\ 0.3177 \end{pmatrix} \begin{pmatrix} 0.0272 \\ 0.3025 \\ 0.1806 \\ 0.7976 \\ 0.3803 \\ 0.3070 \end{pmatrix} \begin{pmatrix} 0.0273 \\ 0.3040 \\ 0.1673 \\ 0.7876 \\ 0.4031 \\ 0.3098 \end{pmatrix}$$

$$\begin{pmatrix} 0.0273 \\ 0.3033 \\ 0.1736 \\ 0.7926 \\ 0.3923 \\ 0.3082 \end{pmatrix} \begin{pmatrix} 0.0273 \\ 0.3037 \\ 0.1706 \\ 0.7903 \\ 0.3974 \\ 0.3089 \end{pmatrix} \begin{pmatrix} 0.0273 \\ 0.3035 \\ 0.1720 \\ 0.7914 \\ 0.3950 \\ 0.3086 \end{pmatrix} \begin{pmatrix} 0.0273 \\ 0.3036 \\ 0.1714 \\ 0.7909 \\ 0.3961 \\ 0.3087 \end{pmatrix} \begin{pmatrix} 0.0273 \\ 0.3036 \\ 0.1717 \\ 0.7911 \\ 0.3956 \\ 0.3086 \end{pmatrix}$$

If we multiply the last vector with A again, we get

$$\begin{pmatrix} 0.5622 \\ 6.2447 \\ 3.5282 \\ 16.2711 \\ 8.1428 \\ 6.3496 \end{pmatrix}$$

To get an approximation of the eigenvalue, we divide the coordinates, or compute the length of this vector. From the first coordinate we get $\lambda \approx 20.59$, from the second $\lambda \approx 20.57$. By computing the length, we get $\lambda \approx 20.57$.

Example 9.3.2. The power method is not guaranteed to work. It if doesn't, the loop won't terminate, so we should have an extra stopping condition in our code.

9.4 Extended example: PageRank

One of the algorithms used by Google to rank web pages, is called *PageRank*, and is named after Larry Page, one of the founders of Google. We will now formulate this algorithm, and see how we can apply the power method in this context.

We want to attach a number $PR(X)$ to any web page X , in such a way that a more popular web page has a higher value than a less popular one. The basic equation that can be used to find $PR(X)$ is

$$PR(X) = \sum_{Y \rightarrow X} \frac{PR(Y)}{\text{links}(Y)}$$

The sum extends over all Y with a link to X . The denominator $\text{links}(Y)$ is the number of links from Y (to any other web page). In Figure 9.3, there are six web pages, and we look at the six equations

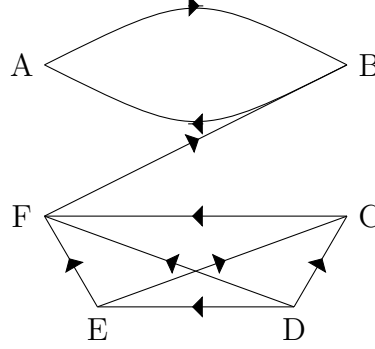
$$\begin{aligned} PR(A) &= \frac{PR(B)}{1} \\ PR(B) &= \frac{PR(A)}{1} + \frac{PR(F)}{1} \\ PR(C) &= +\frac{PR(D)}{3} + \frac{PR(E)}{2} \\ PR(D) &= ? \\ PR(E) &= +\frac{PR(D)}{3} \\ PR(F) &= \frac{PR(C)}{1} + \frac{PR(D)}{3} + \frac{PR(E)}{2} \end{aligned}$$

Note that there is nothing linking to page D , so it is not quite clear what the right hand side of its equation should be. For now, we use zeroes. The matrix form of this equation is

$$\begin{pmatrix} PR(A) \\ PR(B) \\ PR(C) \\ PR(D) \\ PR(E) \\ PR(F) \end{pmatrix} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & \frac{1}{3} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{3} & \frac{1}{2} & 0 \end{pmatrix} \begin{pmatrix} PR(A) \\ PR(B) \\ PR(C) \\ PR(D) \\ PR(E) \\ PR(F) \end{pmatrix} \quad (9.1)$$

It turns out that we should modify these equations a bit, but let us first consider what the strength of this idea is.

Figure 9.3: A small internet



If you want to be ranked as high as possible, you need many links pointing to you, and especially links from popular sites. It is possible to create a lot of pages referring to your homepage, but it is difficult to get popular pages to refer to you. In Figure 9.3, it is obvious that D should be ranked lower than the others, but it is not quite clear if A should be more popular than C in this model.

We now change the setup slightly. Our model so far says something about what happens when someone browses the internet, randomly choosing a link on each page. But it is also possible to arrive at a web page in other ways than following links, for instance by editing the url directly, using a bookmark, or starting a browser from a link that is not on the internet (a local pdf-file, for instance). If we assume that there is an 85% chance that an ordinary link is used, and a 15% chance that a completely random web page is loaded by some other means, we should change our model by multiplying the right hand side of the previous equations by 0.85, and adding 0.15 evenly spread out over all web pages.

This modifies our matrix from (9.1) to

$$\begin{aligned}
 M &= 0.85 \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & \frac{1}{3} & \frac{1}{2} & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{3} & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{3} & \frac{1}{2} & 0 \end{pmatrix} + \frac{0.15}{6} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix} \\
 &= \begin{pmatrix} 0.0250 & 0.8750 & 0.0250 & 0.0250 & 0.0250 & 0.0250 \\ 0.8750 & 0.0250 & 0.0250 & 0.0250 & 0.0250 & 0.8750 \\ 0.0250 & 0.0250 & 0.0250 & 0.3083 & 0.4500 & 0.0250 \\ 0.0250 & 0.0250 & 0.0250 & 0.0250 & 0.0250 & 0.0250 \\ 0.0250 & 0.0250 & 0.0250 & 0.3083 & 0.0250 & 0.0250 \\ 0.0250 & 0.0250 & 0.8750 & 0.3083 & 0.4500 & 0.0250 \end{pmatrix}
 \end{aligned}$$

We are looking for an eigenvector for this matrix. Using the power method, we end up with the vector (where we have divided by the sum of the terms, so that the entries add up to one):

$$\begin{pmatrix} 0.3869 \\ 0.4257 \\ 0.0457 \\ 0.0250 \\ 0.0321 \\ 0.0846 \end{pmatrix}$$

Therefore, the ranked list of web pages is B,A,F,C,E,D. The exact figures tell us the probability that a random web user visits a given web page at a given time. For instance, there is an 8.46% probability that our random user visits web page F. Note in particular that A is a very popular page in this model, even though there is only one link to it.

9.5 Exercises

Exercise 9.5.1. Find the eigenvectors and the eigenvalues for each matrix.

(a). $\begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$

(b). $\begin{pmatrix} 3 & 2 \\ 0 & 3 \end{pmatrix}$

(c). $\begin{pmatrix} 1 & 1 \\ 2 & 4 \end{pmatrix}$

(d). $\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$

Chapter 10

Homogeneous coordinates

Modern GPUs (graphic processing units, also informally known as graphics cards) are streamlined for operations on vectors of length 4, not 3 as you might expect. The reason is that the fourth coordinate allows us to deal with translation (that is, vector addition), in the same way we deal with matrix multiplication.

10.1 Definition of homogeneous coordinates

We know that the linear subspaces of $\mathbb{R}^2$ are the lines containing the origin (in addition to the trivial subspaces $\{\vec{0}\}$ and $\mathbb{R}^2$ itself). We can describe the set of such lines by a single invariant, namely its slope. If the equation of the line is

$$ax + by = 0,$$

the slope is $-a/b$. Perhaps you are more used to seeing the equation for the line “solved for y ”:

$$y = -\frac{a}{b}x$$

There is one line that we are missing, however. The y -axis itself has equation $x = 0$, so if we try to compute the slope, we end up with division by zero. To include this line too, we consider a slight change in our point of view. Starting from $ax + by = 0$, we note that any line apart from the y -axis is represented by such an equation with $b \neq 0$. The y -axis is represented by this equation when $b = 0$. Also, multiplying the equation by any non-zero scalar λ gives us the same line. The only illegal choice for coefficients $(a, b) \in \mathbb{R}^2$ is if both $a = 0$ and $b = 0$. The equation $0 = 0$ does not represent a line.

Definition 10.1.1. Consider the set

$$\{(a, b) \in \mathbb{R}^2 \mid (a, b) \neq (0, 0)\}.$$

Define a relation $\sim$ on this set by

$$(a, b) \sim (\lambda a, \lambda b) \text{ for all } \lambda \in \mathbb{R}, \lambda \neq 0$$

One-dimensional *homogeneous coordinates* are coordinates (a, b) where two representations that are $\sim$ -related are considered equal.

To see why this is one-dimensional, look first at the case $b \neq 0$. Then we can choose $\lambda = 1/b$, and see that

$$(a, b) \sim \lambda(a, b) = \frac{1}{b}(a, b) = \left(\frac{a}{b}, 1\right).$$

So any such point corresponds to an “ordinary” point $x \in \mathbb{R}^1$ by being equivalent to homogeneous coordinates on the form $(x, 1)$. Consider then the case $b = 0$. Then, since $(a, b) \neq (0, 0)$, we must have $a \neq 0$. Choose $\lambda = 1/a$. Then

$$(a, b) = (a, 0) \sim \lambda(1, 0) = \frac{1}{a}(a, 0) = (1, 0).$$

This is just a single point, which we can think about as the point at infinity (note that in this model, we don’t separate between ∞ and $-\infty$).

Definition 10.1.2. The union of the line $\mathbb{R}$ and a point at infinity, given precisely by homogeneous coordinates (a, b) , is called the *projective line*.

This is well-defined because the relation $\sim$ is an equivalence relation. In fact, the projective line contains exactly one point for each *equivalence class*, that is one point for each subset of $\mathbb{R}^2 \setminus (0, 0)$ consisting of pairs that are equivalent to each other. To get a reminder about the notions about equivalence relations, let us prove:

Theorem 10.1.3. *The relation*

$$(a, b) \sim (\lambda a, \lambda b) \text{ for all } \lambda \in \mathbb{R}, \lambda \neq 0$$

is an equivalence relation on $\mathbb{R}^2 \setminus (0, 0)$.

Proof. Recall that we need to check three conditions: reflexivity, symmetry and transitivity. For reflexivity, we need to show that $(a, b) \sim (a, b)$. For this we can choose $\lambda = 1$, so that

$$(a, b) \sim 1 \cdot (a, b).$$

For symmetry, assume $(a, b) \sim (c, d)$. We need to prove that $(c, d) \sim (a, b)$. By definition, $(a, b) \sim (c, d)$ means that

$$(a, b) \sim \lambda(a, b) = (c, d)$$

for some $\lambda \neq 0$. But then we can choose $\mu = 1/\lambda$, and get that

$$(c, d) \sim \mu(c, d) = \frac{1}{\lambda}(\lambda a, \lambda b) = (a, b)$$

as required. Finally, if both $(a, b) \sim (c, d)$ and $(c, d) \sim (e, f)$, we need to show that $(a, b) \sim (e, f)$. By definition, there are constants λ and μ such that

$$(a, b) \sim \lambda(a, b) = (c, d), \quad (c, d) \sim \mu(c, d) = (e, f).$$

Then, choosing $\nu = \mu\lambda$, we get

$$(a, b) \sim \nu(a, b) = \mu\lambda(c, d) = \mu(c, d) = (e, f)$$

Since the relation satisfies these three conditions, it is an equivalence relation. $\square$

We will use matrix multiplication to get new points in homogeneous coordinates from old ones. Then it is more convenient to write (a, b) as a column vector $\begin{pmatrix} a \\ b \end{pmatrix}$. We can achieve three forms of transformations of the line using homogeneous coordinates and matrix multiplication. Let us look at each in turn.

Example 10.1.4. One transformation comes from an ordinary operation on the line, namely *scaling*. To scale with a factor s , consider the matrix product

$$\begin{pmatrix} s & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} sa \\ b \end{pmatrix}$$

When $b \neq 0$, we can divide by it, and write $x = a/b$:

$$\begin{pmatrix} s & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ 1 \end{pmatrix} \sim \begin{pmatrix} s & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} sa \\ b \end{pmatrix} \sim \begin{pmatrix} sx \\ 1 \end{pmatrix}$$

So for ordinary points, scaling acts as usual. If $b = 0$, however, both $(a, b) \sim (1, 0)$ and $(sa, b) \sim (1, 0)$, so the point at infinity is not affected by scaling.

Example 10.1.5. An interesting feature of homogeneous coordinates is that we can formulate translation in terms of matrix multiplication. To translate by t (which means adding the number t for real numbers), consider the matrix product

$$\begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} a + tb \\ b \end{pmatrix}$$

If $b \neq 0$, we can divide by b and get (with $x = a/b$)

$$\begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ 1 \end{pmatrix} \sim \begin{pmatrix} 1 & t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} a + tb \\ b \end{pmatrix} \sim \begin{pmatrix} x + t \\ 1 \end{pmatrix}$$

So for ordinary points, t is added, and we have the ordinary notion of translation. If $b = 0$, both $(a, b) \sim (1, 0)$ and $(a + tb, b) \sim (0, 0)$, so the point at infinity is not moved by translation.

Example 10.1.6. The third, and final, example of a transformation is projection on a point p . This is a linear operation only if $p = 0$, but we don't assume this now. Consider the matrix product

$$\begin{pmatrix} 0 & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} pb \\ b \end{pmatrix}$$

As before, if $b \neq 0$ and $x = a/b$,

$$\begin{pmatrix} 0 & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ 1 \end{pmatrix} \begin{pmatrix} 0 & p \\ 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} pb \\ b \end{pmatrix} \sim \begin{pmatrix} p \\ 1 \end{pmatrix}$$

So all ordinary points are sent to the single point p . If $b = 0$, $(pb, b) = (0, 0)$. The origin in $\mathbb{R}^2$ was excluded when we defined homogeneous coordinates, and therefore this computation doesn't make sense. We say that the projection map is *undefined* at the point of infinity.

Homogeneous coordinates are not restricted to the one-dimensional case. We will now see the general definition, and then look more closely at the two- and three-dimensional cases later.

Definition 10.1.7. Consider the set

$$\{(a_0, a_1, \dots, a_n) \in \mathbb{R}^{n+1} \setminus (0, 0, \dots, 0)\}.$$

Define a relation $\sim$ on this set by

$$(a_0, a_1, \dots, a_n) \sim (\lambda a_0, \lambda a_1, \dots, \lambda a_n) \text{ for all } \lambda \in \mathbb{R}, \lambda \neq 0$$

n -dimensional *homogeneous coordinates* are coordinates $(a_0, a_1, \dots, a_n)$ where two representations that are $\sim$ -related are considered equal. The set of equivalence classes is called the *real projective n -space*.

As in the one-dimensional case, the relation is an equivalence relation. Also, there is now a projective space of dimension one less at infinity, not only a point. For instance, we can talk about the “line at infinity” for the projective plane.

10.2 Homogeneous coordinates in the plane

Homogeneous coordinates in the plane have three coordinates: (a, b, c) . As long as $c \neq 0$, we can divide by it, and get “ordinary” plane coordinates

$$(a, b, c) \sim \frac{1}{c}(a, b, c) = \left(\frac{a}{c}, \frac{b}{c}, 1\right) = (x, y, 1)$$

If $c = 0$, we can think of $(a, b, 0)$ as giving one-dimensional homogeneous coordinates for the line at infinity. It is also convenient to think about $(a, b, 0)$ in a different way, as representing a line through the origin in the plane. Then $(x, y, 1)$ represents the ordinary point (x, y) in the plane. And $(x, y, 0)$ the line through the two points $(0, 0)$ and (x, y) , or perhaps better the direction between the two points.

10.2.1 Rotation and translation

As for the line, there are several aspects of matrix multiplication for homogeneous coordinates in the plane. There is added complexity, coming from the fact that there is more than one point at infinity, so we need to keep track of what happens with these points or directions as well. We will see the interesting phenomena by considering various examples of behaviour.

Example 10.2.1. As our first example, we will see how to extend any usual linear transformation to the homogeneous coordinates. So let A be any 2×2 -matrix, and consider the product

$$\begin{pmatrix} a_{00} & a_{01} & 0 \\ a_{10} & a_{11} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} a_{00}a + a_{01}b \\ a_{10}a + a_{11}b \\ c \end{pmatrix}$$

If $c \neq 0$, we can introduce $x = a/c$, $y = b/c$, and see that

$$\begin{pmatrix} a_{00} & a_{01} & 0 \\ a_{10} & a_{11} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \sim \begin{pmatrix} a_{00} & a_{01} & 0 \\ a_{10} & a_{11} & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} a_{00}a + a_{01}b \\ a_{10}a + a_{11}b \\ c \end{pmatrix} \sim \begin{pmatrix} a_{00}x + a_{01}y \\ a_{10}x + a_{11}y \\ 1 \end{pmatrix}$$

Therefore, this product acts in the same way as the matrix A on ordinary points. If $c = 0$, the third coordinate is also zero after the transformation. Therefore, the points at infinity “remain” at infinity. However, directions are changed in the same way as the ordinary points. The point at infinity corresponding to the line through $(0, 0)$ and (x, y) is transformed into the point corresponding to the line through $(0, 0)$ and $(a_{00}x + a_{01}y, a_{10}x + a_{11}y)$.

Note in particular that any rotation about the origin is extended in this way, and lines through the origin are rotated in the same way as points.

Example 10.2.2. Translation in the plane is adding a vector $\vec{t} = [t_0, t_1]^T$. Consider the matrix product

$$\begin{pmatrix} 1 & 0 & t_0 \\ 0 & 1 & t_1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} a + t_0c \\ b + t_1c \\ c \end{pmatrix}$$

If $c \neq 0$, we can divide by it to get $x = a/c$, $y = b/c$ as before. Then

$$\begin{pmatrix} 1 & 0 & t_0 \\ 0 & 1 & t_1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \sim \begin{pmatrix} 1 & 0 & t_0 \\ 0 & 1 & t_1 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} a + t_0 c \\ b + t_1 c \\ c \end{pmatrix} \sim \begin{pmatrix} x + t_0 \\ y + t_1 \\ 1 \end{pmatrix}$$

Thus this matrix acts as translation on ordinary points. If $c = 0$, the result is just $[a, b, 0]^T$, so the points at infinity are unchanged. We can think about this as saying that the line through $(0, 0)$ and (x, y) is translated. The direction of the line through (t_0, t_1) and $(x + t_0, y + t_1)$ is the same as the direction of the original line, and thus determines the same point at infinity.

One new phenomenon is that rotation and translation can be combined, allowing us to rotate about general points, not only the origin.

Example 10.2.3. We want to rotate the plane an angle $\alpha = 0.3$ (or about 17 degrees) about the point $P = (2, 3)$. Find a matrix representative M for the rotation.

The idea is simple: We translate so that P is transformed into the origin. Rotation about the origin is a linear transformation, so we already know how to do this. Finally, we translate again, so that the origin is transformed back to P .

First move P to the origin:

$$\begin{pmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{pmatrix}$$

Then rotate:

$$\begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Finally, move the origin to P :

$$\begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{pmatrix}$$

Moving any point $[a, b, c]^T$ through these steps correspond to multiplying by each of the matrices in turn. The end result is the same as we get by multiplying the three matrices together first (note the order):

$$M = \begin{pmatrix} 1 & 0 & 2 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 \\ \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{pmatrix}$$

If we multiply the two leftmost matrices first, we get

$$M = \begin{pmatrix} \cos \alpha & -\sin \alpha & 2 \\ \sin \alpha & \cos \alpha & 3 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & -2 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{pmatrix}$$

This always happen when we first apply an ordinary linear transformation, and then translate: The components of the translation appear in the corresponding positions in the rightmost column. The final result is

$$M = \begin{pmatrix} \cos \alpha & -\sin \alpha & -2 \cos(\alpha) + 3 \sin(\alpha) + 2 \\ \sin \alpha & \cos \alpha & -2 \sin(\alpha) - 3 \cos(\alpha) + 3 \\ 0 & 0 & 1 \end{pmatrix}$$

If we find numerical approximations to these trigonometric values, we end up with

$$M = \begin{pmatrix} 0.9553 & -0.2955 & 0.9759 \\ 0.2955 & 0.9553 & -0.4570 \\ 0 & 0 & 1 \end{pmatrix}$$

To get a picture (see Figure 10.1) of how this works, let us use this transformation on the triangle with vertices PQR given by $P = (2, 4)$, $Q = (3, 5)$ and $R = (1, 7)$. We extend to three coordinates by letting the last coordinate be 1, and note that we can multiply the matrix M by the matrix with column vectors given by the coordinates of the vertices. The answer will be the coordinates of the transformed vertices P', Q', R' :

$$MP = \begin{pmatrix} 0.9553 & -0.2955 & 0.9759 \\ 0.2955 & 0.9553 & -0.4570 \\ 0 & 0 & 1.0000 \end{pmatrix} \begin{pmatrix} 2 & 3 & 1 \\ 4 & 5 & 7 \\ 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} 1.7045 & 2.3643 & -0.1374 \\ 3.9553 & 5.2062 & 6.5258 \\ 1 & 1 & 1 \end{pmatrix}$$

Before we leave this example, let us see directly how M transforms the points at infinity. This means that we are considering $(a, b, c) = (a, b, 0)$, so

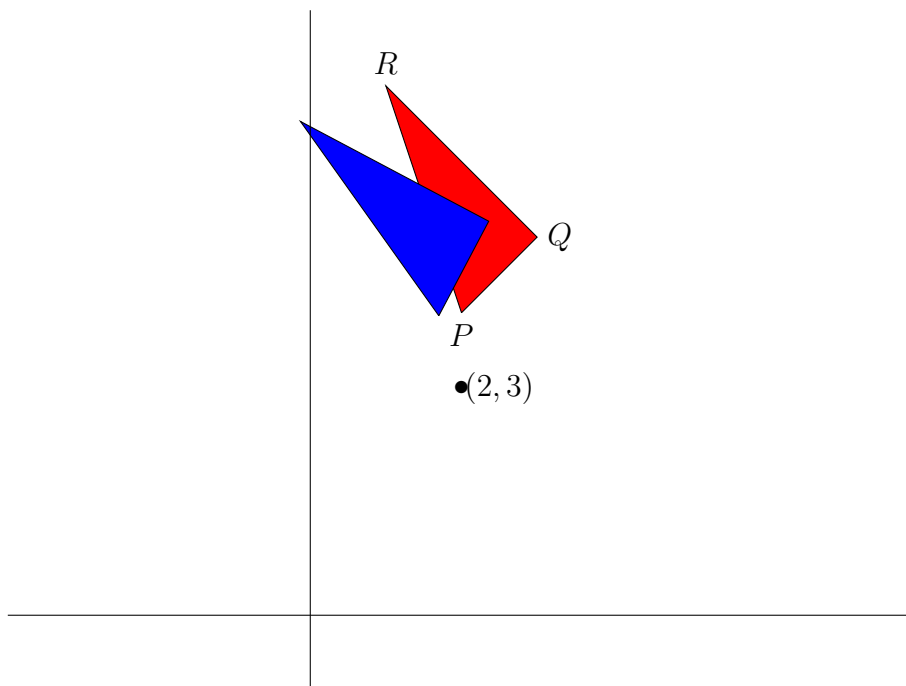
$$M \begin{pmatrix} a \\ b \\ 0 \end{pmatrix} = \begin{pmatrix} a \cos(\alpha) - b \sin \alpha \\ a \sin(\alpha) + b \cos(\alpha) \\ 0 \end{pmatrix}$$

M acts at the points of infinity in the samw way as the rotation matrix, with no effect from the translations. This could also be seen by stepping through the three matrices in turn, using the arguments we have already seen.

10.2.2 Projections

There are two slightly different notions of projections: projections from a point at infinity, and projections from an ordinary point.

Figure 10.1: Rotating the red triangle yields the blue one



Example 10.2.4. Choose a point $(a, b, 0)$ at infinity, and a line given in ordinary coordinates by an equation $sx + ty = u$. Projection from the point to the line means geometrically that any point $P = (P_x, P_y)$ is transformed into the point we get by intersecting the given line and the line through P with direction given by the point at infinity. We want to find a 3×3 -matrix M that represents this transformation.

If the line we project on contains the origin, so that $u = 0$, this projection is *linear*, and can be given by a 2×2 -matrix. In this case, we can find the form of the matrix by noting that the vector $[a, b]^T$ is transformed to the origin, and that the vector $[t, -s]^T$ (the direction vector of the line) is unchanged. This gives four equations in four unknowns (the four elements M_{ij}), and as long as $[a, b]^T$ is not parallel to $[t, -s]^T$, this system has only one solution. We omit the details of the computation, but can check that

$$\frac{1}{sa + tb} \begin{pmatrix} tb & -ta \\ -sb & sa \end{pmatrix}$$

does the job. To apply this to a general line, we need to take into account a translation, as in Example 10.2.3. The matrix we end up with is

$$M = \frac{1}{sa + tb} \begin{pmatrix} tb & -ta & au \\ -sb & sa & bu \\ 0 & 0 & sa + tb \end{pmatrix} = \begin{pmatrix} \frac{tb}{sa+tb} & -\frac{ta}{sa+tb} & \frac{au}{sa+tb} \\ -\frac{sb}{sa+tb} & \frac{sa}{sa+tb} & \frac{bu}{sa+tb} \\ 0 & 0 & 1 \end{pmatrix}$$

Let us look at a concrete example, choosing all the constants as

$$\begin{aligned}a &= 1 \\b &= 5 \\s &= 2 \\t &= 3 \\u &= -1\end{aligned}$$

Then the matrix M becomes

$$\frac{1}{17} \begin{pmatrix} 15 & -3 & -1 \\ -10 & 2 & -5 \\ 0 & 0 & 17 \end{pmatrix}$$

The effect of using this matrix is shown in Figure 10.2. Each point on one of the parallel lines is sent to the same point on the fixed line, namely the intersection. If we compute the transformation P' of $P = (1, 2)$, we find the coordinates as

$$\frac{1}{17} \begin{pmatrix} 15 & -3 & -1 \\ -10 & 2 & -5 \\ 0 & 0 & 17 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix} = \frac{1}{17} \begin{pmatrix} 8 \\ -11 \\ 17 \end{pmatrix}$$

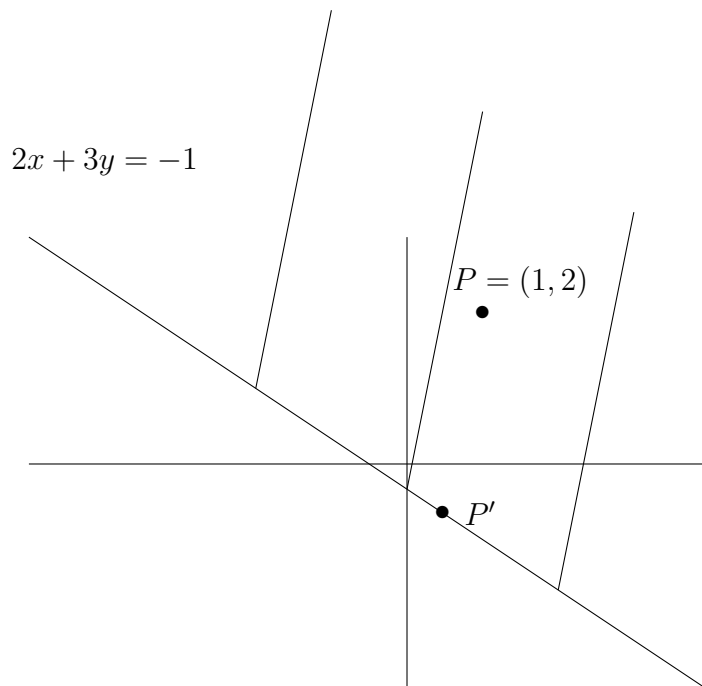
Finally, consider what happens with a point at infinity, say $(c, d, 0)$. Then

$$M = \frac{1}{sa + tb} \begin{pmatrix} tb & -ta & au \\ -sb & sa & bu \\ 0 & 0 & sa + tb \end{pmatrix} \begin{pmatrix} c \\ d \\ 0 \end{pmatrix} = \begin{pmatrix} \frac{t(bc-ad)}{sa+tb} \\ \frac{-s(bc-ad)}{sa+tb} \\ 0 \end{pmatrix}$$

If $bc - ad = 0$, we get all zeroes. Since homogeneous coordinates were defined by excluding $(0, 0, 0)$, this means that the transformation is not defined in this point! If $bc - ad \neq 0$, it is a common factor that can be ignored (think of $\lambda = bc - ad$), and we can also ignore the constant $1/(sa + tb)$. So all other points at infinity are sent to $(t, -s, 0)$, the point at infinity representing the direction of the line we project onto.

Example 10.2.5. Choose an ordinary point $(a, b, 1)$, and a line given in ordinary coordinates by an equation $sx + ty = u$. Projection from the point to the line means that any point $P = (P_x, P_y)$ is transformed into the intersection point we get by intersecting the given line with the line through P and (a, b) . Whether you try to figure out a geometric description of this transformation, or you write down the system of equations for the intersection point, in the end the matrix M representing this transformation is

$$M = \frac{1}{-as - tb} \begin{pmatrix} u - tb & ta & -ua \\ bs & u - as & -ub \\ s & t & -as - tb \end{pmatrix} \quad (10.1)$$

Figure 10.2: Projection from $(1, 5, 0)$ to the line $2x + 3y = -1$ 

To look at a concrete example, we choose our constants as

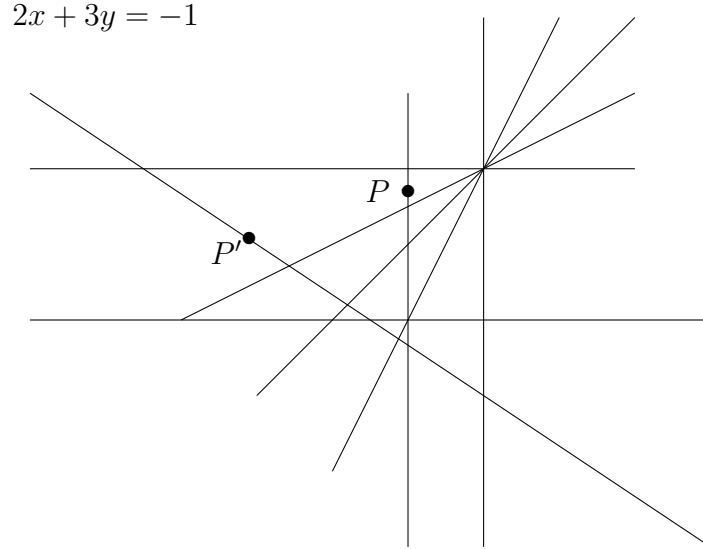
$$\begin{aligned} a &= 1 \\ b &= 2 \\ s &= 2 \\ t &= 3 \\ u &= -1 \end{aligned}$$

Then our matrix becomes

$$M = \frac{1}{-8} \begin{pmatrix} -7 & 3 & 1 \\ 4 & -3 & 2 \\ 2 & 3 & -8 \end{pmatrix}$$

The result of applying this transformation is shown in Figure 10.3. Each point on one of the lines through $(1, 2)$ is sent to the same point on the fixed line, namely the intersection point. If we want to compute the transformation P' the point $P = (1, 2)$, we compute

$$\frac{1}{-8} \begin{pmatrix} -7 & 3 & 1 \\ 4 & -3 & 2 \\ 2 & 3 & -8 \end{pmatrix} \begin{pmatrix} 0 \\ 1.7 \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{6.1}{-8} \\ \frac{-3.1}{-8} \\ \frac{-2.9}{-8} \end{pmatrix} \sim \begin{pmatrix} -2.21 \\ 1.07 \\ 1 \end{pmatrix}$$

Figure 10.3: Projection from $(2, 1, 1)$ to the line $2x + 3y = -1$ 

Note that a point on the line parallel to the line we project upon, through $(1, 2)$, is transformed to the point $(s, t, 0)$. This is the point at infinity in the direction of the line, and is a precise mathematical formulation of the less precise statement that parallel lines meet at infinity. Similarly, a point at infinity should now be thought of as representing a line through $(1, 2)$ with a given slope. It is transformed in the same way as any other point on this line.

10.3 Homogeneous coordinates in three-space

Homogeneous coordinates in space use four coordinates: (a, b, c, d) . As long as $d \neq 0$, we get ordinary space coordinates by dividing by it:

$$(a, b, c, d) \sim \frac{1}{d}(a, b, c, d) = \left(\frac{a}{d}, \frac{b}{d}, \frac{c}{d}, 1\right) = (x, y, z, 1)$$

If $d = 0$, we can think of the remaining three coordinates as homogeneous coordinates for the plane at infinity. Or we can think of $(x, y, z, 0)$ as representing the direction of the line through $(x, y, z, 1)$ and the origin.

10.3.1 Rotations and translations

Linear transformations of $\mathbb{R}^3$ extend to homogeneous coordinates in the same way as in the two-dimensional case.

Example 10.3.1. Let A be any 3×3 -matrix. Then the 4×4 -matrix

$$\left(\begin{array}{c|c} A & 0 \\ \hline 0 & 1 \end{array} \right) = \begin{pmatrix} a_{00} & a_{01} & a_{02} & 0 \\ a_{10} & a_{11} & a_{12} & 0 \\ a_{20} & a_{21} & a_{22} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

transforms the vector $[a, b, c, 1]^T$ in the same way that A transforms $[a, b, c]^T$, keeping the 1 at the end. The effect on the plane at infinity is the same as the effect on the line at infinity in two dimensions.

Example 10.3.2. Translation in space is given by adding a vector $\vec{t} = [t_0, t_1, t_2]^T$. Consider the matrix product

$$\begin{pmatrix} 1 & 0 & 0 & t_0 \\ 0 & 1 & 0 & t_1 \\ 0 & 0 & 1 & t_2 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} = \begin{pmatrix} a + t_0 d \\ b + t_1 d \\ c + t_2 d \\ d \end{pmatrix}$$

If $d \neq 0$, we can divide by it, and see that

$$\begin{pmatrix} 1 & 0 & 0 & t_0 \\ 0 & 1 & 0 & t_1 \\ 0 & 0 & 1 & t_2 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} \sim \begin{pmatrix} x + t_0 \\ y + t_1 \\ z + t_2 \\ 1 \end{pmatrix}$$

On the other hand, if $d = 0$, the product of the matrix and $[a, b, c, 0]^T$ is again $[a, b, c, 0]^T$. Geometrically, the effect of translation on a line is to move it to a parallel line, so the direction is unchanged.

As for the plane, we can apply compositions of transformations to get for instance rotation about an arbitrary point.

Example 10.3.3. The matrix for rotating a degree α about a line parallel to the x -axis about the point $P = (2, 5, -1)$ can be found as a composition of three more easily understood operations.

First, translate so that P is moved to the origin. Then apply the rotation about the x -axis there (this is a linear operation). Finally, translate back again, so that the origin is transformed into P .

The matrix we get by thinking in this way is

$$M = \begin{pmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 5 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\alpha) & -\sin(\alpha) & 0 \\ 0 & \sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & -5 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

10.3.2 Projections

Projections are also similar to what we've already seen in the plane. But because three-dimensional images are more difficult to visualize, it is a bit more complicated to use geometric intuition. Also, the geometry of the plane we project onto is more complicated than the geometry of a line. We start with a simple, but still interesting, case: orthogonal projection on a line through the origin.

Example 10.3.4. Let $\vec{u} = [u_0, u_1, u_2]^T$ be a vector representing the direction of a line through the origin. To project onto this line, note first that the plane with $\vec{u}$ as normal vector is spanned by $[-u_1, u_0, 0]^T$ and $[0, -u_2, u_1]^T$ (if the coordinates are non-zero, there are special cases if one or more coordinates are zero). Both these vectors must be sent to the origin. The simplest matrix sending these to zero is the matrix $[u_0, u_1, u_2]$. Also, we need that the vector $\vec{u}$ is in the image. This means that the matrix for the projection should be

$$\text{a constant times } \begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} \begin{pmatrix} u_0 & u_1 & u_2 \end{pmatrix}$$

The constant should be chosen so that $\vec{u}$ is sent to itself. But $\begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} \cdot 1 = \vec{u}$, so this will work if

$$1 = \text{the constant times } \begin{pmatrix} u_0 & u_1 & u_2 \end{pmatrix} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} = \text{constant times } u_0^2 + u_1^2 + u_2^2.$$

To conclude, the matrix M representing this projection is

$$M = \frac{1}{u_0^2 + u_1^2 + u_2^2} \begin{pmatrix} u_0 \\ u_1 \\ u_2 \end{pmatrix} \begin{pmatrix} u_0 & u_1 & u_2 \end{pmatrix} = \frac{1}{u_0^2 + u_1^2 + u_2^2} \begin{pmatrix} u_0^2 & u_0 u_1 & u_0 u_2 \\ u_1 u_0 & u_1^2 & u_1 u_2 \\ u_2 u_0 & u_2 u_1 & u_2^2 \end{pmatrix}$$

Note finally that the effect of using this matrix on a vector is the same as performing the so-called *vector projection*.

For an application of this projection, see Exercise 10.4.3.

To project onto a plane, we can apply reasoning similar to the previous case.

Example 10.3.5. Projection from a point at the plane of infinity onto a plane is most easily described by picking two vectors spanning the plane.

If the plane doesn't contain the origin, we need to translate it to the origin, project onto it, and translate back. Since we have already seen how to do this several

times, we assume for simplicity that the plane does contain the origin. Then the transformation is linear, and we can consider a 3×3 -matrix representing it.

Let the point at infinity be $(a, b, c, 0)$, the two vectors spanning the plane be $\vec{u}$ and $\vec{v}$. So the projection takes a point on a line with direction vector $[a, b, c]^T$ to the point in the $\vec{u}, \vec{v}$ -plane that intersects the line. This makes sense as long as the vector $[a, b, c]^T$ does lie in this plane. The transformation is not defined at $(a, b, c, 0)$, but is defined at all other points.

We can find the 3×3 -matrix M for this transformation by multiplying three matrices together:

$$M = \begin{pmatrix} u_0 & v_0 \\ u_1 & v_1 \\ u_2 & v_2 \end{pmatrix} A_{2 \times 2} \begin{pmatrix} -b & a & 0 \\ 0 & -c & b \end{pmatrix}$$

The right hand matrix ensures that $[a, b, c]^T$ is transformed to $\vec{0}$. The left hand matrix ensures that $\vec{u}$ and $\vec{v}$ are in the image, so that the image is exactly the plane spanned by them. The matrix A must be included to make sure that $\vec{u}$ is transformed into itself, and $\vec{v}$ into $\vec{v}$. Note that

$$\begin{pmatrix} u_0 & v_0 \\ u_1 & v_1 \\ u_2 & v_2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \vec{u} \text{ and } \begin{pmatrix} u_0 & v_0 \\ u_1 & v_1 \\ u_2 & v_2 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \vec{v}$$

On the other hand,

$$\begin{pmatrix} -b & a & 0 \\ 0 & -c & b \end{pmatrix} \begin{pmatrix} u_0 & v_0 \\ u_1 & v_1 \\ u_2 & v_2 \end{pmatrix} = \begin{pmatrix} -bu_0 + au_1 & -bv_0 + av_1 \\ -cu_1 + bu_2 & -cv_1 + bv_2 \end{pmatrix}$$

By collecting these results, we see that

$$A = \begin{pmatrix} -bu_0 + au_1 & -bv_0 + av_1 \\ -cu_1 + bu_2 & -cv_1 + bv_2 \end{pmatrix}^{-1}$$

Luckily, we have a formula for the inverse of a 2×2 -matrix, and end up with

$$A = \frac{1}{\Delta} \begin{pmatrix} -cv_1 + bv_2 & bv_0 - av_1 \\ cu_1 - bu_2 & -bu_0 + au_1 \end{pmatrix}$$

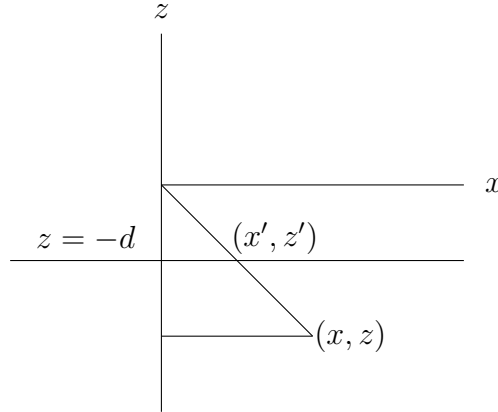
where Δ is the determinant:

$$\Delta = (-bu_0 + au_1)(-cv_1 + bv_2) - (-bv_0 + av_1)(-cu_1 + bu_2)$$

We end up with

$$M = \frac{1}{\Delta} \begin{pmatrix} u_0 & v_0 \\ u_1 & v_1 \\ u_2 & v_2 \end{pmatrix} \begin{pmatrix} -cv_1 + bv_2 & bv_0 - av_1 \\ cu_1 - bu_2 & -bu_0 + au_1 \end{pmatrix} \begin{pmatrix} -b & a & 0 \\ 0 & -c & b \end{pmatrix}$$

Figure 10.4: Consider the two triangles in this figure



Example 10.3.6. Projection from an ordinary point onto a plane really needs homogeneous coordinates, since it is not a linear transformation. The general form of such a matrix can be found by the same reasoning as we used in the plane. Let us now, however, just look at a single case: How we finally project an image onto the computer screen using perspective.

There is a standard way of thinking about this. Recall that in eye coordinates, x and y have their ordinary meaning with regards to the computer screen, whereas the z -axis points towards the viewer. If we assume that the origin is the position of the eye, then the plane containing the screen is perpendicular to the z -axis, so parallel to the xy -plane, at some distance d from the eye. This plane has equation $z = -d$. We can see what happens with x - and y -coordinates separately, making it easier to draw. See Figure 10.4, showing that (x, z) is transformed into (x', z') .

By considering the two triangles, we see that $x'/x = z'/z$. We also see that $z' = -d$. With a similar reasoning applied to y , we see that

$$\begin{pmatrix} x \\ y \\ z \end{pmatrix} \text{ is transformed to } \begin{pmatrix} \frac{-dx}{z} \\ \frac{-dy}{z} \\ -d \end{pmatrix}$$

The 4×4 -matrix we need to get this effect is

$$M = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -\frac{1}{d} & 0 \end{pmatrix}$$

To see that this works, note that

$$M \begin{pmatrix} x \\ y \\ z \\ 1 \end{pmatrix} = \begin{pmatrix} x \\ y \\ z \\ -\frac{z}{d} \end{pmatrix} \sim \begin{pmatrix} \frac{-dx}{z} \\ \frac{-dy}{z} \\ -d \\ 1 \end{pmatrix}$$

Here we used the equivalence relation, multiplying by $-\frac{d}{z}$.

Note that if we apply M to a point with z -coordinate equal to zero, we get a point at infinity. If we apply M to the origin, we get $[0, 0, 0, 0]^T$, so the transformation is not defined at the point where the eye is positioned.

10.4 Exercises

Exercise 10.4.1. Find the matrix representing a rotation in the plane with angle $\alpha = \pi/2$ about the point $(2, -2)$. Compute the transformation of the rectangle with vertices $(2, 0)$, $(2, 6)$, $(4, 6)$ and $(4, 0)$. Draw a figure showing the rectangle before and after the rotation.

Exercise 10.4.2. Find the matrix representing a perspective projection from the origin onto the plane $z = -4$. Compute the image of the triangle with vertices $(2, 3, -6)$, $(4, 5, -6)$ and $(4, 5, -8)$.

Exercise 10.4.3. RGB colour coordinates are triples of numbers, representing the proportion of red, green and blue used to make up a colour. There are different ways to represent these coordinates, we will use `doubles` between 0 and 1. Then the point $(0, 0, 0)$ represents black, $(1, 1, 1)$ represents white, and the line through black and white represents different shades of gray.

The simplest way to get a black-and-white image from a colour image is to project from general RGB coordinates to this line. From Example 10.3.4 we know how to do this.

(a). Find the matrix representing projection onto the line representing shades of gray.

(b). Find some software that allows you to try this out on real images. (MATLAB is an example, using the commands `imread` and `imshow`.)

Chapter 11

Functions of two variables

We know what a function is: it is a rule that for each element in one set associates an element in another set. Starting from the plane $\mathbb{R}^2$, we can consider functions

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}.$$

The subject at hand is the behaviour of this type of function. As for functions in one variable, we also allow a smaller domain of definition. Note that we need two inputs to produce one output, so we will usually write something like

$$f(x, y) = \sin(y) + x^2 \cdot y$$

for a function from $\mathbb{R}^2$ to $\mathbb{R}$. Then the *graph* of f is the set in $\mathbb{R}^3$ consisting of all triples (x, y, z) where $z = f(x, y)$. For instance,

$$f(1, \pi) = \sin(\pi) + 1^2 \cdot \pi = \pi.$$

Therefore the point $(x, y, z) = (1, \pi, \pi)$ lies on the graph of f .

The question of whether such a function is *continuous* is less clear than for a function of one variable. The definition is that f is continuous in (a, b) if

$$f(a, b) = \lim_{(x, y) \rightarrow (a, b)} f(x, y).$$

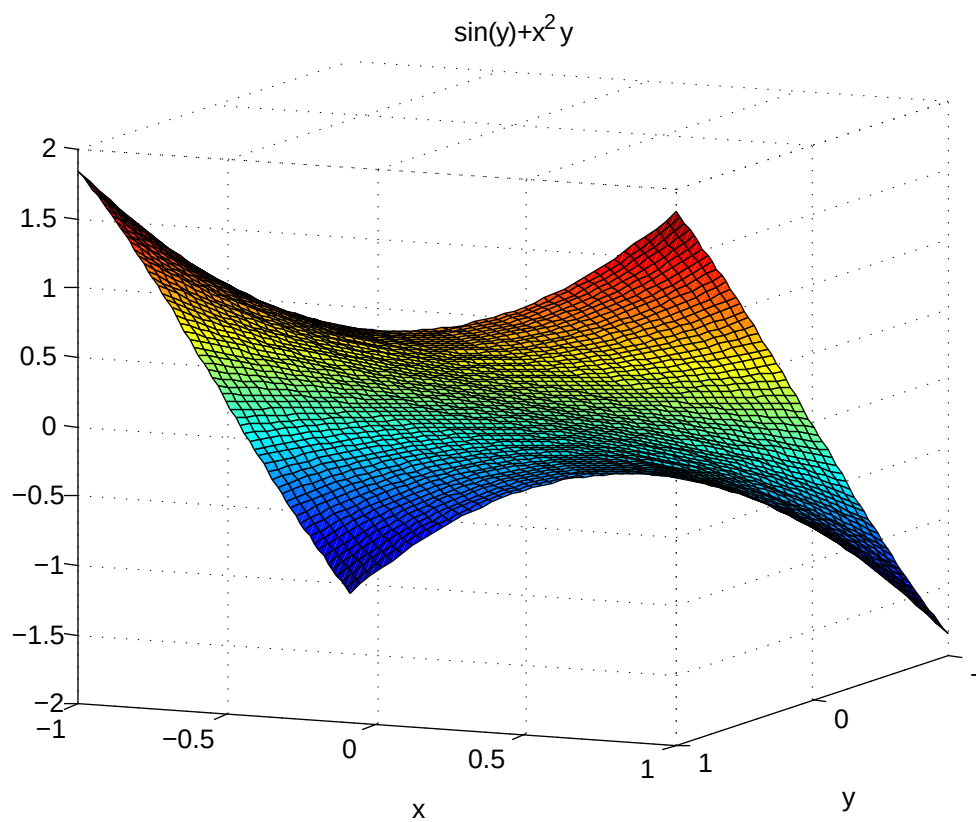
The limit here is taken over all points close to (a, b) in the domain of definition of f . We will mostly ignore the problems with continuity in two variables, but note that most of the functions we are interested in are continuous in every point in their domain of definition. For instance,

$$f(x, y) = \frac{x}{y}$$

is defined for $y \neq 0$, and is continuous. In this case, we can for instance write the domain of definition in symbols as

$$D_f = \{(x, y) \in \mathbb{R}^2 \mid y \neq 0\}.$$

Figure 11.1: The graph of a function of two variables



11.1 Partial derivatives and applications

If we restrict the domain of definition of a function in two variables to a line in $\mathbb{R}^2$ parallel to one of the coordinate axes, we can think of the result as a function in one variable. For instance, we can think of

$$g(x) = f(x, 4)$$

as the restriction of f to the line $y = 4$. As such, we can differentiate g in the usual way. The result is called a *partial derivative* of f .

Definition 11.1.1. Let $f = f(x, y)$ be a function in two variables. We define the *partial derivatives* of f with respect to x and y by

$$\begin{aligned} f_x(x, y) &= \frac{\partial f}{\partial x} = \lim_{h \rightarrow 0} \frac{f(x+h, y) - f(x, y)}{h} \\ f_y(x, y) &= \frac{\partial f}{\partial y} = \lim_{k \rightarrow 0} \frac{f(x, y+k) - f(x, y)}{k} \end{aligned}$$

if the limits exist. The symbol ∂ is pronounced “d”. Partial derivatives can also be defined for functions of n variables. Then there are n choices for the variable we compute the derivative with respect to.

In particular, we can compute f_x by treating y as a constant, as can be seen from the following sample computations:

Example 11.1.2.

$$\begin{aligned} \frac{\partial}{\partial x} (\sin(y) + x^2 y) &= 0 + 2xy \\ \frac{\partial}{\partial y} (\sin(y) + x^2 y) &= \cos(y) + x^2 \cdot 1 \end{aligned}$$

Compare with the following ordinary derivatives:

$$\begin{aligned} (\sin(2) + x^2 \cdot 2)' &= 0 + 2x \cdot 2 \\ (\sin(y) + 3^2 y)' &= \cos(y) + 3^2 \cdot 1 \end{aligned}$$

We see that the actual computation of a partial derivative is no more difficult than the computation of ordinary derivatives. As for a function of one variable, we want to use derivatives to understand phenomena like growth, extremal values and local behaviour of functions. There is some added complexity coming from the interaction of the two partial derivatives, so we need another definition.

Definition 11.1.3. A function of two variables is called *differentiable* in a point (a, b) if

$$\lim_{(x,y) \rightarrow (a,b)} \frac{f(x, y) - f(a, b) - f_x(a, b)(x - a) - f_y(a, b)(y - b)}{\sqrt{(x - a)^2 + (y - b)^2}} = 0.$$

Note that this definition presupposes that the partial derivatives exist.

As for continuity, most of the functions we are interested in will have this property whenever they are defined.

The upshot of the definition is that the function is well approximated by its tangent plane at (a, b) , in the same way that a function of one variable can be approximated by a tangent line.

The two partial derivatives can be combined to form a vector.

Definition 11.1.4. The *gradient vector* ∇f of a function of two variables in a point (a, b) is the vector

$$\nabla f(a, b) = (f_x(a, b), f_y(a, b)).$$

The symbol ∇ is read “nabla” or “del”. The gradient can also be defined for a function of n variables; then it is a vector with n components.

Example 11.1.5. The gradient of the function $f(x, y) = x^4 - 3x^2y + 2y^3$ is in general

$$\nabla f = \nabla f(x, y) = (4x^3 - 6xy, -3x^2 + 6y^2).$$

In the point $(1, 2)$ the gradient is

$$\nabla f(1, 2) = (4 \cdot 1^3 - 6 \cdot 1 \cdot 2, -3 \cdot 1^2 + 6 \cdot 2^2) = (-8, 21).$$

Definition 11.1.6. A point $(x, y) = (a, b)$ is called a *stationary point* (or a critical point) for the function $f = f(x, y)$ if $\nabla f = \vec{0}$.

The most important properties of gradients are summarized in the next theorem.

Theorem 11.1.7. Let ∇f be the gradient vector of a differentiable function $f = f(x, y)$ at (a, b) . Assume that $\nabla f \neq \vec{0}$. Then

∇f points in the direction where f increases fastest.

$-\nabla f$ points in the direction where f decreases fastest.

The directions orthogonal to ∇f are directions where f does not change (infinitesimally).

If a point (in the interior of the domain of definition) is a local maximum or minimum point of a differentiable function, the point is a stationary point, i.e. the gradient is the zero vector.

Example 11.1.8. Find the direction in which $f(x, y) = \sin(2x + y)$ increases fastest in the point $(\frac{\pi}{4}, \frac{\pi}{4})$.

We compute the two partial derivatives, and evaluate at the point:

$$\begin{aligned} f_x(x, y) &= 2 \cos(2x + y) \\ f_x\left(\frac{\pi}{4}, \frac{\pi}{4}\right) &= 2 \cos\left(\frac{3\pi}{4}\right) = -\sqrt{2} \\ f_y(x, y) &= \cos(2x + y) \\ f_y\left(\frac{\pi}{4}, \frac{\pi}{4}\right) &= \cos\left(\frac{3\pi}{4}\right) = -\frac{\sqrt{2}}{2} \end{aligned}$$

The direction we ask for is

$$\nabla f\left(\frac{\pi}{4}, \frac{\pi}{4}\right) = \left(-\sqrt{2}, -\frac{\sqrt{2}}{2}\right).$$

A sufficiently differentiable function in one variable can be approximated linearly, as

$$f(x) \approx f(a) + f'(a)(x - a),$$

or better, with a Taylor expansion of order two, as

$$f(x) \approx f(a) + f'(a)(x - a) + \frac{1}{2}f''(x)(x - a)^2.$$

In two variables, we can write a linear approximation as

$$f(x, y) \approx f(a, b) + f_x(a, b)(x - a) + f_y(a, b)(y - b) = f(a, b) + \nabla f(a, b) \begin{pmatrix} x - a \\ y - b \end{pmatrix}.$$

What should be included to get a second order approximation? Note first that there are four ways of differentiating a function in two variables twice:

$$\begin{aligned} f_{xx} &= \frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x} \left(\frac{\partial f}{\partial x} \right) \\ f_{yy} &= \frac{\partial^2 f}{\partial y^2} = \frac{\partial}{\partial y} \left(\frac{\partial f}{\partial y} \right) \\ f_{xy} &= \frac{\partial^2 f}{\partial x \partial y} = \frac{\partial}{\partial x} \left(\frac{\partial f}{\partial y} \right) \\ f_{yx} &= \frac{\partial^2 f}{\partial y \partial x} = \frac{\partial}{\partial y} \left(\frac{\partial f}{\partial x} \right) \end{aligned}$$

Example 11.1.9. What are the second order derivatives of $f(x, y) = \sin(x) \cos(2y)$?

We compute that

$$\begin{aligned} f_x &= \cos(x) \cos(2y) \\ f_y &= -2 \sin(x) \sin(2y) \end{aligned}$$

This gives us

$$\begin{aligned}f_{xx} &= -\sin(x) \cos(2y) \\f_{yy} &= -4 \sin(x) \cos(2y) \\f_{xy} &= -2 \cos(x) \sin(2y) \\f_{yx} &= -2 \cos(x) \sin(2y)\end{aligned}$$

It was no accident that $f_{xy} = f_{yx}$ in this example:

Theorem 11.1.10. *If the second partial derivatives of a function $f = f(x, y)$ are continuous, the mixed partial derivatives are equal:*

$$f_{xy} = f_{yx}$$

The four (three different) second order derivatives can be conveniently arranged in a matrix:

Definition 11.1.11. The *Hessian* of a function in two variables is the matrix

$$\begin{pmatrix} f_{xx} & f_{yx} \\ f_{xy} & f_{yy} \end{pmatrix}.$$

Example 11.1.12. The Hessian of $f(x, y) = \sin(x) \cos(2y)$ is

$$\begin{pmatrix} f_{xx} & f_{yx} \\ f_{xy} & f_{yy} \end{pmatrix} = \begin{pmatrix} -\sin(x) \cos(2y) & -2 \cos(x) \sin(2y) \\ -2 \cos(x) \sin(2y) & -4 \sin(x) \cos(2y) \end{pmatrix}.$$

The Hessian in the point $(x, y) = (0, \pi/4)$ is

$$\begin{pmatrix} -\sin(0) \cos(2\pi/4) & -2 \cos(0) \sin(2\pi/4) \\ -2 \cos(0) \sin(2\pi/4) & -4 \sin(0) \cos(2\pi/4) \end{pmatrix} = \begin{pmatrix} 0 & -2 \\ -2 & 0 \end{pmatrix}$$

Now the second order approximation for a function in two variables can be expressed as

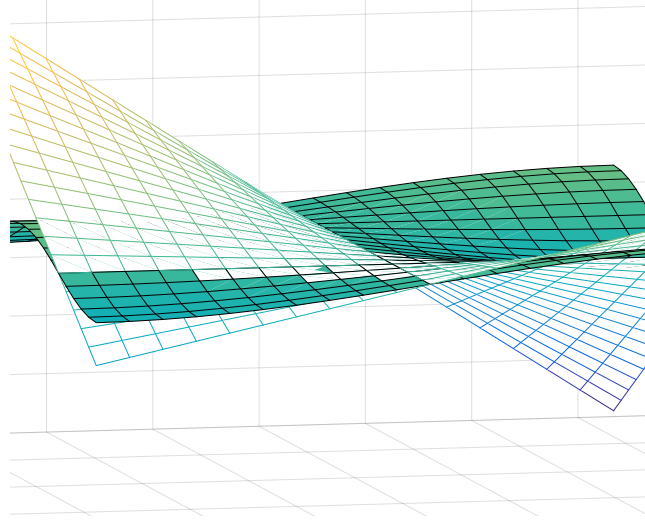
$$f(x, y) \approx f(a, b) + \nabla f(a, b) \begin{pmatrix} x - a \\ y - b \end{pmatrix} + \frac{1}{2} \begin{pmatrix} x - a & y - b \end{pmatrix} \begin{pmatrix} f_{xx}(a, b) & f_{yx}(a, b) \\ f_{xy}(a, b) & f_{yy}(a, b) \end{pmatrix} \begin{pmatrix} x - a \\ y - b \end{pmatrix}.$$

Example 11.1.13. The second order approximation of $f(x, y) = \sin(x) \cos(2y)$ at the point $(x, y) = (0, \pi/4)$ is calculated from the function value, gradient and the Hessian as follows:

$$f(0, \pi/4) = \sin(0) \cdot \cos(2 \cdot \pi/4) = 0$$

$$\nabla f(0, \pi/4) = (\cos(0) \cos(2 \cdot \pi/4), \sin(0) \sin(2 \cdot \pi/2)) = (0, 0)$$

Figure 11.2: Second order approximation



$$\begin{pmatrix} -\sin(0) \cos(2\pi/4) & -2 \cos(0) \sin(2\pi/4) \\ -2 \cos(0) \sin(2\pi/4) & -4 \sin(0) \cos(2\pi/4) \end{pmatrix} = \begin{pmatrix} 0 & -2 \\ -2 & 0 \end{pmatrix}$$

Therefore

$$\begin{aligned} f(x, y) &\approx f(0, \pi/4) + \nabla f(0, \pi/4) \begin{pmatrix} x - 0 \\ y - \pi/4 \end{pmatrix} \\ &+ \frac{1}{2} \begin{pmatrix} x - 0 & y - \pi/4 \end{pmatrix} \begin{pmatrix} f_{xx}(0, \pi/4) & f_{yx}(0, \pi/4) \\ f_{xy}(0, \pi/4) & f_{yy}(0, \pi/4) \end{pmatrix} \begin{pmatrix} x - 0 \\ y - \pi/4 \end{pmatrix} \\ &= 0 + (0, 0) \begin{pmatrix} x \\ y - \pi/4 \end{pmatrix} + \frac{1}{2} \begin{pmatrix} x & y - \pi/4 \end{pmatrix} \begin{pmatrix} 0 & -2 \\ -2 & 0 \end{pmatrix} \begin{pmatrix} x \\ y - \pi/4 \end{pmatrix} \\ &= \frac{1}{2} \begin{pmatrix} -2(y - \pi/4) & -2x \end{pmatrix} \begin{pmatrix} x \\ y - \pi/4 \end{pmatrix} \\ &= \frac{1}{2} (-2(y - \pi/4)x - 2x(y - \pi/4)) \\ &= -2x(y - \pi/4) \end{aligned}$$

The approximation is good close to the point $(x, y) = (0, \pi/4)$. In Figure 11.2, the graph of the function is shown together with its second order approximation (in lighter colour).

For a differentiable function, a local maximum or minimum must be a stationary point. In some cases, the nature of a stationary point can be determined from the Hessian. If $\nabla f(a, b) = \vec{0}$, the second order approximation simplifies to

$$f(x, y) \approx f(a, b) + \frac{1}{2} \begin{pmatrix} x - a & y - b \end{pmatrix} \begin{pmatrix} f_{xx}(a, b) & f_{yx}(a, b) \\ f_{xy}(a, b) & f_{yy}(a, b) \end{pmatrix} \begin{pmatrix} x - a \\ y - b \end{pmatrix}$$

If the second degree term is positive for all (x, y) close to, but not equal to, (a, b) , this point is a minimum. Similarly, if this term is negative, we have a maximum. It is also possible that the term has both positive and negative values. Then the stationary point is neither a maximum nor a minimum. These properties are related to the eigenvalues of the Hessian, but can be formulated more elementary:

Theorem 11.1.14. *Let (a, b) be a stationary point for a sufficiently differentiable function $f = f(x, y)$. Consider the determinant*

$$\Delta = \begin{vmatrix} f_{xx}(a, b) & f_{yx}(a, b) \\ f_{xy}(a, b) & f_{yy}(a, b) \end{vmatrix} = f_{xx}(a, b)f_{yy}(a, b) - f_{yx}(a, b)^2.$$

- a) *If $\Delta > 0$ and $f_{xx}(a, b) > 0$, (a, b) is a local minimum.*
- b) *If $\Delta > 0$ and $f_{xx}(a, b) < 0$, (a, b) is a local maximum.*
- c) *If $\Delta < 0$, (a, b) is neither a maximum nor a minimum. It is a so-called saddle point.*
- d) *If $\Delta = 0$, all possibilities are still open.*

Part d) really means that the point could be a maximum, a minimum, a saddle point, or something completely different.

Example 11.1.15. The following four functions all have $(0, 0)$ as their only stationary point, and correspond to the four different cases of the theorem. All computations are left to the reader. The graphs are displayed in Figure 11.3.

- a) $f(x, y) = x^2 + y^2$.
- b) $g(x, y) = -x^2 - y^2$.
- c) $h(x, y) = x^2 - y^2$.
- d) $k(x, y) = x^3 - 3xy^2$.

Example 11.1.16. Find the only candidate for stationary point for the function

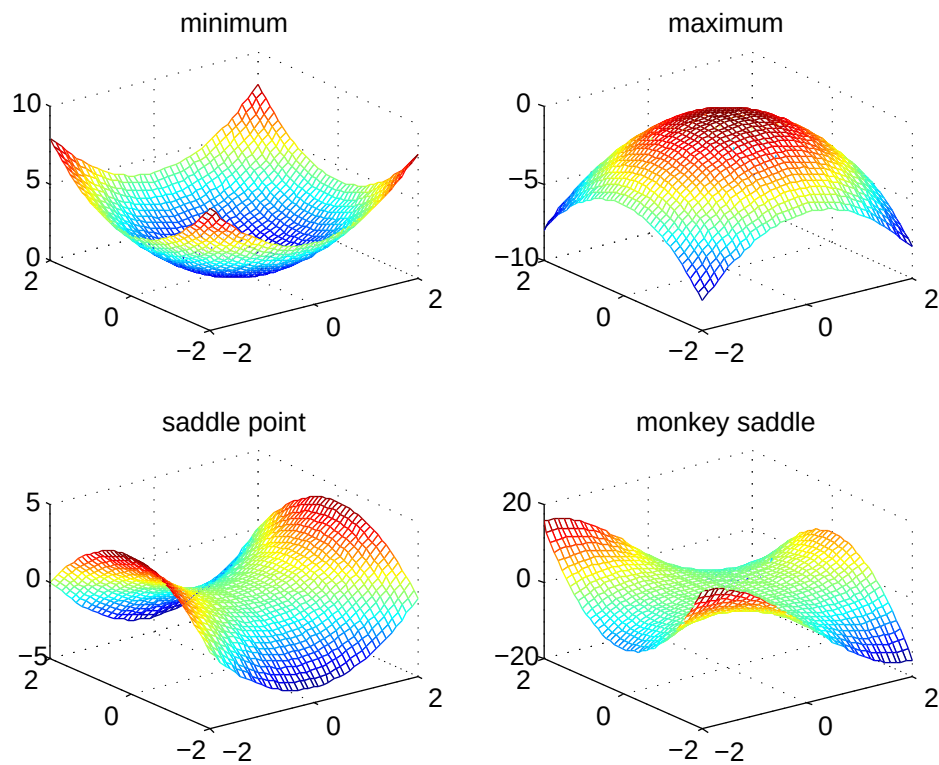
$$f(x, y) = x^2 - 12x + 3y^2 - 12y.$$

Determine whether it is a maximum or minimum, if possible.

To answer these questions, we compute the gradient $\nabla f = [2x - 12, 6y - 12]$ and the determinant of the Hessian

$$\begin{vmatrix} f_{xx}(a, b) & f_{yx}(a, b) \\ f_{xy}(a, b) & f_{yy}(a, b) \end{vmatrix} = \begin{vmatrix} 2 & 0 \\ 0 & 6 \end{vmatrix} = 12.$$

Figure 11.3: Four cases for stationary points



The gradient is zero for $x = 6$, $y = 2$. In the case of our function f , the four functions appearing the Hessian are all constants, so it is easy to evaluate them at $(x, y) = (6, 2)$. The determinant is $12 > 0$ and $f_{xx}(a, b) = 2 > 0$, so (a, b) is a minimum.

Example 11.1.17. Consider the function

$$f(x, y) = x \sin y.$$

Let us find all the stationary points, and determine what kind they are.

The gradient is $\nabla f = [\sin y, x \cos y]$. If $\nabla f = \vec{0}$, the first coordinate tells us that $\sin y = 0$. This is true for $y = n\pi$ for all integers n . Note that $\cos n\pi \neq 0$ (in fact $\cos n\pi = (-1)^n$), so the second coordinate tells us that $x = 0$. Therefore, there are infinitely many stationary points,

$$(x, y) = (0, n\pi) \quad \text{for all integers } n.$$

The determinant of the Hessian becomes

$$\begin{vmatrix} f_{xx}(a, b) & f_{yx}(a, b) \\ f_{xy}(a, b) & f_{yy}(a, b) \end{vmatrix} = \begin{vmatrix} 0 & \cos y \\ \cos y & -x \sin y \end{vmatrix} = -\cos^2 y.$$

For each stationary point, this is negative (in fact equal to -1). Therefore this function has infinitely many saddle points, and no local maximum nor minimum.

11.2 Partial derivatives in image processing

In this section, which is really just an extended example, we will see that partial derivatives are of interest in image processing. The basic example is *intensity*.

Grayscale images

To keep things simple, we will only consider grayscale images. To each pixel we associate a graytone represented by an integer in the interval $[0, 255]$. 0 represents black, 255 represents white. The function that takes a pixel, or rather its coordinates, to the corresponding integer, is denoted $I = I(x, y)$.

For instance, if we look at the picture with $8 = 2 \cdot 4$ pixels, where the values of I are given as (with indexing starting at 1)

$$\begin{array}{cccc} I(1, 1) = 0 & I(1, 2) = 54 & I(1, 3) = 54 & I(1, 4) = 200 \\ I(2, 1) = 20 & I(2, 2) = 150 & I(2, 3) = 80 & I(2, 4) = 250 \end{array}$$

Figure 11.4: Eight-pixel grayscale image



we will get the image in Figure 11.4.

Even though I is just an array of integers, we can think of I as an approximation to a continuous function. Recall that

$$\frac{\partial I}{\partial x}(x, y) = \lim_{h \rightarrow 0} \frac{I(x+h, y) - I(x, y)}{h} \approx \frac{I(x+h, y) - I(x, y)}{h} \quad \text{for } h \text{ small.}$$

We can then choose h as small as possible, which now is $h = 1$ (the difference in index between two neighbouring pixels). We are therefore just left with the approximation

$$\frac{\partial I}{\partial x}(x, y) \approx I(x+1, y) - I(x, y).$$

If the values of I are in the range $[0, 255]$, the range of $I(x+1, y) - I(x, y)$ is $[-255, 255]$. We can compute the difference, and then rescale linearly by the rule $u \rightarrow \frac{u+255}{2}$ to get a new number in the range $[0, 255]$. Round off to ensure that we get an integer. Then we can display the partial derivative as a new image!

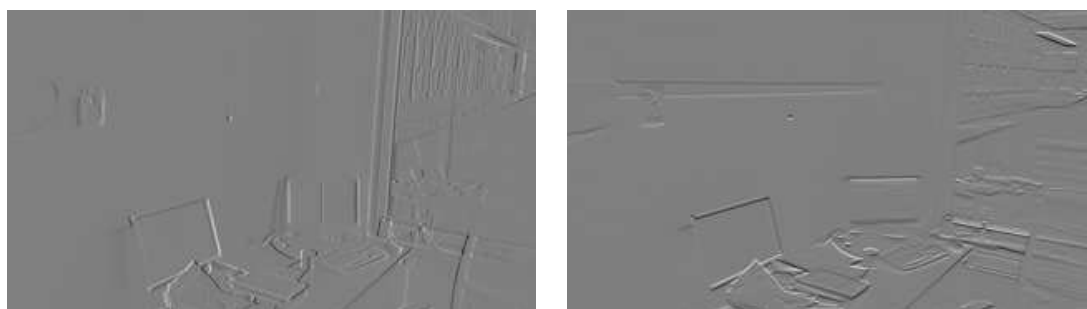
What information do you think is contained in this new image? Recall that the derivative is a measure of change. Therefore, we would expect changes in the image, as we move in the x -direction, to be contained in this new image. The rescaling means that no change (zero before the rescaling) is coded as gray ($255/2$), change from dark to light (positive before rescaling) is coded as closer to white, and change from light to dark (negative before rescaling) is coded as closer to black.

Example 11.2.1. Let us consider the author's office desk, as shown in Figure 11.5. Computing the two partial derivatives produces these two images in Figure 11.6. Can you tell which image corresponds to $\partial I / \partial x$? Note that the information contained in the images is rather coarse compared to the original. Also note that vertical and

Figure 11.5: A black/white image



Figure 11.6: Images from partial derivatives, rescaled



horizontal structures are evidently displayed. For instance, the horizontal beam is apparent in one image, the vertical division line between the wall and the window is apparent in the other.

11.3 Least squares

This topic brings together matrix theory and functions of two variables. We want to find a line approximating a finite number of points, typically coming from measurements. The line should be the best approximation, in some sense. We will describe two equivalent ways of producing this line.

The points we want to approximate are

X	Y
X_1	Y_1
X_2	Y_2
$\vdots$	$\vdots$
X_n	Y_n

Our approximation is

$$Y = AX + B$$

where the constants A and B must be determined. The principle of least squares approximation is that we want the squares

$$(Y(X_i) - Y_i)^2 = (AX_i + B - Y_i)^2$$

to be small. More precisely, we want the sum of all these to be as small as possible! We shall see that this requirement determines A and B uniquely. An important question in statistics is whether the line produced in this way represents a meaningful approximation to the data. For now, we will only focus on producing the line.

11.3.1 Partial derivatives approach

In this approach, we will consider A and B as variables, and we want to minimize the function $E = E(A, B)$ that computes the square error term:

$$E(A, B) = \sum_{i=1}^n (Y_i - (AX_i + B))^2.$$

The (complicated) computation becomes somewhat simpler by introducing $\beta = B + A\bar{X}$, where

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$

is the average (mean) of the measured X_i .

We have

$$AX_i + B = AX_i - A\bar{X} + B + A\bar{X} = A(X_i - \bar{X}) + \beta.$$

Then

$$E(A, \beta) = \sum_{i=1}^n (Y_i - (AX_i + B))^2 = \sum_{i=1}^n (Y_i - (A(X_i - \bar{X}) + \beta))^2.$$

We then form the partial derivatives with respect to A and β :

$$\begin{aligned} \frac{\partial E}{\partial A} &= -2 \sum_{i=1}^n (Y_i - (A(X_i - \bar{X}) + \beta))(X_i - \bar{X}) \\ \frac{\partial E}{\partial \beta} &= -2 \sum_{i=1}^n (Y_i - (A(X_i - \bar{X}) + \beta)) \end{aligned}$$

At a minimum, we will have both partial derivatives equal to zero. Moving all terms with Y s to one side, we get the two equations:

$$\begin{aligned} \sum_{i=1}^n Y_i(X_i - \bar{X}) &= \sum_{i=1}^n A(X_i - \bar{X})^2 + \sum_{i=1}^n \beta(X_i - \bar{X}) \\ \sum_{i=1}^n Y_i &= \sum_{i=1}^n A(X_i - \bar{X}) + \sum_{i=1}^n \beta. \end{aligned}$$

Note that

$$\sum_{i=1}^n (X_i - \bar{X}) = \sum_{i=1}^n X_i - \sum_{i=1}^n \bar{X} = n\bar{X} - n\bar{X} = 0.$$

The two equations are simplified to

$$\begin{aligned} \sum_{i=1}^n Y_i(X_i - \bar{X}) &= A \sum_{i=1}^n (X_i - \bar{X})^2 \\ \sum_{i=1}^n Y_i &= \sum_{i=1}^n \beta = n\beta \end{aligned}$$

Now the result can be read off:

$$A = \frac{\sum_{i=1}^n Y_i(X_i - \bar{X})}{\sum_{i=1}^n (X_i - \bar{X})^2}$$

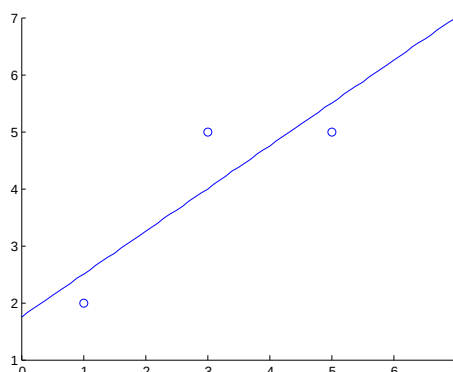
and

$$B = \beta - A\bar{X} = \frac{1}{n} \sum_{i=1}^n Y_i - A\bar{X} = \bar{Y} - A\bar{X}.$$

Theorem 11.3.1. *The line $Y = AX + B$ that minimizes the sum of squares error for the points (X_i, Y_i) , $i = 1, 2, \dots, n$, has coefficients*

$$\begin{aligned} A &= \frac{\sum_{i=1}^n Y_i(X_i - \bar{X})}{\sum_{i=1}^n (X_i - \bar{X})^2} \\ B &= \bar{Y} - A\bar{X}. \end{aligned}$$

Figure 11.7: Best linear approximation to three points



Example 11.3.2. The computations involved generally warrant using a computer, so here we will only consider an unrealistically small example by hand. Assume we have the three points $(1, 2)$, $(3, 5)$ and $(5, 5)$. Then the numbers involved in the computations are

$$\begin{aligned}\bar{X} &= \frac{1}{3}(1 + 3 + 5) = 3 \\ \bar{Y} &= \frac{1}{3}(2 + 5 + 5) = 4 \\ \sum (X_i - \bar{X})^2 &= (1 - 3)^2 + (3 - 3)^2 + (5 - 3)^2 = 8 \\ \sum Y_i(X_i - \bar{X}) &= 2(1 - 3) + 5(3 - 3) + 5(5 - 3) = 6 \\ A &= \frac{6}{8} = 0.75 \\ B &= 4 - 0.75 \cdot 3 = 1.75\end{aligned}$$

Thus, our best approximation is the line

$$Y = 0.75X + 1.75.$$

See Figure 11.7.

11.3.2 Linear algebra approach

The starting point for this approach is to introduce names for the error terms, say ϵ_i , so that we have the exact relationship

$$\begin{aligned}Y_1 &= AX_1 + B + \epsilon_1 \\ Y_2 &= AX_2 + B + \epsilon_2 \\ &\vdots \\ Y_n &= AX_n + B + \epsilon_n\end{aligned}$$

We can write this in matrix form as

$$\begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix} = \begin{pmatrix} X_1 & 1 \\ X_2 & 1 \\ \vdots & \vdots \\ X_n & 1 \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{pmatrix}$$

Introduce notation

$$\vec{Y} = M\vec{p} + \vec{\epsilon}$$

where the symbols exactly match the matrices in the same order above. To minimize the square of the errors now means to minimize the square of the length of the vector $\vec{\epsilon}$ of errors, or equivalently the length. But the vector

$$\vec{\epsilon} = \vec{Y} - M\vec{p}$$

points to the actual value from the measurements from a linear combination of the columns of M , and this is as short as possible when $\vec{\epsilon}$ is orthogonal to the plane spanned by the columns $\vec{x}$ and $\vec{u} = [1, 1, \dots, 1]^T$. This means that the two scalar products $\vec{x} \bullet \vec{\epsilon} = 0$ and $\vec{u} \bullet \vec{\epsilon} = 0$, or that $M^T \vec{\epsilon} = 0$ by definition of matrix multiplication. But then

$$M^T \vec{\epsilon} = M^T (\vec{Y} - M\vec{p}) = 0$$

or

$$M^T M \vec{p} = M^T \vec{y}.$$

This can now be thought of as a set of two equations in two unknowns, and has a unique solution. Let us see what this says, written out with A and B :

$$M^T M \vec{p} = \begin{pmatrix} \sum X_i^2 & \sum X_i \\ \sum X_i & \sum 1 \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix} = M^T \vec{y} = \begin{pmatrix} \sum X_i Y_i \\ \sum Y_i \end{pmatrix}$$

It is an exercise in properties of the mean to check that this again produces Theorem 11.3.1. We will for now be satisfied with checking that we get the same result in an example.

Example 11.3.3. With the three points $(1, 2)$, $(3, 5)$ and $(5, 5)$, the system of equations becomes

$$\begin{pmatrix} 1^2 + 3^2 + 5^2 & 1 + 3 + 5 \\ 1 + 3 + 5 & 1 + 1 + 1 \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix} = \begin{pmatrix} 1 \cdot 2 + 3 \cdot 5 + 5 \cdot 5 \\ 2 + 5 + 5 \end{pmatrix}.$$

The total matrix is thus

$$\begin{pmatrix} 35 & 9 & 42 \\ 9 & 3 & 12 \end{pmatrix}.$$

Perform row-operations (in a non-standard order):

$$\begin{aligned} \begin{pmatrix} 35 & 9 & 42 \\ 9 & 3 & 12 \end{pmatrix} &\sim \begin{pmatrix} 35 & 9 & 42 \\ 3 & 1 & 4 \end{pmatrix} \sim \begin{pmatrix} 8 & 0 & 6 \\ 3 & 1 & 4 \end{pmatrix} \\ &\sim \begin{pmatrix} 1 & 0 & 3/4 \\ 3 & 1 & 4 \end{pmatrix} \sim \begin{pmatrix} 1 & 0 & 3/4 \\ 0 & 1 & 7/4 \end{pmatrix} \end{aligned}$$

As before, we get $A = 3/4 = 0.75$ and $B = 7/4 = 1.75$, so the best line approximation is $Y = 0.75X + 1.75$.

11.4 Exercises

Exercise 11.4.1. Find the gradient of each function, and evaluate it in the given point.

(a). $f(x, y) = x^4 - 2 \sin(xy)$ in $(2, \pi)$

(b). $g(x, y) = e^{x-y}$ in $(3, 3)$

(c). $h(x, y) = x^2 y^2$ in $(1, -1)$

Exercise 11.4.2. Find the linear approximation of each function at the given point.

(a). $f(x, y) = e^{x^2-y^2}$, $(x, y) = (1, 1)$.

(b). $g(x, y) = x^2 y + x^3 y^2$, $(x, y) = (1, -1)$

(c). $h(x, y) = 2x + y$, $(x, y) = (3, 2)$

Exercise 11.4.3. Find the second order approximation of each function at the given point.

(a). $f(x, y) = \ln(x + y)$, $(x, y) = (2, 3)$

(b). $g(x, y) = x^3 y^2 - 2x^5 y$, $(x, y) = (-2, -1)$

(c). $h(x, y) = 2x^2 - xy - 3y^2$, $(x, y) = (1, -1)$

Exercise 11.4.4. Find and classify (if possible) the stationary points of each function.

(a). $f(x, y) = 3x^3 + xy^2 + 12x$

(b). $g(x, y) = \frac{1}{3}x^3 - 4x + \frac{1}{3}y^3 - 9y$

(c). $h(x, y) = 3x^2 + xy^2$

Exercise 11.4.5. In each part, find the best line approximating the given points. Use both the partial derivatives approach and the linear algebra approach!

(a). $(0, 2)$, $(1, 1)$, $(3, -1)$ and $(6, -3)$.

(b). $(2, 2)$, $(1, 2)$ and $(3, 2)$.

(c). $(0, 3)$, $(1, 2)$, $(1, 4)$ and $(2, 5)$.

Chapter 12

Statistical analysis of text

As a simple example of producing an empirical function, we will count some features of ordinary text. Then we will apply our results to one of the simplest forms of encryption, where each letter in the alphabet is exchanged for another. Along the way, we will introduce some vocabulary from statistics and cryptology.

12.1 Counting in texts

A very simple question about a given string is how many times each character appears. For simplicity, our texts will contain only the 26 lowercase letters of the English alphabet. Punctuation and spaces are ignored, and uppercase letters are converted to lowercase. We will therefore associate an array of integers of length 26 to any text. Division by the total number of counted characters will give an array of floats summing to one, which will tell us the percentage of occurrences of a given character. For instance, to the text “A very short example”, we first create the string `averyshortexample`. Then we will associate the integer array of occurrences. Dividing by the number of counted characters (17), and multiplying by 100, we get the array of doubles representing percentages, see Table 12.1.

For reference, we should make a starting array of frequencies from some large text, or database of texts, so that we can compare other texts to these reference frequencies.

Example 12.1.1. We will compile a reference array for frequencies using William Shakespeare’s works (standard modernized version). It is convenient to use as simple a format as possible. Many classical texts, including Shakespeare’s *Macbeth*, is available from <http://www.gutenberg.org/>. For our purposes, the simplest way to read the file is one character at a time. Whenever the character is a letter, we add one in the correct place in an array of the form `int[26]`. We now look at some sample code to do this. There are some possible simplifications, but we want to extend this

Table 12.1: Occurrences and frequencies of characters in `averyshortexample`

letter	number
a	2
b	0
c	0
d	0
e	3
f	0
g	0
h	1
i	0
j	0
k	0
l	1
m	1
n	0
o	1
p	1
q	0
r	2
s	1
t	1
u	0
v	1
w	0
x	1
y	1
z	0

letter	percentage
a	11.8
b	0
c	0
d	0
e	17.6
f	0
g	0
h	5.9
i	0
j	0
k	0
l	5.9
m	5.9
n	0
o	5.9
p	5.9
q	0
r	11.8
s	5.9
t	5.9
u	0
v	5.9
w	0
x	5.9
y	5.9
z	0

code further on. First, we input a file, and find the first letter in the text. Just input your path in this listing.

```
int [] numberOfLetters = new int [26];
    FileReader textFileReader = new FileReader("path\\macbeth");
    BufferedReader textReader = new BufferedReader(textFileReader);
    int c = textReader.read();
    while (!(c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')){
        // ensures that c is a letter
        c = textReader.read();
    }
```

Note that a letter in the English language is either in the range from a to z or A to Z. As long as the character is not in this range, we read another character.

```
while (c != -1){ // c = -1 means end-of-file
    if (c >= 'a' && c <= 'z'){ // if c is a lowercase letter
        numberOfLetters[c-'a']++;
    }
    if (c >= 'A' && c <= 'Z'){ // if c is an uppercase letter
        numberOfLetters[c-'A']++;
    }
    c = textReader.read();
}
```

Using Macbeth, we end up with Table 12.2 of occurrences for each letter.

Using a different play will produce slightly different percentages. For instance, 12.3% of the letters in King Lear are es.

Definition 12.1.2. Our first definition of probability is that the probability of randomly choosing an element from a finite list is given by the relative frequency of that element in the list. We use the notation $P(x)$ for the probability of choosing x .

Example 12.1.3. An example is randomly choosing a letter from Macbeth. The probability that the chosen letter is an s is 0.064, or 6.4%, so $P(s) = 0.064$.

Our next goal is to get more refined information by looking at pairs of consecutive letters, not just single letters. The simplest way to handle this is to write a table in the format `int[26][26]` where we store in place 03, for instance, the number of occurrences of the pair ad. As a simple sanity check, note that almost all occurrences of q in English are followed by u, so when the first index is 16, all entries should be (close to) zero except when the second index is 20. Note also that each letter (except the first and last) is a part of two pairs. For instance, the pairs of “data” are da,at,ta.

Table 12.2: Occurrences and frequencies of characters in Macbeth

letter	number	percentage
a	6141	7.8
b	1505	1.9
c	2241	2.9
d	3379	4.3
e	9403	12.0
f	1723	2.2
g	1355	1.7
h	5324	6.8
i	4744	6
j	38	0.05
k	693	0.9
l	3335	4.2
m	2534	3.2
n	5078	6.5
o	6118	7.8
p	1014	1.3
q	123	0.16
r	4586	5.8
s	5009	6.4
t	7191	9.2
u	2611	3.3
v	667	0.8
w	1948	2.5
x	113	0.1
y	1645	2.1
z	15	0.02

Example 12.1.4. We will modify the above example, Example 12.1.1, to count pairs in addition to just occurrences. In the initialization above, we add our two-dimensional array

```
int [][] dobbelSample = new int [26][26];
```

To keep track of two characters at once, we also include a local `int` called `d` inside the loop.

```
while (c != -1){ // c = -1 means end-of-file
    int d = textReader.read();
    while (!(d >= 'a' && d <= 'z') || (d >= 'A' && d <= 'Z')){
        // so that d is a letter
        d = textReader.read();
        if (d == -1){ // d = -1 means end-of-file
            break;
        }
    }
    if (c >= 'a' && c <= 'z'){ // lowercase letter
        numberOfLetters[c-'a']++;
        if (d >= 'a' && d <= 'z'){
            dobbelSample[c-'a'][d-'a']++;
        }
        if (d >= 'A' && d <= 'Z'){
            dobbelSample[c-'a'][d-'A']++;
        }
    }
    if (c >= 'A' && c <= 'Z'){ // uppercase letter
        numberOfLetters[c-'A']++;
        if (d >= 'a' && d <= 'z'){
            dobbelSample[c-'A'][d-'a']++;
        }
        if (d >= 'A' && d <= 'Z'){
            dobbelSample[c-'A'][d-'A']++;
        }
    }
    c = d;
}
```

Since spaces and punctuation are ignored, there are some examples of pairs that appear in this list, but which will not appear inside a single word. To get a closer look at the results, there are in Macbeth 1505 bs. There are 155 bas, meaning that if we know that a given letter is a b, the probability that the next letter is an a is

$155/1505 = 0.103$. There is not a single occurrence of the pair bn .

Definition 12.1.5. Conditional probability is the probability of one occurrence given another. More precisely, pick at random a pair of consecutive elements from a finite list. The conditional probability that the second element is y given that the first is x , is equal to the proportion of all occurrences of the pair xy to all pairs starting with x . We use the notation $P(y|x)$ for the conditional probability of y occurring given that x has occurred.

Example 12.1.6. In Macbeth, the conditional probability $P(n|b) = 0$ whereas $P(a|b) = 0.103$.

This process can be repeated several times, so that we can get probabilities for what the next letter is based on the two previous letters, the three previous letters and so on. If you understood the Java code for generating the conditional probabilities $P(y|x)$, you will certainly see how to find $P(z|xy)$ (the probability that if we have seen xy the next letter is z) using an array of type `int [26] [26] [26]` of frequencies, and so on.

12.2 Permutations of letters and the Caesar Cipher

The simplest way to encrypt a text is to exchange each letter for another. To decrypt the text, just apply the same procedure backwards.

Definition 12.2.1. A *permutation* of a set is a bijection from the set to itself.

An example of a permutation of the set $\{a, b, c, d\}$ is the bijection

$$f(a) = c$$

$$f(b) = a$$

$$f(c) = d$$

$$f(d) = b$$

The inverse bijection f^{-1} comes from reading this table in reverse.

$$f^{-1}(a) = b$$

$$f^{-1}(b) = d$$

$$f^{-1}(c) = a$$

$$f^{-1}(d) = c$$

Using f to encode, the word *dad* is transformed to the nonsensical string *bc b*. Using f^{-1} to decode, the string *ac b* is transformed to the meaningful word *bad*. In order to crack such a code, we can try decoding a string using all possible permutations, and choose a meaningful decoded word or sentence. However, the number of permutations increases rapidly with the number of allowed characters. There are 24 possible permutations of the set $\{a, b, c, d\}$, listed here by what the four letters are sent to in order:

<i>abcd</i>	<i>abdc</i>	<i>acbd</i>	<i>acdb</i>	<i>adbc</i>	<i>adcb</i>
<i>bacd</i>	<i>badc</i>	<i>bcad</i>	<i>bcda</i>	<i>bdac</i>	<i>bdca</i>
<i>cabd</i>	<i>cadb</i>	<i>cbad</i>	<i>cbda</i>	<i>cdab</i>	<i>cdba</i>
<i>dabc</i>	<i>dacb</i>	<i>dbac</i>	<i>dbca</i>	<i>dcab</i>	<i>dcba</i>

In general, there are $n!$ permutations of a set with n elements, so for our 26 letter alphabet, there are

$$26! \approx 4 \cdot 10^{26}$$

possibilities to try. Obviously, we need a more sophisticated approach to cracking this cipher than simply trying all the possibilities in turn. The simplest answer is to compare the frequency of each letter to the reference frequency that we created in Example 12.1.1, and assume that the most common letter is the encrypted version of e, the second most common is t and so forth. As an interactive method, this usually works if the text is not too short. These types of problems are often encountered in recreational puzzles, where you in addition to frequencies should consider known words or phrases. With the means we currently have at our disposal, general permutations are a bit too difficult to leave to the computer. Let us make a simplification that will allow us to do everything automatically.

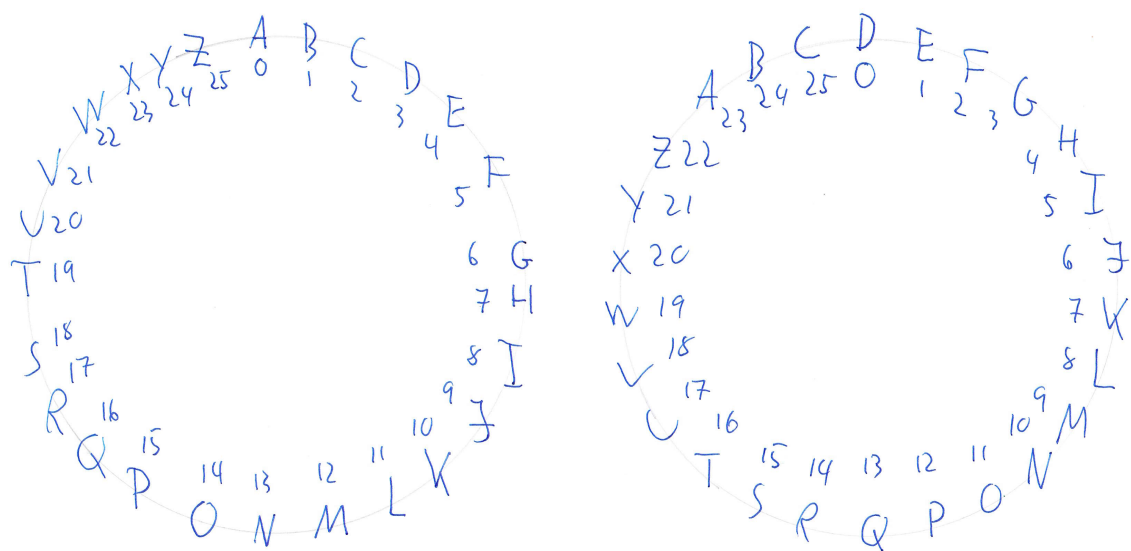
Example 12.2.2 (Caesar Cipher). Note that there is a simple way to make a permutation, simply by writing the letters a to z clockwise in a circle, and rotate a certain number of times. If we for instance rotate our circle three times counterclockwise as in Figure 12.1, we can associate numbers to the letters in our original text as shown on the left, and then associate letters to the numbers as shown on the right. Doing this consistently throughout the text leads to a simple encryption of the text.

This encryption is called the “Caesar cipher”, as Caesar apparently used it to encrypt messages. For instance the phrase “veni, vidi, vici” would be encrypted as “yhql, ylgf, ylfj”. The recipient of the secret message could then decrypt it going backwards, turning the meaningless phrase back into the original.

This procedure is simple, and we can let Java handle it easily. For simplicity, assume the texts are stored as `char[]`. Then the Java code to encrypt the message is

```
char[] ICame = { 'v', 'e', 'n', 'i', ' ', ' ', ' ', 'v', 'i', 'd', 'i', ' ', ' ',
    ' ', 'v', 'i', 'c', 'i' };
```

Figure 12.1: Caesar cipher



```

int rotate = 3; //we have chosen 3 steps
char[] ICameEncrypted = new char[ICame.length];
for (int i = 0; i < ICame.length; i++){
    if (ICame[i] >= 'a' && ICame[i] <= 'z'){
        ICameEncrypted[i] = (char)( 'a' + (ICame[i]
            - 'a' + rotate)% 26);
    }else{
        ICameEncrypted[i] = ICame[i];
    }
}

```

To decrypt the message, change only the integer `rotate` to $26 - 3 = 23$. Note that the code would not work if we used -3 instead, because Java's convention of using negative remainders in the modulo operation would yield incorrect decryption for letters near the end of the alphabet.

Our next task is cracking the Caesar cipher, in the sense that we will try to find out what the value of the variable `rotate` must be. For short texts, this is not always possible to solve completely, since the amount of data is too small to distinguish between possibilities. To give a simple example, if the encrypted word is *aybb*, we can list the 26 possible decryptions and see that only one, *kill*, is a sensible word. Similarly, there is only one possible decryption of *xcy* (try to find it). However, if the encrypted word is *wx*, there are at least two sensible words on the list, *no* and *hi*. There is no way to choose between them without having more information about the context.

Our task is even more difficult, since we will try to let the computer crack the cipher automatically, without the use of a dictionary. We had better assume that our starting text is of reasonable length.

12.3 Comparing frequencies

We have seen how to generate tables of frequencies of letters in text, and how to encrypt using the Caesar cipher. Now we will see how to let the computer crack the Caesar cipher. Imagine that we have intercepted an encrypted message. We want to perform the following steps:

- Count the frequencies of the letters.
- Compare the frequencies in the message to a reference sequence.
- Choose the most likely decryption based on this comparison.

The first part is easy; we have already computed frequencies from Macbeth, so we only input our message instead. The second part is more difficult. How do we compare frequencies? There are several ways to do this, but we will choose one that is widely applicable, based on χ^2 statistics (χ is the Greek letter “chi”).

Definition 12.3.1. Assume we know an expected table of relative frequencies (percentages) of letters based on extensive counting, stored in an array of type `double[26]` called `expectedFrequencies`. Similarly, we have an array `observedFrequencies` from counting in a given text. The χ^2 statistic of the observation is then

$$\sum_{i=0}^{25} \frac{(\text{observedFrequencies}[i] - \text{expectedFrequencies}[i])^2}{\text{expectedFrequencies}[i]} \quad (12.1)$$

Formally, there is an overall constant missing. For now, this is not important to us.

We will come back to this formula later, but for now we all we need is that this sum is small if there is a good match between observations and expectations, large if not. One can form the above sum either for the number of occurrences, or for percentages. Since the only difference will be an overall constant, this will not influence our use of this formula: we are going to find the minimum. The denominator is there to ensure that common letters and uncommon letters both contribute meaningfully to the sum.

The next step is to compute the sum for each possible rotation of the Caesar cipher. In general, it is easier to crack the code the longer the encoded text is.

Example 12.3.2. As an example, we will try to find the meaning of

etmxxuoaybmdqftqqfamegyyqdepnkftagmdfyadqxahqxmzpyadqfqybqdmfq

There are 63 characters here. We compute the frequency table, and for each possible rotation, we compare this to the table of expected frequencies using the χ^2 formula. Do not write down any part of the text until we have found the most likely decryption! Below are some highlights from Java code for this.

```
String encryptedText =
    "etmxxuoaybmdqftqqfamegyyqdepnkftagmdfyadqxahqxmzpyadqfqybqdmfq";
double[] MacbethTable = <frequency table from Macbeth>
double[] rotatedChiSquare = new double[26];
The array rotatedChiSquare will hold the values of  $\chi^2$  for each rotation.
for (int i = 0; i < 26; i++){
    String rotated = encryptCaesar(i, encryptedText);
    double[] rotatedFrequency = computeFrequency(rotated);
    rotatedChiSquare[i] = computeChiSquare
        (rotatedFrequency, MacbethTable);
}
```


The three methods called here are in essence already programmed. `encryptCaesar` was essentially written in Example 12.2.2. `computeFrequency` was essentially written in Example 12.1.1. `computeChiSquare` is an implementation of the formula in Equation (12.1).

We then let `decryptionRotation` be the index of the smallest element in `rotatedChiSquare`, and finally use the decryption algorithm, which in this case is equal to the encryption algorithm:

```
int decryptionRotation = <index of minimum in rotatedChiSquare>
String decryptedText = encryptCaesar(decryptionRotation, encryptedText);
System.out.println(decryptedText);
```

12.4 Sampling text

Now that we have seen how to compute frequencies of letters, and use this as a basis for probabilities, we can reverse the situation and ask for generation of texts that have a similar frequency of letters as our original texts. The simplest way to do this is to choose a random number between 0 and 1 and compare it to the cumulative probability from the frequency table. Cumulative probabilities are meaningful when our input has some natural order, like the order of the letters in the alphabet.

Definition 12.4.1. Starting from a given table of probabilities of letters of the alphabet, we define the cumulative probability function by having the value at a letter equal to the sum of all previous values. If we have a probability function P , then the cumulative probability function F is thus given by

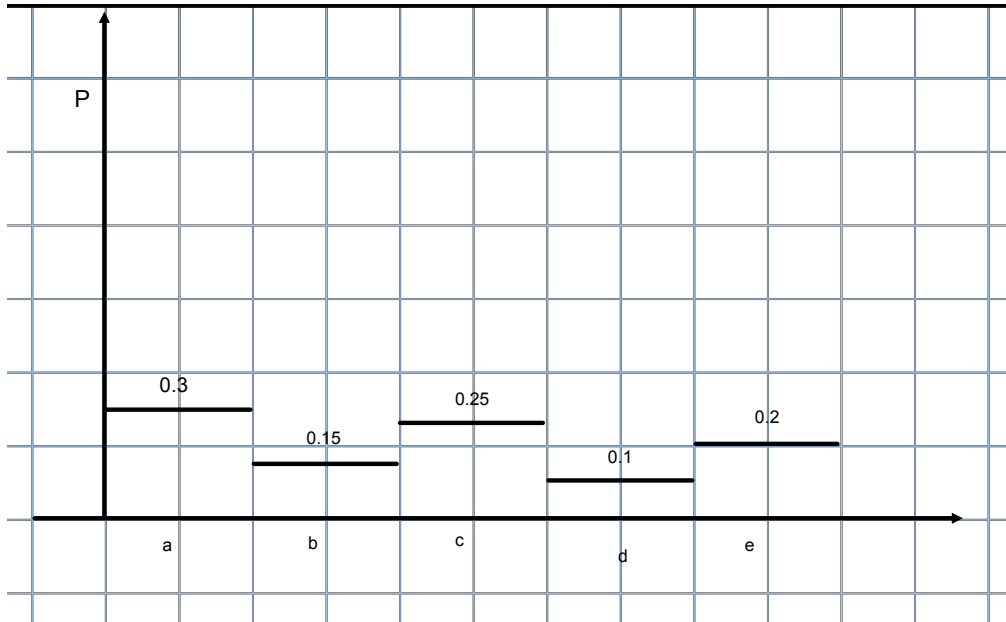
$$\begin{aligned} F(a) &= P(a) \\ F(b) &= P(a) + P(b) \\ F(c) &= P(a) + P(b) + P(c) \\ &\vdots \\ F(z) &= P(a) + P(b) + \cdots + P(z) \end{aligned}$$

Note that it is also possible to define F recursively, by adding the new value to the previous sum, so that for instance $F(m) = F(l) + P(m)$.

Example 12.4.2. To visualize this, we consider a simplified version with only five letters a, b, c, d, e , and the function P assigns 0.3, 0.15, 0.25, 0.1, 0.2 to these letters in order. Figure 12.2 shows this function. Then F is computed as follows:

$$\begin{aligned} F(a) &= P(a) &&= 0.3 \\ F(b) &= F(a) + P(b) &= 0.3 + 0.15 &= 0.45 \\ F(c) &= F(b) + P(c) &= 0.45 + 0.25 &= 0.7 \\ F(d) &= F(c) + P(d) &= 0.7 + 0.1 &= 0.8 \\ F(e) &= F(d) + P(e) &= 0.8 + 0.2 &= 1 \end{aligned}$$

Figure 12.2: A probability distribution



P and F are shown together in Figure 12.3.

To get a random letter, choose a random number (uniformly distributed) between 0 and 1. In Java, this is most easily done by calling `Math.random()`. If the number is in the interval $[0, F(a))$, choose the letter a . If it is in the interval $[F(a), F(b))$, choose b , and so forth.

Example 12.4.3. We choose randomly a number between 0 and 1. Continuing with the numbers from Example 12.4.2, if the number is less than 0.3, we get an a . If it is between 0.3 and 0.45, we get a b , and so forth. This procedure is illustrated in Figure 12.4.

Here is Java code to generate the cumulative probability function, as well as choosing a random letter in this way, given a `double[26]` called `frequency`.

```
// compute the cumulative probability function
double[] cumulativeFrequency = new double[26];
cumulativeFrequency[0] = frequency[0];
for (int i = 1; i < 26; i++){
    cumulativeFrequency[i] = cumulativeFrequency[i-1] + frequency[i];
}
//get a random letter
double randomNumber = Math.random();
```

Figure 12.3: Cumulative probability

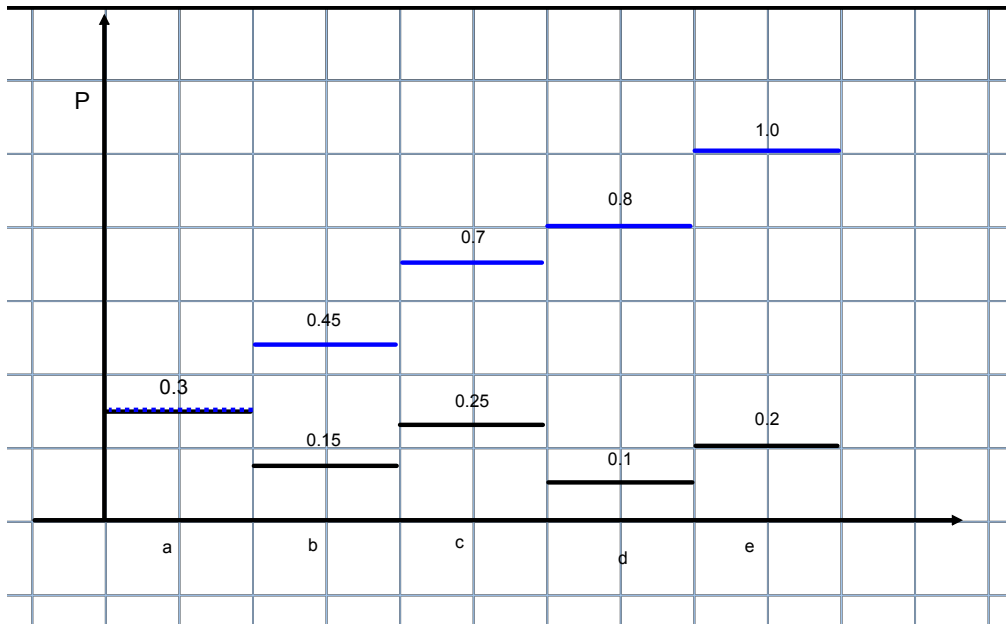
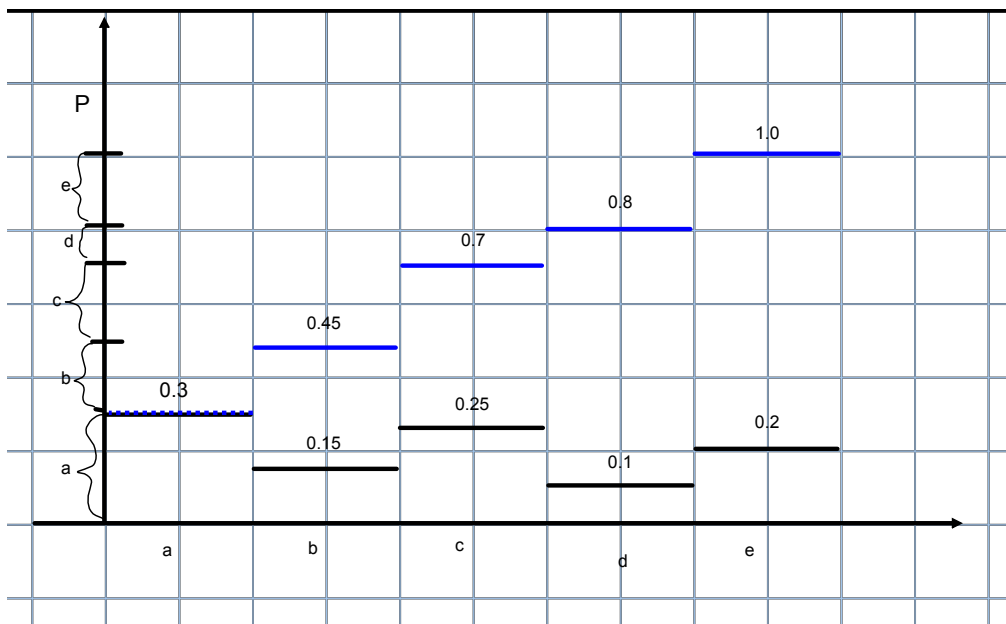


Figure 12.4: Sampling using cumulative probability



```

int whatNumber = 0;
while (cumulativeFrequency[whatNumber] < randomNumber){
    whatNumber ++;
}
char randomLetter = (char) ('a' + whatNumber);

```

Note that the word “random” now has at least two different meanings. The random number is equally likely to be in any part of the interval $[0, 1)$, but the random letter is more likely to be e than x. Running the above code some 50 times, we get something like

```
rrwsmadadiiecerfyttaehrhwstocmrtbimnohrhsahselnrnu
```

The result from randomly choosing letters according to frequencies obviously does not yield real text, but by simply computing frequencies, it is unlikely that the computer would recognize it as fake. However, if we take into account the conditional probabilities, the text seems less likely. How can we improve our fake text to consider pairs? The procedure is simple. Choose the starting letter as above, but from then on choose a random letter using the two-dimensional table of conditional probabilities as a starting point, with first index corresponding to the previous letter.

Example 12.4.4. We start by producing a random letter `char c` according to frequency, as above. Then we choose the next letter by the same procedure, but using the part of `dobbelSample` from Example 12.1.4 where the first index is given by `c-'a'`. Then we repeat as many times as we would like. In pseudocode,

```

randomTextFromPairs[0] = <random letter as above>
for i from 1 to 50
    d = <random letter obtained from dobbelSample[c-'a']>
    randomTextFromPairs[i] = d
    c = d
end

```

Note that the definition of `d` requires us to compute cumulative frequencies for each of the 26 arrays `dobbelSample[i]`, this can be done in advance. Here is a sample obtained from the procedure above:

dimeretbeiwinoobllmofhehwngoomasagsethobelesdswoud

By also taking into account triples or quadruples, we get

isesirrookedorletheedowlesereakingleengsaftenenderpi

and

mselfcountmacbethetheothatwoodtofprophecawdormybrownfat

In some intuitive way, the text more and more resembles English. Imagine this text presented to someone who is starting to learn English! Taking care of spaces leads to better results, but the coding is somewhat more convoluted.

Comment 12.4.5. C.E. Shannon in *A mathematical theory of communication*, 1949, introduced notions for the actual informational content of data. The idea is that there is less extra information if the next character of a text is the most likely character than if it is less likely. To illustrate his ideas, he produced nonsensical texts by the procedure we used above in the same way. The article is also famous for introducing the notion of *bit*.

12.5 Exercises

Exercise 12.5.1. Find the only possible decryption of *xcy* using the Caesar cipher.

Exercise 12.5.2. For comparison, write a simple programme to choose the rotation such that *e* is the commonest letter. This will often work just as well as the more elaborate algorithm presented later in the text.

Exercise 12.5.3. Prove that there are $n!$ permutations of a set with n elements using mathematical induction.

Exercise 12.5.4. We will try to understand how much information there really is in the frequency table of a text.

(a). Compute χ^2 statistics for other plays of Shakespeare, using *Macbeth* as a reference. Are there differences between tragedies and comedies?

(b). It is sometimes possible to distinguish between authors based on frequency tables. Compute χ^2 statistics for texts by other authors, using *Macbeth* as a reference. Search the net for texts in ascii/utf-8 format. A good place to start is <http://www.gutenberg.org>.

Exercise 12.5.5. Crack the following text, which is encrypted using the Caesar cipher:

uibpmuibqkaquwzmnvcbpivxzwoziuuqvo

Exercise 12.5.6. Alter the counting and sampling procedures to take spaces into account. This will yield more interesting results, but is slightly difficult because spaces appear in various guises.

(a). Ignore punctuation (colon, comma, point, apostrophes and so on), but let space include any of the standard whitespace characters (space, line feed, carriage return, tabulator). When several whitespace characters appear consecutively, count the result as only one space. As a test, count the number of spaces in *Macbeth*.

(b). Include space in the above sense, and sample letters (and spaces) randomly, and then using pairs, triples and quadruples. How often do you get real words separated by spaces?

Exercise 12.5.7. A message has been encrypted using the Caesar cipher. The encrypted text is 'tmdde'.

(a). List the 26 values coming from the χ^2 -comparison between frequency tables for the 26 possible decryptions, starting from the frequency table coming from Macbeth.

(b). List the 26 possible decryptions. Determine what the correct decryption is.

(c). What is the explanation for the strange connection between these two lists?

Exercise 12.5.8. Modify the code for frequency and for sampling to include some other interesting characters. Here are some ideas:

- Include the digits 0 to 9 .
- Include the Norwegian letters æ, ø, å, Æ, Ø, Å.
- Include punctuation.

Exercise 12.5.9. In this exercise, two students should cooperate. Student A writes down a sentence, and tells student B the first character. Then B guesses the next. For each character, write down how many guesses B needed to find the correct one. In the end, all ones in the list represent characters that were in some sense unnecessary.

Chapter 13

A very short introduction to MATLAB

With some background in programming, it is likely that the simpler aspects of MATLAB are relatively easy to master. This chapter is meant as a very short introduction, guiding you to try some MATLAB programming but leaving most of the details to your own exploration of the MATLAB documentation. We will perform a couple of computations that are of independent interest, but are included here because they are very simple to see in MATLAB.

13.1 Getting started with MATLAB

Install MATLAB. When you start MATLAB, you will first see the interpreter window in the middle. Arithmetic expressions can be evaluated, such as $2-3$ or 14^3 . Common functions are evaluated in the same way, as in `sin(2)` or `exp(-4)`.

Open the editor, for instance by pressing New Script. Then we can write several lines of code, and run it by pressing F5. This is the preferred way of using MATLAB, using the interpreter sparingly.

MATLAB has a very simple way to create evenly spaced arrays of numbers. Writing

```
x = 1:4  
y = 1:0.1:4
```

is a short way of defining

```
x(1) = 1  
x(2) = 2  
x(3) = 3
```

```

x(4) = 4
y(1) = 1
y(2) = 1.1
y(3) = 1.2
<and so on>
y(30) = 3.9
y(31) = 4

```

Note that arrays are indexed starting from 1, and that assignments are performed using the equality sign.

To input a matrix, write the matrix row by row, elements separated by commas, rows separated by semicolons. The whole list should be enclosed in brackets. For instance,

```
M = [1,2,3;5,6,7;8,9,9]
```

produces the matrix

$$M = \begin{pmatrix} 1 & 2 & 3 \\ 5 & 6 & 7 \\ 8 & 9 & 9 \end{pmatrix}$$

Since indices start from 1, $M(2,3)$ is the number 7.

Functions can be applied to arrays. For instance, `sin(x)` produces the array

```

ans =

    0.8415    0.9093    0.1411   -0.7568

```

and similarly for `sin(M)`. Arithmetic operations are as a rule matrix operations. For instance, `x*x` will produce an error:

```

Error using *
Inner matrix dimensions must agree.

```

To impose elementwise operations, use a point in front of the multiplication symbol. For instance, `x.*x` returns

```

ans =

     1     4     9    16

```

Simple programming elements like `for` and `if` are present as in


```

for i = 1:8
    for j = 0:0.2:1
        if (sin(i) < j)
            j - sin(i)
        end
    end
end
end

```

Note that `end` is used as a common codeword for ending a block.

If you don't want MATLAB to print the result of a calculation, end the statement with a semicolon.

Sometimes MATLAB returns strings even when you would expect numbers. For instance, the command `dec2bin` turns a decimal number into a binary number, returned as a *string*.

```

bin2 = dec2bin(2)
2*bin(2)

```

returns first 10, which is indeed the binary representation of 2, but the second command returns 98 96. This is two times the ascii representations of the characters '1' and '0'.

MATLAB has many shortcuts for efficient programming of mathematics. We will only need a small part of the functionality that MATLAB offers, so that the short introduction above will provide enough information at least to be able to read the relevant MATLAB code.

13.2 Simple plotting in MATLAB

MATLAB has ready-to-use methods for plotting in two and three dimensions. If we want to plot for instance $f(x) = \sin x$ from $x = -2$ to $x = 12$, the way to achieve it is

```

x = -2:0.1:12;
y = sin(x);
plot(x,y)

```

The size of the steps, here 0.1, must be chosen so that the graph looks about right. Conversely, we could choose the number of steps and set the size of the steps accordingly:

```

N = 100;
a = -2;

```

```

b = 12;
h = (b-a)/N;

x = a:h:b;
y = sin(x);
plot(x,y)

```

The command `plot` works on any lists of numbers of the same length, not only for functions as above. For instance

```

x = [0,1,1,0,0];
y = [0,0,1,1,0];
plot(x,y)

```

draws a square. If we want to draw only points, not lines between them, we use the command `scatter`. This could be used, for instance, to draw measured values which might be studied using regression.

This code block plots 100 random points in the unit square:

```

n = 100;
for i = 1:n
    x(i) = rand(1);
    y(i) = rand(1);
end
scatter(x,y)

```

Example 13.2.1. The simplest way to get a three-dimensional plot, is to choose a grid in the xy -plane and compute the values of a function on the grid. Here is a simple example using $f(x, y) = \sin(xy)$.

```

[x,y] = meshgrid(-1:0.1:2, 1:0.1:4); % -1 < x < 2, 1 < y < 4
z = sin(x.*y);
surf(z)

```

Try also `mesh` instead of `surf`.

If we have a parametric curve, say $x(t) = t$, $y(t) = t^2$, $z(t) = t^3$, we can plot it using `plot3`:

```

t = -2:0.05:2;
x = t;
y = t.^2;
z = t.^3;
plot3(x,y,z)

```

Try also `scatter3` instead of `plot3`.

MATLAB also has simple features for producing animations. Here is some simple code for producing a standing wave.

```
x = -15:0.2:15;
for i = 1:63
    y = 5 * cos(i/10) * sin(x);
    plot(x,y)
    axis equal
    M(i) = getframe;
end
movie(M,5)
```

Julia sets

Complex numbers are represented by using `i` as the imaginary unit, as in `z = 0.1 + 1.2*i`. Ordinary operations can then be performed as for real numbers. The norm (or modulus) of a complex number is called simply by `norm(z)`.

As an example of plotting using complex numbers, we will use the `image` command, which produces an image based on rgb values stored in a matrix. The *Julia set* of a function consists of the set of points that don't go to infinity when the function is applied repeatedly.

Example 13.2.2. Let $f(z) = z^2$. To determine if $1/2$ is in the Julia set, consider the sequence produced by applying f several times:

$$\begin{aligned} f(1/2) &= 1/4 \\ f(1/4) &= 1/16 \\ f(1/16) &= 1/256 \\ &\vdots \end{aligned}$$

The sequence $\{\frac{1}{2}, \frac{1}{4}, \frac{1}{16}, \dots\}$ doesn't go to infinity (it goes to zero), so 0.5 is in the Julia set.

Similarly, i is in the Julia set because the sequence $\{i, -1, 1, 1, 1, \dots\}$ doesn't go to infinity.

2 is not in the Julia set, since $2, 4, 16, 256$ does go to infinity.

To approximate the Julia set, we choose a matrix of points and determine whether the sequence of each one gets large in a finite number of steps. If it does, we assume the point is outside the Julia set. If it doesn't, we assume the point is in the Julia set.

Example 13.2.3. We will consider the function

$$f(z) = z^2 + \mu/z^2 = z^2 + (-0.02 + 0.01i)/z^2$$

Feel free to try other values of μ . Let us stop after a maximal number of iterations, say 50, and guess that the sequence goes to infinity if at any time the norm is greater than 2, say. We compute the number of iterations to come over this threshold, and store it. This number can be used to get colours in the image; this is not important. This is done for each grid point where we choose approximately a million points in the square where both the real and the imaginary part are between -1 and 1. Here is a complete listing:

```
mu = -0.02 + 0.01*i;
N = 500;
maxIterations = 50;
J = zeros(2*N+1); % a square matrix of size 2N+1
for x = -N:N
    for y = -N:N
        z = x/N + i*y/N;
        iterations = 0;
        while (norm(z) < 2 & iterations < maxIterations)
            z = z^2 + mu/z^2;
            iterations = iterations + 1;
        end
        J(x+N+1,y+N+1) = iterations*255/50;
    end
end
image(J);
```

13.3 Sound in MATLAB

MATLAB has a simple way to record sound and play sound. At the same time, we can see the underlying numerical representation of the sound.

Example 13.3.1. An A with frequency 440 hertz is produced like this:

```
t = 1:8192; %sample rate 8192 is default
noteA = sin(2*pi*440*t/8192);
sound(noteA)
```

Putting together several sounds in this manner produces some resemblance of music.

```

t = 0:(1/8192):1; %sample rate 8192
C = sin(2*pi*261.6*t);
D = sin(2*pi*293.7*t);
E = sin(2*pi*329.6*t);
F = sin(2*pi*349.2*t);
Helt = 0:(1/8192):4; % 4 times longer
CHel = sin(2*pi*261.6*Helt);
sover = [C,C,C,E,D,D,D,F,E,E,D,D,CHel];
sound(sover)

```

Typing `plot(sover)` doesn't produce anything meaningful. Try it! See for instance

<http://liutaiomottola.com/formulae/freqtab.htm>
for frequencnies of musical notes.

To get something more interesting, either import a file or record something.

Example 13.3.2. `handel.mat` is a MATLAB sample file. Importing it by typing `load handel` gives two variables, but we will only consider the one named `y`. Plot it using `plot(y)`. Play it using `sound(y)`. Note that `y` is a table, which we can think of as a sampled function of time with 8192 entries per second. To get an approximation of the derivative of this function, consider the definition

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

The smallest possible value we have for h is $1/8192$, which means that we can approximate the derivative by subtracting two neighbouring values in the table `y`, and scale it by dividing by $h = 1/8192$. However, it turns out that we should let MATLAB handle scaling to get to the sound, so in the end we will actually approximate a multiple of the derivative by running this code:

```

for i=1:(length(y)-1)
    z(i)=y(i+1)-y(i);
end
sound(z)

```

Plot and listen to `y` and `z`, and try to understand the similarities and differences.

Example 13.3.3. Recording sound in MATLAB is a good way to get interesting examples. Using default values, so we don't have to specify anything, we can record ten seconds of sound from the computer's microphone like this:

```
numberOfSeconds = 10;
recorder = audiorecorder;
recordblocking(recorder, numberOfSeconds);
```

To play it back, use `play(recorder)`. To access the numbers behind the recording, and plot it as above, use

```
y = getaudiodata(recorder);
plot(y)
```

13.4 User-defined functions

To define a new function in MATLAB, click on New Function, and fill in the information in the header. Then save it as a .m file with the same name as the function. When MATLAB tries to understand a function call, it will look for .m files in its default directory (or any other folder in its path). A function can have many inputs and many outputs.

Here is an example, finding the maximum and the minimum in a vector.

```
function [ max, min ] = MaxMin( x )
%MaxMin finds the maximum and the minimum in the vector x.
%   Jon Eivind Vatne, August 2013
max = x(1);
min = x(1);
for i = 2:length(x)
    if (x(i) > max)
        max = x(i);
    end
    if (x(i) < min)
        min = x(i);
    end
end
end
```

Note that the function returns two values, which are the values of the local variables `max` and `min` when the execution of the code in the function is finished. There is no `return` statement.

Save the file as `MaxMin.m`. Here is how to call the function (from a script, another function or directly in the interpreter):

```
prices =[54,12,46,23,72,24,46,2,33,53];  
[maxPrice, minPrice] = MaxMin(prices)
```

There is a large community of MATLAB users worldwide, and many interesting functions are available as .m files for instances at

<http://www.mathworks.com/matlabcentral/fileexchange/>

13.4.1 Anonymous functions and function handles

For simple functions, it is sometimes easier to define them in a different way, inside a script. This is called an anonymous function, and the resulting structure has the `function_handle` data type. For instance, consider a function you want to perform some operations on:

$$f(x) = 2x^2 - \sin(1/\sqrt{1+x^2})$$

Instead of creating a function file for this f , MATLAB allows to define it like this:

```
f = @(x) 2*x.^2 + 1./\sqrt{1+x.^2}
```

Depending on how you plan to use it, it is possible to define it without points before the operations. One practical benefit of this definition is that `f` can now be passed as input to other functions, both user-defined and built-in MATLAB functions. It is also possible to create function handles for other functions, without specifying arguments. For example, we can create a function handle for the sine function by writing

```
f = @sin
```

Later we will use function handles to specify statistical experiments, and then compute statistical information in functions taking the function handle as input.

Chapter 14

Sampling from discrete distributions

We have seen how to sample letters based on given frequency tables. Now we proceed to give some standard classes of frequencies that are particularly simple or interesting. Some examples are given in Matlab, because Matlab easily allows us to produce graphical descriptions of the results. Many other programming languages and software systems have similar capabilities.

14.1 Expectation and variance

We will look consistently at two important measures associated to finite sequences of numbers. The first is the mean, which is the sum divided by the number of elements. The other is the empirical variance, which is the sum of squares of distances between the numbers and the mean, divided by the number of elements minus one. When we use specific rules to generate numbers, we similarly associate two numbers to the rule. The expected value is the value we expect that the mean of numbers generated by the same rule many times will have. The variance is what we expect the empirical variance will be in the same manner.

Definition 14.1.1. Let $X_1, X_2, \dots, X_n$ be a finite sequence of numbers. The *mean* is

$$\bar{X} = \frac{X_1 + X_2 + \dots + X_n}{n} = \frac{1}{n} \sum_{i=1}^n X_i.$$

The *empirical variance* is

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2.$$

The *standard deviation* is $S = \sqrt{S^2}$.

Here the exponent 2 in S^2 is there to remind us that we use squares in the definition of this number, and that we need to compute the square root if we want to compare to the numbers X_i .

Example 14.1.2. Given numbers $X_1 = 3, X_2 = 4, X_3 = 6$. The *mean* is

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i = \frac{1}{3}(3 + 4 + 6) = \frac{13}{3} \approx 4.33.$$

The variance is

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2 = \frac{1}{3-1} \left(\left(3 - \frac{13}{3}\right)^2 + \left(4 - \frac{13}{3}\right)^2 + \left(6 - \frac{13}{3}\right)^2 \right) \approx 2.33.$$

The standard deviation is $S \approx \sqrt{2.33} \approx 1.53$. It is customary to write equality signs instead of approximation signs in this setting, so we write $S = 1.53$. Also, to help us remember that S^2 is a squared identity, we usually also write $S^2 = 1.53^2$ rather than $S^2 = 2.33$.

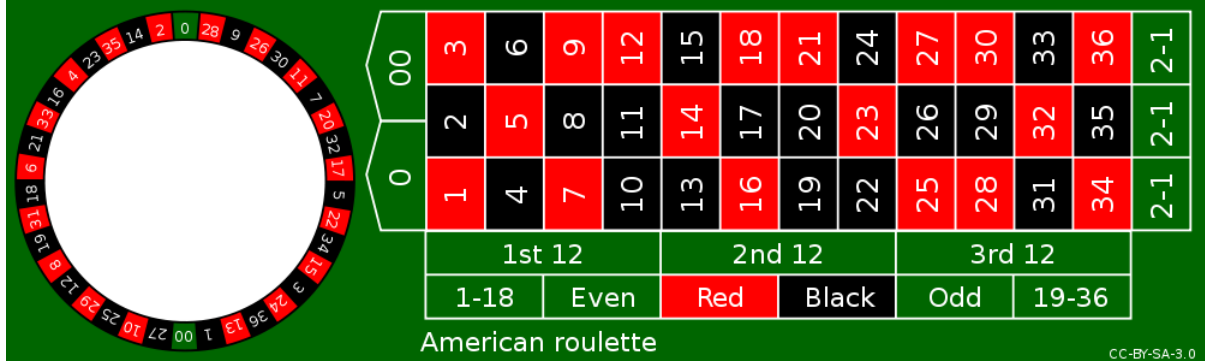
Comment 14.1.3. The mean represents a typical value in some sense. Usually, many of the numbers we start with will be close to the mean, but not always. For example, in a class of students the mean age is typically around 22. However, this could also be the mean age in a family of four, with two small children and two parents close to 40. Obviously, the mean is a better representative of typical values in the first case than in the second.

The variance and the standard deviation are measures of the spread of numbers. If the mean is a good representative for typical values, these numbers will be relatively small. If the values are far from the mean, the variance will be relatively large.

Comment 14.1.4. The variance is usually easier to compute directly than the standard deviation. On the other hand, the standard deviation is usually the preferred measure of spread in further computations. Consider again the numbers in Example 14.1.2. If these numbers represent measured distances in kilometres, the mean is also measured in kilometres. Since the differences are squared when computing variance, the unit is kilometres squared! The standard deviation is again measured in kilometres, so that it can be used in formulas together with the mean or the original measurements.

Let us now turn around, and consider the situation where we have a rule for picking random numbers. This rule can be expressed by listing a number of possible values and the probability that each value is chosen. If we call the rule X and the values x_i , we write $P(X = x_i)$ for the probability that the random number is x_i .

Figure 14.1: Roulette (American version), from Wikimedia commons



Example 14.1.5. In roulette, see Figure 14.1, assume that you bet 10 on red and 15 on odd numbers. The possible winnings that can be achieved are 0, 10, 15, 25. The probability for red is $18/38$, the probability for odd is $18/38$, the probability for both is $9/38$ and the probability for neither is $11/38$ (0 and 00 are neither red, black, odd nor even). The table of probabilities for the winnings, X , is thus

$$\begin{aligned} P(X = 0) &= 11/38 \\ P(X = 10) &= 9/38 \\ P(X = 15) &= 9/38 \\ P(X = 25) &= 9/38 \end{aligned}$$

There are some slight variations in the layout of roulette tables, so if you look at a different example there could be minor changes to the numbers in this example.

Definition 14.1.6. An X that gives certain values with certain probabilities is called a (discrete) stochastic variable. It can be described for instance by giving a table of its probabilities or by explaining the rule that X uses to produce the random numbers.

Definition 14.1.7. The expectation $E(X)$ of X is

$$E(X) = \sum_x x \cdot P(X = x).$$

Example 14.1.8. Continuing with roulette from Example 14.1.5, we see that the expectation is

$$E(X) = 0 \cdot \frac{11}{38} + 10 \cdot \frac{9}{38} + 15 \cdot \frac{9}{38} + 25 \cdot \frac{9}{38} = \frac{450}{38} = 11.84.$$

What does this number signify? The stakes are also returned when you win. If we were to make another stochastic variable $Y = \text{net winning}$, which is winnings minus

stakes, the values would be -25 for losing both bets, -5 for winning on red, losing on odd, 5 for winning on odd and losing on red, and 25 for winning on both. We would then get the expectation

$$E(Y) = -25 \cdot \frac{11}{38} - 5 \cdot \frac{9}{38} + 5 \cdot \frac{9}{38} + 25 \cdot \frac{9}{38} = -\frac{50}{38} = -1.32.$$

This means that on average, you lose 1.32 each time you play using this particular bet. Note that it is more difficult to give a meaningful description of what $E(X)$ actually mean: Since the stakes are returned as well, you get twice the stipulated value of X . Thus the return is $2 \cdot 11.84 = 23.68$ from an investment of 25. We can write this as $E(2X) = 2 \cdot E(X) = 23.68$. Again we see that you lose $25 - 23.68 = 1.32$ each time you play, on average.

Definition 14.1.9. The *variance* of a stochastic variable X is

$$\text{Var}(X) = \sum_x (x - E(X))^2 \cdot P(X = x).$$

The *standard deviation* of X is $\sqrt{\text{Var}(X)}$.

As for empirical variance, it is customary to write $\text{Var}(X)$ as a square.

Example 14.1.10. The variance of X and Y above are computed as

$$\begin{aligned} \text{Var}(X) &= (0 - \frac{450}{38})^2 \cdot \frac{11}{38} + (10 - \frac{450}{38})^2 \cdot \frac{9}{38} + (15 - \frac{450}{38})^2 \cdot \frac{9}{38} + (25 - \frac{450}{38})^2 \cdot \frac{9}{38} = 9.21^2 \\ \text{Var}(Y) &= (-25 + \frac{50}{38})^2 \cdot \frac{11}{38} + (-5 + \frac{50}{38})^2 \cdot \frac{9}{38} + (5 + \frac{50}{38})^2 \cdot \frac{9}{38} + (25 + \frac{50}{38})^2 \cdot \frac{9}{38} = 18.41^2 \end{aligned}$$

The variance of Y is higher since its values are more spread out than the values of X . If we were to compute $\text{Var}(2X)$, similarly to how we computed $E(2X)$ above, we would also get $\text{Var}(2X) = 18.41^2$. This is not a coincidence, but is a formal consequence of properties of variance. See the appendix to this section.

In the rest of this chapter, we will look at various examples of rules where we can implement the rules on the computer to get many samples, and give a theoretical explanation of the rules as well. Then we can see that the mean and the expectation are connected, and that the two meanings of variance are connected.

Appendix: Properties of expectation and variance

For convenience, some properties of the functions E and Var are collected here.

Let X be a stochastic variable, a and b are real numbers.

$$E(aX + b) = aE(X) + b \tag{14.1}$$

$$\text{Var}(aX + b) = a^2 \text{Var}(X) \quad (14.2)$$

These are the two properties used in Example 14.1.8 and Example 14.1.10.

There is also a more general formula, related to a function applied to a stochastic variable:

$$E(g(X)) = \sum g(x)P(X = x) \quad (14.3)$$

This is not as directly applicable as the two formulas above, but can be used to deduce them.

If Y is a second stochastic variable, then

$$E(X + Y) = E(X) + E(Y). \quad (14.4)$$

Similarly, if $X_1, \dots, X_n$ is a sequence of variables, we have that

$$E(X_1 + \dots + X_n) = E(X_1) + \dots + E(X_n). \quad (14.5)$$

We say that stochastic variables are *independent* if values of one (or more) have no bearing on others. Let X and Y be two independent stochastic variables. Then

$$\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y). \quad \text{NB: Only for independent variables.} \quad (14.6)$$

Similarly, if $X_1, \dots, X_n$ is a sequence of independent variables, we have that

$$\text{Var}(X_1 + \dots + X_n) = \text{Var}(X_1) + \dots + \text{Var}(X_n) \quad \text{NB: Only for independent variables} \quad (14.7)$$

Example 14.1.11. Let $X_1, \dots, X_n$ be independent stochastic variables with the same probabilities for all results (for instance n different dice tossed). Let μ be the common expectation, and σ^2 the common variance, of these variables. Let also $\bar{X} = (X_1 + \dots + X_n)/n$ be the average of them. Then

$$E(\bar{X}) = \frac{\mu + \dots + \mu}{n} = \mu \quad (14.8)$$

and

$$\text{Var}(\bar{X}) = \frac{\sigma^2 + \dots + \sigma^2}{n^2} = \frac{\sigma^2}{n} \quad (14.9)$$

Example 14.1.12. We want to prove that the variance has a different description, as

$$\text{Var}(X) = \sum x^2 P(X = x) - (E(X))^2 \quad (14.10)$$

By definition,

$$\begin{aligned} \text{Var}(X) &= \sum (x - E(X))^2 P(X = x) \\ &= \sum (x^2 P(X = x) - 2xE(X)P(X = x) + (E(X))^2 P(X = x)). \end{aligned}$$

Since the summation index x does not appear in $E(X)$, we get:

$$\begin{aligned}\sum -2xE(X)P(X=x) &= -2E(X)\sum xP(X=x) \\ \sum (E(X))^2P(X=x) &= (E(X))^2\sum P(X=x)\end{aligned}$$

Now $\sum xP(X=x) = E(X)$ and $\sum P(X=x) = 1$, so

$$\begin{aligned}\text{Var}(X) &= \sum (x^2P(X=x) - 2xE(X)P(X=x) + (E(X))^2P(X=x)) \\ &= \sum x^2P(X=x) - 2E(X) \cdot E(X) + (E(X))^2 \cdot 1 \\ &= \sum x^2P(X=x) - 2(E(X))^2 + (E(X))^2 \\ &= \sum x^2P(X=x) - (E(X))^2.\end{aligned}$$

This is what we wanted to prove.

14.2 Biased coin flips - random walks

A very simple rule for producing a number is to say that 1 should be produced with a given probability p , and 0 with a given probability $1-p$. No other values are allowed. For historical reasons, this is sometimes called a *Bernoulli* trial. 1 represents success, 0 failure. If we want to model the flip of a coin, we could assign 1 to heads, 0 to tails and use $p = 0.5$. If the coin were *biased*, meaning that the probability is not the same for both results, we could use a different p .

As long as we have a random generator of some kind, it is easy to set up simulations of this sort. In MATLAB, the `rand`-function returns a number between 0 and 1. Thus, we can get results this way, assuming that $p = 0.8$ is the value we want.

```
p = 0.8;
t = rand; % random number between 0 and 1
if t < p
    % success
end
```

In Java, we can use `Math.Random()` in the same way.

Using one or the other, we can generate many values by looping. Counting the number of successes in n trials ought to give a result so that

$$\frac{\text{number of successes}}{n} \approx p.$$

Running

```

p = 0.8;
n = 10000;

numberOfSuccesses = 0;
for i = 1:n
    t = rand;
    if t < p
        numberOfSuccesses = numberOfSuccesses + 1;
    end
end
numberOfSuccesses

```

a few times gave the results 8025, 8015, 8018, 7986, 8055, 8013.

Random walks

One way to get a nice picture of what happens when we repeat a Bernoulli trial many times is to move a point based on it. We will set up this trial in the plane, so that both the x and the y coordinates are changed in a random manner. At each step, there is a fixed probability 0.5 that each coordinate will increase by one; otherwise it will decrease by one. We start at the origin. Plotting the result is quite easy e.g. in MATLAB, so we will use MATLAB for the computations. It is instructive to change one or both of the values 0.5 that appear in code below.

```

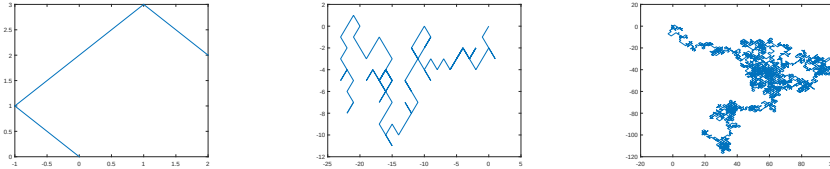
n = 1000 % number of iterations

x(1) = 0; % Our first point is at the origin
y(1) = 0;

for i = 2:n
    t = rand; % will determine change in x
    if t < 0.5
        x(i) = x(i-1) + 1;
    else
        x(i) = x(i-1) - 1;
    end
    t = rand; % will determine change in y
    if t < 0.5
        y(i) = y(i-1) + 1;
    else
        y(i) = y(i-1) - 1;
    end
end

```

Figure 14.2: Random walks with 5, 100 and 4000 steps



```

end
end
plot(x,y)

```

The resulting graph is an example of what is called a *random walk* for obvious reasons.

Example 14.2.1. Test runs of random walks with $n = 5$, $n = 100$ and $n = 4000$ produced the three results in Figure 14.2.

If we perform this operation many times, and look at the ending point (which is not easy to see in the graph), what will the average coordinates be?

Let $X(n)$ be the stochastic variable that returns the x -coordinate after $n - 1$ steps (the end point after the code above is run, $n - 1$ steps since MATLAB starts indexing at 1). The average x -coordinate is close to the expectation of $X(n)$, which we can compute by thinking about how $X(n)$ comes from $X(n - 1)$.

$$X(n) = \begin{cases} X(n-1) + 1 & \text{with probability } 0.5 \\ X(n-1) - 1 & \text{with probability } 0.5 \end{cases}$$

This means that the expectation

$$E(X(n)) = 0.5(E(X(n-1) + 1) + 0.5(E(X(n-1) - 1) = E(X(n-1))$$

Since the expectation does not change from one iteration to the next, it must be equal to the starting point, which is 0. Similarly, the average y -coordinate is 0, which can be computed as $E(Y(n))$.

If we perform this operation many times, and look at the ending point, what will the average distance from the origin be?

The distance from the origin is $\sqrt{E(X^2) + E(Y^2)}$. We first compute $E(X(n)^2)$.

$$E(X(n)^2) = 0.5(E(X(n-1) + 1)^2 + 0.5(E(X(n-1) - 1)^2 = E(X(n-1)^2) + 1$$

Since the value increases with 1 in each iteration, and we start at the origin, we get that $E(X(n)^2) = n - 1$. Thus the average distance is $\sqrt{2(n-1)}$.

14.3 Drawing randomly from a finite set

We will now look at two ways of drawing random elements from a finite set with a special subset.

Example 14.3.1. One way to draw a random integer between 1 and 6 is to roll a die. If we roll the die again, it is possible to get the same result as in the first trial. We can ask for the probability of getting a certain number of fives or sixes (the special subset) when the die is rolled n times. The probability distribution obtained in this case is an example of a binomial distribution.

Another way to draw a random integer between 1 and 6 is to write each number on a piece of paper, and pick one with your eyes closed. If you pick a second piece, without replacing the first, you will get two different elements. We can ask for the probability of getting zero, one or two of the numbers 5, 6 in this way. This probability distribution is an example of a hypergeometric distribution.

Let us be more precise about the setup, and see if we can understand the situation through computer simulation.

Binomial distribution

We let X be the stochastic variable that counts the number of successes in a trial of n independent Bernoulli trials with the same p . We write

$$X \sim \text{bin}(n, p)$$

in this situation, where *bin* is short for *binomial distribution*. This name will be explained later. To simulate one such trial, we can use this MATLAB code.

```
p = 0.8 % probability of success
n = 14 % the number of trials

numberOfSuccesses = 0;

for i = 1:n
    t = rand;
    if (t < p)
        numberOfSuccesses = numberOfSuccesses + 1;
    end
end
```

Since we are going to do many experiments with varying parameters, it is convenient to wrap this in a user-defined function in MATLAB. The code below should be saved in a file called `BinomialDistribution.m`.

```
function [ numberOfSuccesses ] = BinomialDistribution( n,p )

numberOfSuccesses = 0;
for i = 1:n
    t = rand;
    if (t < p)
        numberOfSuccesses = numberOfSuccesses + 1;
    end
end
end
```

To run an experiment many times, using this distribution, we can use a function handle (without inputs) to run the experiment.

```
sixes = @() BinomialDistribution1(15, 1/6) % sixes simluates the experiment
sixes() % produces a number
```

Since the probability for getting a six when you roll a die is $1/6$, the experiment above simluates counting sixes in 15 rolls.

To find approximately the expectation of $X \sim \text{bin}(n, p)$, we can run it many times, say N , count the total number of successes, and divide by N . This can be automated by a MATLAB function taking a function handle and the number of trial runs as input, see Exercise 14.5.9.

This number turns out to be approximately np . Assuming this is the true value of the expectation, we can approximate the variance by definition:

```
variance = 0;

for i=1:numberOfTrials
    numberOfSuccesses = 0;
    for i=1:n
        t=rand;
        if (t < p)
            numberOfSuccesses = numberOfSuccesses + 1;
        end
    end
    variance = variance + (numberOfSuccesses - n*p)^2;
end

variance = variance/(numberOfTrials -1)
```

Running several simulations, we see that the variance will be $np(1-p)$, approximately. Again this process can be automated, see Exercise 14.5.10.

Using the properties of expectation and variance, it is not so difficult to see that this is true in a more formal way.

Theorem 14.3.2. *Let $X \sim \text{bin}(n, p)$. Then $E(X) = np$ and $\text{Var}(X) = np(1 - p)$.*

Proof. We will prove the statement by induction on n , letting p be any fixed number. The case $n = 1$ is simple, since there are only two outcomes from the trial:

$$E(X) = 1 \cdot p + 0 \cdot (1 - p) = p = 1 \cdot p$$

Then the variance is

$$\text{Var}(X) = (1 - p)^2 \cdot p + (0 - p)^2 \cdot (1 - p) = p(1 - p) = 1 \cdot p(1 - p)$$

Assume that the formulas are correct for some n , we want to prove that for $X \sim \text{bin}(n + 1, p)$ they are also correct. Note that X is the sum of $X_1 \sim \text{bin}(n, p)$ and $X_2 \sim \text{bin}(1, p)$, where X_2 represents the final Bernoulli trial. By definition, X_1 and X_2 are independent. Using the induction hypothesis and the case $n = 1$, we see that

$$E(X) = E(X_1) + E(X_2) = np + p = (n + 1)p$$

and

$$\text{Var}(X) = \text{Var}(X_1) + \text{Var}(X_2) = np(1 - p) + p(1 - p) = (n + 1)p(1 - p).$$

Therefore, the formulas are true for all values of n , and we have proved the theorem. $\square$

Example 14.3.3. Toss 100 dice. You expect to get about $100/6 \approx 16.67$ sixes.

When running 1000 such trials in MATLAB, we got a table like this:

number of sixes	occurrences
< 5	0
6	3
7	4
8	4
9	13
10	24
11	44
12	54
13	76
14	74
15	96
16	99
17	110
18	93
19	91
20	72
21	50
22	37
23	22
24	14
25	5
26	9
27	2
28	2
29	1
30	1
> 31	0

As we might have guessed, the commonest result is 17 sixes, with 16 in second place. It is so uncommon to get results bigger than 27, say, that we cannot be very sure about what happens from just 1000 trials. But how sure can we really be of anything here?

Based on the above table, we could guess that we should expect that 17 sixes should occur approximately 11% of the time, 16 approximately 10%. There are formulas that can be derived for these values, and the theoretically correct numbers will show that there is a slightly larger chance for getting 16 than 17. This is another indication that 1000 trials is not enough to see clearly what happens for “many” trials.

Hypergeometric distribution

The hypergeometric distribution is similar to the binomial distribution, but we will not allow repetitions. Think about it as drawing, and keeping, some elements from a set.

Example 14.3.4. Draw ten cards from a full deck of 52 cards. How many of them will be diamonds?

There are $M = 13$ diamonds among the $N = 52$ cards, and we are to draw $n = 10$ cards. Think about it as drawing one card at the time. The probability that we draw a diamond is equal to M/N . If the card we draw is a diamond, the number of diamonds left in the deck is reduced by one; otherwise it is unchanged. In either case, the total number of cards left is reduced by one. Here is some MATLAB code based on this observation:

```
N = 52; % cards in total
M = 13; % diamonds
n = 10; % cards to be drawn

drawnDiamonds = 0;

for i = 1:n
    t = rand;
    if (t < M/N)
        % we pick a diamond
        drawnDiamonds = drawnDiamonds + 1;
        M = M-1;
        N = N-1;
    else
        % we pick something else
        N = N-1;
        % M is unchanged
    end
end
```

Running the above code several times often produce 2, 3 and 4, but among 100 trials all numbers from 0 to 6 appeared.

The code above can also be used as a basis for producing a function in MATLAB that simulates the experiment, see Exercise 14.5.11.

Let X be the stochastic variable that counts the number of “special” elements drawn in a trial where n elements are drawn, and not replaced, from a set of N , when

M of the N are special. We sometimes write

$$X \sim \text{hypergeom}(N, M, n)$$

in this situation. Sampling in the same way as we already did for the binomial distribution, we can guess that

$$E(X) = nM/N.$$

The variance is trickier, as the formula contains several factors:

$$\text{Var}(X) = n \frac{M}{N} \left(1 - \frac{M}{N}\right) \frac{N-n}{N-1}$$

Theorem 14.3.5. *Let $X \sim \text{hypergeom}(N, M, n)$ be a stochastic variable with hypergeometric distribution. Then*

$$E(X) = nM/N.$$

and

$$\text{Var}(X) = n \frac{M}{N} \left(1 - \frac{M}{N}\right) \frac{N-n}{N-1}$$

We will not prove the formula for the variance. This is difficult since the addition property for variance (in Equation (14.7)) is valid only for independent variables, and the individual drawings in the hypergeometric setup are not independent. However, the addition property for expectation (in Equation (14.5)) is valid in general, and since the expectation of each drawing of the n drawings is M/N , the formula for the expectation follows.

You could try to get numerical evidence for the formulas in the theorem by performing experiments using a large number of trials, following the setup of Exercises 14.5.9, 14.5.10 and 14.5.11.

Binomial coefficients without overflow

We will now take a brief detour, showing how to compute binomial coefficients without integer overflow, if possible. We will also present some formulas for the binomial and the hypergeometric distribution, and use this to check some claims made earlier.

Recall that by definition, the binomial coefficients are defined as

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad \text{for integers with } 0 \leq k \leq n.$$

Theorem 14.3.6. *There is a recursion relation*

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

valid for $1 \leq k \leq n-1$.

Since $\binom{n}{n} = \binom{n}{0} = 1$, we can write down a table for all binomial coefficients by starting with 1s, and adding the numbers above. This diagram is known as Pascal's triangle.

Example 14.3.7. Let us use the recursion to compute $\binom{10}{3}$ as an example. Imagine that the top and the left are filled with ones first, and then we get the rest of the table by addition.

1	1	1	1	1	1	1	1
1	2	3	4	5	6	7	8
1	3	6	10	15	21	28	36
1	4	10	20	35	56	84	120

The answer is 120. Note that all the numbers involved in the computation are smaller than the final answer. In particular, if the final answer is representable as an integer type with a fixed number of bits, we can compute it using only that type without fearing overflow. Compare Example 6.2.3 and Exercise 6.2.4.

In Exercise 14.5.12, you will be asked to write a programme to compute binomial coefficients using these ideas.

Proof of the binomial recursion. A very boring proof of the theorem is to add the summands and manipulate the fractions. Recall that $n! = n \cdot (n-1)!$.

$$\begin{aligned}
 \binom{n-1}{k-1} + \binom{n-1}{k} &= \frac{(n-1)!}{(k-1)!(n-1-(k-1))!} + \frac{(n-1)!}{k!(n-1-k)!} \\
 &= \frac{(n-1)!}{(k-1)!(n-k)!} \frac{k}{k} + \frac{(n-1)!}{k!(n-1-k)!} \frac{n-k}{n-k} \\
 &= \frac{(n-1)!(k+n-k)}{k!(n-k)!} \\
 &= \frac{n!}{k!(n-k)!} \\
 &= \binom{n}{k}
 \end{aligned}$$

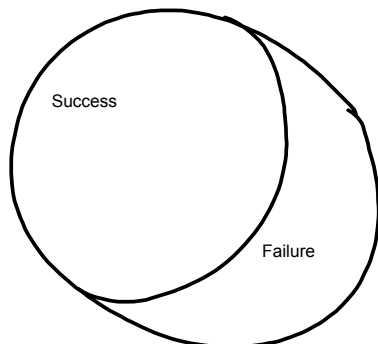
A conceptual proof goes like this. The binomial coefficient $\binom{n}{k}$ counts the number of ways we can pick k objects from n (can you see why?). Think about the last object. If we pick it, we are left with picking $k-1$ among $n-1$. If we do not pick it, we are left with picking k among $n-1$. These are the only possibilities, so we have the formula in the theorem. $\square$

We end with stating the formulas for the probability that binomially or hypergeometrically distributed stochastic variables have a specific value.

Theorem 14.3.8. Let $X \sim \text{bin}(n, p)$. Then

$$P(X = x) = \binom{n}{x} p^x (1-p)^{n-x}.$$

Figure 14.3: Drawing a success or a failure from a random set



Proof. Here is an informal sketch of a proof. To get exactly x successes, there are $\binom{n}{x}$ possibilities for picking which x of the n are successes. For each element, there is a probability p for a success. Therefore we get the factor p^x . Similarly, there is a probability $1 - p$ for not having a success, and this happens for the remaining $n - x$ occurrences. This explains the factor $(1 - p)^{n-x}$. $\square$

Theorem 14.3.9. *Let $X \sim \text{hypergeom}(N, M, n)$. Then*

$$P(X = x) = \frac{\binom{M}{x} \binom{N-M}{n-x}}{\binom{N}{n}}.$$

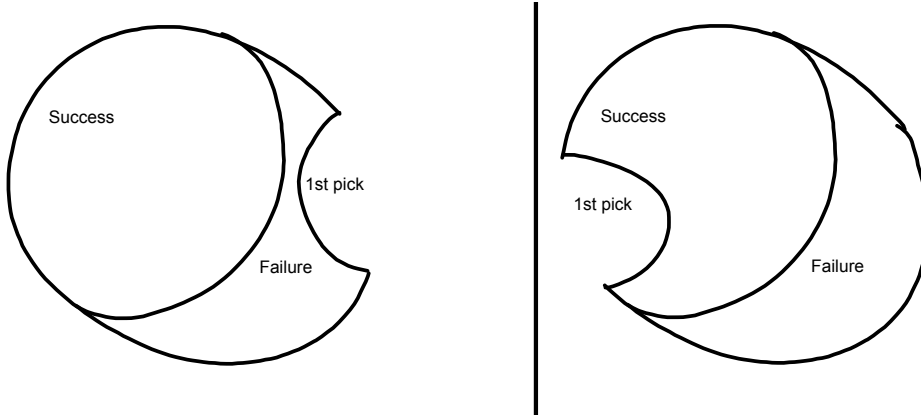
Proof. A quick argument for this formula goes as follows. There are $\binom{N}{n}$ possible ways to draw the entire set of n elements from N . If one such drawing should be counted, we want to draw exactly x from among the smaller set. This gives the factor $\binom{M}{x}$. Similarly, the remaining $n - x$ must be drawn from the other $N - M$ elements, explaining the factor $\binom{N-M}{n-x}$. $\square$

Comment 14.3.10. As a simple visual aid to remember the difference between the set-up for the binomial and the hypergeometric cases, we can use Venn diagrams. We will simply divide our possible choices into two parts, success and failure as in Figure 14.3.

In the binomial set-up, the Venn diagram stays the same for each pick, regardless of the outcome of previous picks. For the hypergeometric set-up, the diagram changes; see Figure 14.4.

If the first pick was a failure, the successes form a larger part (or percentage) for the second pick, as you can see on the left. Conversely, if the first pick is a success, the successes form a smaller part for the second pick, as illustrated on the right.

Figure 14.4: Hypergeometric: probabilities change



14.4 The Poisson distribution

The Poisson distribution is used to simulate finding objects that are randomly distributed in a continuous setting. Examples are molecules of nitrogen in a volume of air, calls to an internet server over a period of time, or trees in a forest area. We will concentrate on the case of time, even though the other cases are quite similar.

Our starting point will be that there are λt occurrences on average in a period of length t , where $\lambda > 0$ is some parameter, and that all occurrences happen at different times, are independent of each other, and that the probability is equal for all periods of the same length. In this situation, it can be shown that if X is the number of occurrences, then

$$P(X = x) = \frac{(\lambda t)^x}{x!} e^{-\lambda t}.$$

We write $X \sim \text{Po}(\lambda t)$. Can we sample from this distribution?

Example 14.4.1. Focus on the time before the first occurrence happens. The probability that there is not any occurrence before time t , say, is equal to $P(X = 0)$ with the parameter λt . So

$$P(X = 0) = \frac{(\lambda t)^0}{0!} e^{-\lambda t} = e^{-\lambda t}.$$

Recall that we can pick a random (uniformly distributed) number u from the interval $[0, 1)$. Note that we use the letter u for a random number now, since t is usually reserved for time. We want to choose the time t so that $P(X = 0)$ with this parameter is exactly equal to u . The computation goes like this:

$$\begin{aligned} e^{-\lambda t} &= u \\ -\lambda t &= \ln u \\ t &= -\frac{\ln u}{\lambda} \end{aligned}$$

To sample from the Poisson distribution, we use this idea repeatedly, and add the times at each step. Once we pass the final limit time, we stop. Here is some MATLAB code for doing this, written as a function.

```
function [ numberOfOccurrences ] = Poisson( lambda, stopTime )
totalTime = 0; % running total
numberOfOccurrences = -1; % in the loop, one is counted after stopTime
while (totalTime < stopTime)
    u = rand;
    t = -log(u)/lambda;
    totalTime = totalTime + t;
    numberOfOccurrences = numberOfOccurrences + 1;
end
end
```

Running this function many times will indicate that the expected value is approximately λt . Doing computations similar to the computations involved in guessing the variance of the binomial distribution, we will also guess that the variance is λt . You can estimate the expectation and variance of a stochastic variable following the Poisson distribution by using the setup from Exercises 14.5.9 and 14.5.10.

Comment 14.4.2. For this function, there could be a problem in some programming languages. The MATLAB documentation states that the random function produces a number in the open interval $(0, 1)$. The Java documentation states that `Math.random()` produces a number in the half-open interval $[0, 1)$. The difference is that zero is included. Since the logarithm of zero is undefined, some care should be exercised in a Java implementation. One work-around is to take the logarithm of $1 - u$ instead. Do you understand why this will solve the problem? For Python, the documentation states that both ends of the interval may or may not be included, depending on rounding of floating points.

Let us formally state and prove the properties deduced in the previous example.

Theorem 14.4.3. *Let $X \sim \text{Po}(\lambda t)$. Then*

1. *X is a probability distribution, that is the total probability is 1.*
2. *$E(X) = \lambda t$.*
3. *$\text{Var}(X) = \lambda t$*

All of these properties can be deduced from the formula, using series expansion of the exponential function. Not all students are familiar with these series, so feel free to skip this proof if you lack the prerequisites for it (see also Chapter 20).

Proof. We first need to show that $\sum_{x=0}^{\infty} P(X = x) = 1$. Recall that

$$e^y = \sum_{n=0}^{\infty} \frac{y^n}{n!}.$$

We will use this formula with $y = \lambda t$.

$$\begin{aligned} \sum_{x=0}^{\infty} P(X = x) &= \sum_{x=0}^{\infty} \frac{(\lambda t)^x}{x!} e^{-\lambda t} \\ &= e^{-\lambda t} \sum_{x=0}^{\infty} \frac{(\lambda t)^x}{x!} \\ &= e^{-\lambda t} e^{\lambda t} \\ &= 1 \end{aligned}$$

Thus, we have proved that it is in fact a probability distribution. Next, for the expectation, we need to prove that $\sum_{x=0}^{\infty} xP(X = x) = \lambda t$. This comes from series manipulations:

$$\begin{aligned} E(X) &= \sum_{x=0}^{\infty} xP(X = x) \\ &= \sum_{x=0}^{\infty} x \frac{(\lambda t)^x}{x!} e^{-\lambda t} \\ &= e^{-\lambda t} \sum_{x=1}^{\infty} \frac{(\lambda t)^x}{(x-1)!} \\ &= e^{-\lambda t} \lambda t \sum_{x=1}^{\infty} \frac{(\lambda t)^{x-1}}{(x-1)!} \\ &= e^{-\lambda t} \lambda t e^{\lambda t} \\ &= \lambda t \cdot 1 = \lambda t \end{aligned}$$

The series is reindexed to start from $x = 1$.

Finally, the variance can be computed similarly, using a reindexing of the series starting from $x = 2$. We use the formula for the variance from Example 14.1.12. Let us therefore first compute $\sum x^2 P(X = x)$:

$$\begin{aligned} \sum x^2 P(X = x) &= \sum_{x=0}^{\infty} x^2 \frac{(\lambda t)^x}{x!} e^{-\lambda t} \\ &= e^{-\lambda t} \sum_{x=1}^{\infty} x \frac{(\lambda t)^x}{(x-1)!} \\ &= e^{-\lambda t} \sum_{x=1}^{\infty} (x-1+1) \frac{(\lambda t)^x}{(x-1)!} \\ &= e^{-\lambda t} \left(\sum_{x=2}^{\infty} \frac{(\lambda t)^x}{(x-2)!} + \sum_{x=1}^{\infty} \frac{(\lambda t)^x}{(x-1)!} \right) \\ &= e^{-\lambda t} \left((\lambda t)^2 \sum_{x=2}^{\infty} \frac{(\lambda t)^{x-2}}{(x-2)!} + \lambda t \sum_{x=1}^{\infty} \frac{(\lambda t)^{x-1}}{(x-1)!} \right) \\ &= e^{-\lambda t} \left((\lambda t)^2 e^{\lambda t} + (\lambda t) e^{\lambda t} \right) \\ &= (\lambda t)^2 + \lambda t \end{aligned}$$

Therefore the variance is

$$\text{Var}(X) = -\left(E(X)\right)^2 + \sum x^2 P(X = x) = -(\lambda t)^2 + (\lambda t)^2 + \lambda t = \lambda t$$

□

14.5 Exercises

Exercise 14.5.1. Perform a random walk many times, and plot only the endpoints. The command to plot a set of points is `scatter` in MATLAB. Recall also that we can find the last element of an array using the index `end`. In the same figure draw a circle with radius $\sqrt{2n-2}$ and center at the origin.

Exercise 14.5.2. Let $X \sim \text{bin}(n, p)$ with $n = 20$ and $p = 0.2$.

- (a). What is the expectation and the variance of X ?
- (b). Compute $P(X = 1)$ and $P(X = 4)$.
- (c). Compute $P(X \geq 2)$.

Exercise 14.5.3. Fourteen dice are rolled, and we are interested in the number of sixes, call it X .

- (a). What is the probability distribution of X ?
- (b). What is the expectation and the variance of X ?
- (c). What is the probability of getting exactly four sixes?
- (d). What is the probability of getting at least two sixes?

Exercise 14.5.4. Let $X \sim \text{hypergeom}(N, M, n)$ where $N = 8$, $M = 3$ and $n = 4$.

- (a). What is the expectation and the variance of X ?
- (b). Write down the full table of probabilities for X .

Exercise 14.5.5. In a group of 105 students, 42 commutes to campus. In a poll, 8 students are randomly drawn. Let X be the number of commuters drawn.

- (a). What is the distribution of X ?
- (b). What is the expectation of X ?
- (c). What is the probability that exactly four commuters are drawn?
- (d). What is the probability that less than or equal to 7 commuters are drawn?

Exercise 14.5.6. Let X be given by its probability distribution:

x	0	2	3	4	6	7
$P(X = x)$	0,2	0,2	0,1	0,2	0,05	0,25

Compute expectation and variance of X .

Exercise 14.5.7. A cell phone base station receives SMS with an intensity of $\lambda = 4$ SMS per minute. Let X be the number of received SMS over a period of 20 seconds.

(a). What is the probability that the base station receives exactly two SMS in this period?

(b). What is the probability that at least one SMS is received?

Exercise 14.5.8. Based on observations over 20 years, fatal horse-kickings result in about 0.7 dead soldiers per army corps per year in the Prussian army.

(a). What conditions must be fulfilled if this situation is to be described by a Poisson distribution? What is λ ?

(b). What is the probability that no-one is kicked to death in any army corps over a six month period (there were 14 corps in the Prussian army)?

Exercise 14.5.9. In this exercise, you are asked to write a MATLAB function to estimate the expectation of a discrete stochastic variable. We require that the inputs are a function handle and the number of trials, and the output is the value you get by computing the mean of the results of performing the experiment this number of times. The first line of the file should look like this:

```
function [expected] = GuessExpectation(X, trials)
```

Warning: Make sure that the experiment is actually performed the required number of times. This means that `X()` should appear within the code.

Exercise 14.5.10. In this exercise, you are asked to write a MATLAB function to estimate the variance of a discrete stochastic variable.

(a). We first require that the inputs are a function handle. the (previously) computed or estimated expectation (Exercise 14.5.9), and the number of trials, and the output is the value you get by computing the empirical variance of the resulting values. The first line of the file should look like this:

```
function [expected] = GuessVariance(X, trials, mu)
```

Warning: Make sure that the experiment is actually performed the required number of times. This means that `X()` should appear within the code.

(b). Modify the code so that `mu` is an optional parameter. If it is not given, its value should be estimated using the results from Exercise 14.5.9. The first line of the file should look like this:

```
function [expected] = GuessVariance(X, trials, varargin)
```

You should look in the documentation for how to handle a variable number of arguments to a MATLAB function, e.g. for `varargin`.

Exercise 14.5.11. Write a MATLAB function that performs experiments with the hypergeometric distribution, based on the code in Example 14.3.4. The first line should be

```
function [ numberOfSuccesses ] = HypergeometricDistribution(N, M, n)
```

Exercise 14.5.12. Write a programme, e.g. in Java or MATLAB, to compute binomial coefficients using the ideas from Theorem 14.3.6 and Example 14.3.7.

Chapter 15

Graphs

In this chapter, we will introduce the notion of *graphs* as an abstraction of the notion of being *connected*. A typical example comes from a railroad network. A graph describing the Norwegian railroads would have a *node* for each city, and an *edge* between two cities if there is a rail line connecting them. For instance, there would be edges between Bergen and Oslo, and between Oslo and Trondheim, but not between Bergen and Trondheim, as there is no direct rail line between these two cities.

15.1 Definition of a graph

We draw a graph as a collection of nodes and edges, as in Figure 15.1. The nodes here are labelled with letters from the beginning of the alphabet, there are eight of them in this example. There are edges between some of the nodes. For instance, there are edges between B and A , C and E , three edges between A and D , and even an edge between D and itself. Note also that there isn't any edge connecting H to any other node. There are nine edges in total.

The exact location of the nodes is not a part of the data, we are only interested in the connections between them. Therefore we regard the graph in Figure 15.2 as being the same as the graph in Figure 15.1.

Figure 15.1: Nodes and edges make a graph

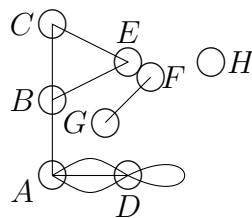
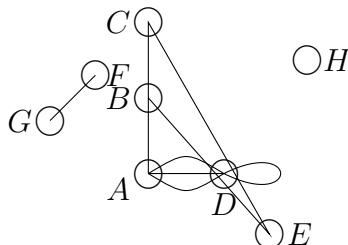


Figure 15.2: The same graph with some nodes moved



Note that the drawing in Figure 15.2 is a bit confusing, in general we would try to avoid drawings where it is unclear whether two vertices are connected. In this example, it is not clear if there is an edge between E and D (false), or if the edge between E and B just happens to pass very close to D (true).

Definition 15.1.1. A graph $\mathcal{G}$ consists of two sets, the set of *nodes* and the set of *edges*. If we write $\mathcal{N} = \mathcal{N}_{\mathcal{G}}$ for the set of nodes and $\mathcal{E} = \mathcal{E}_{\mathcal{G}}$ for the set of edges, we demand that each edge in $\mathcal{E}$ determines a pair of nodes, not necessarily distinct, in $\mathcal{N}$. If the two nodes are equal, the edge is called a *loop*. A graph is called *simple* if any pair of nodes has at most one edge between them, and there are no loops. A *path* consists of several edges in order, such that the second node incident to each edge is the same as the first node incident to the next edge. A *connected component* of a graph consists of a subset of $\mathcal{N}$, such that all nodes in the subset are connected with paths, and there is no edge between a node in the component and a node outside.

In the graph in Figure 15.1, the sets of nodes and edges can be written

$$\begin{aligned}\mathcal{E} &= \{A, B, C, D, E, F, G, H\} \\ \mathcal{N} &= \{AB, AD_1, AD_2, AD_3, BC, BE, CE, DD, FG\}\end{aligned}$$

Note the three edges between A and D , and the loop DD . An example of a path is $AB, BC, CE, EB = BE, BA = AB, AD_2, DD, DD, DA_1 = AD_1$. The connected components are $ABCDE$, FG and H .

Example 15.1.2. The graph $\mathcal{G}$ in Figure 15.3 is simple. The graph $\mathcal{H}$ in Figure 15.4 is not, as there are two edges between A and C . Since we don't need names for the nodes in $\mathcal{G}$, they are not included.

Furthermore, $\mathcal{G}$ has six nodes, five edges and two components, whereas $\mathcal{H}$ has four nodes, six edges and only one component.

Definition 15.1.3. Here is a more formal, less verbose, definition of a graph:

A *graph* is a triple $(\mathcal{N}, \mathcal{E}, \psi)$ where $\mathcal{N}$ (nodes) and $\mathcal{E}$ (edges) are sets, and ψ is a function from $\mathcal{E}$ to the set of unordered pairs $\{\{n_1, n_2\} | n_1, n_2 \in \mathcal{N}\}$.

Figure 15.3: A simple graph

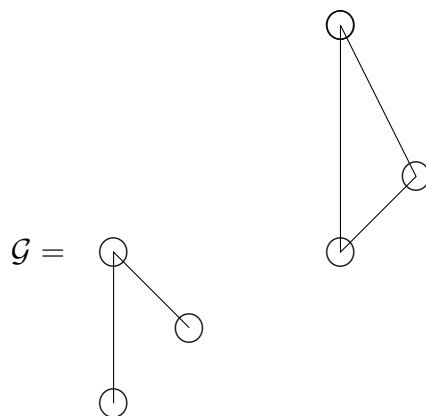
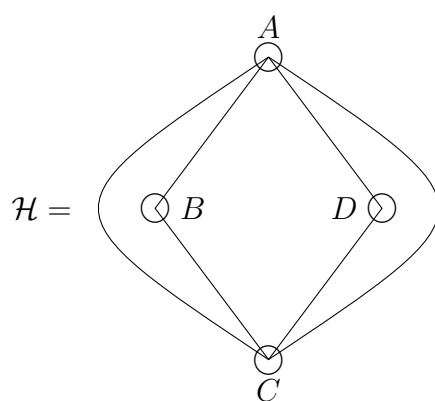
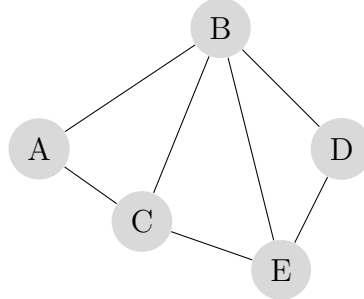
Figure 15.4: Two edges between A and C 

Figure 15.5: A graph with several cycles



Compared to our previous definition, the formality lies in formulating the condition that each edge connects two nodes using the function ψ . We will mostly deal with the less formal definition, though some exceptional cases are easier to understand by appealing to such a formal definition. For instance, since the empty set $\emptyset$ is a set, the formal definition doesn't exclude the empty graph. The informal is equivalent, but it might be less clear that the empty graph is allowed.

15.2 Examples of graphs

We will see several classes of graphs that are interesting for various purposes. In order to gain familiarity with the basic definitions of graph theory, it is essential to use the terms for different graphs.

15.2.1 Trees

A tree is a graph where each pair of vertices can be connected with a path in (essentially) exactly one way. To give a rigorous definition, it is easiest to define what should not happen first.

Definition 15.2.1. A *cycle* in a graph is a path starting and ending in the same node. We usually demand that the path is not merely going through a set of edges first in one direction, then in the other; this would be a trivial cycle.

Example 15.2.2. Consider the graph in Figure 15.5. The path $ABDEBA$ is a cycle, whereas $EDBDE$ is not. Can you find some more cycles?

Definition 15.2.3. A *tree* is a non-empty connected simple graph without cycles.

Example 15.2.4. In Figure 15.6 three trees have been drawn. The two leftmost have a node at the bottom, and all edges go upwards from this bottom node. It is common to refer to a tree with such a structure as a *rooted tree*, and the bottom

Figure 15.6: Three trees

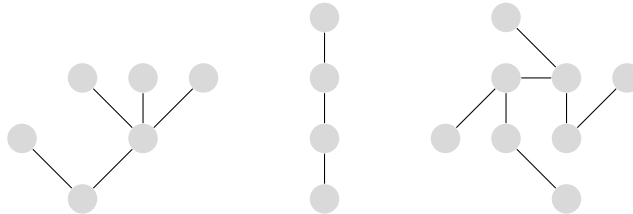
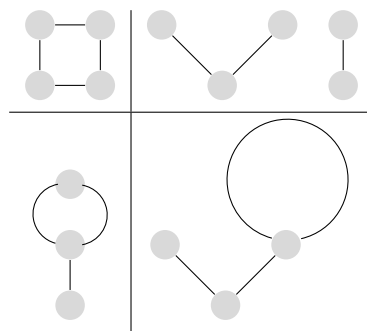


Figure 15.7: Four graphs that are not trees



node is then called the *root*. Note that in computer science, many trees are drawn with the root at the top (the Australian convention).

In Figure 15.7 there are four graphs that aren't trees. These graphs are not trees because (from left to right) of having a cycle, not being connected, having a double edge, and having a loop.

Theorem 15.2.5. *Let $\mathcal{T}$ be a tree with n nodes and e edges. Then $n = e + 1$.*

Proof. Informally, starting with a single node, the tree can be drawn by connecting a new node by a new edge, one node at a time. Thus the numbers n and e are both increased by one equally many times starting from $n = 1$ and $e = 0$. $\square$

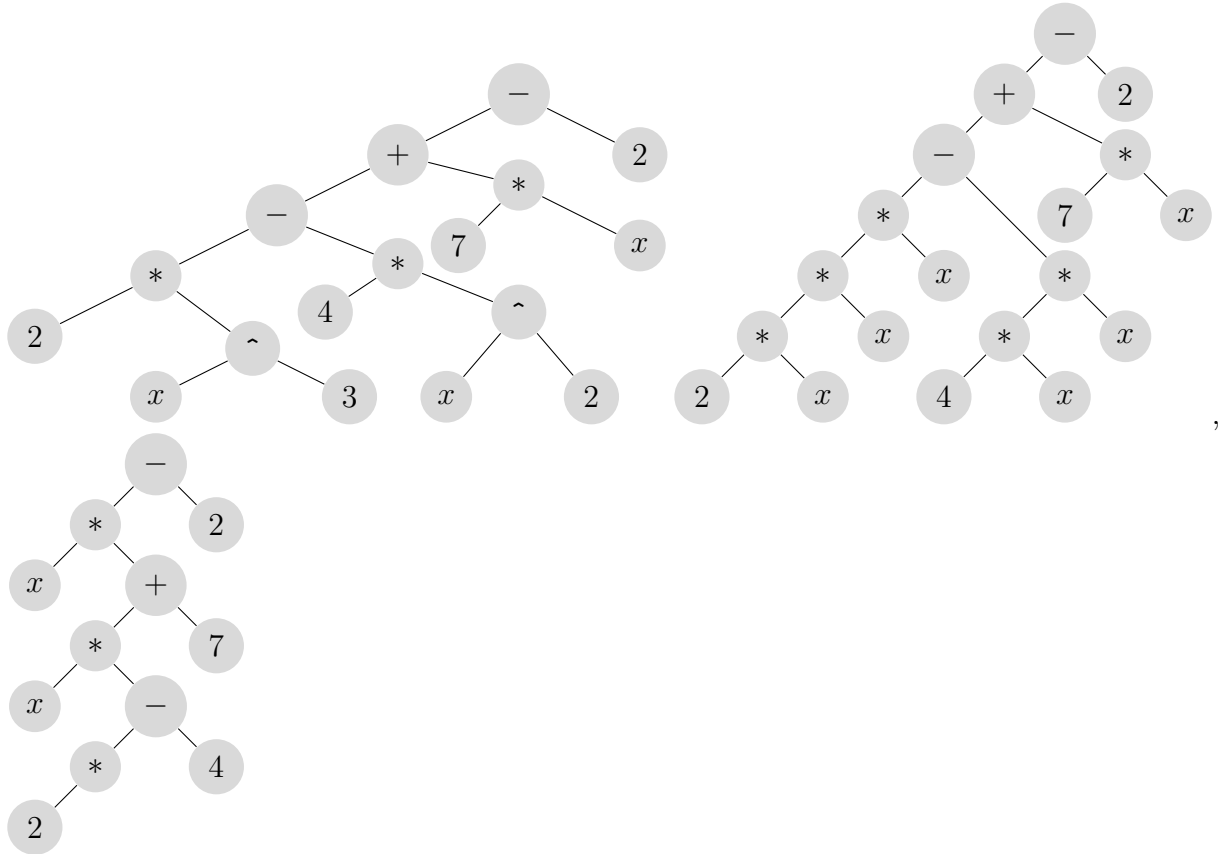
Example 15.2.6. Trees can be used to see more clearly the execution sequence of arithmetic expressions. How would you compute the expression $2x^3 - 4x^2 + 7x - 2$ for many values of x ? Here are three different MATLAB expressions for computing this expression:

$$2*x^3 - 4*x^2 + 7*x - 2$$

$$2*x*x*x - 4*x*x + 7*x - 2$$

$$x*(x*(2*x - 4) + 7) - 2$$

Figure 15.8: Three execution orders for the same mathematical expression



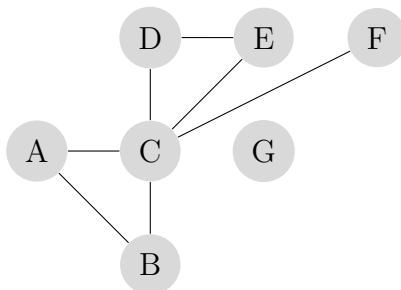
Running each of these 10^8 times for the same values of $\mathbf{x}$, the first line took about 27 seconds to run, the second 8.6 and the third 6.5. To express these as trees, make a tree (customarily with the root at the top) where each leaf contains one of the values $x, 2, 3, 4, 7$ that appear in the expressions, and each other node contains the (binary) operation using the two inputs to that leaf. See Figure 15.8.

The first expression has eight operations, the second has nine, whereas the third only has six. It seems that exponentiation is expensive, but that it is otherwise better to have fewer operations. If efficiency is important, deciding the order of evaluation is one aspect to consider.

15.2.2 Regular graphs

Definition 15.2.7. The degree of a node in a graph is the number of edges incident to the node. Imagine drawing a small circle around the node, then the degree is the number of lines intersecting the small circle. In particular, with this definition, a

Figure 15.9: Degrees 2, 2, 5, 2, 2, 1, 0



loop counts as two edges when computing the degree.

A graph in which all nodes have the same degree is called a *regular* graph.

Example 15.2.8. Look at the graph in Figure 15.9. The degree of the nodes A through G are 2, 2, 5, 2, 2, 1, 0.

Example 15.2.9. The graphs in Figure 15.10 are all regular. What is the common degree of the nodes in each graph?

Theorem 15.2.10. Assume that the degree of each node in a regular graph is d , and let n be the number of nodes and e the number of edges. Then

$$e = \frac{nd}{2}$$

Proof. The number nd is the total number of incidences between edges and nodes. Since each edge is incident to two nodes, we have to divide by two to get the number of edges. $\square$

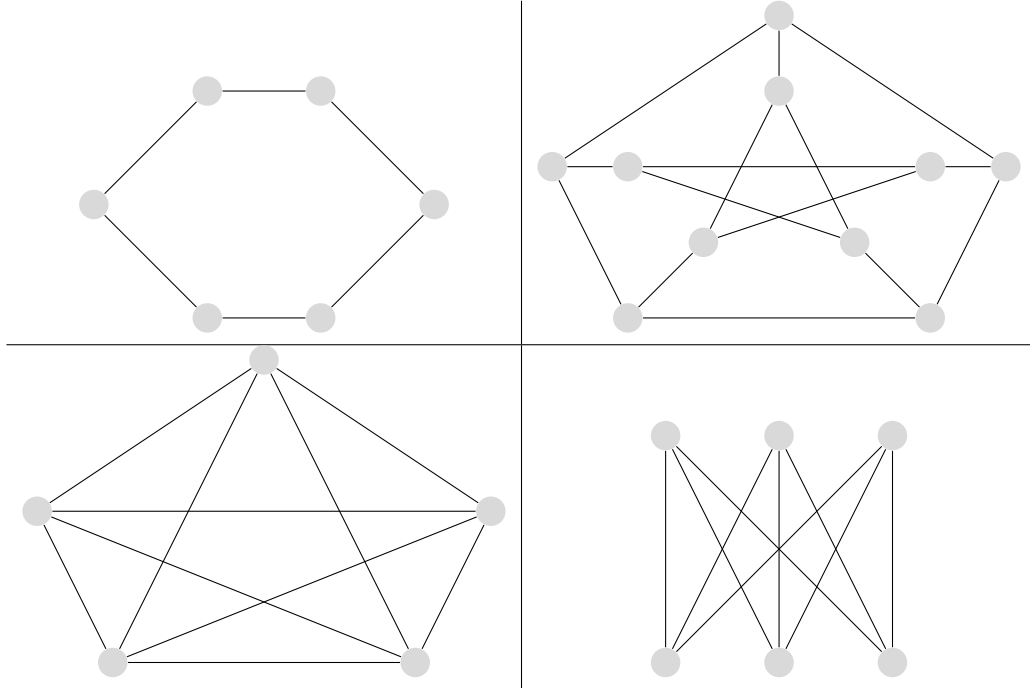
Example 15.2.11. Can a tree be regular? We have two theorems relating the number of edges to the number of nodes. For trees, $n = e + 1$. For regular graphs, $e = \frac{nd}{2}$. If we substitute the second equation into the first, and solve for n , we get that

$$n = \frac{2}{2-d}$$

Since both n and d are integers, we only get two possibilities: $(n, d) = (2, 1)$ and $(n, d) = (1, 0)$. Therefore, there are only two regular trees. Can you draw them?

Example 15.2.12. A *complete* graph is a simple graph where each pair of (distinct) nodes is connected by an edge. This means that if the number of nodes is n , the degree of each node is $n - 1$ since it is connected to all the other nodes. A triangle is

Figure 15.10: Some regular graphs



a complete graph with three nodes, the complete graph with five nodes is the third of the graphs in Figure 15.10. By the formula in the theorem, the number of edges is

$$e = \frac{nd}{2} = \frac{n(n-1)}{2} = \binom{n}{2}$$

The binomial coefficient $\binom{n}{k}$ can be interpreted as counting the number of ways to choose k objects among n . In our case, choosing any two nodes among the total number n gives an edge.

15.3 Special cycles

A path starting and ending in the same node is called a *cycle*. A trivial example is going an edge back and forth, but we are now mostly interested in examples where an edge is traversed only once. There are two special types of cycles that are particularly interesting, the *Eulerian* and the *Hamiltonian* cycles.

Definition 15.3.1. An *Eulerian* cycle is a cycle in a graph that includes each edge exactly once. A *Hamiltonian* cycle is a cycle in a graph that includes each node exactly once. Depending on the graph, these may or may not exist.

Figure 15.11: Eulerian cycle ADCEBIFCBAGCHA and graph without Eulerian cycle

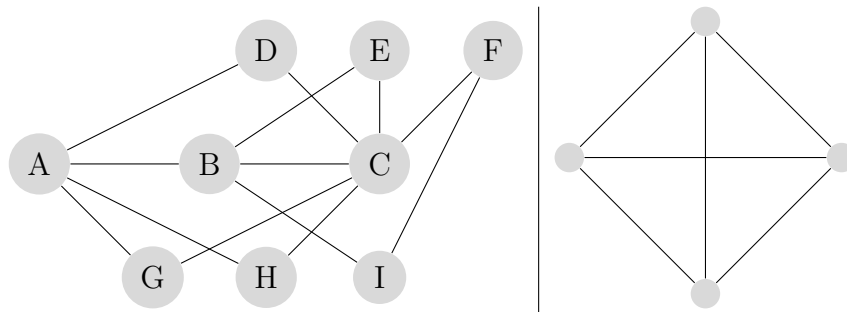
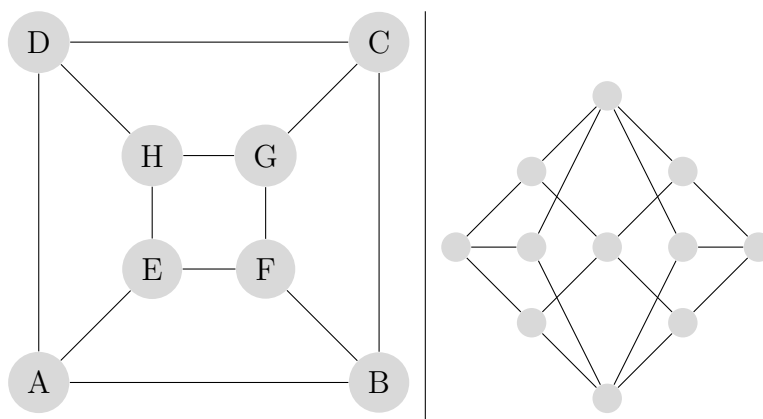


Figure 15.12: Hamiltonian cycle ABFEHGCDA and graph without Hamiltonian cycle



It turns out that determining if there is an Eulerian cycle in a graph is simple, whereas determining if there is a Hamiltonian cycle is hard (NP-complete).

Example 15.3.2. The leftmost graph in Figure 15.11 has an Eulerian cycle as indicated. The rightmost graph doesn't have any Eulerian cycle.

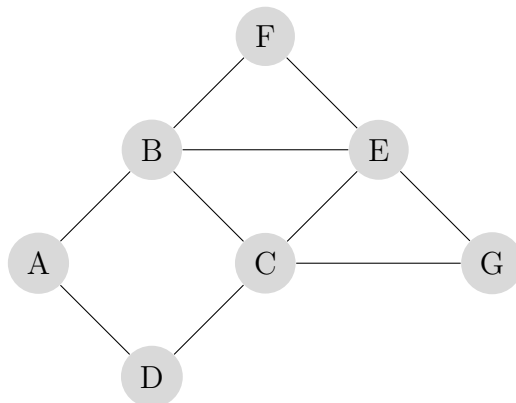
Example 15.3.3. The leftmost graph in Figure 15.12 has a a Hamiltonian cycle as indicated. The rightmost graph doesn't have any Hamiltonian cycle.

It is impossible for a graph to have a cycle of either of these types unless it is connected. Here is a simple criterion for determining whether a graph has an Eulerian cycle.

Theorem 15.3.4. *A connected graph has an Eulerian cycle if and only if the degree of each node is even.*

Proof. If there is an Eulerian cycle, the cycle will arrive at and leave each node equally many times. Thus the degree of each vertex is even.

Figure 15.13: We can find an Eulerian cycle



Conversely, suppose a graph has even degree for each node. Start with any node v . Construct a cycle by going from each node to a new one by going through an arbitrary edge. This process can only stop at v , since the degree of each node is even. When it stops, erase all edges that have been traversed so far. If there are edges not used, at least one of the nodes in the first cycle has more edges, call it w . Start the same procedure with w as a starting point, and insert the new cycle into the first one at w . Repeat until there are no more edges. $\square$

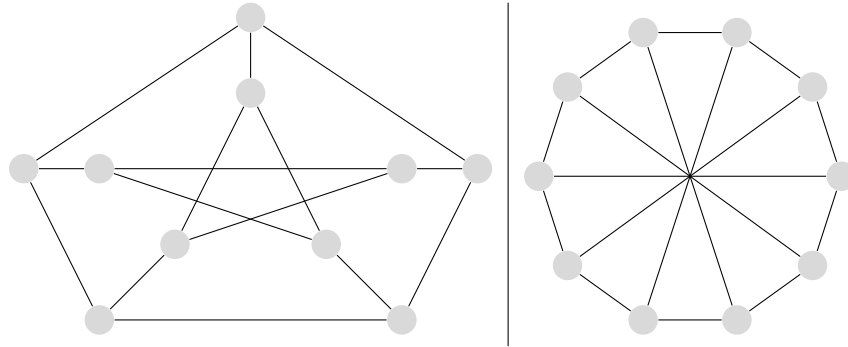
The procedure in the proof is called *Hierholzer's algorithm*. It is illustrated in the next example.

Example 15.3.5. By the theorem, the graph in Figure 15.13 has an Eulerian cycle. We can find one by the procedure in the proof. For instance, the process can have three steps. Starting with $v = A$, we first find for instance the cycle $ABCD A$. There are two nodes in this cycle with more edges, so we choose $w = B$ as the new starting point. This could give us the cycle $BFEB$. Inserting this into the first cycle, we get $ABFEBCDA$. Again, there are two nodes with more edges. Choosing E , we find a cycle $ECGE$. Inserting this too, we get the final answer:

$$ABFECEGBCDA$$

Example 15.3.6. There is no way to determine whether a graph has a Hamiltonian cycle by considering only degrees of vertices as for the Eulerian case. For instance, the two graphs in Figure 15.14 are both regular graphs with ten nodes, each node with degree 3. One of them obviously has a Hamiltonian cycle, the other doesn't. Note that it is hard to explain why there isn't a Hamiltonian cycle.

Figure 15.14: Only one of these graphs has a Hamiltonian cycle



15.4 Directed graphs, weighted graphs

One way of encoding a simple graph, is to make a two-dimensional array where each dimension is indexed by the set of nodes. If there is an edge between two nodes, put a 1 in the corresponding spot in the array, otherwise a 0. For instance in Figure 15.13, using alphabetical order, the matrix is

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 0 \end{pmatrix}$$

In this way, it is easier to write down a specification for a graph, we don't need a drawing, only the corresponding matrix.

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Such an array is sometimes referred to as an *adjacency matrix*. Note that we could have used an array of the form `boolean[][]`, but if we use a numerical type we can easily record some extra information in the array.

Directed graphs

A *directed graph* is similar to an ordinary graph, but all edges are oriented: they point from one of the nodes to the other. We draw this using small arrows.

The matrix representation of a simple directed graph has a 1 in place (i, j) if there is an edge from node i to node j . More generally, we can let the number in place (i, j) be the number of edges from node i to node j .

Example 15.4.1. A tree with a choice of root can be interpreted as a directed graph. Orient all edges towards the root, away from the leaves.

Example 15.4.2. In many situations, it is reasonable to think about edges in a directed graph as representing the flow of time. If the graph doesn't have any directed cycles (meaning that you cannot follow a directed path from a node back to itself), we will progress through the graph. In the end, we will end up in some final node.

To give an example, consider a programme that handles internet sales. The input to the programme is an interactive form containing information about the buyer (name, address, credit card information) and the product to be bought. Let us consider a simple scheme showing a few of the choices to be made. Time flows from the top to the bottom. Each node represents a question the buyer must answer (with a question mark), or some action that needs to be taken because of some earlier choice, each edge from one node to another represents a direction to go based on the previous answer. We assume that the store is located in the US, but also sends packages to Norway.

Look at the graph in Figure 15.15.

Each question answered takes us a step closer to actually sending the package. Based on information sent in by the buyer, choices are made afterwards. For instance, if the package is to be sent internationally (to Norway), customs information must be printed.

Weighted graphs

Consider the classical *travelling salesman* problem. A salesman should go to a number of different cities to promote his goods. For each pair of cities there is a given travelling time between them. The problem is to determine the route that will take the shortest overall travelling time.

The standard approach to this problem is to make a graph whose nodes are the relevant cities. If it is possible to travel from one city to another, draw an edge between them. For each edge, write the time it takes to travel along the edge. For instance, the graph in Figure 15.16 shows an example with five cities. The travelling time between E and B , for instance, is 24.

Figure 15.15: A simplified internet sales chart

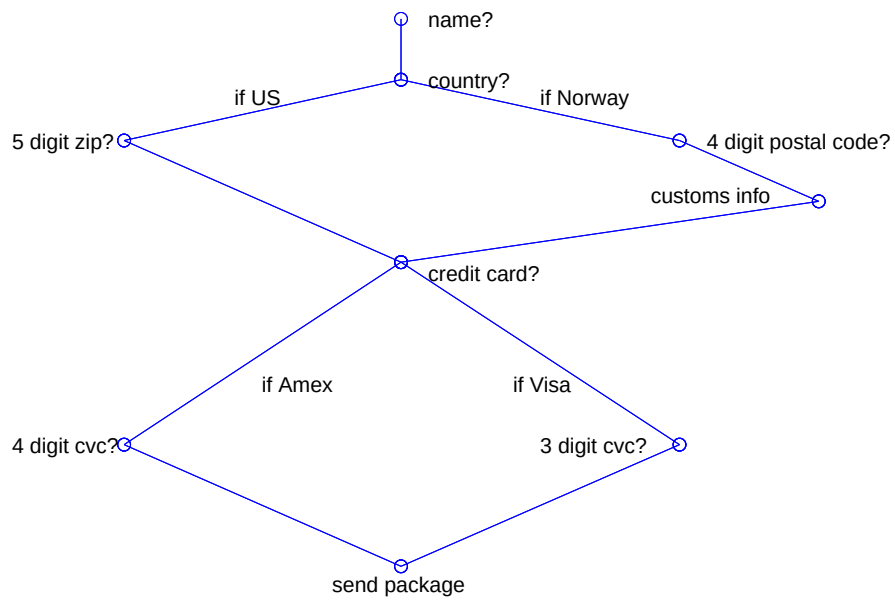
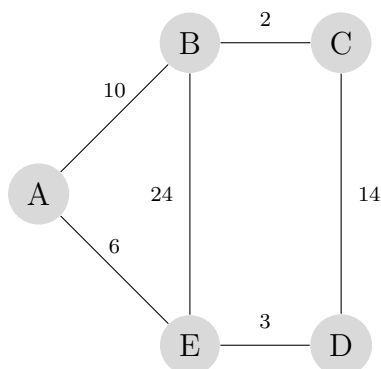


Figure 15.16: A salesman must visit five cities



Definition 15.4.3. A *weighted graph* is a graph where each edge has an associated number (the weight).

Example 15.4.4. We either draw the graph, writing the weight next to the edge, or we represent the information in the style of an adjacency matrix, but where we write the weight instead of the number of edges at a given place. It is also convenient to use ∞ as label where there isn't any edges, at least if we represent the numbers as floating points. For instance, the matrix representation of the weighted graph in Figure 15.16 is

$$\begin{pmatrix} \infty & 10 & \infty & \infty & 6 \\ 10 & \infty & 2 & \infty & 24 \\ \infty & 2 & \infty & 14 & \infty \\ \infty & \infty & 14 & \infty & 3 \\ 6 & 24 & \infty & 3 & \infty \end{pmatrix}$$

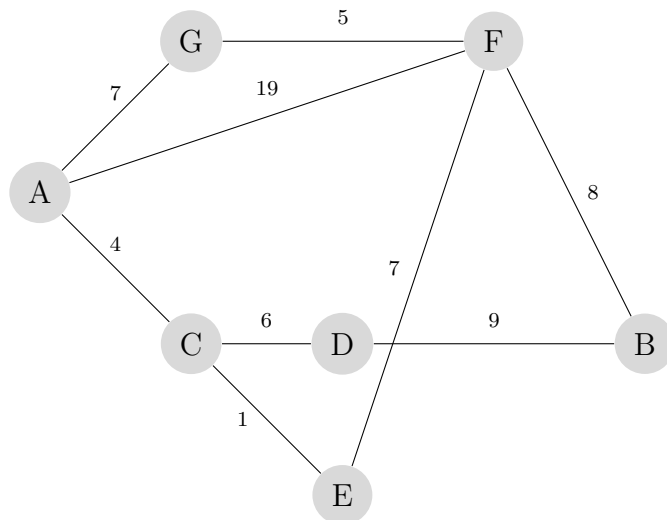
As an example of what we can do with weighted graphs, we will consider the shortest path problem.

Example 15.4.5 (Shortest path). Let A and B be two of the nodes of a weighted, connected graph (all weights are positive). For each path from A to B , let its length be the sum of the weights as we go along. Choose the path with the minimal length; this is the *shortest path* between the two nodes.

How can we find this path? One algorithm for finding it goes as follows. We start at A , and consider all nodes connected to A . Find the edge with the smallest weight, and store the connected node, say C . Obviously, the weight we have found must be the smallest possible length of a path from A to C , since any other path from A is longer. If $C = B$ we have solved the problem, otherwise consider all the nodes connected to A or C , and find the one (excluding C) with the shortest path from A . The length of such a path is either the weight of an edge from A , or the sum of the weights from A to C and from C to a new node. Again, choose the node with the smallest path-length from A , say D . The path we have chosen must be the shortest path from A to D , since the only path starting at A which is shorter, is the one ending at C . If $D = B$, we are finished, otherwise consider nodes connected to A , C or D and repeat the above process.

At each repetition, we add one more node which is guaranteed to have its shortest path found by the procedure so far. We consider all the nodes connected to one of the nodes we have already found, and choose the one with the shortest path from A . There are no shorter paths from A , except paths to nodes already found, so the path we have constructed must necessarily be the shortest path from A to the new node. We repeat until we have finally chosen our ending point B in this procedure.

Here is some pseudo-code for this algorithm:

Figure 15.17: We will find the shortest path from A to B 

```

<weights of all edges are known>
finished = {A}
for each vertex connected to some node in finished
    compute shortest path from A
end
finished = finished union the node with shortest path
<store the length of the shortest path to the new node>
repeat until B is in finished
output the length of the shortest path to B

```

Let us do this for the graph in Figure 15.17.

Since we have seven nodes, there will be no more than six steps where we choose a new node. Here are the steps, at each step showing which numbers we need to find the minimum of, and the shortest path leading to the new node at each step.

1. A is the starting point.
2. C, F and G are connected to A , so we choose one of these. The edge to C has the lowest weight, so we choose C and remember that the shortest path from A is AC of length 4.
3. D, E, F and G are connected to A or C . The one with the shortest path from A is E , so we choose E . The shortest path is ACE of length 5.
4. D, F and G are connected to A, C or E . When we try to find the shortest path to F , note that there are two ways to come from A to F , one directly and the

other through C and E . The first choice has length 19, the second has length 12. Still, 12 is greater than the length of the path to G , so at this point we choose G . The path is AG of length 7.

5. B, D and F are all connected to A, C, E or G . For F , there is now another path of length 12 from A , going through G , but the shortest path from A is now to D . The path is ACD and it has length 10.
6. B and F are connected to A, C, D, E or G . The shortest path is to F , and there are two possible shortest paths: AGF and $ACEF$, both of length 12.
7. B is the only one left. We can come to B from D , length 19, from F , length 20, or from G , length 21. The shortest is via D , and we record this.

At this point, when the shortest path to B has been found, we stop. If there were more nodes, we would still stop now.

Here is the list of nodes found, with paths and their lengths, conveniently summarized for us.

node	shortest path	path length
A		0
C	AC	4
E	ACE	5
G	AG	7
D	ACD	10
F	AGF or $ACEF$	12
B	$ACDB$	19

Anyway, the conclusion is that the shortest path from A to B is through C and D , and it has length 19.

Exercise 15.4.6. It is time to implement the algorithm for finding the shortest path outlined vaguely above.

- (a). Write pseudo-code for the algorithm. The input should be the matrix representation of the weighted graph, the output the shortest path and its length.
- (b). Implement the algorithm in MATLAB.
- (c). Implement the algorithm in Java.

15.5 Exercises

Exercise 15.5.1. Let a graph G have n vertices and m edges.

- (a). If $m = n$, can G be a tree?

(b). If $m = n$, can G be a disconnected graph all of whose components are trees?

(c). If $m = n$, can G be a disconnected graph one of whose components is a tree?

(d). If $m = n$, can G be a disconnected graph none of whose components is a tree?

Exercise 15.5.2. Draw the graph associated with the matrix.

(a).

$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

(b).

$$\begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

(c).

$$\begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Exercise 15.5.3. How many ones are there in the matrix representation of a tree?

Exercise 15.5.4. For each graph in Figure 15.18, determine whether there is an Eulerian cycle. If there is one, write one down.

Exercise 15.5.5. Each of the graphs in Figure 15.19 has a Hamiltonian cycle. Write one down.

Exercise 15.5.6. Explain why the graph in Figure 15.20 does not have a Hamiltonian cycle.

Exercise 15.5.7. Find the shortest path from A to B in Figure 15.21.

Figure 15.18: Eulerian cycle?

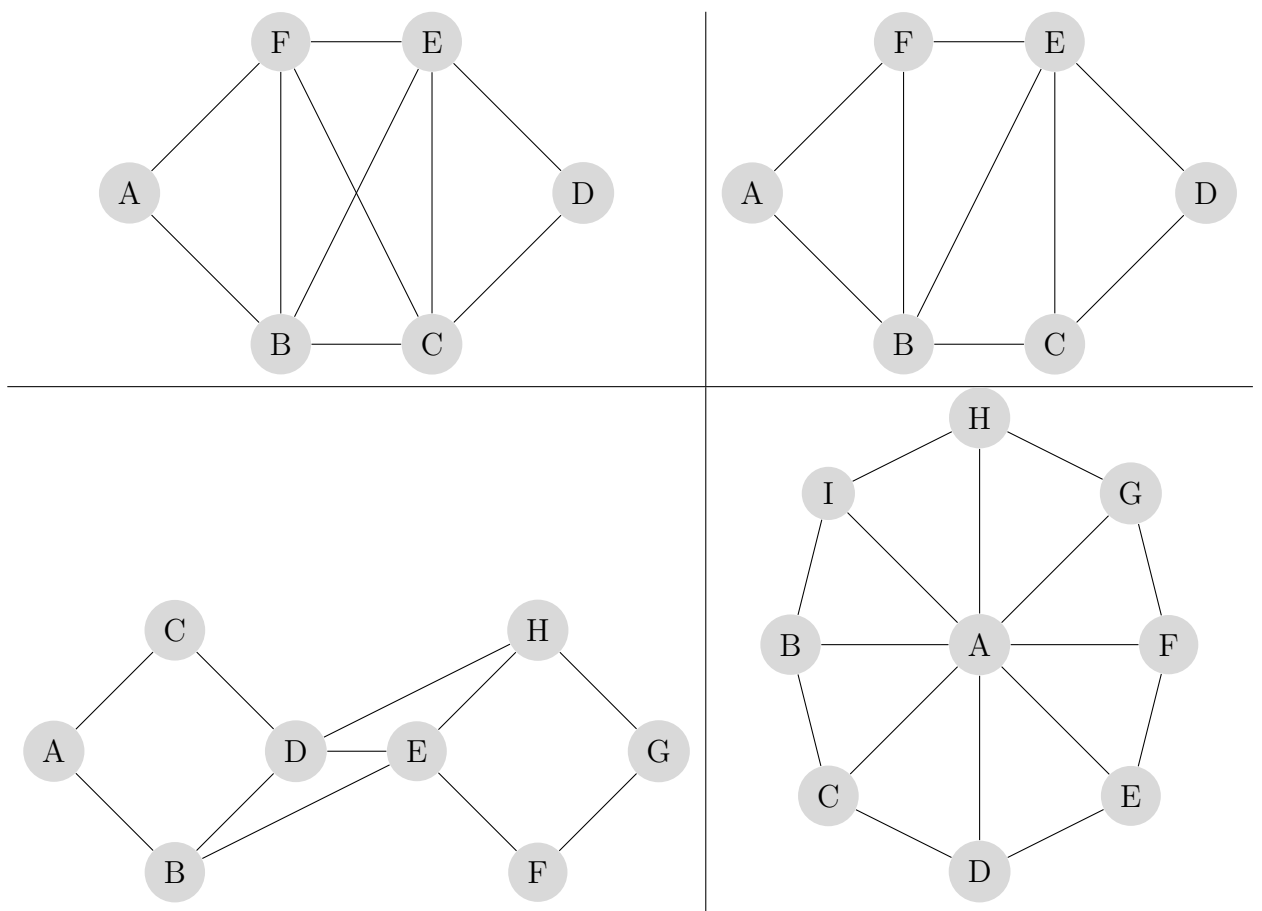


Figure 15.19: Hamiltonian cycle?

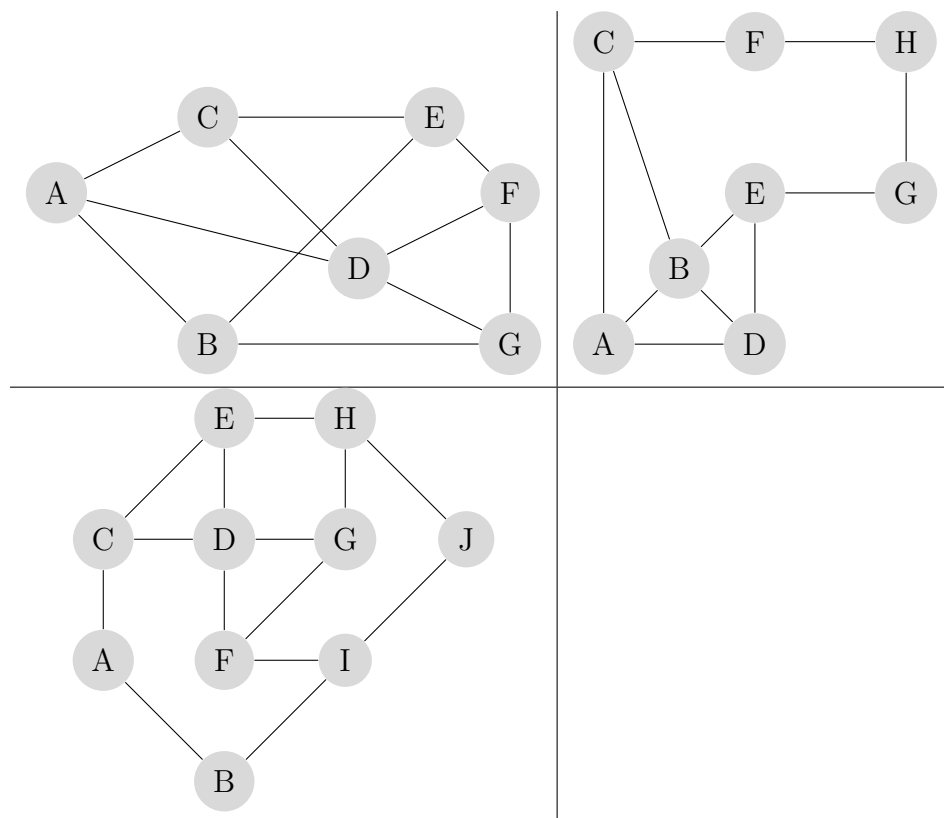


Figure 15.20: Not Hamiltonian

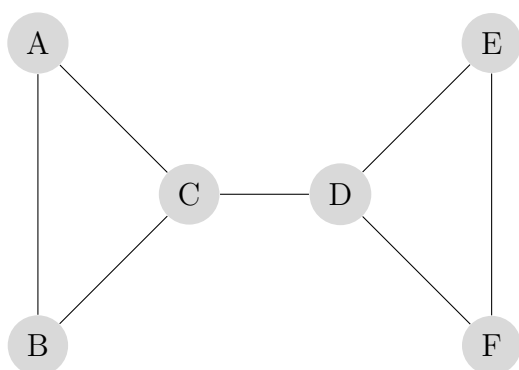
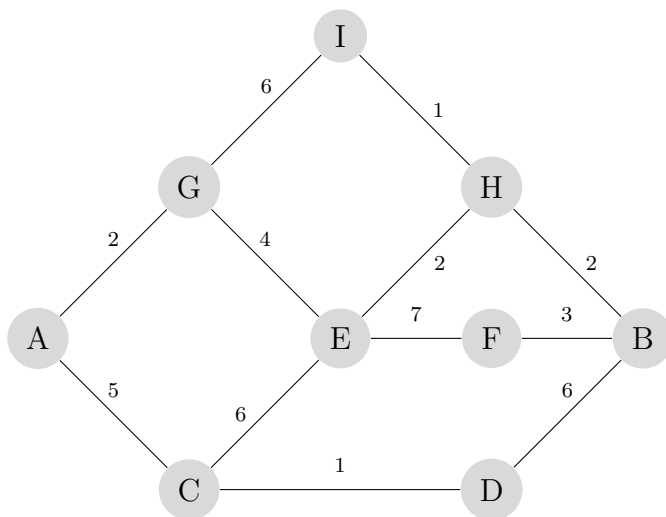


Figure 15.21: Find the shortest path from A to B 

Chapter 16

Probability

In this chapter, we will be a bit more formal about probabilities, as well as commenting on some basic properties. It is helpful to keep the examples we have already seen in mind, like the binomial and the hypergeometric distribution. However, the present set-up is much more general.

16.1 Stochastic variables

Probability is formalized using the notion of *stochastic variables*, also known as *random variables*. We have already seen some examples, but not really a definition of what a stochastic variable is.

Definition 16.1.1. A *measure space* is a set with a notion of size for subsets. Examples are finite sets, with size given by the number of elements in a subset, and intervals, with size given by the length of a subinterval. These examples can be modified by dividing by the total number of elements, or by the length of the whole interval, to produce a measure space of total size one.

A stochastic variable is a function from a measure space of size one. The values of a stochastic variable are called outcomes. If X is a stochastic variable and x is an outcome, we write

$$P(X = x) = \text{size of the set that will give the outcome } x.$$

We say that $P(X = x)$ is the probability that the stochastic variable X will have the value x .

We will look at the case of intervals in the end of this chapter, but for now we take another look at finite sets.

Example 16.1.2. If you roll a die, each of the six outcomes is equally likely. Therefore the measure space is a set of six elements, each with size $1/6$, and the stochastic variable $X = \text{number of eyes}$. Then $P(X = 4) = 1/6$, for instance.

Example 16.1.3. The Norwegian lottery Flax prints the winnings listing on the back of each ticket. The listing gives the number of winning tickets for the various returns, from a total of 4.8 million tickets. Here is the table:

Number	Return
8	1 000 000
2	100 000
56	10 000
4000	1 000
6920	500
16 000	250
12 000	150
16 800	125
124 800	100
232 000	50
856 000	25

The lowest possible return, 25, is equal to the price of the ticket. Let X be the winnings from a randomly drawn ticket. Drawing a random ticket gives a probability of $124\,800/4\,800\,000 = 0.026$ for getting a return of 100 (a net winning of 75). With symbols,

$$P(X = 100) = 0.026.$$

Example 16.1.4. A very common situation is that there are a large number of equally likely possible results from some procedure, and the stochastic variable counts the number that fulfill some conditions. Then we can compute the probability by computing the ratio of results that fulfill the conditions to the total number of results. Sometimes the results that fulfill the conditions are called *successes*.

For example, what is the probability of getting at least 9 when you roll two dice? The total number of possible results is 36, as in this table:

11	12	13	14	15	16
21	22	23	24	25	26
31	32	33	34	35	36
41	42	43	44	45	46
51	52	53	54	55	56
61	62	63	64	65	66

There are ten results where the sum is nine or more, so the probability of getting a result fulfilling this condition is the ratio $10/36 \approx 0.278$.

If you look back to the reasoning behind the formula for the hypergeometric distribution in Theorem 14.3.9, you will see that we used this method.

Comment 16.1.5. We will sometimes write $P(A)$ if A is a subset of a measure space. Then $P(A)$ denotes its “size”. We think of this as the probability that a randomly chosen element in the measure space lies in A . Since a stochastic variable is formally defined as a function from a measure space, we can interpret the notation $P(X = x)$ as being the size of the set of elements in the measure space that are sent to x by the function X . Using the set theoretic notion of inverse image, we can relate these two ways of using P by the equality

$$P(X = x) = P(X^{-1}(\{x\}))$$

and similarly for other conditions on X .

Example 16.1.6. The probability for getting at least 9 when you roll two dice can now be written more formally in two ways. Let X be the outcome. Then

$$P(X \geq 9) = P(\{36, 45, 54, 63, 46, 55, 64, 56, 65, 66\}) = \frac{10}{36}.$$

In some cases, there are finitely many results of varying probabilities, for instance given approximately by a large number of experiments or by decree.

Example 16.1.7. The description of various grades in higher education in Norway was formulated with the goal of getting a distribution which is similar to this: Picking a random exam (that gets a passing grade), the probability that the grade is A, B, C, D, E is respectively about 0.1, 0.25, 0.3, 0.25, 0.1.

One is often interested in stochastic experiments, which are “real-world” situations that produce measure spaces. A stochastic experiment typically has the following features.

1. There is a set of possible outcomes (this is our measure space).
2. The experiment can be repeated “many” times, independently of previous experiments.
3. The result of the experiment is unknown before the experiment is performed.

If these conditions are fulfilled, we can approximate the probability that a given outcome happens by counting the number of occurrences of that result divided by the number of experiments run. The actual probability will then be the limit as the number of experiments goes to infinity. We used this idea implicitly when we considered various properties of the discrete distributions like the binomial distribution or the hypergeometric distribution.

Example 16.1.8. Typical examples of stochastic experiments are measurements. In our present context, measurements from counting are perhaps more natural than other types of measurements.

Consider a street with a pedestrian crossing, regulated by traffic lights. Let X be the number of cars passing the pedestrian crossing in one cycle, between two consecutive red lights. X can be measured by counting the cars passing the light. The set of outcomes is the possible number of cars passing, $0, 1, 2, \dots$. To repeat the experiment, just wait for the light to change again, and count once more.

Similarly, one could count the number of requests made to a web server per hour, the number of ducks in a pond or the number of people on a bus.

Example 16.1.9. How wide is a sheet of A4 paper? We can measure this by using a ruler. The theoretical result is 210 mm, but the result of a measurement can be slightly different for a number of reasons:

- Difficulty with the equipment. The ruler is perhaps inaccurate.
- Human error. The person conducting the experiment might read off the number incorrectly, use the ruler incorrectly (reading from the end of the ruler rather than from the zero spot) or make mistakes for some other reason.
- Natural variation. Producers of paper might have some small changes in width between sheets of paper in the same production batch, or between batches. This could be caused by inaccuracies in their equipment, or it could be a feature (maybe slight variation in paper width makes it easier for a printer to pick up sheets).
- External circumstances. Paper width might change due to temperature or humidity.
- Other causes that we didn't think about.

Definition 16.1.10. Let X be a stochastic variable. We define the expectation

$$\mu_X = E(X) = \sum_x xP(X = x)$$

and the variance

$$\text{Var}(X) = E((X - \mu_X)^2) = \sum_x (x - \mu_X)^2 P(X = x).$$

Many properties of these notions have already been introduced.

16.2 Sets, counting and probabilities

We will now look at some elementary counting arguments, link them to probabilities, and deduce connections between probabilities analogous to those coming from counting. The elementary and necessary observation is that if all outcomes are equally likely, the probability of getting an outcome of a given type is equal to the number of outcomes of that type divided by the total number of outcomes. We will use, and have used, this observation without comment.

Example 16.2.1. In set theory, we have operations like intersection and union. Here are some simple counting examples using a standard deck of cards, that illustrate how probabilities are related to these operations.

We consider a deck of 52 cards, with four suits (spades, hearts, diamonds and clubs), each containing 13 cards from one (ace) to thirteen (king). We draw a random card, and assume that each card is equally likely to be drawn. Thus the probability that our card is the four of diamonds, for instance, is $1/52$.

We will answer these questions.

1. What is the probability that our card is a diamond?
2. What is the probability that our card is a face card (11,12 or 13)?
3. What is the probability that our card is a diamond or a face card?
4. What is the probability that our card is both a diamond and a face card?

In each case, we can count the number of cards that fulfill the conditions. Note that “or” is used in the logical sense (one or the other or both).

1. There are 13 diamonds, so the probability is $13/52 = 1/4$.
2. There are 12 face cards, so the probability is $12/52 = 3/13$.
3. There are 13 diamonds and 9 face cards that are not diamonds, so there is a total of 22 cards fulfilling this condition, so the probability is $22/52$.
4. There are three diamond face cards, so the probability is $3/52$.

How are these probabilities related to each other?

Let $|S|$ be the notation for the number of elements in a set S .

Theorem 16.2.2. *There are connections between set theoretic operations and probabilities as follows:*

a) Let A and B be subsets of some finite set. Then

$$|A \cup B| = |A| + |B| - |A \cap B|.$$

b) Let A and B be subsets of a measure space. Then

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

c) Let A be a subset of a set with n elements. Then

$$|\bar{A}| = n - |A|.$$

d) Let A be a subset of a measure space of size 1. Then

$$P(\bar{A}) = 1 - P(A).$$

Proof. The two set theoretic statements were already discussed in Section 2.5, and are repeated here only to show the analogy between counting and probability. The arguments are also analogous. Note in particular that if we consider probabilities in a finite set where each element is equally likely to be drawn, the equation in part b) is just the equation in part a) divided by the number of elements, and similarly for d) and c).

To prove part a), note that if we count $|A| + |B|$ the elements in $|A \cap B|$ are counted twice, so we must subtract them. To prove part b), note that if we compute $P(A) + P(B)$, the probability that we draw from $A \cap B$ is included twice, so that we must subtract it. $\square$

Example 16.2.3. What is the probability of getting at least one six when you roll two dice? Let A be the set of pairs where the first result is a six, B the set of pairs where the second result is a six. Then we ask for $P(A \cup B)$. We know that $P(A) = 1/6 = P(B)$. Using this to correctly compute $P(A \cup B)$ requires us to understand $A \cap B$ as well. This is the probability that both are sixes, so that $P(A \cap B) = 1/36$. The end result is that

$$P(A \cup B) = P(A) + P(B) - P(A \cap B) = \frac{1}{6} + \frac{1}{6} - \frac{1}{36} = \frac{11}{36}$$

What is the probability that neither of the two dice is a six? This is complementary to the case of at least one being a six, so we are looking at the set $\overline{A \cup B}$. Therefore, the answer is

$$P(\overline{A \cup B}) = 1 - P(A \cup B) = 1 - \frac{11}{36} = \frac{25}{36}$$

Example 16.2.4. In an accident at sea, females and children are evacuated first. Of 200 people on board, 100 are female and 85 are children. In the first round of evacuation, 145 people are evacuated. How many girls were on board? How many grown women were on board? What is the probability that a random person among the evacuated turns out to be male?

Let A represent the set of females, B the set of children. The numbers given can be expressed in terms of these sets:

$$\begin{aligned} |A| &= 100 \\ |B| &= 85 \\ |A \cup B| &= 145 \end{aligned}$$

The set of girls is the intersection between females and children. Turning the connection between $\cap$ and $\cup$ around, we see that

$$|A \cap B| = |A| + |B| - |A \cup B| = 100 + 85 - 145 = 40.$$

So the number of girls is 40. The set of grown women is $A \setminus B$. If we think of A as the new universe, this means that we can use the formula for complements in this form:

$$|A \setminus B| = |A| - |A \cap B| = 100 - 40 = 60.$$

There are 60 grown women on board.

Finally, the set of males among the evacuated, that is the set of boys, can be represented in several ways. One is as $B \setminus A \cap B$. The number of boys is therefore

$$|B \setminus A \cap B| = |B| - |A \cap B| = 85 - 40 = 45.$$

Among the 145 evacuated, 45 are boys, so the probability of choosing one of these is

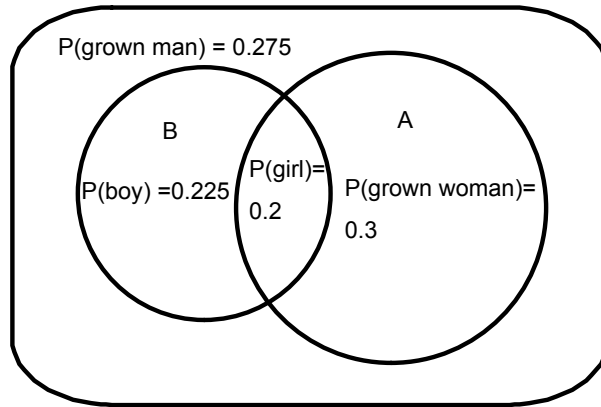
$$P(\text{choosing a boy}) = \frac{45}{145} = 0.31$$

In the next section, we will think about this as a conditional probability $P(\bar{A}|A \cup B)$.

Comment 16.2.5. As for sets, it is common to draw Venn diagrams when reasoning about probabilities. If the parts of the Venn diagram are labelled with probabilities, we should see that the total probability is one.

Example 16.2.6. Figure 16.1 shows a labelled Venn diagram with probabilities, using the set-up from Example 16.2.4.

Figure 16.1: Probabilities for choices male/female and adult/child



16.3 Conditional probabilities

We have seen examples of conditional probabilities when counting letters in text. Now we will be more systematic about these notions, and deduce a few practical formulas.

Definition 16.3.1. Let A and B be two sets of possible outcomes in the same experiment (or subsets of a given measure space). Then we define the conditional probability of A given B by the formula

$$P(A|B) = \frac{P(A \cap B)}{P(B)}.$$

This formula makes sense as long as $P(B) \neq 0$, and we require this to define the conditional probability.

Think about this definition as restricting the universe to consist of B only, discarding anything outside B . We ask how large the part of A that is inside B is compared to the size of B .

Example 16.3.2. When we roll two dice, let A be the outcome that the sum is at least 8. Let B be the outcome that the first die is a six, and let C be the outcome

that the first die is a one. Counting the number of possibilities, we see that

$$\begin{aligned} P(A) &= \frac{15}{36} \\ P(B) &= \frac{6}{36} \\ P(C) &= \frac{6}{36} \\ P(A \cap B) &= \frac{5}{36} \\ P(A \cap C) &= \frac{0}{36} \end{aligned}$$

By definition

$$P(A|B) = \frac{P(A \cap B)}{P(B)} = \frac{5/36}{6/36} = \frac{5}{6}$$

and

$$P(A|C) = \frac{P(A \cap C)}{P(C)} = \frac{0/36}{6/36} = \frac{0}{6} = 0.$$

If we consider only the elements in B , we have six elements, and five of them gives a sum of at least 8. If we consider only the elements in C , we have six elements, and none of these results gives a sum larger than 7.

Example 16.3.3. If we consider also $P(B|A)$, the elements in $A \cap B$ are the same, but now we compare to the size of A , not the size of B . Fiddling with the defining equations gives a connection between the two conditional probabilities.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad \text{gives} \quad P(A \cap B) = P(A|B)P(B)$$

and

$$P(B|A) = \frac{P(A \cap B)}{P(A)} \quad \text{gives} \quad P(A \cap B) = P(B|A)P(A).$$

Therefore

$$P(A|B)P(B) = P(A \cap B) = P(B|A)P(A) \tag{16.1}$$

which is the multiplicative property for $P(A \cap B)$, or finally

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}. \tag{16.2}$$

This equation is often referred to as *Bayes' theorem*.

Example 16.3.4. Since A and $\bar{A}$ together cover the whole space of outcomes, it also covers B . We have that

$$P(\bar{A}|B) = \frac{P(\bar{A} \cap B)}{P(B)}.$$

Since $(A \cap B) \cup (\bar{A} \cap B) = B$, and the union is disjoint, we see that

$$P(\bar{A}|B) + P(A|B) = 1.$$

In the same way we can reason about B and $\bar{B}$, and end up with the formula

$$P(A) = P(A|B)P(B) + P(A|\bar{B})P(\bar{B}).$$

Example 16.3.5. We continue with the numbers from Example 16.2.4. We draw random people from the ship in question. The probability of choosing a female is $P(A) = \frac{100}{200} = 0.5$. The set of evacuated people is $A \cup B$, a set of size 145, so the probability that a random person is evacuated is $P(A \cup B) = \frac{145}{200} = 0.725$. The conditional probability that an evacuee is female is

$$P(A|A \cup B) = \frac{P(A \cap (A \cup B))}{P(A \cup B)} = \frac{P(A)}{P(A \cup B)} = \frac{0.5}{0.725} = 0.69.$$

Comment 16.3.6. It can be difficult to distinguish between $P(A \cap B)$, $P(A|B)$ and $P(B|A)$. These three numbers measure the size of the same set, but relative to different sets. We can visualize the difference using Venn diagrams of different sizes, see Figure 16.2.

16.4 Correlation and independence

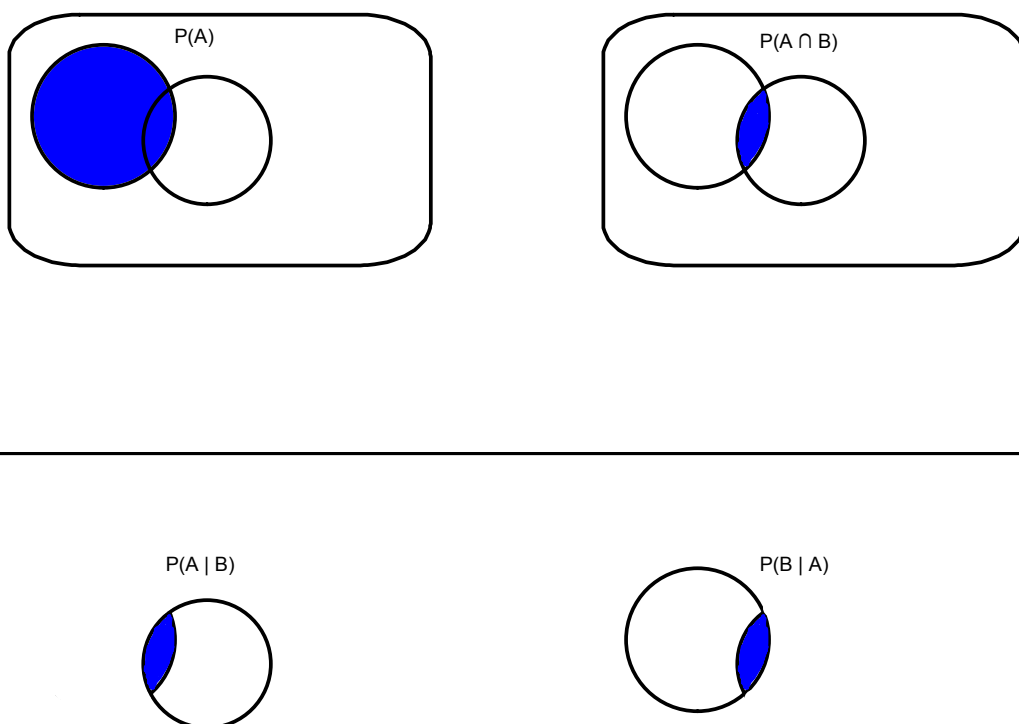
Let X and Y be two stochastic variables. If we know the outcome of X , will that alter our understanding of Y ? Or are the two variables completely unrelated? Consider two very simple examples.

Example 16.4.1. You see the outline of a person in a dark alley. Is the person a man or a woman? As a starting point, the probability for either possibility is $1/2$. This describes the stochastic variable Y , which has the two values “man” and “woman”.

Then you realize that the person is approximately 190 cm tall. Based on this extra information, you would conclude that it is more likely that the person is a man than a woman. So if X is the stochastic variable which assigns the height to a person, we assume that large values of X will change the value we expect to get from Y . In this case we say that X and Y are dependent, or that they are correlated.

Assume that you notice something else, instead of the height. You notice that the person has dark hair. This extra piece of information does not help you determine whether you see a man or a woman. The stochastic variable that assigns hair colour to a person is *independent* of Y .

Figure 16.2: $P(A \cap B)$, $P(A|B)$ and $P(B|A)$ measure the same set inside different sets



These properties can be formalized by giving a mathematical description of the terms. It is simpler to assume that all our stochastic variables have numerical values (rather than vague descriptions like hair colour).

Definition 16.4.2. Two stochastic variables X and Y are *independent* if

$$P(X = x|Y = y) = P(X = x)$$

for all x and y .

Read the equation carefully. The probability that X gets a certain value never changes because of values of Y .

Being independent can be rephrased as

$$P(X = x \wedge Y = y) = P(X = x) \cdot P(Y = y) \quad \text{for all } x, y.$$

So we see that independence is a symmetric relation on stochastic variables. This condition is really a reformulation of the multiplicative property in Equation (16.1) for stochastic variables. The reformulation is valid only for independent variables, whereas Equation (16.1) holds in general.

Example 16.4.3. Two different dice produce independent stochastic variables X and Y for the number of eyes. Therefore, the probability for getting two sixes is

$$P(X = 6 \wedge Y = 6) = P(X = 6)P(Y = 6) = \frac{1}{6} \cdot \frac{1}{6} = \frac{1}{36}.$$

Draw two cards from a deck of cards, and ask whether both the cards are aces or not. The probability that the first card is an ace is $1/13$, and the probability that the second card is an ace is also $1/13$. However, this is not an example of independent variables, so the probability that both are aces is not $1/13^2$.

Let A be the outcome that the first card is an ace, B that the second is. Then

$$\begin{aligned} P(A) &= 1/13 \\ P(B) &= 1/13 \\ P(B|A) &= \frac{3}{51} < \frac{1}{13} \\ P(B|\bar{A}) &= \frac{4}{51} > \frac{1}{13} \\ P(A \cap B) &= P(B|A)P(A) = \frac{3}{51} \cdot \frac{1}{13} = 4.5 \cdot 10^{-3} \end{aligned}$$

For comparison, $1/13^2 = 5.9 \cdot 10^{-3}$.

Independence, and lack of independence, is closely connected to *correlation*.

Definition 16.4.4. Let X and Y be two stochastic variables defined on the same measure space. Let furthermore μ_X, μ_Y be the expectations of X and Y , and σ_X and σ_Y the standard deviations. Then we define the *covariance* between X and Y by

$$\text{Cov}(X, Y) = E((X - \mu_X)(Y - \mu_Y))$$

and the *correlation coefficient*

$$\rho(X, Y) = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y}.$$

Not that the formula for the covariance needs a double sum, we sum over all possible pairs (x, y) to compute the expectation $E((X - \mu_X)(Y - \mu_Y))$.

There is a lot of small computations involved in producing a correlation coefficient. Let us look at a very small example where we can compute everything by hand, keeping in mind that more meaningful examples will demand using a computer.

Example 16.4.5. Consider a small game where we roll a die. There are two ways to win. X returns 1 if the result is odd, 0 if the result is even. Y returns 2 if the result is ≥ 4 , 1 if the result is 2 or 3, and 0 if the result is 1. The probabilities for each pair of results can be found by counting. The information is conveniently organized in a table as follows:

y \ x	x		sum Y
	0	1	
0	0	1/6	1/6
1	1/6	1/6	1/3
2	1/3	1/6	1/2
sum X	1/2	1/2	1

The sets that are counted are $\emptyset$ for the zero, $\{4, 6\}$ for the 1/3, and singletons for the rest of the entries. The four numbers $\mu_X, \mu_Y, \sigma_X, \sigma_Y$ are computed from the bottom row and the rightmost column only. We find:

$$\begin{aligned} \mu_X &= \sum_x xP(X = x) = 0 \cdot 1/2 + 1 \cdot 1/2 = 1/2 \\ \mu_Y &= \sum_y yP(Y = y) = 0 \cdot 1/6 + 1 \cdot 1/3 + 2 \cdot 1/2 = 4/3 \\ \sigma_X^2 &= \sum_x (x - \mu_X)^2 P(X = x) = (0 - 1/2)^2 \cdot 1/2 + (1 - 1/2)^2 \cdot 1/2 = (1/2)^2 \\ \sigma_Y^2 &= \sum_y (y - \mu_Y)^2 P(Y = y) = (0 - 4/3)^2 \cdot 1/6 + (1 - 4/3)^2 \cdot 1/3 + (2 - 4/3)^2 \cdot 1/2 \\ &= 5/9 = 0.745^2 \end{aligned}$$

Let us then compute the two newly defined numbers. To find the covariance, note that there are six pairs (x, y) to consider, corresponding to the six positions in the middle of the table above.

$$\begin{aligned}
Cov(X, Y) &= \sum_{x,y} (x - \mu_X)(y - \mu_Y)P(X = x \wedge Y = y) \\
&= (0 - 1/2)(0 - 4/3) \cdot 0 \\
&+ (1 - 1/2)(0 - 4/3) \cdot 1/6 \\
&+ (0 - 1/2)(1 - 4/3) \cdot 1/6 \\
&+ (1 - 1/2)(1 - 4/3) \cdot 1/6 \\
&+ (0 - 1/2)(2 - 4/3) \cdot 1/3 \\
&+ (1 - 1/2)(2 - 4/3) \cdot 1/6 \\
&= -1/6
\end{aligned}$$

Finally, we compute ρ :

$$\rho(X, Y) = \frac{Cov(X, Y)}{\sigma_X \sigma_Y} = \frac{-1/6}{0.5 \cdot 0.745} = -0.447$$

Comment 16.4.6. Covariance and correlation tell us about how two stochastic variables are connected. It is easier to compare the correlation coefficient, since dividing by the standard deviation normalizes the numbers:

The correlation coefficient satisfies $-1 \leq \rho \leq 1$.

If X and Y are large at the same time, small at the same time, ρ will be positive. The better X and Y fit together, the closer ρ will be to 1. If X is small when Y is large, and vice-versa, ρ will be negative. The stronger this effect is, the closer ρ will be to -1 . Finally, if there is no connection between large values of X and Y , ρ will be (close to) zero. If X and Y are independent, $\rho = 0$. It is also possible that terms of different signs cancel out when X and Y are dependent, so we can not conclude from $\rho = 0$ that X and Y are independent.

Comment 16.4.7. Let us consider how units fit with the various numbers we have introduced. If X is measured in meters, say, then

$$\mu_X = E(X) = \sum_x xP(X = x)$$

will be a sum of x s, measured in meters, multiplied by numbers $P(X = x)$. Therefore the expectation has the same unit as X . If you consider the variance instead,

$$\text{Var}(X) = \sum_x (x - \mu_X)^2 P(X = x),$$

we see that $(x - \mu_X)^2$ has meters times meters, square meters, as unit. Therefore the variance also has unit m^2 . So variance cannot be compared directly to the original variable. Standard deviation, on the other hand, is the square root of the variance,

and therefore has unit $\sqrt{\text{m}^2} = \text{m}$. So the standard deviation can be compared to the original variable. This will be an important topic later.

The covariance has unit equal to the product of the units of X and Y . The same is true for $\sigma_X\sigma_Y$, so in the fraction defining the correlation coefficient, the numerator and the denominator have the same unit. The result is that ρ is unitless, it is just a number. See also Comment 14.1.4 where similar remarks were made about the empirical versions.

Theorem 16.4.8. *Let X and Y be two stochastic variables. Then*

$$\text{Var}(X + Y) = \text{Var}(X) + 2\text{Cov}(X, Y) + \text{Var}(Y).$$

Note that if X and Y are independent, so that $\text{Cov}(X, Y) = 0$, this reduces to the simpler formula $\text{Var}(X + Y) = \text{Var}(X) + \text{Var}(Y)$.

Proof. The proof of the above theorem is an exercise in manipulating the properties of variance and expectation. By definition of variance,

$$\begin{aligned} \text{Var}(X + Y) &= E(((X + Y) - (\mu_X + \mu_Y))^2) \\ &= E(X^2 + 2XY + Y^2 - 2X\mu_X - 2X\mu_Y - 2Y\mu_X - 2Y\mu_Y + \mu_X^2 + 2\mu_X\mu_Y + \mu_Y^2) \\ &= E(X^2 - 2X\mu_X + \mu_X^2) \\ &\quad + E(2(XY - X\mu_Y - Y\mu_X + \mu_X\mu_Y)) \\ &\quad + E(Y^2 - 2Y\mu_Y + \mu_Y^2) \\ &= E(X^2) - 2\mu_X E(X) + \mu_X^2 \\ &\quad + 2E((X - \mu_X)(Y - \mu_Y)) \\ &\quad + E(Y^2) - 2\mu_Y E(Y) + \mu_Y^2 \\ &= \text{Var}(X) + 2\text{Cov}(X, Y) + \text{Var}(Y) \end{aligned}$$

In the last step, we used that $\mu_X = E(X)$ to simplify

$$E(X^2) - 2\mu_X E(X) + \mu_X^2 = E(X^2) - 2\mu_X^2 + \mu_X^2 = E(X^2) - \mu_X^2 = \text{Var}(X)$$

□

16.5 Continuous distributions and integration

Some types of stochastic variables can have any real value. For these situations, probabilities are expressed in terms of integrals rather than sums, and are assigned to intervals, not single values. Since the assumed background in integration is limited, many details are skipped in this topic. In addition, many interesting stochastic variables are connected to integrals that we are unable to compute exactly, so that proficiency in techniques of integration is of limited use.

Example 16.5.1. Attach probabilities to an interval in this way. Integrate the function f over the interval, where

$$f(x) = \begin{cases} 0 & x < 0 \\ x & 0 \leq x < 1 \\ 2 - x & 1 \leq x < 2 \\ 0 & x \geq 2 \end{cases}$$

If we let X be the stochastic variable described in this way, we will write either $P(X \in [a, b])$ or $P(a \leq X < b)$ for the probability that X lies in the interval. For instance

$$\begin{aligned} P(0.5 \leq X < 1.5) &= \int_{0.5}^{1.5} f(x) dx \\ &= \int_{0.5}^1 x dx + \int_1^{1.5} (2 - x) dx \\ &= \left. \frac{x^2}{2} \right|_{0.5}^1 + \left(2x - \frac{x^2}{2} \right) \Big|_1^{1.5} \\ &= 3/8 + 3/8 = 3/4. \end{aligned}$$

For another sample computation,

$$P(3 \leq X < 5) = \int_3^5 f(x) dx = \int_3^5 0 dx = 0.$$

Comment 16.5.2. In connection with probability and statistics, integrals are always definite. The indefinite integral, or anti-derivative, has no independent meaning other than as a computational tool for definite integrals.

Definition 16.5.3. Assume that X is a stochastic variable for which we can discuss the probability that X lies in an interval (any interval). Then we define the cumulative distribution function of X by

$$F(x) = P(X \leq x) = P(X \in (-\infty, x]).$$

If F' exists, we call it the density function of X , usually written with the same lowercase letter:

$$f(x) = F'(x).$$

The expectation of X is then given by

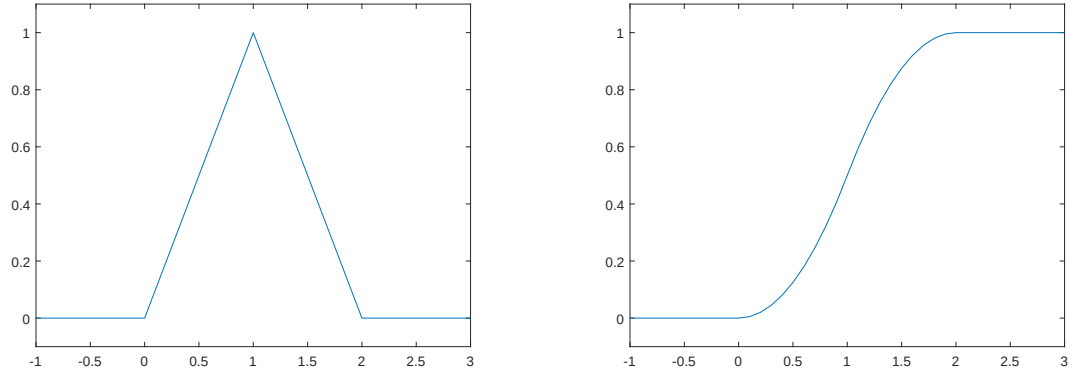
$$\mu = E(X) = \int_{-\infty}^{\infty} x f(x) dx.$$

The variance is

$$\sigma^2 = \text{Var}(X) = \int_{-\infty}^{\infty} (x - \mu)^2 f(x) dx.$$

In this case we sometimes call X a *continuous* stochastic variable. Similarly, f is called a continuous probability distribution.

Figure 16.3: A continuous distribution and its cumulative probability function



Comment 16.5.4. Since we are only interested in integration of f , everything works if there are a few points where f is not defined, for instance points where F has a “corner”.

Example 16.5.5. Continuing with the function from Example 16.5.1, we can compute the function F explicitly. Since f is given by different formulas in different parts of its domain of definition, the computation of F is partitioned similarly.

$$\begin{aligned}
 F(x) &= \int_{-\infty}^x 0 \, dt = 0 && \text{for } x < 0 \\
 F(x) &= \int_{-\infty}^0 0 \, dt + \int_0^x t \, dt = \frac{x^2}{2} && \text{for } 0 \leq x < 1 \\
 F(x) &= \int_{-\infty}^0 0 \, dt + \int_0^1 t \, dt + \int_1^x (2-t) \, dt \\
 &= \frac{1}{2} + \left(2t - \frac{t^2}{2}\right)\Big|_1^x && \text{for } 1 \leq x < 2 \\
 &= -1 + 2x - \frac{x^2}{2} \\
 F(x) &= 1 && \text{for } x \geq 2
 \end{aligned}$$

Example 16.5.6. In Section 14.4, we looked at the Poisson distribution. We also thought about how to sample from the distribution, leading to the stochastic variable that returns the time before the first occurrence. This is a continuous distribution.

Recall that if $X \sim \text{Po}(\lambda t)$, the probability that there hasn’t been any occurrence until time t is

$$P(X = 0) = e^{-\lambda t}.$$

We now let T be the time before the first occurrence, so that $T < t$ is the same as $X \geq 1$. This means that

$$F(t) = P(T < t) = P(X \geq 1) = 1 - P(X = 0) = 1 - e^{-\lambda t} \quad \text{for } t > 0.$$

If we differentiate, we get that

$$f(t) = F'(t) = \lambda e^{-\lambda t} \quad \text{for } t > 0.$$

Formally, we can extend the domain of definition of f to the whole real line by declaring it to be zero for negative values. As an example, if there are on average $\lambda = 3$ requests per minute to a server, the probability that the first request comes in the first minute is

$$\int_0^1 f(t) dt = \int_0^1 \lambda e^{-\lambda t} dt = -e^{-\lambda t} \Big|_0^1 = -e^{-3} + e^0 = 1 - e^{-3} = 0.95$$

Theorem 16.5.7. *Some properties of continuous variables, which can be proved by being more formal about the definitions:*

1. The total probability is 1, $\int_{-\infty}^{\infty} f(x) dx = 1$.
2. Probability is non-negative, $f(x) \geq 0$.
3. $F(x) = \int_{-\infty}^x f(t) dt$
4. $\lim_{x \rightarrow -\infty} F(x) = 0$.
5. Variance has a slightly different formula as well:

$$\text{Var}(X) = \int x^2 f(x) dx - \mu^2$$

Other formulas that we have seen already are also valid for continuous stochastic variables, but some are more difficult. For instance, in the definition of covariance, the formula

$$\text{Cov}(X, Y) = E((X - \mu_X)(Y - \mu_Y))$$

is valid, but the right hand side involves double integration, a topic for advanced calculus classes.

16.6 Exercises

Exercise 16.6.1. A stochastic variable X is defined by this table of probabilities:

x	0	2	3	4	6	7
$P(X = x)$	0.2	0.2	0.1	0.2	0.05	0.25

Compute the expectation and the variance of X .

Exercise 16.6.2. Show that

$$P(A) = P(A|B)P(B) + P(A|\overline{B})P(\overline{B}).$$

Exercise 16.6.3. Below is a joint probability distribution for the two stochastic variables X and Y .

$y \backslash x$	5	6	7	sum Y
1	0.0194	0.5510	0.1471	0.7176
2	0.0626	0.1148	0.1051	0.2824
sum X	0.0820	0.6658	0.2522	1

(a). Compute the correlation coefficient between X and Y . Can you conclude from this whether X and Y are independent?

(b). Determine, in some other way, whether X and Y are independent.

Exercise 16.6.4. Below is a joint probability distribution for the two stochastic variables X and Y .

$y \backslash x$	1	2	3	sum Y
0	0.02	0.06	0.02	0.1
1	0.1	0.3	0.1	0.5
2	0.08	0.24	0.08	0.4
sum X	0.2	0.6	0.2	1

(a). Compute the correlation coefficient between X and Y . Can you conclude from this whether X and Y are independent?

(b). Determine, in some other way, whether X and Y are independent.

Exercise 16.6.5. In this exercise, we consider a set of 200 people. Here is some information about these people:

- 40 are retired (call this set A), 120 are women (call this set B) and 115 use public transport (C).
- 75% of the retired people use public transport
- Being retired is equally likely for men and women.
- 65 women use public transport.

We consider probabilities by picking a person from this group at random. I recommend that you draw a Venn diagram, and mark various parts of it with probabilities or the number of people as you go along.

- (a). What is $P(A)$, $P(B)$ and $P(C)$?
- (b). What is $P(A \cap B)$, $P(A \cup C)$ and $P(B \cap C)$?
- (c). What is $P(A|B)$, $P(B|A)$ and $P(A|C)$?
- (d). If there are 15 retired women using public transport, can you compute the probability of picking a working man that doesn't use public transport?

Exercise 16.6.6. Consider the function

$$f(x) = \begin{cases} c \cdot x^2 & \text{if } 0 \leq x < 1 \\ 0 & \text{otherwise} \end{cases}$$

Here c is some constant. We want f to be the density function of a probability distribution, and X is a stochastic variable with this distribution.

- (a). Determine the value of c that makes f the density function of a probability distribution. Use this value in the rest of the exercise.
- (b). Compute the expectation of X .
- (c). Compute the variance of X .

Chapter 17

The normal distribution

One of the most important questions in science or engineering is how accurate a measurement is. There will almost always be some possible sources of error. In addition, the phenomenon that is measured can also fluctuate, so that even a completely correct measurement at a point only approximates the behaviour in a region. In statistics, we will quantify the margin of error. The goal is to be able to say precisely how much we can trust a measurement.

In this chapter, we will look at the most important distribution in statistics, the normal, or Gaussian, distribution. This is the default tool to address the question of accuracy of measurements.

17.1 The normal distributions

The normal distributions are especially important in statistics. The definition is not easy to understand, and the main properties are difficult to obtain, but the distribution is not so difficult to use in practical situations.

Definition 17.1.1. A stochastic variable is *normally distributed* with expectation μ and variance σ^2 if its density function is

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

If we say only *the normal distribution*, we think of the case $\mu = 0$, $\sigma = 1$ (the standard normal distribution). We write $X \sim N(\mu, \sigma^2)$ to signify that X is a stochastic variable with this distribution. The standard normal distribution is then $X \sim N(0, 1)$.

The normal distribution is also sometimes called the *Gaussian* distribution.

The function f is not easy to deal with directly. In particular, the indefinite integral of f can not be found within the class of *elementary functions*. Nevertheless,

the function is so important that a lot of numerical information is available for the integral. For instance, a web search will easily produce tables of values for the normal distribution.

Here is a short example showing how to use the function `erf`, which is a built-in function in MATLAB.

Example 17.1.2. The definition of the `erf`-function is

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt$$

Comparing this to the definition of the normal distribution, we see that $\mu = 0$ and $\sigma = 1/\sqrt{2}$ almost fits, but we have to multiply by a constant. Also, integration starts at 0, not $-\infty$. `erf` is short for “error function”.

Let $X \sim N(0, 1)$ be a standard normally distributed variable. To compute $P(X \leq x)$ we can do the following:

$$P(X \leq x) = 0.5 + 0.5 \cdot \operatorname{erf}(x/\sqrt{2}).$$

To see that this makes sense, note first that because of symmetry,

$$\int_{-\infty}^0 \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} dx = \int_0^{\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}} dx = \frac{1}{2}.$$

Also, using the substitution $u = \sqrt{2}t$, we get that

$$\operatorname{erf}(x/\sqrt{2}) = \frac{2}{\sqrt{\pi}} \int_0^{x/\sqrt{2}} e^{-t^2} dt = \frac{2}{\sqrt{\pi}} \int_0^x e^{-u^2/2} du / \sqrt{2}.$$

Dividing by two and adding 1/2 gives the equation we are interested in. For example,

$$P(X \leq 0.8) = 0.5 + 0.5 \cdot \operatorname{erf}(0.8/\sqrt{2}) = 0.7881.$$

Now that we know how to compute values of $P(X \leq x)$, we can look at some important properties.

Let $X \sim N(0, 1)$.

1. $P(X \geq x) = P(X \leq -x)$.
2. $P(X \leq 0) = 0.5$.
3. $P(-x \leq X \leq x) = 1 - 2P(X \leq -x)$.

All these properties are consequences of the symmetry of the defining integral. Note also that we can form similar expressions using symmetry around μ for $X \sim N(\mu, \sigma^2)$.

Some especially interesting values for $X \sim N(0, 1)$ are

1. $P(X \leq 2.326) = 0.99$.
2. $P(X \leq 1.96) = 0.975$.
3. $P(X \leq 1.645) = 0.95$.
4. $P(X \leq 1.282) = 0.9$.

Example 17.1.3. Let $Z \sim N(\mu, \sigma^2)$ and $X = \frac{Z-\mu}{\sigma}$. Then $X \sim N(0, 1)$. More generally, if Z is a normal distribution, $aZ + b$ is also. The exact parameters for X follow from standard properties of expectation and variance. Turning around the equation for X , we see that

$$Z = \sigma X + \mu.$$

Using this, we can also compute things like

$$P(Z \leq \mu + \sigma) = P(\sigma X + \mu \leq \mu + \sigma) = P(X \leq 1) = 0.8413.$$

Similarly, we can show that

$$P(\mu - \sigma \leq Z \leq \mu + \sigma) = 0.683$$

and

$$P(\mu - 2\sigma \leq Z \leq \mu + 2\sigma) = 0.954.$$

These properties might help us understand what standard deviation really is.

Finally, we will state the equation we need to compute values of normally distributed variables in MATLAB for future reference. Let $X \sim N(\mu, \sigma^2)$.

$$P(X \leq x) = 0.5 + 0.5 \cdot \operatorname{erf}\left(\frac{x - \mu}{\sigma\sqrt{2}}\right). \quad (17.1)$$

We will need to look at simplified pictures of the graph of the density function f . The shaded area in Figure 17.1 represents the probability $P(X \leq x)$.

Similarly, the area in Figure 17.2 represents the probability $P(X > x)$.

Finally, in Figure 17.3, the shaded area shows the probability that X lies in the given interval.

17.2 Confidence intervals

Normally distributed variables often arise from measurements. If we want to measure some unknown quantity, like the distance between two mountain peaks, there will always be some margin of error. The idea is to model the measurement by a normally distributed variable. The expectation is the unknown quantity that we want to compute. The variance is a number related to the quality of the measurement (quality of the equipment used, deviation due to human error and so forth).

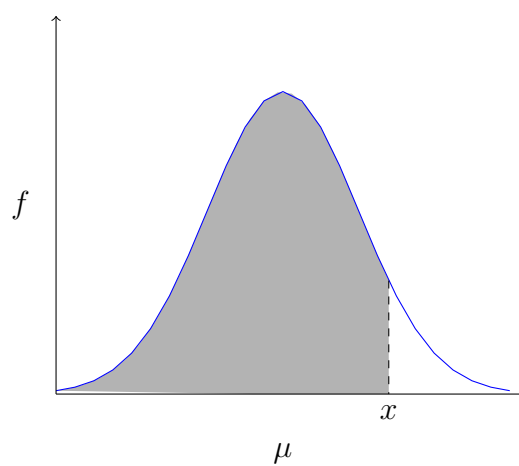
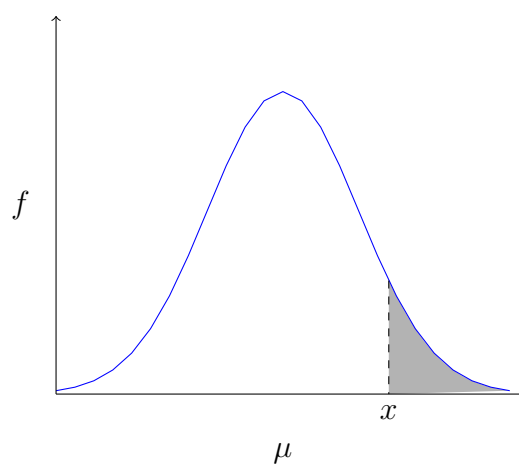
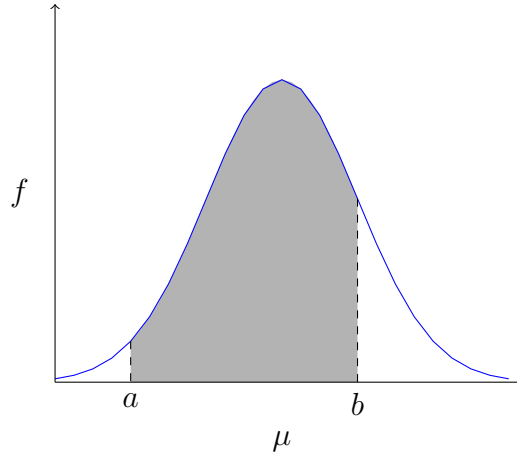
Figure 17.1: $P(X \leq x)$ Figure 17.2: $P(X > x)$ 

Figure 17.3: $P(a < X \leq b)$ 

Example 17.2.1. Let μ be the distance between two mountain peaks. We don't know what this number is. The distance is measured by some standard technique, using well-tested equipment. The variance of a measurement is known to be $\sigma^2 = 3^2$ meters. We assume that the measurement is given by a stochastic variable X with $X \sim N(\mu, \sigma^2)$. Say first that we measure the distance, yielding a result of $X = 3245$ meters. Then we would guess that μ is close to 3245, and the margin of error is in some sense given by σ . For instance, the probability that such a measurement is between $\mu - 2\sigma$ and $\mu + 2\sigma$ is 0.954. In other words, there is a 95.4% probability that if we construct an interval $X \pm 2\sigma$, this interval will contain μ . If we put in the numbers, our guess is that μ can be found in the interval $[3245 - 2 \cdot 3, 3245 + 2 \cdot 3] = [3239, 3251]$.

What can we deduce about the unknown μ if three independent measurements produce the numbers

$$X_1 = 3245, X_2 = 3247, X_3 = 3245?$$

Let us consider the mean of the three measurements, $\bar{X} = \frac{X_1 + X_2 + X_3}{3}$. Then $\bar{X} \sim N(\mu, \sigma^2/3)$! We can therefore guess the value of μ based on the three measurements, with a smaller margin of error. We usually write $\hat{\mu}$ for a stochastic variable used to guess the value of μ . Based on this series of measurements, we get

$$\hat{\mu} = \bar{X} = \frac{X_1 + X_2 + X_3}{3} = \frac{3245 + 3247 + 3245}{3} = 3245.67$$

Reasoning with intervals as before, our guess is that the true distance μ can be found in the interval $\bar{X} \pm 2\sigma/\sqrt{3} = [3242.2, 3249.1]$. This interval is smaller, since we believe we can guess the true value of μ more accurately based on three measurements than on only one.

Let us formalize this a little bit.

Definition 17.2.2. Let μ be an unknown quantity. We say that a stochastic variable $\hat{\mu}$ is an estimator for μ if we use this stochastic variable to guess the value of μ . Typically, we want $E(\hat{\mu}) = \mu$ and $\text{Var}(\hat{\mu})$ should be small.

The most important example is if μ can be measured by n independent stochastic variables $X_i \sim N(\mu, \sigma^2)$. Then we set

$$\hat{\mu} = \bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \sim N(\mu, \sigma^2/n).$$

Sometimes $\hat{\mu}$ is called a point estimator, to underline that there is a single value, and to distinguish it from a confidence interval:

Definition 17.2.3. Let μ be an unknown quantity, $\hat{\mu}$ an estimator for μ . An interval $[a, b]$ constructed from $\hat{\mu}$ is called a $1 - \alpha$ confidence interval, if the probability that such an interval actually contains μ is $1 - \alpha$.

In the most important case

$$\hat{\mu} = \bar{X} = \frac{1}{n} \sum_{i=1}^n X_i \sim N(\mu, \sigma^2/n).$$

A $1 - \alpha$ confidence interval can be chosen as

$$\bar{X} \pm u_{\alpha/2} \frac{\sigma}{\sqrt{n}} = \left[\bar{X} - u_{\alpha/2} \frac{\sigma}{\sqrt{n}}, \bar{X} + u_{\alpha/2} \frac{\sigma}{\sqrt{n}} \right] \quad (17.2)$$

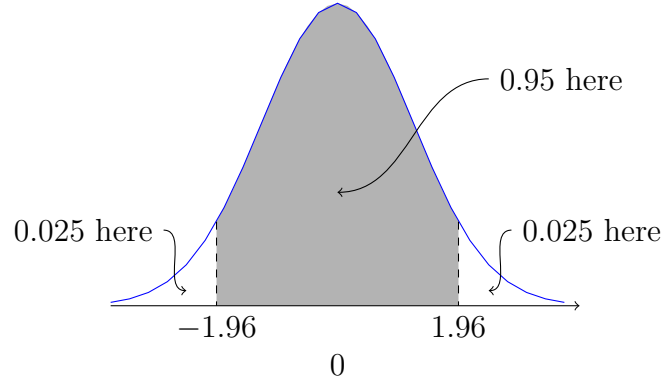
Here $u_{\alpha/2}$ is defined by the equation that if $Z \sim N(0, 1)$,

$$P(-u_{\alpha/2} < Z \leq u_{\alpha/2}) = \int_{-u_{\alpha/2}}^{u_{\alpha/2}} f(x) dx = 1 - \alpha.$$

Comment 17.2.4. It is difficult to compute the u -values exactly, so we will need a table of precomputed values. Here are some of them:

1. $u_{0.005} = 2.576$ corresponding to 99% confidence interval.
2. $u_{0.025} = 1.96$ corresponding to 95% confidence interval.
3. $u_{0.05} = 1.645$ corresponding to 90% confidence interval.

$u_{0.05}$ is called the 0.05-quantile of the standard normal distribution. Alternatively, we can compute them in MATLAB using the inverse of the **erf**-function, **erfinv** (also

Figure 17.4: $1 = 0.025 + 0.95 + 0.025$, split at $\pm u_{\alpha/2} = \pm u_{0.025} = \pm 1.96$ 

a built-in function). Turning around the equation combining **erf** and probabilities in the normal distribution, we get that

$$u_{\alpha} = \sqrt{2} \cdot \text{erfinv}(1 - 2 \cdot \alpha)$$

or directly for $u_{\alpha/2}$,

$$u_{\alpha/2} = \sqrt{2} \cdot \text{erfinv}(1 - \alpha)$$

Comment 17.2.5. Since μ is a fixed quantity, it is formally wrong to say that there is a 95.4% probability that μ lies in the interval $[3242.2, 3249.1]$. There are no stochastic variables in such a statement, so we can not talk about probabilities! The correct formulation is that the probability that an interval made in this way contains μ is 95.4%. The probability is attached to the interval's construction!

The geometric meaning of $u_{0.025} = 1.96$ as related to a 95% confidence interval is given in Figure 17.4.

Let us consider a couple of typical examples.

Example 17.2.6. A series of measurements is sampled from a stochastic variable $X \sim N(\mu, \sigma^2)$, with $\sigma^2 = 4.3^2$. The measurements are:

17.9 12.1 13.3 12.1 17.5 11.7 10.4 17.4 13.2 16.6

Compute a 95% confidence interval for the unknown quantity μ .

The formula says that a 95% confidence interval is of the form $\bar{X} \pm u_{0.025} \frac{\sigma}{\sqrt{n}}$. We first compute the mean $\bar{X}$ of the 10 numbers:

$$\bar{X} = \frac{17.9 + 12.1 + 13.3 + 12.1 + 17.5 + 11.7 + 10.4 + 17.4 + 13.2 + 16.6}{10} = 14.22$$

Since $\sigma = 4.3$ and $n = 10$, we get the interval

$$\bar{X} \pm u_{0.025} \frac{\sigma}{\sqrt{n}} = 14.22 \pm 1.96 \frac{4.3}{\sqrt{10}} = 14.22 \pm 2.655 = [11.55, 16.89].$$

In many examples, we will suppress the actual measurements and start from the mean.

Example 17.2.7. A doctor will typically make three measurements for blood pressure. There are two similar measurements performed, systolic and diastolic. They are measured in the same way at different times in the pulse cycle, so we will only consider one of them. The unit of measurement is *mmHg*, millimeters of mercury (one millimeter of mercury is roughly 133 Pascal). We will assume that a measurement of blood pressure is given as a stochastic variable $X \sim N(\mu, \sigma^2)$ where $\sigma = 10$ and μ is the unknown blood pressure of the patient.

If the result of the measurement is 135, a 95% confidence interval for the blood pressure is

$$X \pm u_{\alpha/2} \sigma = 135 \pm 1.96 \cdot 10 = 135 \pm 19.6 = [115.4, 154.6]$$

If the mean of three measurements is 137, a 95% confidence interval is

$$\bar{X} \pm u_{\alpha/2} \sigma / \sqrt{3} = 137 \pm 1.96 \cdot 10 / \sqrt{3} = 137 \pm 11.3 = [125.7, 148.3]$$

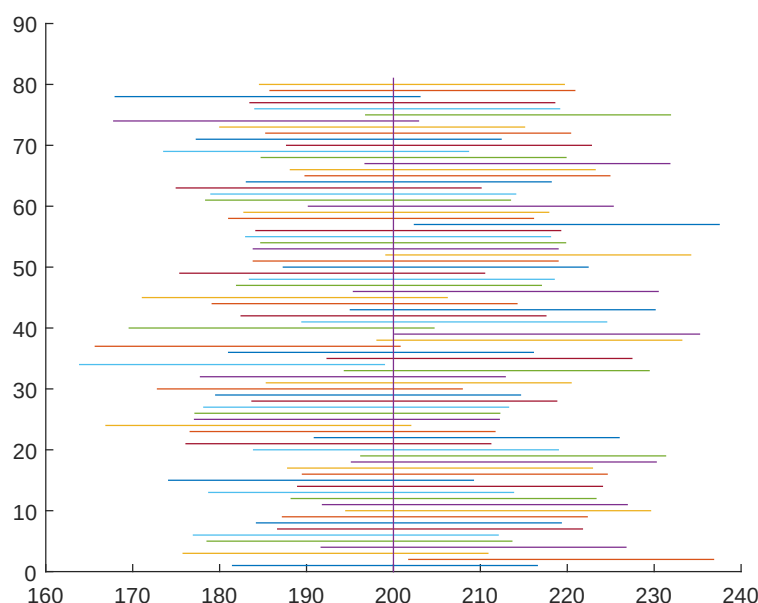
Example 17.2.8. Assume that the *pH* level of a lake is measured to determine whether it is suitable for introducing trout. A series of ten measurements is performed. Assume that the *pH* level is given by a stochastic variable $X \sim N(\mu, \sigma^2)$ where μ is the unknown we want to measure, and $\sigma = 0.5$. If the mean $\bar{X}$ is 6.2, we can make 90%, 95% and 99% confidence intervals:

$$\begin{aligned} 90\% : \quad & \bar{X} \pm u_{0.05} \frac{\sigma}{\sqrt{10}} = 6.2 \pm 1.645 \frac{0.5}{\sqrt{10}} = [5.94, 6.46] \\ 95\% : \quad & \bar{X} \pm u_{0.025} \frac{\sigma}{\sqrt{10}} = 6.2 \pm 1.96 \frac{0.5}{\sqrt{10}} = [5.89, 6.51] \\ 99\% : \quad & \bar{X} \pm u_{0.005} \frac{\sigma}{\sqrt{10}} = 6.2 \pm 2.576 \frac{0.5}{\sqrt{10}} = [5.79, 6.61] \end{aligned}$$

A *pH* level above 6 is considered suitable for trout. Do you think trout should be introduced in this lake?

Comment 17.2.9. *pH* is actually a logarithmic scale, and it is more reasonable to assume that there is a normal distribution before the logarithm is applied. In this case, using the normal distribution is an approximation already at the theoretical level!

Figure 17.5: 80 confidence intervals (95%)



Example 17.2.10. If we compute 95% confidence intervals many times, we will on average catch the correct value in 95% of the trials. Also, we will miss 5% of the time! To test our set-up, we can choose values for μ and σ , sample from $N(\mu, \sigma^2)$, and construct confidence intervals based on the samples. Figure 17.5 shows a typical outcome. It was constructed using MATLAB with $\mu = 200$, $\sigma = 40$, $n = 20$ and 80 runs. For 95% confidence intervals, we would expect around 4 mistakes in 80 runs. In the figure, it seems that three, possibly four, miss the correct value, represented by the vertical line.

17.3 Hypothesis testing

In this section, we will discuss whether it is possible to give a clear answer to a simple yes/no question.

Example 17.3.1. A company is considering changing their software suite to the product of a new distributor. To decide whether the change is worthwhile, a series of typical jobs are performed on the new system. If the time spent is lower when using the new system, a change is warranted. However, there is a cost attached to adapting new software, for instance for installation and training. Thus, a slight improvement is not enough to change software, the gain must be substantial.

In the field of hypothesis testing, a conservative approach is the norm. This means that we have a starting point, and any changes from this starting point must be substantial or significant, or they will not be used. We will formulate this mathematically by saying that there is a high probability that a change is for the better. Staying with the current choice is *the null hypothesis*, H_0 . The other choice is called *the alternative hypothesis*, H_1 . Unless there is strong evidence for H_1 , we stay with H_0 . We also need to attach numbers to how strong the evidence must be.

Comment 17.3.2. It is very important to note that the two hypotheses H_0 and H_1 play very different roles. We will make computations based on assuming that H_0 is true, and then we will compare actual measurements to the results of these computations. The result will either be that the result is so unlikely that we reject the zero hypothesis or that the result seems to be within acceptable statistical variation, and we opt to stay with the zero hypothesis: we do not reject the null hypothesis. We never make computations based on the alternative hypothesis, but it will help us to find out what computations we should perform assuming H_0 .

Definition 17.3.3. The set-up for a *hypothesis test* is the following. Let μ be an unknown quantity. We want to compare it to a limit value μ_0 , and we say that *the null hypothesis*, H_0 , and *the alternative hypothesis*, H_1 , are

$$H_0 : \mu = \mu_0 \qquad H_1 : \mu > \mu_0.$$

Assume that we have an estimator $\hat{\mu}$, for instance $\hat{\mu} = \bar{X}$, where $\bar{X} \sim N(\mu, \sigma^2/n)$ is the average of n independent normally distributed variables $X_i \sim N(\mu, \sigma^2)$. If we want level of significance α , we will reject H_0 if the conditional probability of getting at least as high a value as $\hat{\mu}$, if the real value is $\mu = \mu_0$, is less than α .

The number α represents the maximal chance of a false positive: if the real, unknown μ is less than or equal to μ_0 , H_0 should ideally not be rejected. The probability that we will wrongfully reject H_0 is then less than or equal to α .

This will be clearer after an example.

Example 17.3.4. For an unknown μ we will test whether $\mu > \mu_0 = 15$. We want level of significance $\alpha = 0.05$. The measurement is based on $n = 12$ independent measurements, yielding $\hat{\mu} = 16.1$. In addition, $\sigma^2 = 3 = \sqrt{3}^2$.

So $H_0 : \mu = 15$, $H_1 : \mu > 15$. The probability we want to compute is

$$P(\hat{\mu} > 16.1 | \mu = 15)$$

Note that if $\mu = 15$, $\hat{\mu} \sim N(15, 3/12) = N(15, 0.5^2)$, so by Equation (17.1) we can compute

$$P(\hat{\mu} > 16.1 | \mu = 15) = 1 - P(\hat{\mu} \leq 16.1 | \mu = 15) = 1 - \left(0.5 + 0.5 \cdot \text{erf}\left(\frac{16.1 - 15}{0.5\sqrt{2}}\right)\right) = 0.014.$$

Since there is a probability of $1.4\% < 5\%$ of getting a result at least this high, there is enough evidence to reject H_0 . The measurements are so high that it is unlikely that μ is 15 (or lower).

Note that there is still a 1.4% chance that we are mistaken. This number is often called the p -value of the experiment, and can be used as basis for setting up hypothesis testing. We will use a mathematically equivalent formulation using the notion of critical value.

Example 17.3.5. How high does the measurement have to be if we are to reject H_0 ? We would need that $\hat{\mu} > k$, where k satisfies

$$P(\hat{\mu} > k | \mu = \mu_0) = \alpha$$

or

$$P(\hat{\mu} > k | \mu = 15) = 0.05.$$

Recall that $\hat{\mu} \sim N(15, 0.5^2)$, so using **erf**, we get that

$$P(\hat{\mu} > k | \mu = 15) = 1 - \left(0.5 + 0.5 \cdot \text{erf}\left(\frac{k - 15}{0.5\sqrt{2}}\right)\right) = 0.05.$$

To solve for k , we first isolate **erf** to get

$$\text{erf}\left(\frac{k - 15}{0.5\sqrt{2}}\right) = 0.9.$$

Then we apply **erfinv**, and finally solve for k . This gives us

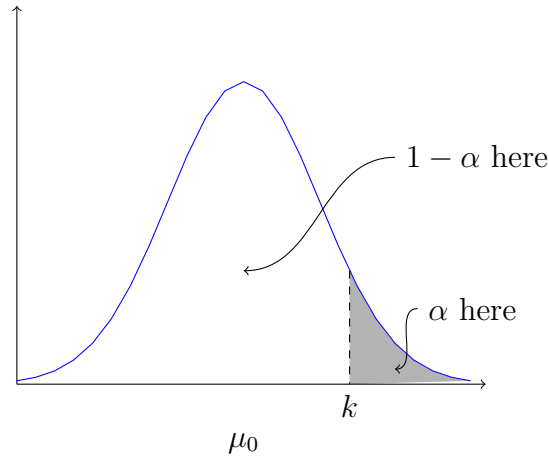
$$k = \text{erfinv}(0.9) \cdot 0.5\sqrt{2} + 15 = 15.82.$$

Any result at least as high as 15.82 would give us enough evidence to reject H_0 . A graphical representation of the meaning of k can be found in Figure 17.6.

In general, we would like to use the notation with u_α , since this is more customary. Unwrapping the essence of the computations above yields the first part of this theorem, the two other parts are similar. Note that H_1 tells us what inequalities to use for testing rejection of H_0 (H_1 determines the *rejection region*).

Theorem 17.3.6. *We want to compare an unknown quantity μ to a value μ_0 , using the mean of n independent stochastic variables $X_i \sim N(\mu, \sigma^2)$ using level of significance α . In each part, $H_0 : \mu = \mu_0$.*

Figure 17.6: The grey region represents acceptable false positive level



- a) If $H_1 : \mu > \mu_0$ we reject H_0 if $\bar{X} > k = \mu_0 + u_\alpha \frac{\sigma}{\sqrt{n}}$.
- b) If $H_1 : \mu < \mu_0$ we reject H_0 if $\bar{X} < k = \mu_0 - u_\alpha \frac{\sigma}{\sqrt{n}}$.
- c) If $H_1 : \mu \neq \mu_0$ we reject H_0 if $|\bar{X} - \mu_0| > u_{\alpha/2} \frac{\sigma}{\sqrt{n}}$.

In the third part, the $\alpha/2$ represents that the probability α is split in two, one near infinity and one near minus infinity.

Definition 17.3.7. The value k in parts a) and b) of Theorem 17.3.6 is called the critical value of the test.

Example 17.3.8. With the set-up from Example 17.3.5, we can compute that

$$k = \mu_0 + u_\alpha \frac{\sigma}{\sqrt{n}} = 15 + 1.645 \frac{\sqrt{3}}{\sqrt{12}} = 15.82$$

as before.

Example 17.3.9. We will test whether new software has significantly shorter running time than existing solutions, using part b) of the theorem.

A typical job for the company takes $\mu_0 = 100$ s to perform, with a variance of $\sigma^2 = 10^2$. Our set-up is thus:

$$H_0 : \mu = 100 \quad H_1 : \mu < 100$$

By the theorem, we reject H_0 if the mean of n runs is less than

$$k = \mu_0 - u_\alpha \frac{\sigma}{\sqrt{n}}.$$

Choosing level of significance $\alpha = 0.05$, and conducting $n = 8$ runs, we get

$$k = \mu_0 - u_\alpha \frac{\sigma}{\sqrt{n}} = 100 - u_{0.05} \frac{10}{\sqrt{8}} = 100 - 1.645 \frac{10}{\sqrt{8}} = 94.2.$$

If the mean of 8 runs is less than 94.2s, we believe that the new software is significantly better. In stricter statistical parlance, we reject the hypothesis that the old software is as good as the new one.

Example 17.3.10. A machine produces a liquid, filling containers with 500 ml of the liquid. The amount of liquid in a single container is assumed to be normally distributed with $X \sim N(500, 1^2)$. After routine maintenance work, a series of containers are measured precisely to make sure that the machine still produces the right amount of liquid. We will analyze this situation using part c) of the theorem. Let μ be the unknown amount of liquid.

Our set-up is:

$$H_0 : \mu = 500 \quad H_1 : \mu \neq 500$$

With a level of significance $\alpha = 0.01$, and a series of 10 runs, we shall reject H_0 if

$$|\bar{X} - 500| > u_{0.005} \frac{1}{\sqrt{10}} = 2.576 \frac{1}{\sqrt{10}} = 0.81.$$

In other words, if the mean of 10 runs is larger than 500.81 or smaller than 499.19, we have enough evidence to believe that the machine produces a different amount. Then more calibration is needed.

Example 17.3.11. A brand of beer is supposed to contain 4.5 % alcohol. From experience, it is known that the actual alcohol percentage in a bottle is normally distributed with $\mu_0 = 4.5$ and $\sigma^2 = 0.05^2$. A batch of produced beer is tested to see if the alcohol level is correct by choosing ten random bottles. Formulate a hypothesis test in this situation, using $\alpha = 0.05$.

Since we are interested in whether 4.5% is correct, we should allow for both smaller and larger values. Therefore, the two-sided test in part c) of the theorem applies. Our set-up is thus

$$H_0 : \mu = 4.5 \quad H_1 : \mu \neq 4.5.$$

With level of significance $\alpha = 0.05$ and $n = 10$ tested bottles, we shall reject H_0 if

$$|\bar{X} - 4.5| > u_{0.05/2} \frac{0.05}{\sqrt{10}} = 1.96 \frac{1}{\sqrt{10}} = 0.033.$$

Example 17.3.12. We continue with the previous example, but now the beer is tested by the authorities to make sure that the alcohol level is not too high. A higher alcohol level than stated would affect taxation and public health issues. In this case, we are only interested in whether the level is too high, whereas too low levels are acceptable. We are therefore in the situation of part a) in the theorem. The hypothesis test is

$$H_0 : \mu = 4.5 \quad H_1 : \mu > 4.5.$$

With level of significance $\alpha = 0.05$ and $n = 10$ tested bottles, we shall reject H_0 if

$$\bar{X} > k = \mu_0 + u_\alpha \frac{\sigma}{\sqrt{n}} = 4.5 + 1.645 \frac{0.05}{\sqrt{10}} = 4.526.$$

Compare with the result in the previous example. What would be the conclusion if the mean of the ten measurements were 4.53? Try to understand why the results are different in the two cases. It is helpful to make one of our standard drawings, putting in both critical values in the same figure.

17.4 Student's t-distribution

Unless the measurements follow very well tested guidelines, with high quality equipment, it is uncommon to know beforehand what the variance should be. Therefore, the set-up earlier in this chapter is insufficient to understand more general phenomena. In this section, we will briefly look at the changes that must be made to incorporate the case of unknown variance. To use this theory in practice requires appropriate tables, so we will only look at a few examples.

Definition 17.4.1. Let $\hat{\mu} = \bar{X}$ be the mean of n independent, equally distributed normal stochastic variables $X_i \sim N(\mu, \sigma^2)$. We want to estimate μ , but now we also assume that σ is unknown. Therefore, we consider the estimator

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2$$

for the variance. Then

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}}$$

has a new distribution, the t -distribution with $n - 1$ degrees of freedom, $T \sim t_{n-1}$. This is sometimes called *Student's t* distribution since its definition was first published (by Gosset) under the pseudonym Student.

The formulas for t are much more difficult than the formulas for the normal distribution. Therefore, they are typically found in tables. An internet search for *t distribution tables* will give you a lot of information. Note in particular that for large values of n , t_n resembles the normal distribution.

Much of what we have already seen for the normal distribution, like confidence intervals and hypothesis testing, carries over to the t -distribution. We will not go further than to confidence interval, comparing the result to the confidence interval we would get from the case of known variance.

Theorem 17.4.2. *Assume that an unknown quantity μ is measured by n independent, equally distributed normal stochastic variables $X_i \sim N(\mu, \sigma^2)$, where σ is unknown as well. A $1 - \alpha$ confidence interval for μ is given by*

$$\bar{X} \pm t_{\alpha/2, n-1} \frac{S}{\sqrt{n}},$$

where the value of $t_{\alpha/2, n-1}$ must be found from a table.

We will not prove this theorem.

Example 17.4.3. We want to find a 95% confidence interval for an unknown quantity μ , measured as above. Since we don't know the value of σ , we will have to use the t -distribution. Let us assume that $n = 6$ measurements produced a mean of $\bar{X} = 8.7$, and that the measurements further provided the value $S^2 = 0.3^2$. From a table, we find the value $t_{\alpha/2, n-1} = t_{0.025, 5} = 2.571$. Then a 95% confidence interval has the form

$$\bar{X} \pm t_{\alpha/2, n-1} \frac{S}{\sqrt{n}} = 8.7 \pm 2.571 \frac{0.3}{\sqrt{6}} = 8.7 \pm 0.31.$$

Example 17.4.4. We want a 90% confidence interval for an unknown quantity μ , measured in such a way that σ^2 is also unknown. In a series of measurements, assumed to be represented by a normal stochastic variable, the following numbers are obtained:

$$14.5 \quad 15.2 \quad 15.1 \quad 16.9 \quad 14.1 \quad 13.7$$

We compute the mean

$$\bar{X} = \frac{1}{n} \sum X_i = (14.5 + 15.2 + 15.1 + 16.9 + 14.1 + 13.7)/6 = 14.92.$$

This will be the center of our confidence interval. We then compute S^2 , the estimate for the variance:

$$S^2 = \frac{1}{n-1} \sum (X_i - \bar{X})^2 = \frac{1}{5} ((14.5 - 14.92)^2 + \cdots + (13.7 - 14.92)^2) = 1.13^2.$$

From a table, we find that

$$t_{\alpha/2, n-1} = t_{0.05, 5} = 2.015.$$

Putting everything together, we get our confidence interval as

$$\bar{X} \pm t_{\alpha/2, n-1} \frac{S}{\sqrt{n}} = 14.92 \pm 2.015 \cdot 1.13\sqrt{6} = 14.92 \pm 0.93 = [13.99, 15.85].$$

17.5 The central limit theorem and normal approximations

In statistics, the normal distribution is a powerful computational tool. In addition to phenomena that naturally follow a normal distribution, there are many distributions that are only approximately normal. From a theoretical point of view, the *central limit theorem* says that if we have enough measurements, most distributions can be approximated by a normal distribution with the same expectation and variance. From a practical point of view, the computational cost of e.g. a binomial distribution $X \sim \text{bin}(n, p)$ with large n is prohibitive. However, the corresponding normal distribution is as easy to work with as any other normal distribution.

Theorem 17.5.1 (Central limit theorem). *Let $X_1, X_2, X_3, \dots$ be a sequence of independent stochastic variables, each with the same distribution, in particular the same expectation μ and variance σ^2 . Then, for large n , the averages*

$$\frac{1}{n} \sum_{i=1}^n X_i \approx N\left(\mu, \frac{\sigma^2}{n}\right)$$

and the sum

$$\sum_{i=1}^n X_i \approx N(\mu, n\sigma^2).$$

There are many different formulations and generalizations of this theorem. Our formulation is not very precise, in the sense that it doesn't say anything about the quality of the approximation. For our purposes, we will take the simplistic point of view that we should exchange the sum or the average with the approximating normal distribution whenever the numbers involved make it difficult to compute exact probabilities.

One of the many applications of this theorem is that other distributions can be approximated by the normal distribution, meaning that we can exchange costly

computations by cheaper computations that provide an almost correct answer. It is possible to give some guidelines about the applicability of these approximations, but we will follow the maxim that we should use the approximation when the original computation is too expensive. The rest of this section is devoted to examples of this theorem. Note that this is the first time we really use the variance of the discrete distributions we have learnt about before.

Example 17.5.2. Assume that $X \sim \text{bin}(n, p)$. Then we know that $E(X) = np$ and $\text{Var}(X) = np(1-p)$. To get conditions for a good approximation by normal distributions, we typically need that n is large since the binomial distribution is based on n repeated experiments (each with the same distribution, so the central limit theorem applies). However, it turns out that the value of n needed also depends on p . Values of p close to $1/2$ work better, whereas values closer to 0 or 1 generally require higher n . If you look for information about this online, you will often find that there are rules of thumb about the variance of X . We will not be more formal about this, but a high value for $\text{Var}(X)$ means that the normal distribution works better than a lower value for $\text{Var}(X)$.

Let us proceed with some calculations for different values of n and p . First, assume that $X \sim \text{bin}(10, 1/6)$. For instance, X can count the number of sixes if we throw 10 dice. We can compute for instance that

$$P(X \leq 1) = \binom{10}{0}(1/6)^0(5/6)^{10} + \binom{10}{1}(1/6)^1(5/6)^9 = 0.4845.$$

If we let $Y \sim N(10/6, 10 \cdot (1/6) \cdot (5/6)) = N(1.67, 1.18^2)$ we get the approximation

$$P(Y \leq 1) = 0.5 + 0.5 \cdot \text{erf}\left(\frac{1 - 1.67}{1.18 \cdot \sqrt{2}}\right) = 0.29.$$

This is not very good. Let us increase n and look at $X \sim \text{bin}(1000, 1/6)$. Let us ask for the probability $P(X \leq 140)$. By the same formula as above, but using the computer for all computations, we get that

$$P(X \leq 140) = 0.01189.$$

For $Y \sim N(167, 11.8^2)$ we get

$$P(Y \leq 140) = 0.01183.$$

The approximation is much better. For very high values of n , it is very time consuming to perform exact computations. Then we will have to rely on the normal approximation. Figure 17.7 shows that the density function of Y only vaguely approximates the binomial distribution X for $n = 10$. For $n = 100$ it is much better, whereas for $n = 1000$ we would have to use a better resolution to see the difference.

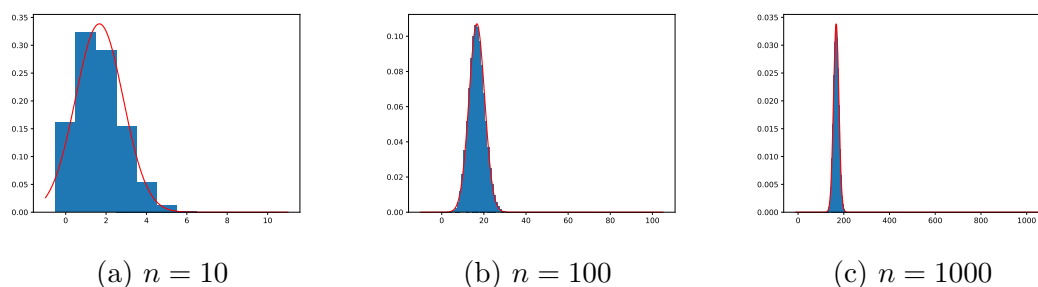


Figure 17.7: The bars show the distribution of $X \sim \text{bin}(n, 1/6)$ and the curves show the densities of the corresponding normal approximations

Example 17.5.3. When one die is thrown, the result X is a number from the set $\{1, 2, 3, 4, 5, 6\}$. Each outcome is equally probable, with probability $1/6$ (uniform distribution). Thus the expectation is

$$\mu = \sum_x xP(X = x) = \frac{1 + 2 + 3 + 4 + 5 + 6}{6} = \frac{21}{6} = 3.5$$

and the variance is

$$\sigma^2 = \sum_x (x - \mu)^2 P(X = x) = \frac{(1 - 3.5)^2 + \cdots + (6 - 3.5)^2}{6} = \frac{35}{12} = 2.9167 = 1.7078^2.$$

If we use a number n of dice, the results should be independent and identically distributed. Therefore, the central limit theorem applies, and for large n we can expect to get good approximations to the average of n dice by using the normal distribution with the same expectation and variance. Let $\bar{X} = \frac{X_1 + \cdots + X_n}{n}$ be this sum. Then

$$E(\bar{X}) = \frac{1}{n} \sum E(X_i) = \mu$$

and

$$\text{Var}(\bar{X}) = \frac{1}{n} \sum \text{Var}(X_i) = \frac{\sigma^2}{n}.$$

For small values of n , we can compute both the exact probabilities for $\bar{X}$ and the probabilities of the normal distribution $N(\mu, \sigma^2/n)$. For higher values of n , the normal distribution is still as easy to work with, but it is maybe easier to run experiments for $\bar{X}$. If we want to know the probability that the average of 1000 dice lies between 3.4 and 3.6, this is easily computed by the normal approximation. Let $Y \sim N(3.5, 2.9167/1000) = N(3.5, 0.054^2)$. Then $P(3.4 \leq \bar{X} \leq 3.6) \approx P(3.4 \leq Y \leq 3.6)$

is computed by

$$\begin{aligned} P(3.4 \leq Y \leq 3.6) &= P(Y \leq 3.6) - P(Y \leq 3.4) \\ &= 0.5 + 0.5 \cdot \operatorname{erf}\left(\frac{3.6-3.5}{0.054\sqrt{2}}\right) - (0.5 + 0.5 \cdot \operatorname{erf}\left(\frac{3.4-3.5}{0.054\sqrt{2}}\right)) \\ &= 0.936. \end{aligned}$$

The last step in the computation used the computer. This answer is reasonable as we expect a high probability to get close to the expectation with many tries. By running a few thousand computer experiments and checking the percentage that lies in the indicated interval, we get some lower and some higher values than 0.936.

However, let us try to do the same for $n = 2$. Then we see that the probability for $P(3.4 \leq \bar{X} \leq 3.6) = 1/6$ (if the average of two dice is between 3.4 and 3.6, the sum has to be exactly 7). Now $Y \sim N(3.5, 2.9167/2) = N(3.5, 1.208^2)$ and

$$P(3.4 \leq Y \leq 3.6) = 0.066.$$

The answer to this computation is so far from the true value that it has no practical value.

Example 17.5.4. The Poisson distribution $X \sim \operatorname{Po}(\lambda t)$ can be approximated by the corresponding normal distribution $N(\lambda t, \lambda t)$ if λt is large. This can be seen from the central limit theorem by thinking of independent experiments in small intervals for t . We will look at some computations to see that the approximations get better as λt increases.

Let $X \sim \operatorname{Po}(\lambda t) = \operatorname{Po}(4)$, and consider the probability $P(X \leq 3)$. Let $Y \sim N(4, 2^2)$ be the normal approximation. We get

$$P(X \leq 3) = \sum_{x=0}^3 \frac{4^x}{x!} e^{-4} = 0.43$$

and

$$P(Y \leq 3) = 0.5 + 0.5 \cdot \operatorname{erf}((3-4)/(2\sqrt{2})) = 0.31.$$

With larger numbers, say $X \sim \operatorname{Po}(12)$ and $Y \sim N(12, \sqrt{12}^2)$, we get via a similar calculation that

$$P(X \leq 7) = 0.32$$

and

$$P(Y \leq 7) = 0.31.$$

As for the binomial distribution, the numbers work better with higher variance.

Finally, we should mention that there are several further possibilities to consider. First, it is possible to improve the approximations by thinking about integers. From the continuous point of view, there is nothing special about the integers. But since we know that e.g. the binomial distribution only takes integer values, we can adjust the parameters of the normal distribution so that end points are $1/2$ more than the integer we are interested in, see Exercise 17.6.17. Also, we can approximate distributions by simpler distributions other than the normal distribution. One example is that the hypergeometric distribution is similar to the binomial distribution when N is much larger than n , see Exercise 17.6.18.

17.6 Exercises

Exercise 17.6.1. Deduce the following equations for $Z \sim N(\mu, \sigma^2)$ by writing $Z = \sigma X + \mu$ where $X \sim N(0, 1)$, and using properties of the standard normal distribution.

- (a). $P(Z \geq \mu + x) = P(Z \leq \mu - x)$
- (b). $P(Z \leq \mu) = 0.5$
- (c). $P(\mu - x \leq Z \leq \mu + x) = 1 - 2P(Z \leq \mu - x)$

Exercise 17.6.2. Let Z be a stochastic variable with expectation μ and variance σ^2 , and let $X = (Z - \mu)/\sigma$. Compute the expectation and variance of X .

Exercise 17.6.3. Let $X \sim N(0, 1)$. Use *e.g.* MATLAB to compute the following probabilities.

- (a). $P(X \leq 1.2)$
- (b). $P(X > 0.3)$
- (c). $P(-0.2 < X \leq 1.5)$

Exercise 17.6.4. Let $X \sim N(0, 2^2)$. Use *e.g.* MATLAB to compute the following probabilities.

- (a). $P(X \leq 1.2)$
- (b). $P(X > 0.3)$
- (c). $P(-0.2 < X \leq 1.5)$

Exercise 17.6.5. A series of experiments is performed, resulting in independent identically distributed measurements of the form $X \sim N(\mu, \sigma^2)$ where $\sigma^2 = 3^2$ and μ is unknown. You will need the constants $u_{0.025} = 1.96$ and $u_{0.05} = 1.645$.

(a). If there are $n = 3$ measurements with results 105, 112, 109, find a 95% confidence interval for μ .

(b). If there are $n = 6$ measurements with average result 178, find a 90% confidence interval for μ .

(c). If there are $n = 7$ measurements with average result -38 , find a 95% confidence interval for μ .

Exercise 17.6.6. A series of experiments is performed, resulting in independent identically distributed measurements of the form $X \sim N(\mu, \sigma^2)$ where $\sigma^2 = 3^2$ and μ is unknown. We wish to form a 95% confidence interval with total length at most 1.5 (so it is of the form $\bar{X} \pm 0.75$). How many measurements must be performed in order to achieve this? You will need the constant $u_{0.025} = 1.96$.

Exercise 17.6.7. A radar measurement is used to determine whether cars are speeding. On a road with speed limit 80 (kilometers per hour), we will formulate the question of speeding as a hypothesis test.

(a). Determine what μ_0 and μ mean for this experiment. What is H_0 and H_1 ?

(b). We want level of significance $\alpha = 0.01$. There is a risk that something unwanted happens with probability 1%. What is it?

You will need the constant $u_{0.01} = 2.326$.

(c). Let $\sigma^2 = 5^2$ be the known variance. What is the critical level k ? What is the condition for rejecting H_0 ?

(d). The measurement is 87. How do you conclude?

(e). In a different set-up, three independent radar measurements are performed simultaneously. What will change? If the mean of the three measurements is 87, how would you conclude?

Exercise 17.6.8. In each part, determine if the data allows you to reject H_0 . You will need the constants $u_{0.05} = 1.645$ and $u_{0.1} = 1.282$. The variance is always $\sigma^2 = 4^2$ and the zero hypothesis is $H_0 : \mu = 124$.

(a). $\alpha = 0.05$, $H_1 : \mu < 124$, $n = 7$, $\bar{X} = 118$.

(b). $\alpha = 0.05$, $H_1 : \mu < 124$, $n = 7$, $\bar{X} = 122$.

(c). $\alpha = 0.1$, $H_1 : \mu < 124$, $n = 7$, $\bar{X} = 118$.

(d). $\alpha = 0.1$, $H_1 : \mu < 124$, $n = 7$, $\bar{X} = 122$.

Exercise 17.6.9. In one of the points in the previous exercise, you could conclude without any new computations, based only on results already computed. Which one?

Exercise 17.6.10. In each part, determine if the data allows you to reject H_0 . The variance is always $\sigma^2 = 3.3^2$ and the zero hypothesis is $H_0 : \mu = -35$. You will need the constants $u_{0.05} = 1.645$ and $u_{0.025} = 1.96$.

(a). $\alpha = 0.05$, $H_1 : \mu < -35$, $n = 9$, $\bar{X} = -37$.

(b). $\alpha = 0.05$, $H_1 : \mu \neq -35$, $n = 9$, $\bar{X} = -37$.

Exercise 17.6.11. How will you explain the difference between a and b in the previous exercise?

Exercise 17.6.12. Confidence intervals and two-sided hypothesis tests look similar. Try to prove this statement:

In a hypothesis test of the form $H_0 : \mu = \mu_0$ against $H_1 : \mu \neq \mu_0$, we reject H_0 at significance level α if and only if μ_0 is outside a $(1 - \alpha)$ confidence interval for μ based on the same measurements.

Exercise 17.6.13. Based on 6 measurements, assumed to be independent and $N(\mu, \sigma^2)$ -distributed, we want to find a 95% confidence interval for the unknown μ . We will also assume that the standard deviation σ is unknown. The measurements are:

6.3 6.6 5.9 6.1 6.2 6.1

(a). Compute the estimators $\hat{\mu}$ and S .

(b). Find the correct t -value and write down the 95% confidence interval.

(c). For comparison, if we had known that $\sigma = 0.24$, use our earlier set-up to produce a 95% confidence interval. Why is it natural that this confidence interval is smaller?

Exercise 17.6.14. Let $X \sim \text{bin}(n, p)$ where $n = 250$ and $p = 0.4$.

(a). Compute the expectation and variance of X , and identify the approximating normal distribution Y .

(b). Compute the approximate probability that X is less than 110.

(c). Compute the approximate probability that X is between 90 and 110.

Exercise 17.6.15. Let $X \sim \text{Po}(\lambda t) = \text{Po}(150)$.

(a). Identify the normal approximation $Y \sim N(\mu, \sigma^2)$ to X .

(b). Compute an approximation to $P(X < 135)$.

(c). Compute an approximation to $P(X > 145)$.

Exercise 17.6.16. Assume that $X \sim \text{bin}(n, p) = \text{bin}(200, p)$ where p is unknown. Based on an experiment we get $X = 62.3$. Use the ideas of normal approximations to produce an approximate 95% confidence interval for p .

Exercise 17.6.17. Many interesting distributions only take integer values. This simple observation can be used to improve the accuracy of normal approximations. As an example, we will look at $X \sim \text{bin}(n, p) = \text{bin}(25, 0.5)$. Look at $P(X \leq 11)$ and $P(X \geq 12)$.

(a). What is the sum of $P(X \leq 11)$ and $P(X \geq 12)$?

(b). Let $Y \sim N(np, np(1-p))$ be the normal approximation to X . Compute $P(Y \leq 11) + P(Y \geq 12)$.

(c). The problem is that the normal approximation fails to see anything between the values 11 and 12. The idea of *integer correction* is to approximate computation of probabilities by splitting the interval between integers in two, so that $P(X \leq 11) \approx P(Y \leq 11 + 0.5)$ and $P(X \geq 12) \approx P(Y \geq 12 - 0.5)$. Compute $P(Y \leq 11 + 0.5) + P(Y \geq 12 - 0.5)$.

Exercise 17.6.18. We will look at a two-step approach to approximate computations of the hypergeometric distribution. First, we will approximate by a binomial distribution, then we will approximate by a normal distribution. In order to fix our ideas, we will consider a political question that will be determined by a vote among one million voters. A poll is conducted among one hundred randomly chosen voters. Assume that the true number that will vote in favour of the given question is 550000. Let the stochastic variable X be the number of voters in the poll that are in favour.

(a). Explain why it is natural to assume that X has a hypergeometric distribution. Identify the correct parameters for the distribution.

(b). Explain why it is natural to approximate X by a binomial distribution Y . Identify the correct parameters for the distribution Y .

(c). Finally, identify the normal distribution Z that approximates the binomial distribution Y .

(d). Assume that the number of voters in favour of the given question was much lower, just 5000. How would that influence the normal distribution Z ?

Chapter 18

Regression

We have earlier seen how to find a best linear approximation to a set of points in the plane, using the method of least squares. We will take this line of thought a bit further, using different sorts of approximations, and try to quantify how good the approximation is.

18.1 Regression in different guises

We will extend the method of least squares to cover more general situations. The examples include other types of curves in the plane, and higher dimensions.

Linear approximation

Recall the one type of regression we have already seen, the line fitting a set of measured points best. It is best to look at the linear algebra approach if we want other sorts of models.

Example 18.1.1. As in Section 11.3, we consider a set of points (maybe obtained from measurements in some experiment) $\{(X_i, Y_i) | i = 1, \dots, n\}$. To find the line $y = Ax + B$ fitting the measurements best, we saw that we had to solve a linear algebra problem

$$M^T M \vec{p} = M^T \vec{y}$$

where

$$M = \begin{pmatrix} X_1 & 1 \\ X_2 & 1 \\ \vdots & \vdots \\ X_n & 1 \end{pmatrix}, \quad \vec{p} = \begin{pmatrix} A \\ B \end{pmatrix} \quad \text{and} \quad \vec{y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}.$$

For instance, let the points be

(X_1, Y_1)	$(2.1376, 3.9447)$
(X_2, Y_2)	$(2.9388, 6.6958)$
(X_3, Y_3)	$(3.0238, 5.8318)$
(X_4, Y_4)	$(4.6742, 9.9759)$
(X_5, Y_5)	$(5.3244, 11.2515)$
(X_6, Y_6)	$(7.5886, 15.5039)$
(X_7, Y_7)	$(7.6224, 14.6720)$
(X_8, Y_8)	$(9.0571, 16.1542)$

Then the linear algebra formulation of the problem becomes

$$\begin{pmatrix} 270.2655 & 42.3669 \\ 42.3669 & 8.0000 \end{pmatrix} \begin{pmatrix} A \\ B \end{pmatrix} = \begin{pmatrix} 528.0798 \\ 84.0298 \end{pmatrix}$$

No matter how many points there are, we will always get a system with two equations in two unknowns. Solving this system yields

$$A = 1.8099 \quad \text{and} \quad B = 0.9185$$

so the line best approximating this data is

$$y = 1.8099x + 0.9185.$$

See Figure 18.1.

How good is this linear approximation to the data? From the graph, it seems to be a good fit. There are ways to measure how good the approximation is.

Coefficient of determination

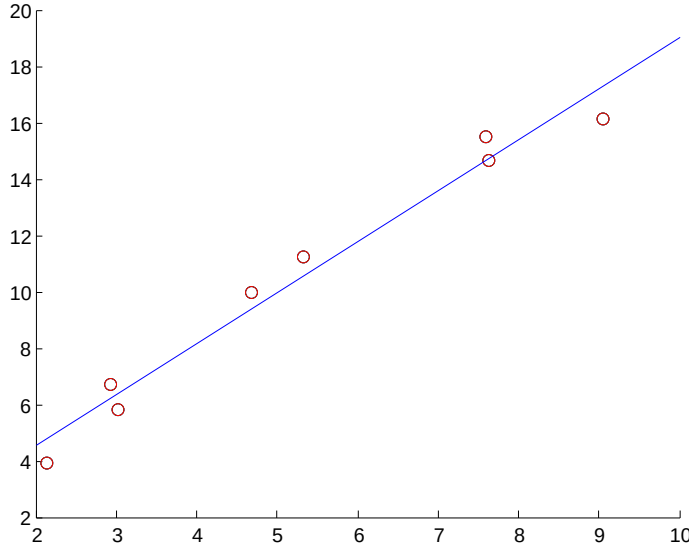
Definition 18.1.2. We measure the quality of the linear approximation $y = ax + b$ to a set of measurements $\{(X_i, Y_i) | i = 1, \dots, n\}$ by considering how large a portion of the variation of Y that is explained by the approximation. Define

$$S_y^2 = \sum_{i=1}^n (Y_i - \bar{Y})^2$$

and

$$SSE = \sum_{i=1}^n (Y_i - (aX_i + b))^2.$$

Figure 18.1: The best linear fit to the data



SSE is short for “sum of squares of error”. Finally, let

$$r^2 = \frac{S_y^2 - SSE}{S_y^2}.$$

Then r^2 is called the *coefficient of determination*. It tells us how many percent of the variation of Y that is explained by the linear approximation.

Example 18.1.3. In Example 18.1.1, we compute that

$$\begin{aligned} S_y^2 &= 154.4899 \\ SSE &= 4.1389 \\ r^2 &= 0.9732 \end{aligned}$$

In this case, 97% of the variation of y is explained by the linear relationship.

There are several other formulas that can be used to compute this number. The number r^2 is the natural square of the *sample correlation coefficient*

$$r = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2} \sqrt{\sum (Y_i - \bar{Y})^2}}.$$

Another formula is

$$r^2 = \frac{a^2 S_x^2}{S_y^2} = \frac{a^2 \sum (X_i - \bar{X})^2}{\sum (Y_i - \bar{Y})^2}.$$

Here a is the coefficient of x in the linear approximation (the slope of the line).

Comment 18.1.4. If the points are in fact collinear, all the error terms $Y_i - (aX_i + b)$ will be zero, and the coefficient of determination will be 1.

Quadratic approximation

If the coefficient of determination from a linear model is small, the linear approximation tells us little about the data. Perhaps another sort of relationship exists between the data points? If we suspect another pattern, we can deduce a set of equations determining coefficients similar to the coefficients of a line in about the same way. Let us first look at the case of a quadratic pattern.

Example 18.1.5. We consider a set of points $\{(X_i, Y_i) | i = 1, \dots, n\}$. By inspecting the plot (or for some theoretical reason), we suspect that the points are close to the graph of some quadratic function

$$y = ax^2 + bx + c.$$

We want to find the values of a, b, c such that

$$\sum_{i=1}^n (Y_i - (aX_i^2 + bX_i + c))^2$$

is as small as possible. By arguing as in Section 11.3, we can reduce this problem to solving the linear algebra problem

$$M^T M \vec{p} = M^T \vec{y}$$

where

$$M = \begin{pmatrix} X_1^2 & X_1 & 1 \\ X_2^2 & X_2 & 1 \\ \vdots & \vdots & \vdots \\ X_n^2 & X_n & 1 \end{pmatrix}, \quad \vec{p} = \begin{pmatrix} A \\ B \\ C \end{pmatrix} \quad \text{and} \quad \vec{y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}.$$

Since there are now three coefficients to determine, we need the three equations in the system above.

Here is a small example with integer points.

$$\begin{aligned} (X_1, Y_1) &= (0, 1) \\ (X_2, Y_2) &= (1, 2) \\ (X_3, Y_3) &= (2, 2) \\ (X_4, Y_4) &= (3, 1) \\ (X_5, Y_5) &= (4, 1) \end{aligned}$$

The matrix M is therefore

$$M = \begin{pmatrix} X_1^2 & X_1 & 1 \\ X_2^2 & X_2 & 1 \\ \vdots & \vdots & \vdots \\ X_n^2 & X_n & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 1 & 1 \\ 4 & 2 & 1 \\ 9 & 3 & 1 \\ 16 & 4 & 1 \end{pmatrix}.$$

We get

$$M^T M = \begin{pmatrix} 354 & 100 & 30 \\ 100 & 30 & 10 \\ 30 & 10 & 5 \end{pmatrix}$$

and

$$M^T y = \begin{pmatrix} 35 \\ 13 \\ 7 \end{pmatrix}.$$

Solving the system

$$\begin{pmatrix} 354 & 100 & 30 \\ 100 & 30 & 10 \\ 30 & 10 & 5 \end{pmatrix} \begin{pmatrix} A \\ B \\ C \end{pmatrix} = \begin{pmatrix} 35 \\ 13 \\ 7 \end{pmatrix}$$

yields

$$A = -0.2143, B = 0.7571, C = 1.1714.$$

Therefore, the best quadratic fit to the data is

$$y = -0.2143x^2 + 0.7571x + 1.1714.$$

As with the linear approximation, we can compute the coefficient of determination

$$r^2 = \frac{S_y^2 - SSE}{S_y^2} = 0.62.$$

The quadratic approximation will in this case describe 62% of the variation of Y . Note that

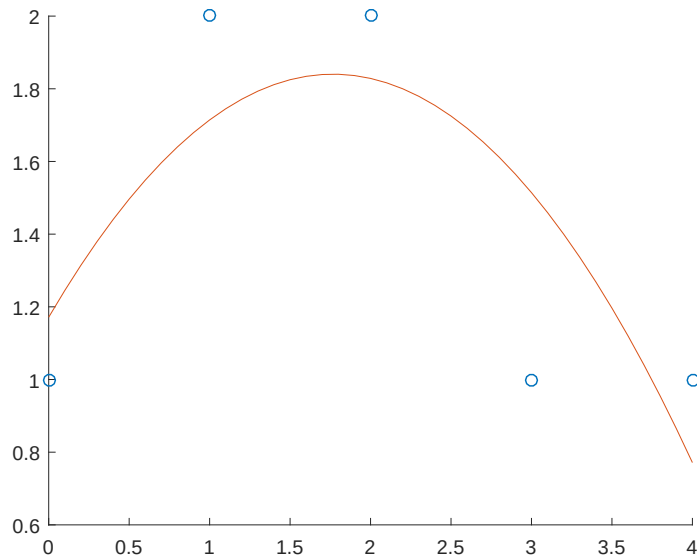
$$SSE = \sum (Y_i - (aX_i^2 + bX_i + c))^2$$

since it is a quadratic approximation, but that S_y^2 has the same meaning as before. The points, along with the approximating curve, are shown in Figure 18.2.

Comment 18.1.6. It is somewhat inconvenient to perform these sort of computations line by line, so we present a MATLAB recipe for computing the above. Since we will do many computations of this sort, it will be practical to wrap the computations in user-defined functions, and the exercises will provide a guide for doing this (see Exercise 18.3.8). Simple modifications will produce other approximations that we will consider later.

First, initialization.

Figure 18.2: The best quadratic fit to the data



```
x = [0;1;2;3;4]
y = [1;2;2;1;1]
M = zeros(length(x), 3)
for i = 1:length(x)
    M(i,1) = x(i)^2;
    M(i,2) = x(i);
    M(i,3) = 1;
end
```

We can use the operator `\` to solve systems of n linear equations in n unknowns in MATLAB as long as there is a single solution. So the coefficients A, B, C will be stored in the vector N produced by this code:

```
C = transpose(M)*M;
D = transpose(M)*y;
p = C\D % solves linear equations
```

Finally, to compute S_y^2 and SSE , it is convenient to store the values that approximate Y_i , say in a new table z :

```
for i = 1:length(x)
    z(i) = p(1)*x(i)^2 + p(2)*x(i) + p(3);
end
sy2 = 0;
```

```

sse = 0;
for i = 1:length(x)
    sy2 = sy2 + (y(i) - mean(y))^2;
    sse = sse + (y(i) - z(i))^2;
end
(sy2-sse)/sy2 % this is r^2

```

To modify this for other types of models, change only the definitions of M and z .

The pattern above is easily extended to curves of higher degree. As an example, for cubic, or third order, curves, we want a model of the form $y = ax^3 + bx^2 + cx + d$. Then

$$M = \begin{pmatrix} X_1^3 & X_1^2 & X_1 & 1 \\ X_2^3 & X_2^2 & X_2 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ X_n^3 & X_n^2 & X_n & 1 \end{pmatrix} \quad \text{and} \quad \vec{p} = \begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix}.$$

To find the best solution we solve $M^T M \vec{p} = \vec{y}$, which now is a system of four equations in four variables.

Sinusoidal approximation

The example in this section uses trigonometric functions instead of polynomial functions as approximations. A vast generalization of this example is handled by *Fourier analysis*, we can think of our case as one of the simplest examples in this topic.

Example 18.1.7. As before, we consider points $\{(X_i, Y_i) | i = 1, \dots, n\}$, and let this be measurements of some phenomenon that varies over the days of a year. So let x be measured in days, starting from new year. We might expect that there is a good approximation to the measurements of the form

$$y = a_0 + a_1 \cos(2\pi x/365) + b_1 \sin(2\pi x/365).$$

If this is the case, our setup will be similar to what we have seen before: Solve the linear algebra problem

$$M^T M \vec{p} = M^T \vec{y}$$

where

$$M = \begin{pmatrix} 1 & \cos(2\pi X_1/365) & \sin(2\pi X_1/365) \\ 1 & \cos(2\pi X_2/365) & \sin(2\pi X_2/365) \\ \vdots & \vdots & \vdots \\ 1 & \cos(2\pi X_n/365) & \sin(2\pi X_n/365) \end{pmatrix}, \quad \vec{p} = \begin{pmatrix} A_0 \\ A_1 \\ B_1 \end{pmatrix} \quad \text{and} \quad \vec{y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}.$$

To get some actual numbers, here are some values for time of sunrise at various dates in 2013 in Bergen, Norway. As before, we measure x in days. To get simple numbers to work with, y is measured in minutes after midnight. Here is a short table:

date	day number	sunrise	minutes
5th May	125	5.23	323
3rd June	154	4.21	261
12th June	163	4.12	252
4th July	185	4.20	260
18th Sept.	261	7.13	433

All these dates are in the period of daylight savings time, so we will get consistent data. On the other hand, we will be one hour off during winter. Compared to the general set-up, we have

$$\{(X_i, Y_i)\}_{i=1}^5 = \{(125, 323), (154, 261), (163, 252), (185, 260), (261, 433)\}.$$

Feeding this data into MATLAB gives the matrices $M^T M$ and $M^T \vec{y}$, and we are left with the matrix equation

$$\begin{pmatrix} 5.0000 & -3.5919 & 0.6175 \\ -3.5919 & 3.0163 & -0.9299 \\ 0.6175 & -0.9299 & 1.9837 \end{pmatrix} \begin{pmatrix} A_0 \\ A_1 \\ B_1 \end{pmatrix} = \begin{pmatrix} 1529 \\ -999.5 \\ 42.19 \end{pmatrix}.$$

Solving this yields $A_0 = 450.8$, $A_1 = 197.3$ and $B_1 = -26.6$. The trigonometric curve fitting with this data is then

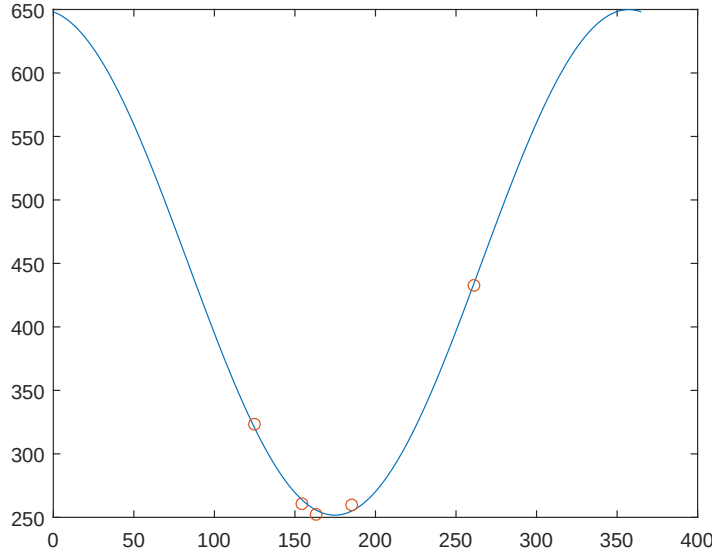
$$y = 450.8 + 197.3 \cos\left(\frac{2\pi x}{365}\right) - 26.6 \sin\left(\frac{2\pi x}{365}\right).$$

You can see this plot in Figure 18.3. The input data has been rounded off to whole minutes. Even for the dates we have used, the predicted value of y will be a little bit off. For instance for 18th September, we get $y(261) = 433.8$. For 1st January, the prediction is $y(1) = 647$ rather than the correct value 644 (remember that we are an hour off during winter). The latest sunrise occurs, according to the model, on 23rd December. This is not quite correct, but it is close. Nevertheless, the coefficient of determination (calculated in Exercise 18.3.2) together with the graph shows that the approximation is a very good one.

Definition 18.1.8. The (first order) sinusoidal approximation to a given data set $\{(X_i, Y_i) | i = 1, \dots, n\}$ with a given ω is the best fitting curve of the form

$$y = a_0 + a_1 \cos\left(\frac{2\pi x}{\omega}\right) + b_1 \sin\left(\frac{2\pi x}{\omega}\right).$$

Figure 18.3: Sunrise in Bergen, minutes past midnight (daylight savings time)



The coefficients are computed by solving

$$M^T M \vec{p} = M^T \vec{y}$$

where

$$M = \begin{pmatrix} 1 & \cos(2\pi X_1/\omega) & \sin(2\pi X_1/\omega) \\ 1 & \cos(2\pi X_2/\omega) & \sin(2\pi X_2/\omega) \\ \vdots & \vdots & \vdots \\ 1 & \cos(2\pi X_n/\omega) & \sin(2\pi X_n/\omega) \end{pmatrix}, \quad \vec{p} = \begin{pmatrix} A_0 \\ A_1 \\ B_1 \end{pmatrix} \quad \text{and} \quad \vec{y} = \begin{pmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{pmatrix}.$$

Note that ω is a part of the given information. It represents the period before the observed phenomenon is expected to repeat itself (the wavelength).

Example 18.1.9. We will now consider four different models for approximating the same data. To keep things simple, we will consider only five points:

x	1	-0.5	0	0.5	1
y	-0.2760	-0.9297	-0.7551	0.7874	1.8810

By successively computing linear, quadratic, sinusoidal and cubic regression curves, we find the following data. Note that for the sinusoidal, we want an approximation based on a given ω , which we will take as $\omega = 2.6$ in this example. Then our building

blocks for the sinusoidal approximation will be $\cos(2\pi x/2.6)$ and $\sin(2\pi x/2.6)$.

	expression	coeff. of determination
linear	$1.2062x + 0.1415$	0.65
quadratic	$1.3893x^2 + 1.2062x - 0.5531$	0.9565
sinusoidal	$0.1781 - 0.862 \cos(2\pi x/2.6) + 1.155 \sin(2\pi x/2.6)$	0.9451
cubic	$-0.8515x^3 + 1.3893x^2 + 1.93x - 0.5531$	0.9858

Which model is better? The coefficient of determination tells us part of the answer. The linear model is not good enough, as only 65 percent of the variation is explained by this model. The quadratic and sinusoidal models are similar, they both explain about 95 percent of the variation. The cubic has a slightly higher coefficient of determination.

Since 95 percent is a good result, each of the three last models is an acceptable candidate. The improvement to 98 percent is not so important, and could be a result of *overfitting*, meaning that the model reflects the observed points very well, but could be poorer for other values. Since the cubic model is more complex, we will probably discard it.

We are therefore left with the quadratic and the sinusoidal models. To choose between them, we would have to know if it is likely that a periodic function is better. The data itself is insufficient to conclude that one is better than the other.

Note also that in order to choose between the two, we could request more observations. A new observation outside the given interval could help in determining whether the observed phenomenon repeats itself or not.

Two-dimensional linear approximation

Our setup will also allow us to consider problems where a quantity z depends on several input variables. Examples of this phenomenon are found everywhere in nature. For example, the constant g we use for the earth's gravity is actually not constant, but depends on latitude and elevation. The limit of how much water air can absorb depends on temperature and pressure. The gravitational force between two bodies depends on the two masses and the distance between them.

We will consider only one case here, where z depends linearly on two other variables x and y .

Example 18.1.10. If z is related to two different variables x and y , we can still use the ideas from regression. The point we start with will then be points in $\mathbb{R}^3$. Assume

that we are given points:

$$\begin{aligned} &(X_1, Y_1, Z_1) \\ &(X_2, Y_2, Z_2) \\ &\vdots \\ &(X_n, Y_n, Z_n) \end{aligned}$$

and we suspect that there is an approximate linear relation of the form $z = ax + by + c$. To get the best fit using the method of least squares, we must solve

$$M^T M \vec{p} = M^T \vec{z}$$

where

$$M = \begin{pmatrix} X_1 & Y_1 & 1 \\ X_2 & Y_2 & 1 \\ & \vdots & \\ X_n & Y_n & 1 \end{pmatrix}, \quad \vec{p} = \begin{pmatrix} A \\ B \\ C \end{pmatrix} \quad \text{and} \quad \vec{z} = \begin{pmatrix} Z_1 \\ Z_2 \\ \vdots \\ Z_n \end{pmatrix}.$$

By considering dimensions, we see that we need to solve a system of three equations in the three unknowns A, B and C .

Assume we have $n = 7$ points:

$$\begin{aligned} &(0, 0, 4) \\ &(1, 0, 6) \\ &(0, 1, 5) \\ &(2, 1, 7) \\ &(2, 2, 5) \\ &(4, -1, 16) \\ &(-1, -1, 3) \end{aligned}$$

Then M and y are

$$M = \begin{pmatrix} 0 & 0 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 2 & 1 & 1 \\ 2 & 2 & 1 \\ 4 & -1 & 1 \\ -1 & -1 & 1 \end{pmatrix} \quad \text{and} \quad y = \begin{pmatrix} 4 \\ 6 \\ 5 \\ 7 \\ 5 \\ 16 \\ 3 \end{pmatrix}.$$

Solving

$$M^T M \vec{p} = M^T \vec{z}$$

now yields

$$p = \begin{pmatrix} 2.3471 \\ -1.5911 \\ 4.3436 \end{pmatrix}.$$

The best linear approximation is then

$$z = 2,3471x - 1.5911y + 4.3436.$$

Computing the coefficient of determination, we see that $r^2 = 0.935$. The linear approximation explains 93.5% of the variation of z .

18.2 χ^2 -statistics and quality of fits

We know how to make a best possible fit to given data using a given type of curve. The coefficient of determination tells us how large a portion of the variation that is explained by the model, but leaves us with the unclear task of assessing the choice between different models. Here we will look at a way to be more explicit about a given type of model. It will allow us to be precise, as in hypothesis testing, about whether a series of observations conforms to a given model. The distributions we consider now are discrete, so the situation has changed compared to the previous section.

Example 18.2.1. A badly manufactured die might have probabilities slightly different from $1/6$ to each of the six outcomes. How can we test if this is true? Obviously, we must roll the die many times and record the results, but how do we assess the results?

Say that we roll the die 6000 times, so that we would expect about 1000 of each value. Assume that the values we get are 972, 891, 1003, 1076, 1073, 985. Then the number that will tell us how far this is from the expected value is

$$\begin{aligned} Q &= \frac{(972-1000)^2}{1000} + \frac{(891-1000)^2}{1000} + \frac{(1003-1000)^2}{1000} \\ &+ \frac{(1076-1000)^2}{1000} + \frac{(1073-1000)^2}{1000} + \frac{(985-1000)^2}{1000} \\ &\approx 24. \end{aligned}$$

Is this number large or small? Are the results bad enough to claim that the die is biased?

Definition 18.2.2. χ^2 (chi-square) distributions are by definition sums of squares of independent standard normal distributions: If $X_i \sim N(0,1)$, and the X_i are independent, we say that $\sum_{i=1}^n X_i^2$ is a χ^2 distribution. The expectation is n , the variance $2n$.

A χ^2 distribution depends on a number n , the degrees of freedom. The exact nature of this distribution is difficult to grasp, but its quantiles are found in tables as for other common distributions. If Z is χ^2 -distributed with n degrees of freedom, $Z \sim \chi_n^2$, we have $P(Z > z_{\alpha,n}) = \alpha$.

The example we have in mind is that

$$Q = \sum_{i=1}^m \frac{(\text{observed}(i) - \text{expected}(i))^2}{\text{expected}(i)} \approx \chi_{m-1}^2.$$

If we measure m different probability values (for instance $m = 6$ when we count the different outcomes when rolling a die), there are $n = m - 1$ degrees of freedom. This distribution is approximately χ^2 . If $Q > z_{\alpha, n}$, we conclude that the measurement is so unlikely that our starting model is incorrect.

Example 18.2.3. Using the set-up above, we formulate our question as a hypothesis test. The null hypothesis is

$$H_0 : \text{The probability for each result is } 1/6.$$

The alternative hypothesis is

$$H_1 : \text{Not all probabilities are } 1/6.$$

With level of significance α , we will reject the zero hypothesis if $Q > z_{\alpha, m-1} = z_{\alpha, 5}$. Otherwise, there will not be enough evidence to reject H_0 .

We have already computed $Q = 24$ in Example 18.2.1. Let us choose level of significance $\alpha = 0.05$. A table will tell us that $z_{0.05, 5} = 11.07$. Since $24 > 11.07$, we reject H_0 : we conclude that there is sufficient evidence to believe that the die is bad.

Example 18.2.4. When we consider the frequencies of letters in this setting, we must modify the computations from Chapter 12 a little bit. We compared lists of percentages, but to use χ^2 we must use the numbers of letters, both for the observed and the expected values. In Example 12.3.2, we wanted to decrypt

etmxxuoaybmdqftqqfamegyyqdepnkftagmdfyadqxahqxkmzpyadqfqybqdmfq

If you look at the formula for Q , you will see that this means only that the values computed before, in that example, must be multiplied by the total number of characters, 63. For instance, without rotations, the computation of Q starts by comparing the number of **a**s in our text with the expected number. We observe six **a**s in the text. Based on Macbeth, the expected number is $63 \cdot 0.078 \approx 4.9$, so

$$Q = \frac{(6 - 4.9)^2}{4.9} + \text{terms for the other 25 letters}.$$

Since we compare 26 different letters, there are 25 degrees of freedom. Here are some quantiles found from a table:

$$\begin{aligned} z_{0.10, 25} &= 34.38 \\ z_{0.05, 25} &= 37.65 \\ z_{0.025, 25} &= 40.65 \\ z_{0.01, 25} &= 44.31 \end{aligned}$$

If we compute χ^2 for each of the 26 different rotations, we will get the following list of Q s (keep in mind that we have multiplied by 63 compared to how we looked at it earlier, if you want to modify the code to see this)

1330,9	3107,3	1438,4	258,7	1823,8	657,6	2068,1	1182,4	304,3
9612,9	535,2	2240,6	798,4	4531,5	25,9	555,9	827,0	173,1
928,5	3922,8	3731,8	898,7	3142,2	2175,8	523,8	3645	

For each of the rotations, we can formulate a hypothesis test. The null hypothesis is

H_0 : The frequencies of letters in the rotated text matches real text.

The alternative hypothesis is

H_1 : The frequencies of letters in the rotated text doesn't match real text.

For each of these 26 rotations, there is enough evidence to reject H_0 except in one case. Only the correct decryption (with $Q = 25.9$) is small enough that H_0 wouldn't be rejected. All the other possibilities are so far from the expected distribution of letters that we would reject that they come from the same distribution as the reference. The second smallest is ≈ 300 , much larger than what we would accept with $\alpha = 0.001$.

In words, for all the incorrect suggested decryptions, the frequencies of letters are so different from ordinary text that we reject the hypothesis that it really is text: it is gibberish!

Example 18.2.5. χ^2 is also used to test for independence. If two variables X and Y are independent, the combined probability matrix is the product of the probability vectors of X and Y . If we actually compute values by experiment, we will only get an approximation to this strict mathematical condition. To test whether the variables are independent based on experiments, we use χ^2 to see if the approximation is good enough to believe that the variables are independent, or if the difference is too large. As before, this is because the estimator is approximately χ^2 .

The null hypothesis is

H_0 : X and Y are independent.

The alternative hypothesis is

H_1 : X and Y are not independent.

With level of significance α , we will reject H_0 if $Q > z_{\alpha, m}$ where $Q = \sum_{i,j} \frac{(obs_{ij} - exp_{ij})^2}{exp_{ij}}$ and $m = (r - 1)(s - 1)$ if there are r choices for X and s choices for Y .

Let us look at a small numerical example. Say that X can have the two values 0, 1 and Y can have the three values 2, 3, 4, and assume that the results we get from a series of test runs is as follows (as probabilities):

	1	0	Y
2	0.15	0.25	0.4
3	0.04	0.16	0.2
4	0.11	0.29	0.4
X	0.3	0.7	1

If the variables are independent, we expect that the difference from the table produced by multiplication is only due to natural variation:

	1	0	Y
2	0.12	0.28	0.4
3	0.06	0.14	0.2
4	0.12	0.28	0.4
X	0.3	0.7	1

Are the results so close to the expected table that independence seems reasonable, or should we conclude that they are in fact dependent? As before, to use χ^2 we have to consider expected numbers, not probabilities. If we assume that the numbers above were produced by a run of 100 tests, we get

obs_{ij}	1	0	Y		exp_{ij}	1	0	Y
2	15	25	40	and	2	12	28	40
3	4	16	20		3	6	14	20
4	11	29	40		4	12	28	40
X	30	70	100		X	30	70	100

The test-variable is

$$\begin{aligned}
 Q &= \sum_{i,j} \frac{(obs_{ij} - exp_{ij})^2}{exp_{ij}} \\
 &= \frac{(15-12)^2}{12} + \frac{(4-6)^2}{6} + \frac{(11-12)^2}{12} \\
 &\quad + \frac{(25-28)^2}{28} + \frac{(16-14)^2}{14} + \frac{(29-28)^2}{28} \\
 &= 2.143.
 \end{aligned}$$

When we have a table of $r \times s$ cases, the approximate χ^2 distribution has $(r-1)(s-1)$ degrees of freedom. We therefore have $(2-1)(3-1) = 2$ degrees of freedom, and find in a table that $z_{0.05,2} = 5.99$. Since $2.143 < 5.99$, we don't have enough evidence to reject H_0 . The results are not indicating that the two variables are dependent, so we stick to the hypothesis that they are independent.

18.3 Exercises

Exercise 18.3.1. In each part, a list of points is given. Compute the linear regression (best fitting line) and the quadratic regression (best fitting second degree curve). Compute the coefficient of determination in each part, and consider whether the increase is large enough to conclude that the quadratic regression is preferable to the linear regression. There are many calculations in this exercise, so it is a good idea to use a computer (e.g. MATLAB).

(a). $(0, 3), (1, 2), (1, 4), (2, 5)$.

(b). $(-2, 4.2), (-1, 0.8), (0, -0.3), (1, 0.7), (2, 3.7)$

(c). $(2, 3.4), (3, 4.3), (5, 8.6), (7, 10)$

Exercise 18.3.2. Compute the coefficient of determination in the sunrise-example, Example 18.1.7.

Exercise 18.3.3. We will consider temperature $T = T(t)$ as a function of time, changing over the hours of a day. As an approximation (sinusoidal regression), we assume that T is similar to a trigonometric expression of the form

$$T(t) \approx a_0 + a_1 \cos\left(\frac{2\pi t}{24}\right) + b_1 \sin\left(\frac{2\pi t}{24}\right).$$

Temperature is measured in $^{\circ}C$ and time in hours; $t = 0$ corresponds to midnight. There are five measurements.

time	6	11	14	18	23
temperature	10	13	14	14	7

There are many calculations in this exercise, so it is a good idea to use a computer (e.g. MATLAB).

(a). Use the set-up for sinusoidal regression to compute the best fitting curve of the form $y = a_0 + a_1 \cos\left(\frac{2\pi t}{24}\right) + b_1 \sin\left(\frac{2\pi t}{24}\right)$.

(b). Compute the coefficient of determination.

(c). According to this approximation, what is the maximum and minimum temperatures, and at what time do they occur?

(d). Why are the coefficients for cosine and sine negative?

Exercise 18.3.4. Below is a joint table of observations of two variables X and Y . There are four choices a, b, c, d for X , two choices p, q for Y .

	a	b	c	d	Y
p	12	14	12	21	59
q	6	9	15	6	36
X	18	23	27	27	95

Determine whether there is sufficient evidence to conclude that X and Y are not independent. Formulate the question as a hypothesis test, and conclude using $\alpha = 0.05$. You can use that $z_{0.05,3} = 7.81$.

Exercise 18.3.5. A series of observations is collected for populations of six different species of migratory birds in a small coastal area. From earlier years, there is a known probability distribution for the relative sizes of the populations. Our task is to determine whether the new observations follow the same distribution, or whether there is sufficient evidence to conclude that there is a shift in the relative sizes. The expected values are

- Species 1: 25%
- Species 2: 10%
- Species 3: 15%
- Species 4: 30%
- Species 5: 5%
- Species 6: 15%

The number of observed birds:

- Species 1: 32
- Species 2: 14
- Species 3: 15
- Species 4: 42
- Species 5: 8
- Species 6: 22

- (a). Formulate the question as a hypothesis test.
- (b). Compute the expected number of birds of each species. Note that you will need the total number of observed birds now!
- (c). Conclude using $\alpha = 0.05$. You can use the number $z_{0.05,5} = 11.07$.

Exercise 18.3.6. Our setup works for models that are linear combinations of the coefficients. We will now look at an example that is different: Exponential approximations. As before, we have a set of points $\{(X_1, Y_1), \dots, (X_n, Y_n)\}$. We suspect that the underlying relationship between the coordinates is exponential,

$$y = Ae^{BX}$$

for some coefficients A and B , and we want the best such curve.

- (a). Introduce $z = \ln y$. Using properties of the logarithm, find a linear approximation problem equivalent to the exponential approximation problem.
- (b). Let there be 4 points $(0, 2)$, $(0.5, 5)$, $(1, 15)$, $(1.5, 38)$. Compute logarithms and solve the linear approximation problem from the previous question.
- (c). Translate back to an exponential approximation. In the same figure, plot the points and the exponential approximation. Is it a good fit?

Exercise 18.3.7. An experiment is believed to adhere to a binomial distribution, $X \sim \text{bin}(n, p) = \text{bin}(3, 0.2)$. In this exercise, we will formulate this question as a hypothesis test using χ^2 . One hundred experiments are to be performed.

- (a). Compute the expected table of observations of each possible value of X .
- (b). The observed numbers of each observation are 55, 37, 8, 0. Compute Q as in the set-up for χ^2 -testing.
- (c). Formulate the question as a hypothesis test. Using $\alpha = 0.05$, use a table to find the correct z -value and conclude.

Exercise 18.3.8. In this exercise we will collect our computations of linear, quadratic, cubic and sinusoidal regression curves into suitable user-defined MATLAB functions. This will allow for simplifications of earlier computations.

- (a). Write a user-defined function that reads two column vectors $\mathbf{X}$ and $\mathbf{Y}$ (assumed to be of the same length), and returns the column vector $\mathbf{p}$ containing the coefficients of the best linear approximation to the set of points with coordinates given by $\mathbf{X}$ and $\mathbf{Y}$. The first line should look like this:

```
function [p] = LinearApproximation(X,Y)
```

- (b). Write a user-defined function that reads two column vectors $\mathbf{X}$ and $\mathbf{Y}$ (assumed to be of the same length), and returns the column vector $\mathbf{p}$ containing the coefficients of the best quadratic approximation to the set of points with coordinates given by $\mathbf{X}$ and $\mathbf{Y}$. The first line should look like this:

```
function [p] = QuadraticApproximation(X,Y)
```

- (c). Write a user-defined function that reads two column vectors $\mathbf{X}$ and $\mathbf{Y}$ (assumed to be of the same length), and returns the column vector $\mathbf{p}$ containing the coefficients of the best cubic approximation to the set of points with coordinates given by $\mathbf{X}$ and $\mathbf{Y}$. The first line should look like this:

```
function [p] = CubicApproximation(X,Y)
```


(d). Write a user-defined function that reads two column vectors $\mathbf{X}$ and $\mathbf{Y}$ (assumed to be of the same length) and a frequency ω , and returns the column vector $\mathbf{p}$ containing the coefficients of the best sinusoidal approximation to the set of points with coordinates given by $\mathbf{X}$ and $\mathbf{Y}$. The first line should look like this:

```
function [p] = SinusoidalApproximation(X,Y,omega)
```

(e). Write a user-defined function that reads three column vectors $\mathbf{X}$, $\mathbf{Y}$ and $\mathbf{Z}$ (assumed to be of the same length), and returns the column vector $\mathbf{p}$ containing the coefficients of the best approximation plane to the set of points with coordinates given by $\mathbf{X}$, $\mathbf{Y}$ and $\mathbf{Z}$. The first line should look like this:

```
function [p] = LinearApproximation2D(X,Y,Z)
```


Chapter 19

RSA cryptography

In this chapter we will introduce a real-world cryptography example, the *RSA*-cryptography of Ron Rivest, Adi Shamir and Leonard Adleman. The strength of the RSA algorithm lies in the difficulty of factoring large numbers, combined with the ease of producing large primes. We will need to understand a few aspects of arithmetic modulo primes and products of two primes in order to give correct encryption and decryption programmes. Also, we will see how to crack RSA encryption by performing computationally intensive procedures.

19.1 Modulo a prime

The most important difference between primes and other numbers, when we do arithmetic modulo the number, is that we can form multiplicative inverses. Call the prime p . If we choose any number m between 1 and $p - 1$, multiplication by it, and then taking remainders modulo p , will give a bijection from the set $\{0, 1, 2, \dots, p - 1\}$ to itself. The inverse bijection will be given by multiplication and taking remainders modulo p by a unique m' between 1 and $p - 1$. For instance, multiplication by $m = 2$ modulo $p = 5$ gives the bijection

$$\begin{aligned} 0 &\rightarrow 0 \\ 1 &\rightarrow 2 \\ 2 &\rightarrow 4 \\ 3 &\rightarrow 1 \\ 4 &\rightarrow 3 \end{aligned}$$

Multiplication by $m' = 3$ gives

$$\begin{aligned} 0 &\rightarrow 0 \\ 1 &\rightarrow 3 \\ 2 &\rightarrow 1 \\ 3 &\rightarrow 4 \\ 4 &\rightarrow 2 \end{aligned}$$

By composing these two bijections, we see that they are inverse operations. Therefore, we also say that 2 and 3 are inverses modulo 5. As for ordinary arithmetic, the product of inverses is one: $2 \cdot 3 = 6 = 1 \bmod 5$.

How can we compute inverses, short of trying all alternatives? To get to it, we need to find the greatest common divisor, gcd, between two numbers. This is computed using the Euclidean algorithm.

The Euclidean algorithm

Let n and m be two positive integers. Division with remainders means that there are unique integers q and r such that

$$n = qm + r \quad \text{where } 0 \leq r < m.$$

The numbers q and r can be computed by built-in commands or operators in many programming languages. In Python, for instance, the notation is $q = n//m$ and $r = n\%m$, whereas MATLAB uses the commands `mod(n,m)` to find r and `idivide(n,m)` to find q . Note that in MATLAB you will have to use integer types for some of the integer operations to be defined (`x = 2` gives a 64 bit floating point, `x = int64(2)` gives a 64 bit integer).

If we want to find the greatest common divisor of n and m , note that if d divides both n and m , it is also a divisor of $r = n - qm$. Similarly, if d divides both r and m , it is also a divisor of $n = qm + r$. So we see that the greatest common divisor of n and m is equal to the greatest common divisor of m and r , which are smaller.

Repeat with the new pair (m, r) instead of the original (n, m) , and stop when you come to a remainder that is zero. Then the previous remainder is the greatest common divisor of the two numbers we started with.

Here is an example.

Example 19.1.1. Find the greatest common divisor of 234 and 6066. We get the following list of divisions

$$\begin{aligned} 6066 &= 25 \cdot 234 + 216 \\ 234 &= 1 \cdot 216 + 18 \\ 216 &= 12 \cdot 18 + 0 \end{aligned}$$

The conclusion is that the greatest common divisor of 234 and 6066 is 18.

It is relatively easy to implement this in most programming languages. Here is an example in MATLAB, written as a recursive function:

```
function [gcd] = EuclideanAlgorithm(a,b)
if a == 0
    gcd = b;
else
    gcd = EuclideanAlgorithm(mod(b,a),a);
end
```

The recursion stops when a remainder is zero, so this code follows the mathematical description precisely.

Computing multiplicative inverses modulo a number

Theorem 19.1.2. *Let m and n be positive integers. There is a number m' such that*

$$mm' = 1 \bmod n \quad (19.1)$$

if and only the greatest common divisor of m and n is 1.

Proof. We will first prove that if m' exists, the greatest common divisor is 1. The equality (19.1) can be rewritten in the form

$$\begin{aligned} mm' - 1 &= xn \quad \text{for some integer } x \quad \text{or} \\ mm' - xn &= 1. \end{aligned}$$

From this we see that any common divisor of m and n is also a divisor in 1, so the only possibility is 1.

The proof in the other direction is constructive; it will lead to a computation of the inverse when it exists. Assume that the greatest common divisor is 1. We want to find m' such that $mm' = 1 \bmod n$. Stated differently, we want two integers m' and x so that

$$mm' - xn = 1.$$

Perform the Euclidean algorithm on m and n :

$$\begin{aligned} m &= q(1)n + r(1) \\ n &= q(2)r(1) + r(2) \\ r(1) &= q(3)r(2) + r(3) \\ &\vdots \\ r(n-3) &= q(n-1)r(n-2) + r(n-1) \\ r(n-2) &= q(n)r(n-1) + r(n) = q(n)r(n-1) + 1 \end{aligned}$$

Since the greatest common divisor is 1, the last remainder $r(n)$ in the Euclidean algorithm is 1. Read the algorithm from bottom to top, and substitute the successive remainders as follows:

$$\begin{aligned}
 1 &= 1 \cdot r(n-2) - q(n)r(n-1) \\
 &= 1 \cdot r(n-2) - q(n)(r(n-3) - q(n-1)r(n-2)) \\
 &= -q(n)r(n-3) + (1 - q(n-1))r(n-2) \\
 &\vdots \\
 &= \text{something} \cdot r(1) + \text{something} \cdot r(2) \\
 &= \text{something} \cdot n + \text{something} \cdot r(1) \\
 &= \text{something} \cdot m + \text{something} \cdot n
 \end{aligned}$$

Finally, we get an equation like

$$1 = ym + xn.$$

This means that $ym = 1 \pmod n$, so that setting $m' = y$ solves the original problem. $\square$

Example 19.1.3. To find the multiplicative inverse of 1919 modulo 8945 using these ideas, we first go through the steps of the Euclidean algorithm, then back-track through it. Read the left half from the top down, then the right half from the bottom up:

$$\begin{array}{l|l}
 8945 = 4 \cdot 1919 + 1269 & 1 = 41 \cdot 1919 - 62 \cdot (8945 - 4 \cdot 1919) = 289 \cdot 1919 - 62 \cdot 8945 \\
 1919 = 1 \cdot 1269 + 650 & 1 = 41 \cdot (1919 - 1 \cdot 1269) - 21 \cdot 1269 = 41 \cdot 1919 - 62 \cdot 1269 \\
 1269 = 1 \cdot 650 + 619 & 1 = 20 \cdot 650 - 21 \cdot (1269 - 1 \cdot 650) = 41 \cdot 650 - 21 \cdot 1269 \\
 650 = 1 \cdot 619 + 31 & 1 = 20 \cdot (650 - 1 \cdot 619) - 1 \cdot 619 = 20 \cdot 650 - 21 \cdot 619 \\
 619 = 19 \cdot 31 + 30 & 1 = 31 - 1 \cdot (619 - 19 \cdot 31) = 20 \cdot 31 - 1 \cdot 619 \\
 31 = 1 \cdot 30 + 1 & 1 = 31 - 1 \cdot 30
 \end{array}$$

The conclusion from this computation is that

$$1 = 289 \cdot 1919 - 62 \cdot 8945.$$

Thus the multiplicative inverse of 1919 modulo 8945 is 289.

It is a bit tricky to keep track of the coefficients in the above argument, but by thinking recursively we can solve the problem by modifying the Euclidean algorithm. Recall first that at the last stage in the Euclidean algorithm, we will get a remainder zero. This can be still be used as the stopping criterion. To look at the recursion, say that the first computation in the algorithm is

$$n = qm + r$$

and that

$$1 = y \cdot r + x \cdot m.$$

Then we get that

$$1 = y(n - qm) + x \cdot m = y \cdot n + (x - yq) \cdot m.$$

So we get our coefficients by the recursive computation.

Comment 19.1.4. If the greatest common divisor is $d > 1$, the above procedure will also write $d = ym + xn$. However, this will not provide a multiplicative inverse (which doesn't exist by Theorem 19.1.2).

It is therefore sensible to change the Euclidean algorithm to have three outputs: the greatest common divisor and the two coefficients. We can use this code:

```
function [gcd,x,y] = EuclideanAlgorithm2(a,b)
%gcd = ax+by
if a == 0
    gcd = b;
    x = 0;
    y = 1;
else
    [gcd,y,x0] = EuclideanAlgorithm2(mod(b,a),a);
    x = x0 - (idivide(b,a))*y;
end
```

The three outputs will satisfy

$$g = ax + by$$

The stopping point of the algorithm is $a = 0$. Then the output $(b, 0, 1)$ gives the trivial equation

$$b = a \cdot 0 + b \cdot 1.$$

When $g = 1$, a multiplicative inverse exists and is found by reduction modulo b , say:

$$1 = ax \bmod b$$

The inverse of a modulo b is the output called $x \bmod b$ from our improved code.

19.2 Modulo pq

Let p and q be two different primes. We will study the modulo operation for the product pq .

First we will introduce a general function on positive integers, Euler's ϕ function.

Definition 19.2.1. Euler's ϕ (phi) function is the function that to the positive integer n returns $\phi(n)$ = the number of integers less than n without common factors with n . More precisely,

$$\phi(n) = |\{k | 1 \leq k \leq n \text{ and } k \text{ and } n \text{ have no common factor}\}|.$$

Here are the first few values of ϕ , together with the sets of k s that are counted.

$$\begin{aligned}\phi(1) &= 1 & \{1\} \\ \phi(2) &= 1 & \{1\} \\ \phi(3) &= 2 & \{1, 2\} \\ \phi(4) &= 2 & \{1, 3\} \\ \phi(5) &= 4 & \{1, 2, 3, 4\} \\ \phi(6) &= 2 & \{1, 5\} \\ \phi(7) &= 6 & \{1, 2, 3, 4, 5, 6\}\end{aligned}$$

Theorem 19.2.2. *The function ϕ satisfies these conditions, and is determined by them.*

1. $\phi(1) = 1$.
2. If p is prime, $\phi(p) = p - 1$ and $\phi(p^r) = p^r - p^{r-1}$.
3. If m and n are integers without common factors, $\phi(mn) = \phi(m)\phi(n)$.

Proof. $\phi(1) = 1$ by definition. If p is prime, the numbers less than or equal to p^r that have common factors with p^r are exactly the multiples of p that are less than or equal to p^r . These numbers are

$$p, 2p, 3p, \dots, p^{r-1}p = p^r.$$

There are p^{r-1} of these, and the remaining $p^r - p^{r-1}$ have no common factor with p^r . Therefore $\phi(p^r) = p^r - p^{r-1}$. Setting $r = 1$ yields $\phi(p) = p - 1$.

The general case of $\phi(mn) = \phi(m)\phi(n)$ is usually proved by a method called the Chinese remainder theorem. We will not give this argument, but rather give a short and direct proof in the case that m and n are two different primes. This is the case we need for the RSA algorithm. To keep in mind that we are using primes, we change the names to p and q .

So we want to prove that if $p \neq q$ are two primes, $\phi(pq) = \phi(p)\phi(q) = (p-1)(q-1)$. The set of numbers with a common factor with pq , less than pq , are exactly the multiples of p or q . Also, a number less than pq can only be a multiple of one of these (why?). So the set of numbers we need to exclude when computing ϕ is split into three parts as

$$\begin{aligned}&\{p, 2p, 3p, \dots, (q-1)p\} \\ &\{q, 2q, 3q, \dots, (p-1)q\} \\ &\{pq\}\end{aligned}$$

We considered pq by itself to make sure it isn't counted twice. All in all we exclude $(q - 1) + (p - 1) + 1$ and are left with

$$\phi(pq) = pq - ((q - 1) + (p - 1) + 1) = (p - 1)(q - 1).$$

The part that ϕ is determined by these properties is proved constructively, by using these rules to compute ϕ . Assume that the prime factorization of n is given by

$$n = p_1^{e_1} \cdot p_2^{e_2} \cdots p_s^{e_s}.$$

Then we can compute $\phi(n)$ as a product of $\phi(p_i^{e_i})$ by the third rule, and each of these is given by the second rule, so that

$$\phi(n) = \phi(p_1^{e_1}) \cdot \phi(p_2^{e_2}) \cdots \phi(p_s^{e_s}) = (p_1^{e_1} - p_1^{e_1-1}) \cdots (p_s^{e_s} - p_s^{e_s-1}).$$

□

As a consequence, we now know how many integers less than pq have inverses modulo pq , namely $\phi(pq) = (p - 1)(q - 1)$.

Example 19.2.3. Here is a list for $p = 5$ and $q = 7$, where there are $(5 - 1)(7 - 1) = 24$ integers that have inverses. We list these, together with their inverses modulo $pq = 35$.

$1 \cdot 1 = 1 \bmod 35$	$9 \cdot 4 = 1 \bmod 35$	$18 \cdot 2 = 1 \bmod 35$	$27 \cdot 13 = 1 \bmod 35$
$2 \cdot 18 = 1 \bmod 35$	$11 \cdot 16 = 1 \bmod 35$	$19 \cdot 24 = 1 \bmod 35$	$29 \cdot 29 = 1 \bmod 35$
$3 \cdot 12 = 1 \bmod 35$	$12 \cdot 3 = 1 \bmod 35$	$22 \cdot 8 = 1 \bmod 35$	$31 \cdot 26 = 1 \bmod 35$
$4 \cdot 9 = 1 \bmod 35$	$13 \cdot 27 = 1 \bmod 35$	$23 \cdot 32 = 1 \bmod 35$	$32 \cdot 23 = 1 \bmod 35$
$6 \cdot 6 = 1 \bmod 35$	$16 \cdot 11 = 1 \bmod 35$	$24 \cdot 19 = 1 \bmod 35$	$33 \cdot 17 = 1 \bmod 35$
$8 \cdot 22 = 1 \bmod 35$	$17 \cdot 33 = 1 \bmod 35$	$26 \cdot 31 = 1 \bmod 35$	$34 \cdot 34 = 1 \bmod 35$

In the RSA algorithm, we will also need to work modulo $\phi(pq) = (p - 1)(q - 1)$. This number often has many factors. For instance, we will only consider the case that both p and q are odd primes, so that $(p - 1)(q - 1)$ has 4 as a factor. We want to find a number that has an inverse modulo $\phi(pq)$, and this can be found by trial and error.

The theorem that we need to make the RSA encryption work, is now stated without proof:

Theorem 19.2.4. *For a pair of primes $p \neq q$ and numbers e, d such that $ed = 1 \bmod \phi(pq)$, and for any number T , we have*

$$T = (T^e)^d \bmod pq.$$

19.3 The RSA algorithm

We now have the necessary ingredients in place for actually describing the RSA algorithm. There are three steps. One is to compute the ingredients that appear in the algorithm. This is performed only once. The second is encryption, the third is decryption. These steps are performed each time a message is sent.

Steps to set up the algorithm

1. Choose two large primes p and q . As an example, we will consider two not-so-large primes to understand the procedure.
2. Compute $\phi(pq) = (p - 1)(q - 1)$.
3. Choose a (positive) number $e < \phi(pq)$ with $\gcd(e, \phi(pq)) = 1$. e is chosen by trial and error.
4. Compute the (positive) number $d < \phi(pq)$ such that $de = 1 \bmod \phi(pq)$.
5. $n = pq$ and e are distributed to anyone you want to receive secret messages from, for instance by publishing these two numbers online or in your local newspaper. d , on the other hand, is to be kept *very* safe.

Since the pair of numbers n and e are publically available, this is an example of a *public key encryption algorithm*.

Steps to encrypt a message

1. Break your message into parts so that each can be described by a number less than n .
2. For each part T , compute $U = T^e \bmod n$.
3. Send U to the recipient, for instance by e-mail or by publishing it in the local newspaper.

Steps to decrypt a message

1. Compute $U^d \bmod n$, this is the original T .
2. Assemble the original message by putting together all these parts.

Note that Theorem 19.2.4 tells us that $U^d \bmod n = (T^e)^d \bmod n$ is in fact the original T .

Example 19.3.1. With $p = 7$ and $q = 13$, we have $n = pq = 91$ and $\phi(91) = (7 - 1)(13 - 1) = 72$. We can then choose for instance $e = 11$, since $\gcd(11, 72) = 1$. Using the Euclidean algorithm, we compute d . It is maybe instructive to do this computation by hand:

$$\begin{aligned} 72 &= 6 \cdot 11 + 6 \\ 11 &= 1 \cdot 6 + 5 \\ 6 &= 1 \cdot 5 + 1 \end{aligned}$$

Reversing the process, we see that

$$\begin{aligned} 1 &= 6 - 5 \\ &= 6 - (11 - 6) = 2 \cdot 6 - 11 \\ &= 2(72 - 6 \cdot 11) - 11 = 2 \cdot 72 - 13 \cdot 11 \end{aligned}$$

This means that $d = -13 \bmod 72$, but we want the positive representative, so we use $d = -13 + 72 = 59 \bmod 72$.

To encrypt the message 16, we compute $U = 16^{11} \bmod 91$, so that $U = 74$. To decrypt the message 74, compute $74^{59} \bmod 91$. This is 16, so we see that the original message is returned.

We could associate the numbers 1 through 26 to the letters of the English alphabet, and maybe let 27 be the representation of space. Then, for instance, the message “attack to the north” is exchanged for the list

$$\{1, 20, 20, 1, 3, 11, 27, 20, 15, 27, 20, 8, 5, 27, 14, 15, 18, 20, 8\}.$$

For each of these 19 numbers, we use the same procedure as above. The encrypted list becomes

$$\{1, 41, 41, 1, 61, 58, 27, 41, 85, 27, 41, 57, 73, 27, 14, 85, 86, 41, 57\}.$$

This list has the weakness that each letter is always encrypted in the same way, so that we could break it by some sort of frequency analysis, for instance. When we choose larger primes, we can use a more sensible approach by using a single number for a larger chunk of letters.

In the next section we will look at some of the problems that might arise when implementing this algorithm. For instance, 74^{59} is too large to be represented as a 64 bit integer (the data type we will use in MATLAB is called `int64`, similar to `long` in Java), so some care has to be taken when computing the powers.

19.3.1 Finding large primes

For now, we will look at the problem of choosing p and q , that is of finding large primes. Also the problem of cracking the encryption is related to factoring large numbers, and these two problems are related. We will look at a rather naïve method, but it illustrates the point we are making.

Example 19.3.2. Let us find the list of all primes less than 100. First, list all the numbers (1 is omitted, by definition 1 is not a prime).

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50
51	52	53	54	55	56	57	58	59	60
61	62	63	64	65	66	67	68	69	70
71	72	73	74	75	76	77	78	79	80
81	82	83	84	85	86	87	88	89	90
91	92	93	94	95	96	97	98	99	100

The first prime is 2, and any number that is a multiple of 2 is not a prime. We store 2, and remove these multiples.

	2	3	5	7	9
11		13	15	17	19
21		23	25	27	29
31		33	35	37	39
41		43	45	47	49
51		53	55	57	59
61		63	65	67	69
71		73	75	77	79
81		83	85	87	89
91		93	95	97	99

The next prime is 3. We store 3 and remove all multiples of 3. Note that $6 = 2 \cdot 3$

has already been removed, as has all multiples of 6.

	2	3	5	7
11		13		17
		23	25	29
31			35	37
41		43		47
		53	55	59
61			65	67
71		73		77
		83	85	89
91			95	97

Continue with first 5 and then 7:

2	3	5	7		2	3	5	7
11	13		17	19	11	13		17
	23			29		23		29
31			37		31			37
41	43		47	49	41	43		47
	53			59		53		59
61			67		61			67
71	73		77	79	71	73		79
	83			89		83		89
91			97					97

The next prime is 11. The smallest integer that is divisible by 11, but which is not a multiple of a smaller number (and thus already removed), is $11^2 = 121 > 100$. So there are no numbers left that are multiples of 11. Similarly, we see that all the numbers we are left with are primes! It is enough to remove multiples of the primes less than $\sqrt{100} = 10$. There are 25 primes less than 100, and the list we end up with is

2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97

This method for finding primes is known as *Erathostenes' sieve*. Here is some pseudo-code for it:

```

numbers = 2:n
primes = <empty list>
for i from 2 to sqrt(n) <or the largest integer smaller than sqrt(n)>
    if i is in numbers
        store i in primes

```

```

        remove all multiples of i from numbers
    end if
end for

```

A refinement of this method is to use it twice. Assume we want to find the primes in the range $9950 - 10000$. First use Eratosthenes' sieve to find the primes less than $\sqrt{10000} = 100$ (we just did this). Any number less than 10000 that is not a prime must necessarily have a prime factor less than 100 (why?), so we remove multiples of the primes in our list that are in the given range. The numbers we are left with are primes.

Example 19.3.3. Starting from the list of numbers 9950, 9951, 9952, $\dots$, 10000, after we remove multiples of 2, 3, 5 we are left with

9953, 9959, 9961, 9967, 9971, 9973, 9977, 9979, 9983, 9989, 9991, 9997

Multiples of 7 are $9961 = 7 \cdot 1423$ and $9989 = 7 \cdot 1427$. There is one multiple of 11, namely $9977 = 11 \cdot 907$. Also, $9971 = 13 \cdot 767$, $9997 = 13 \cdot 769$ and $9979 = 17 \cdot 587$. None of the numbers left are divisible by 19. After removing the numbers found so far, we are left with

9953, 9959, 9967, 9973, 9983, 9991

Continuing with the rest of the primes less than 100, we remove 9959, a multiple of 23, 9953, a multiple of 37, 9983, a multiple of 67, and 9991, a multiple of 97. There are only two primes in this range, 9967 and 9973.

Example 19.3.4. $9967 \cdot 9973 = 99400891$. How much work is it for someone else to factor this number? In principle, if we know that this number is the n in RSA, there are two prime factors. One of them will be less than $\sqrt{99400891} \approx 9969$ (rounded down), but we don't know how much less. To factor it, we need to check all primes less than 9969, but to produce the number, we only needed the list of primes up to 100. There is a huge difference in the amount of work!

Comment 19.3.5. Since e and n are public, anyone who can factor $n = pq$ can compute $\phi(n) = (p-1)(q-1)$ and hence find d using the Euclidean algorithm applied to e and $\phi(n)$. The reason RSA encryption is safe, is that factoring n is infeasible. We have only considered a very simple way to factor large numbers, and better procedures exist. However, the main idea that producing two large primes is much easier than factoring their product holds true in general.

Quantum computers might change this, though. If (or when) such computers are produced, RSA will be compromised due to *Shor's algorithm*, an efficient quantum algorithm for factoring large numbers.

19.4 Implementation of the RSA algorithm

We now proceed to implement the RSA algorithm, both the process of finding good primes, encoding, decoding and eventually cracking the algorithm. All the ingredients are already in place, although there are several problems because we want to compute with rather large numbers. If we were working with fixed-size integers, some of the simplifications in the implementation would be absolutely necessary to get a correct answer. However, several of the improvements in this section will also improve the efficiency significantly for computations involving variable-sized integers.

Choosing the public key

We will implement the procedure from Eratosthenes' sieve. Say that we want the number n in the public key to have approximately 8 digits. Then we need two primes with approximately 4 digits, so that we need all the primes with less than 2 digits. Using `int64`, this will work well for the RSA algorithm. Unfortunately, it is necessary to use other data types to get a higher degree of security in the RSA encryption scheme.

First we must implement Eratosthenes' sieve. Then we must use the similar procedure to find primes in a given range, and choose two of them. We could also find primes in different ranges. Then choose some number e and check that $\gcd(e, \phi(p, q)) = 1$ by running the Euclidean algorithm. If this fails, try another e . Finally, compute d by running the Euclidean algorithm backwards.

Implementing the sieve of Eratosthenes First, we produce a list of all primes less than n . Here is some MATLAB code for this task:

```
function [PrimeList] = Eratosthenes(n)
%Eratosthenes returns the list of primes less than n
PrimeList = 1:n;
PrimeList(1) = 0; % 0 will indicate not a prime
for i = 2:sqrt(n)
    if PrimeList(i)~=0
        j = i;
        while (j*i<=n)
            PrimeList(j*i)=0;
            j = j+1;
        end
    end
end
PrimeList = PrimeList(PrimeList~=0);
```

end

We change all the non-primes to zero, and then make a new list containing the non-zero elements.

Our next task is to make a list of primes in the vicinity of a larger number. We will require two inputs for the lower and upper range of the interval we are searching for primes, and then we will check for division by all primes less than the square root of the upper limit.

```
function [PrimeList] = PrimesInInterval(lower, upper)
%Returns the list of primes in the interval from lower to upper
% lower must be greater than the square root of upper
smallPrimes = Eratosthenes(sqrt(upper));
numberOfPrimes = length(smallPrimes)
PrimeList = lower:upper;
for i =1:numberOfPrimes
    currentPrime = smallPrimes(i);
    % j*currentPrime is a non-prime in the interval
    for j = ceil(lower/currentPrime):floor(upper/currentPrime)
        PrimeList(1+currentPrime*j-lower) = 0;
    end
end
PrimeList = PrimeList(PrimeList~=0);
end
```

Warning: For our use, this is meant to find primes in a small interval of large numbers. The code will not correctly return small primes, i.e. primes smaller than the square root of `upper`, if `lower` has been chosen too small.

Now just run this function to choose two primes p and q between `start` and `stop`; that is any numbers in the list returned. Let $n = pq$, and choose a medium-sized number e that is coprime to $\phi(n) = (p-1)(q-1)$. If we use the Euclidean algorithm backwards, we find d from the same computation that checks if ϕ and e are coprime.

Example 19.4.1. Running the procedure above starting with primes less than 800, we can choose two primes $p = 639671$ and $q = 638893$. Then $\phi = 408680045640$. Trying for instance $e = 41357$ works, and tells us that $d = 53638206053$.

Encryption and decryption

Since the encryption and the decryption actually uses the same type of computations, raising to a power modulo a number, we can implement one method for both

operations. To mitigate the problem of overflow, we will implement our own power modulo a number operation, as just computing T^e and then reduce modulo n leads to overflow in fixed-precision integers, and is also quite inefficient. In Python, integers are variable-sized. In other languages, we would need to be careful with the types used. To make a point of this, we will look at integers with 64 bits (as `long` in Java and `int64` in MATLAB). Even in systems with variable-sizes integers (default in Python, the BigInteger-library in Java), the cost of performing the arithmetical operations necessary for RSA is so high that sensible coding is important.

Powers modulo a number We will use n about as large as the square root of the largest integer we can handle with 64 bits. This means that $n < 2^{32}$. If we want to compute $M^e \bmod n$, first write e in binary, and compute the powers of M in the following way: First, compute $M^2 \bmod n$. Since $n < 2^{32}$, $M^2 < 2^{64}$ and is thus representable as an `int64`. Taking the remainder modulo m yields a new number less than n . Therefore $M^4 = (M^2)^2 \bmod n$ is the square of a number less than n , so is representable as a `long`, and so on. In this way we can make a list of powers $M^{2^k} \bmod n$ without overflow. Then we use the binary expansion to write M^e as a product of the powers we already computed, again remembering to take the remainder modulo n in each step. For instance, if $e = 38 = 100110_2 = 2^5 + 2^2 + 2$, we get that

$$M^e = M^{2^5+2^2+2} = M^{2^5} M^{2^2} M^2$$

Here is some MATLAB code for a function called `powerMod` that uses this idea.

```
function [M] = powerMod(N,e,m)
%powerMod returns M = N^e mod m
if N ==0
    M=0;
elseif e==1
    M = mod(N,m);
else
    M = 0;
    if (mod(e,2)==0) % e even
        M = powerMod(N, e/2,m);
        M = mod(M*M,m);
    else % e odd
        M = mod(N,m);
        M = mod(M*powerMod(N,e-1,m),m)
    end
end
end
end
```

Encryption and decryption For each positive number, representing non-encrypted information, $T < n$, we can encrypt it by calling $U = \text{powerMod}(T, e, n)$. For each encrypted number $U < n$, we can decrypt it by calling $T = \text{powerMod}(U, d, n)$. To use longer messages, write T , and therefore also U , as a table.

Example 19.4.2. For instance, if the public key is $(n, e) = (653859799, 235)$, we can store a string of four characters as a number less than n by assigning a number to each letter (00 through 25 for a through z), and using the eight character number we get as one of the entries in T .

Assume we want to encrypt the message “attack south”. Let us assign 27 to space. Then we break the message into parts as follows:

<i>atta</i>	00, 19, 19, 00	$T(1) = 00191900$
<i>ck s</i>	02, 10, 27, 18	$T(2) = 02102718$
<i>outh</i>	14, 20, 19, 07	$T(3) = 14201907$

We then call $U(i) = \text{powerMod}(T(i), e, n)$ in a loop, getting the encrypted message

$$U = 51284341, 528003199, 329628520.$$

Finally, the recipient can decrypt the message by calling $\text{powerMod}(U(i), d, n)$ with the (secret) $d = 267077827$.

Cracking RSA The set-up we have already developed also allows us to crack the RSA encryption scheme. To decrypt an intercepted message, the only number we need in addition to the public key (n, e) is the number d . The difficult part is factoring n , but once we have found the two factor p and q , we compute d from e in the same way that the party setting up the communication would.

This boils down to producing a list of primes less than $\sqrt{n}$.

For each of these primes, check if n can be divided by it. When the answer is yes, store that prime as p and let $q = n/p$.

19.5 Exercises

Exercise 19.5.1. Use the Euclidean algorithm to compute the greatest common divisor of the given pair of numbers.

(a). 17 000 and 19 000.

(b). 2415 and 1624.

(c). 2178309 and 3524578.

Exercise 19.5.2. Use the Euclidean algorithm with backtracking to compute the multiplicative inverse of a modulo n in each case.

(a). $a = 12$, $n = 35$.

(b). $a = 1097$, $n = 65537$.

Exercise 19.5.3. Using the set-up for Eratosthenes' sieve (the refined version, doing it twice), find all primes in the range 12500 to 12600.

Exercise 19.5.4. In this exercise, we will consider an RSA set-up where everything can be computed using an ordinary calculator.

(a). The public key is $(n, e) = (35, 5)$. Explain why this is a legal choice.

(b). What is d ?

(c). Code letters as $A \rightarrow 0$, $B \rightarrow 1$, ..., $Z \rightarrow 25$. Code and encrypt the word 'south'.

(d). Decrypt, and write as a word, the encrypted string 1, 0, 32, 5.

Exercise 19.5.5. Implement the Euclidean algorithm in Java. Make sure that it works even if the first remainder is zero.

Exercise 19.5.6. Using the method of Eratosthenes, compute all the primes in the interval $[24500, 24600]$.

Exercise 19.5.7. In this exercise, we will consider RSA in a larger scale setting, using numbers that are close to the maximum we can handle using 64 bit integers. The public key is $(n, e) = (115912873, 133)$. Since $n > 10^8$ we can use a simple coding scheme, grouping four characters together by assigning $A \rightarrow 00$, $B \rightarrow 01$, ..., $Z \rightarrow 25$ and space $\rightarrow 99$.

The final goal of this exercise is to crack this encryption scheme, and to decrypt the secret message

$$88709091|115282881|44754833|5274204|37162740|69569552|73765472 \quad (19.2)$$

We will perform a series of computations to achieve this.

(a). Code and encrypt the word "dead".

(b). Use the method of Eratosthenes to find the list of all primes less than $\sqrt{n}$.

(c). For each prime in the list, check if n is divisible by the prime. If yes, call it p and let $q = n/p$.

(d). Check that (n, e) is a legal choice for public key using the Euclidean algorithm, and compute d .

(e). As a test, decrypt your encryption of "dead". Then decrypt the secret message from (19.2), and convert back to text.

Chapter 20

Generating functions

If we have a finite sequence of numbers, we can “store” them in a polynomial. For instance, the sequence $\{1, 4, 6, 4, 1\}$ can be stored as coefficients in the polynomial $1 + 4x + 6x^2 + 4x^3 + 1 \cdot x^4$. If we want to consider infinite sequences, we have to allow infinite sums. Power series are exactly what we need. These series are usually introduced in calculus classes, but are also applicable in discrete mathematics.

20.1 Power series

We have seen earlier that the geometric series can be summed if the quotient is small. If we write x for the quotient, the result is that

$$\frac{1}{1-x} = \sum_{n=0}^{\infty} x^n = 1 + x + x^2 + x^3 + \cdots \quad \text{for } |x| < 1.$$

The idea of connecting a function, like $f(x) = 1/(1-x)$, to a series, like $\sum x^n$, is a powerful tool in calculus. Such a series is called a *power series*, or in this context a *Taylor series*. Since the series resembles an infinite polynomial, we can hope that differentiation and integration can be performed easily for such series. This is indeed true, and the coefficients of the series are also related to repeated differentiation:

Theorem 20.1.1. *If we have an equality*

$$f(x) = \sum_{n=0}^{\infty} a_n x^n$$

valid for all x in an (open) interval around zero, then:

a) f can be differentiated and

$$f'(x) = \sum_{n=1}^{\infty} n a_n x^{n-1}.$$

b) f can be integrated, and

$$\int_0^x f(t) dt = \sum_{n=0}^{\infty} \frac{a_n}{n+1} x^{n+1}.$$

Both equations are valid in the same open interval that the starting equation was valid in.

c) The coefficient $a_n = \frac{f^{(n)}(0)}{n!}$, where $f^{(n)}$ is the n th derivative of f .

We will not give a proof of this theorem here; you can find the proof in most calculus books. Note that we will often want to reindex the series so that the summation index is the same as the exponent.

Example 20.1.2. Let $f(x) = 1/(1-x)$. Then

$$f'(x) = \frac{1}{(1-x)^2} = \left(\sum_{n=0}^{\infty} x^n \right)' = \sum_{n=1}^{\infty} n x^{n-1} = \sum_{m=0}^{\infty} (m+1) x^m.$$

Here we have reindexed the series by writing $m = n - 1$. Similarly,

$$-\ln(1-x) = \int_0^x f(t) dt = \sum_{n=0}^{\infty} \frac{1}{n+1} x^{n+1} = \sum_{m=1}^{\infty} \frac{1}{m} x^m.$$

Here we reindexed using $m = n + 1$. Note that the constant term is zero, consistent with $\ln 1 = 0$.

Example 20.1.3. Since $(e^x)' = e^x$ and $e^0 = 1$, we can find a series that is equal to $f(x) = e^x$ (if it exists; in fact it does) as follows: Assume that

$$f(x) = \sum_{n=0}^{\infty} a_n x^n$$

for a sequence of numbers $\{a_n\}$. Then $f(0) = e^0 = 1 = a_0 + a_1 \cdot 0 + a_2 \cdot 0 + \cdots = a_0$, so the first number is $a_0 = 1$. From then on, we can use $f'(x) = f(x)$ to find the rest of the coefficients:

$$\begin{array}{l|l} f(x) = & f'(x) = \\ a_0 + & 1 \cdot a_1 + \\ a_1 x + & 2 \cdot a_2 x + \\ a_2 x^2 + & 3 \cdot a_3 x^2 + \\ a_3 x^3 + & 4 \cdot a_4 x^3 + \\ \vdots & \vdots \end{array}$$

We see that $a_n = \frac{a_{n-1}}{n}$, so that

$$\begin{aligned} a_0 &= 1 \\ a_1 &= \frac{a_0}{1} = 1 \\ a_2 &= \frac{a_1}{2} = \frac{1}{2} \\ a_3 &= \frac{a_2}{3} = \frac{1}{2 \cdot 3} \\ a_4 &= \frac{a_3}{4} = \frac{1}{2 \cdot 3 \cdot 4} \\ a_5 &= \frac{a_4}{5} = \frac{1}{2 \cdot 3 \cdot 4 \cdot 5} \end{aligned}$$

and in general, $a_n = 1/n!$.

We see that this is consistent with the last statement in the theorem, since $(e^x)' = (e^x)'' = \dots$, we should have $a_n = e^0/n! = 1/n!$ for all n . To conclude,

$$e^x = \sum_{n=0}^{\infty} \frac{1}{n!} x^n.$$

There is a way to find the open interval around zero where a series can be summed, so that the series converges for any x in that interval.

Theorem 20.1.4. *Consider a power series*

$$\sum_{n=0}^{\infty} a_n x^n$$

Assume that the limit

$$L = L(x) = \lim_{n \rightarrow \infty} \left| \frac{a_{n+1} x^{n+1}}{a_n x^n} \right|$$

exists. For those x such that $L < 1$, the series converges. For those x such that $L > 1$, the series diverges. For those x such that $L = 1$, the series could converge or diverge.

Since we take the limit of the ratio of two terms in the series, this theorem is known as the ratio test for convergence.

Proof. By considering only positive or only negative terms, it is possible to drop signs altogether. Therefore, we can assume that all terms are positive below.

Assume first that $L < 1$. Then there is a number M between L and 1, for instance $M = (1 + L)/2$, and a number N such that $|a_{n+1} x^{n+1}| \leq M |a_n x^n|$ for all $n \geq N$. But then

$$\sum_{m=0}^{\infty} a_{n+m} x^{n+m} \leq \sum_{m=0}^{\infty} M^m a_N x^N = a_N x^N \sum_{m=0}^{\infty} M^m = \frac{a_N x^N}{1 - M}.$$

In particular, the sum is finite, so the series converges.

Assume that $L > 1$. Then there is a number M between 1 and L , for instance $M = (1 + L)/2$, and a number N such that $|a_{n+1}x^{n+1}| \geq M|a_nx^n|$ for all $n \geq N$. But then

$$\sum_{m=0}^{\infty} a_{n+m}x^{n+m} \geq \sum_{m=0}^{\infty} M^m a_N x^N = \infty,$$

so the series diverges.

When $L = 1$, to see that both cases can occur, it is sufficient to give two examples of series with $L = 1$, one convergent and one divergent. We leave the details to be checked by the interested reader, but use two series that we have encountered before. An example of a convergent series with $L = 1$ is $\sum \frac{1}{n(n-1)}$. An example of a divergent series with $L = 1$ is $\sum \frac{1}{n}$. $\square$

Example 20.1.5. Let us check a few examples. For the series of the logarithm,

$$\ln(1-x) = -\sum_{n=0}^{\infty} \frac{1}{n+1} x^{n+1},$$

we can compute

$$L = \lim_{n \rightarrow \infty} \left| \frac{x^{n+1}/(n+1)}{x^n/n} \right| = |x| \lim_{n \rightarrow \infty} \frac{n}{n+1} = |x|.$$

By the theorem, the series converges when $L < 1$, which means that $|x| < 1$, and diverges when $|x| > 1$. In the two remaining cases, $x = \pm 1$, the series converges for $x = -1$ and diverges for $x = 1$, though it is more difficult to see this.

For the series for the exponential function, $e^x = \sum_{n=0}^{\infty} x^n/n!$, we compute

$$L = \lim_{n \rightarrow \infty} \left| \frac{x^{n+1}/(n+1)!}{x^n/n!} \right| = |x| \lim_{n \rightarrow \infty} \frac{1}{n+1} = 0.$$

No matter what x is, the limit is zero (less than one), so the series converges for all values of x .

20.2 Sequences as coefficients

We now turn around, and start with a sequence and ask for a series with the sequence as coefficients. The series $\sum a_n x^n$ is called the *generating series* for the

sequence $\{a_n\}$. If the series converges to some function, this function is known as the *generating function*. In other sources, you might see the series itself is referred to as the generating function, regardless of the convergence. We will keep the distinction between the series and the function.

Example 20.2.1. Consider the sequence $\{5, 8, 11, 14, \dots\}$, or with a formula, $a_n = 5 + 3n$ for $n \geq 0$. The generating series associated to this sequence is the series

$$\sum_{n=0}^{\infty} a_n x^n = \sum_{n=0}^{\infty} (5 + 3n)x^n = 5 + 8x + 11x^2 + 14x^3 + \dots$$

In general, generating series are more useful when we can connect them to functions that we are already familiar with. For this to be possible, we will at least need a series that converges (in an open interval). It is often possible to find such an interval by using the ratio test.

Example 20.2.2. Using the ratio test for

$$\sum_{n=0}^{\infty} a_n x^n = \sum_{n=0}^{\infty} (5 + 3n)x^n = 5 + 8x + 11x^2 + 14x^3 + \dots$$

we need the limit

$$L = \lim_{n \rightarrow \infty} \left| \frac{a_{n+1}x^{n+1}}{a_n x^n} \right| = |x| \lim_{n \rightarrow \infty} \frac{3(n+1) + 5}{3n + 5} = |x| \cdot 1 = |x|.$$

We see that the series converges for $|x| < 1$, so there is a function, still unknown, that corresponds to this series.

Based on the computations from the previous section, we can make a short list of generating functions:

sequence	series	function
$\{1, 1, 1, 1, \dots\}$	$\sum_{n=0}^{\infty} 1 \cdot x^n$	$\frac{1}{1-x}$
$\{1, 2, 3, 4, 5, \dots\}$	$\sum_{n=0}^{\infty} (n+1)x^n$	$\frac{1}{(1-x)^2}$
$\{0, 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots\}$	$\sum_{n=1}^{\infty} \frac{1}{n} x^n$	$-\ln(1-x)$
$\{1, 1, \frac{1}{2}, \frac{1}{3!}, \frac{1}{4!}, \dots\}$	$\sum_{n=0}^{\infty} \frac{1}{n!} x^n$	e^x

If we multiply a function by a constant, each coefficient in the corresponding series is multiplied by the same constant. Similarly, if we add two series, corresponding coefficients are added.

Example 20.2.3. The generating function for the sequence $\{2, 2, 2, \dots\}$ is twice the generating function for $\{1, 1, 1, 1, \dots\}$, so it is equal to $2/(1-x)$.

The generating function for the sequence $\{3, 6, 9, 12, \dots\}$ is three times the generating function for $\{1, 2, 3, 4, \dots\}$, so it is $3/(1-x)^2$.

If we sum the two series, we see that the generating function for $\{2+3, 2+6, 2+9, 2+12, \dots\} = \{5, 8, 11, 14, \dots\}$ is equal to $2/(1-x) + 3/(1-x)^2$.

Example 20.2.4. Find the generating function for the sequence $\{-3, 1, 5, 9, 13, \dots\}$.

The generating series is

$$-3 + x + 5x^2 + 9x^3 + 13x^4 + \dots = \sum_{n=0}^{\infty} (4n-3)x^n.$$

We are interested in finding the function that this series converges to (if there is one).

We start by noticing that this is an arithmetic sequence with difference 4. Comparing to the list of known functions, it is clear that the generating function is of the form

$$4 \cdot \frac{1}{(1-x)^2} + \text{other terms}.$$

Since $4 \cdot \frac{1}{(1-x)^2}$ corresponds to $\{4, 8, 12, \dots\}$ we see that we will need to subtract 7 in each position. This means that the generating function is

$$4 \cdot \frac{1}{(1-x)^2} - 7 \cdot \frac{1}{1-x}.$$

Let us see how this fits with the generating series as well:

$$4 \cdot \frac{1}{(1-x)^2} - 7 \cdot \frac{1}{1-x} = \sum_{n=0}^{\infty} 4(n+1)x^n - \sum_{n=0}^{\infty} 7x^n = \sum_{n=0}^{\infty} (4n-3)x^n$$

Example 20.2.5. There are generating functions for binomial coefficients, if we arrange them in the correct way. Consider the finite sequence

$$\left\{ \binom{n}{0}, \binom{n}{1}, \binom{n}{2}, \dots, \binom{n}{n} \right\}.$$

The *binomial theorem* says that the generating function

$$\binom{n}{0} + \binom{n}{1}x + \binom{n}{2}x^2 + \dots + \binom{n}{n}x^n = (1+x)^n.$$

Among our generating functions, this is special since the sequence is finite: The resulting series is a polynomial. There is thus no question about convergence, the equation is valid for all x .

Another way to organize binomial coefficients is by increasing both the numbers at the same time. Consider the sequence

$$\left\{ \binom{n}{0}, \binom{n+1}{1}, \binom{n+2}{2}, \binom{n+3}{3}, \dots \right\}.$$

The series (note that the summation index is k)

$$\sum_{k=0}^{\infty} \binom{n+k}{k} x^k$$

is its generating function. An explicit formula for this function can be found by repeatedly differentiating $1/(1-x)$:

$$\sum_{k=0}^{\infty} \binom{n+k}{k} x^k = \frac{1}{(1-x)^{n+1}} \quad (20.1)$$

Example 20.2.6. We can substitute a new expression into a known series to get a new expression. Sometimes this will give us new generating functions. Consider the function

$$f(x) = \frac{1}{1-2x}.$$

If we substitute $u = 2x$, we get a series we already know, so that

$$f(x) = \frac{1}{1-2x} = \frac{1}{1-u} = \sum_{n=0}^{\infty} u^n = \sum_{n=0}^{\infty} (2x)^n = \sum_{n=0}^{\infty} 2^n x^n.$$

As a result, we see that $1/(1-2x)$ is the generating function for the sequence $\{1, 2, 4, 8, 16, \dots\}$.

In principle, we should always include the interval where such an equation is valid, see Exercise 20.3.5.

Example 20.2.7. Because of the connection between generating functions and operations in calculus, we can sometimes solve problems about sequences for instance by solving differential equations. This example uses more calculus than the rest of this chapter.

Recall that the Fibonacci numbers $F(i)$ are defined recursively by $F(0) = 0$, $F(1) = 1$ and

$$F(n+2) = F(n+1) + F(n) \quad \text{for } n \geq 0.$$

It turns out to be easier to collect this information in the sequence $\{F(n)/n!\}$. We therefore introduce the generating function

$$f(x) = \sum_{n=0}^{\infty} \frac{F(n)}{n!} x^n.$$

Then we find (let $m = n - 1$)

$$f'(x) = \sum_{n=1}^{\infty} \frac{F(n)}{n!} n x^{n-1} = \sum_{n=1}^{\infty} \frac{F(n)}{(n-1)!} x^{n-1} = \sum_{m=0}^{\infty} \frac{F(m+1)}{m!} x^m$$

and (let $l = m - 1$)

$$f''(x) = \sum_{m=1}^{\infty} \frac{F(m+1)}{m!} m x^{m-1} = \sum_{l=0}^{\infty} \frac{F(l+2)}{l!} x^l.$$

If we write out the first few terms of each of these series, we find

series	constant	x	x^2	x^3
f	$F(0)$	$F(1)$	$F(2)/2$	$F(3)/6$
f'	$F(1)$	$F(2)$	$F(3)/2$	$F(4)/6$
f''	$F(2)$	$F(3)$	$F(4)/2$	$F(5)/6$

In general, the terms for x^n are $F(n)/n!$, $F(n+1)/n!$ and $F(n+2)/n!$ respectively. So we see that the original equation for $F(n+2)$ gives us that

$$f''(x) = f'(x) + f(x).$$

This is a differential equation, which can be solved by various methods. The result is that

$$f(x) = A e^{\frac{1+\sqrt{5}}{2}x} + B e^{\frac{1-\sqrt{5}}{2}x}$$

The coefficients A and B can be determined from $f(0) = F(0) = 0$ and $f'(0) = F(1) = 1$. Therefore,

$$f(x) = \frac{1}{\sqrt{5}} e^{\frac{1+\sqrt{5}}{2}x} - \frac{1}{\sqrt{5}} e^{\frac{1-\sqrt{5}}{2}x}.$$

Finally, applying the method of substitution, we can expand the two exponential functions into series and add the results. The coefficients are the Fibonacci numbers divided by $n!$, so by omitting the denominators we get the formula

$$F(n) = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n.$$

20.3 Exercises

Exercise 20.3.1. Find the open interval of convergence for each series.

(a).

$$\sum_{n=1}^{\infty} \frac{x^n}{\sqrt{n}}$$

(b).

$$\sum_{n=0}^{\infty} (-2)^n x^n$$

(c).

$$\sum_{n=1}^{\infty} \frac{x^n}{3^n}$$

Exercise 20.3.2. Find functions representing the generating series for each sequence. In each case, the terms a_0, a_1, a_2 and a_3 are given.

(a). $\{4, 4, 4, 4, \dots\}$

(b). $\{2, 5, 8, 11, \dots\}$

(c). $\{10, 14, 18, 22, \dots\}$

(d). $\{-4, -1, 2, 5, \dots\}$

Exercise 20.3.3. Prove the formula (valid for $|x| < 1$)

$$\sum_{k=0}^{\infty} \binom{n+k}{k} x^k = \frac{1}{(1-x)^{n+1}}$$

by induction on n .

Exercise 20.3.4. Find the pair of a sequence and a generating function that comes from

$$f(x) = \frac{1}{1-2x} = \sum_{n=0}^{\infty} 2^n x^n$$

by differentiating.

Exercise 20.3.5. Find an open interval where the series $\sum_{n=0}^{\infty} 2^n x^n$ converges using the ratio test. Check your answer by comparing to the known open interval where $1/(1-u)$ converges, when $u = 2x$.

Exercise 20.3.6. Find the generating series for the sequence $\{1, 3, 9, 27, 81, \dots\}$.

Exercise 20.3.7. In this exercise, we will compute a generating function for $\{2, 3, 7, 14, 24, 37, \dots\}$.

(a). Write down the first few terms of the series expansion of $\frac{1}{(1-x)^3}$ using Equation (20.1).

(b). Compare the first three terms in the sequence $\{2, 3, 7, 14, 24, 37, \dots\}$ with the first three terms in

$$\frac{A}{(1-x)^3} + \frac{B}{(1-x)^2} + \frac{C}{1-x}$$

Show that this produces a set of three equations in the three unknowns A, B and C .

(c). Solve the system of equations.

(d). It is a fact that

$$\frac{A}{(1-x)^3} + \frac{B}{(1-x)^2} + \frac{C}{1-x}$$

is the generating series for $\{2, 3, 7, 14, 24, 37, \dots\}$ using the computed values of A, B and C . Check that we get the correct values 14 and 24 as well.

Exercise 20.3.8. Consider the generating function for the Fibonacci numbers (not the version where we divide by $n!$), $g(x) = \sum F(n)x^n$. Compute term for term

$$g(x) - xg(x) - x^2g(x).$$

Note that multiplying by a power of x “shifts” the sequence. Use the result to find an expression for g .

Chapter 21

Principal component analysis

Principal component analysis (PCA) is a technique for reducing the dimension of a data set. The goal is (at least) two-fold: To get a more manageable size, and to enhance the most interesting features of the data set. A drawback with this method is a real loss of information. Furthermore, the meaning of the original variables is obscured in the process. As a consequence, the method is more suitable for describing phenomena than for explaining them.

After starting with an overview of PCA, we will introduce the mathematics needed to deal with it. Then we will consider results and analysis from this method using real meteorological data.

21.1 Introduction to principal component analysis

Consider a large set of heterogeneous data, for instance arising from a large number of experiments. An example to keep in mind is meteorological data: air pressure, temperature, precipitation, visibility. The data could be measured for many sites at many times. How could this data be described and analyzed?

In general, we are interested in values (what is the typical temperature in Paris in May) and variation (what is the range of temperatures we would expect in Paris in May). Principal component analysis (PCA) is about variation and covariation. We want to extract vectors or directions that explain as much as possible of the variation of the data, and approximate the whole data set by only considering these vectors. Assume that X is an $n \times m$ -matrix representing the data. Here we want that

$$\begin{array}{c|c} n & \text{the number of objects we want to study} \\ \hline m & \text{the number of measurements for each object} \end{array}$$

Figure 21.1: T and P are small compared to X

$$X = T P^T + E$$

For instance, if we want to compare temperatures in different French cities, we want n cities and m times to compute the temperature. The matrix X would look like this:

$$\begin{array}{c}
 \text{Paris} \\
 \text{Lyon} \\
 \text{Marseille} \\
 \vdots
 \end{array}
 \begin{pmatrix}
 \text{May 1} & \text{May 2} & \text{May 3} & \dots \\
 \text{temp. Paris May 1} & \text{temp. Paris May 2} & \text{temp. Paris May 3} & \dots \\
 \text{temp. Lyon May 1} & \text{temp. Lyon May 2} & \text{temp. Lyon May 3} & \dots \\
 \text{temp. Marseille May 1} & \text{temp. Marseille May 2} & \text{temp. Marseille May 3} & \dots \\
 \vdots & \vdots & \vdots & \vdots
 \end{pmatrix}$$

We want to express X as (recall that A^T is the transpose of the matrix A)

$$X = TP^T + E$$

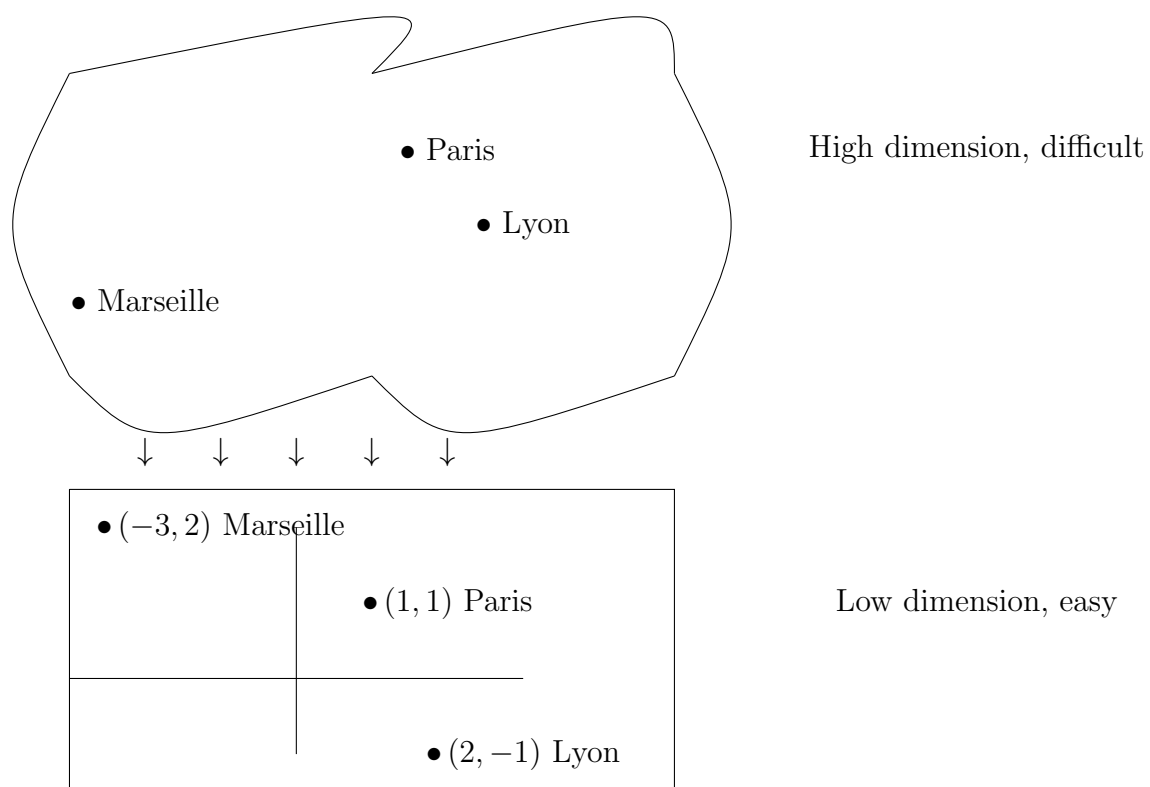
where T is an $n \times a$ -matrix, P is of size $m \times a$, and E has the same size as X . We want a to be small compared to n and m . See Figure 21.1 for an intuitive feeling for what this means.

Here are the names for the various matrices:

X	$n \times m$	data matrix
T	$n \times a$	scores matrix
P	$m \times a$	loadings matrix
E	$n \times m$	noise or error

In the original format, each object is represented by m measurements, so that the data points are n points in $\mathbb{R}^m$. For instance, with $m = 31$ days in May, Paris is represented by a point in $\mathbb{R}^{31}$. This is difficult to visualize! If we let $a = 2$, PCA chooses a two-dimensional plane inside $\mathbb{R}^{31}$, and projects each point into this plane. Then the objects are represented by ordinary plane points, and we can easily visualize them, see Figure 21.2! The plane is spanned by the two columns in P . The coordinates of the n projected points in this plane are given by the n rows of T .

What is lost? Obviously, we can't keep all the information from the large space in the small space, so there is a substantial loss of information. Furthermore, the coordinates in the high dimensional space can be understood directly as results from

Figure 21.2: PCA projects from $\mathbb{R}^m$ to $\mathbb{R}^a$ 

measurements. The coordinates in the low-dimensional space are linear combinations of the original measurements (the technical term is *latent variable*), but have no direct meaning themselves. So it is more difficult to understand the result.

What is gained? The low-dimensional space is chosen so that the points are as far away from each other as possible. Technically, as much variation as possible is kept. Similarly to regression, we can talk about how large a percentage of the variation that is explained by our low-dimensional space. So this is a simpler model that still contains a lot of information. In addition, the result is much less influenced by noise in the measurements than the original high-dimensional model. In principle, this could mean that the model captures the essence of the phenomena we are interested in. The original variables could be correlated, but the new ones (represented by the columns of P) aren't. Also, since we can visualize the results, we can see features of the data set that would otherwise be invisible. For instance, a group of objects might be very close together in the low dimensional space, but difficult to put together in the high-dimensional space.

21.2 Mathematical preliminaries

The processes involved in PCA depend on a number of mathematical results that are difficult to prove. We will now take a look at some of this machinery, but accept that we can't prove everything. It is easier to use PCA than to explain why it works!

We let X be a matrix containing the data we are interested in. If we have n different experiments, each producing m values, X will be an $n \times m$ matrix. For instance, if we measure temperature, air pressure and humidity (three values) in eight different cities, X will be an 8×3 -matrix. In order to compare values in different units, we perform columnwise standardizations. In each column, we subtract the average value of the column in each spot. The effect is that the average value (hence also the sum) in each column is zero, so that we discard information about typical values, retaining information about variation. Assume that X has this property. Furthermore, we can standardize each column by dividing by sample standard deviation. This step ensures that our variables are independent on choice of units (at least linearly). For instance, temperature measurements in Celsius, Kelvin or Fahrenheit will give the same standardized matrix. Also, this allows us to compare variables with incomparable units (precipitation in millimeters is otherwise incomparable to temperature) in a meaningful way.

Definition 21.2.1. Let X be a data matrix, normalized so that the mean in each

column is zero. Then the (sample) covariance matrix of X is the matrix

$$\frac{1}{n-1}X^T X.$$

In words, it is the product of the transpose of X and X , divided by one less than the number of rows.

Example 21.2.2. Consider the matrix

$$\begin{pmatrix} 1 & 1 & 2 \\ 0 & 2 & 3 \\ 1 & 3 & 2 \\ -1 & 2 & 3 \end{pmatrix}.$$

Subtracting the mean from each column ($-1/4$, 2 and $11/4$ respectively) results in the matrix

$$X = \begin{pmatrix} 1 - \frac{1}{4} & 1 - 2 & 2 - \frac{10}{4} \\ 0 - \frac{1}{4} & 2 - 2 & 3 - \frac{10}{4} \\ 1 - \frac{1}{4} & 3 - 2 & 2 - \frac{10}{4} \\ -1 - \frac{1}{4} & 2 - 2 & 4 - \frac{10}{4} \end{pmatrix} = \begin{pmatrix} 0.7500 & -1.0000 & -0.5000 \\ -0.2500 & 0 & 0.5000 \\ 0.7500 & 1.0000 & -0.5000 \\ -1.2500 & 0 & 0.5000 \end{pmatrix}.$$

Furthermore,

$$\frac{1}{n-1}X^T X = \frac{1}{3} \begin{pmatrix} 2.7500 & 0 & -1.5000 \\ 0 & 2.0000 & 0 \\ -1.5000 & 0 & 1.0000 \end{pmatrix}.$$

The terms along the diagonal are the variances of the three columns in X . The off-diagonal element in position (i, j) is the covariance between the i th and j th column of X .

Definition 21.2.3. Let X be a data matrix. The standardized data matrix is obtained from X by first subtracting the mean from each column of X , so that the columns have mean 0. Then we divide by the standard deviation in each column. The resulting matrix then has columns with mean zero and variance 1.

So for each column X_j in X , we first compute

$$\mu_j = \frac{1}{n} \sum_{i=1}^n X_{ij}$$

and let the new X_j be the old X_j minus μ_j . We then compute

$$\sigma_j = \sqrt{\left(\frac{1}{n-1} \sum_{i=1}^n X_{ij}^2 \right)}$$

and let the new X_j be the old X_j divided by σ_j .

Example 21.2.4. Continuing with the matrix from Example 21.2.2, we find that

$$\sigma_1 = \sqrt{\left(\frac{1}{n-1} \sum_{i=1}^n X_{ij}^2\right)} = \sqrt{\left(\frac{1}{3}(0.75^2 + (-0.25)^2 + 0.75^2 + (-1.25)^2)\right)} = 0.9574.$$

Similarly, $\sigma_2 = 0.8165$ and $\sigma_3 = 0.5774$. Dividing the three columns of X by these three values gives us the matrix

$$X_{new} = \begin{pmatrix} 0.7500/\sigma_1 & -1.0000/\sigma_2 & -0.5000/\sigma_3 \\ -0.2500/\sigma_1 & 0 & 0.5000/\sigma_3 \\ 0.7500/\sigma_1 & 1.0000/\sigma_2 & -0.5000/\sigma_3 \\ -1.2500/\sigma_1 & 0 & 0.5000/\sigma_3 \end{pmatrix} = \begin{pmatrix} 0.7833 & -1.2247 & -0.8660 \\ -0.2611 & 0 & 0.8660 \\ 0.7833 & 1.2247 & -0.8660 \\ -1.3056 & 0 & 0.8660 \end{pmatrix}.$$

If we now compute the covariance matrix, we get

$$\frac{1}{n-1} X_{new}^T X_{new} = \begin{pmatrix} 1 & 0 & -0.9045 \\ 0 & 1 & 0 \\ -0.9045 & 0 & 1 \end{pmatrix}.$$

From now on, we will assume that our data matrix X is standardized so that the mean of each column is zero. Whenever X contains measurements in incomparable units, we also assume that the variance of each column is one.

Recall that we are interested in writing

$$X = TP^T + E$$

where T is an $n \times a$ -matrix, P is of size $m \times a$, and E has the same size $n \times m$ as X . Furthermore, we want to choose T and P so that as much of the variation in X as possible is kept. The following definition gives a mathematical description of the best choice for T and P .

Definition 21.2.5. Let X be a standardized matrix of size $n \times m$. The principal components are the eigenvectors of $X^T X$, ordered by the size of the eigenvalues (largest first). We can choose m independent such eigenvectors, and we normalize them to unit vectors. Set $a \leq \min(n, m)$, and choose the first a principal components. These are stored as columns in the matrix of loadings P . Similarly, we store a eigenvectors of XX^T in a matrix T , connected to P by $T = XP$.

We will compute principal components using an algorithm known as *NIPALS*, or *Non-linear Iterative Partial Least Squares*, which resembles the power method for computing eigenvectors. The power method computes an eigenvector corresponding

to the largest eigenvalue. In our case, we can compute eigenvectors (loadings) p for $X^T X$, starting from the largest eigenvalue, then the second largest and so on. We keep normalizing (dividing by the norm, or length, of the vector), so that our vectors p are unit vectors. We then form $t = Xp$ (scores), which will in general not be unit vectors.

Here is pseudo-code for the algorithm:

```

input X % nxm-matrix with mean zero columns
input a % the number of principal components wanted
input error_tolerance % bound for numerical aproximation
for j from 1 to a
    t_old = random vector with n elements % <Note 1>
    p = normalize( transpose(X) * t_old) % length one vector with m elements
    t = X*p;
    while norm( t - t_old) > error_tolerance
        p = normalize( transpose(X) * t)
        t_old = t
        t = X*p % <Note 2>
    end
    store t as the jth column in T
    store p as the jth column in P
    X = X - t*transpose(p) % <Note 3>
end
output T
output P

```

At Note 1 in the code, we choose a random vector. It is possible to make a more intelligent choice for starting point, so you could see other variants of this algorithm. At Note 2 in the code, if the old t were already an eigenvector of XX^T , this new t would be exactly equal to the old t . At Note 3, the matrix X is replaced by a new matrix which has the same principal components as the old X , apart from p . A different way to say this is that $X^T X$ has the same eigenvectors as before (not strictly true since linear combinations of eigenvectors with the same eigenvalue, e.g. 0, are also eigenvectors), but the eigenvalue of p has been reduced to zero. Therefore the largest eigenvalue after Note 3 is equal to the second largest before.

We have now produced (up to the error tolerance) a matrix P containing as columns a basis of an a -dimensional subspace of $\mathbb{R}^m$ and a matrix T containing as rows the coordinates of the n points projected into this subspace.

Definition 21.2.6. We will use the explained variation as one way to assess the number of principal components we need. Let X be a mean-centered data matrix of size $n \times m$. Then the total variation is the trace of $X^T X$, the sum of the diagonal elements. To understand this, write out the diagonal elements of this matrix product. Recall that if $X = (X_{ij})$, $X^T = (X_{ji})$ with the indices swapped. We get

$$\begin{aligned}
 X^T X &= \begin{pmatrix} X_{11} & X_{21} & \cdots & X_{n1} \\ X_{12} & X_{22} & \cdots & X_{n2} \\ \vdots & \vdots & \ddots & \vdots \\ X_{1m} & X_{2m} & \cdots & X_{nm} \end{pmatrix} \begin{pmatrix} X_{11} & X_{12} & \cdots & X_{1m} \\ X_{21} & X_{22} & \cdots & X_{2m} \\ \vdots & \vdots & \ddots & \vdots \\ X_{n1} & X_{n2} & \cdots & X_{nm} \end{pmatrix} \\
 &= \begin{pmatrix} X_{11}^2 + X_{21}^2 + \cdots + X_{n1}^2 & * & \cdots & * \\ * & X_{12}^2 + X_{22}^2 + \cdots + X_{n2}^2 & \cdots & * \\ \vdots & \vdots & \ddots & \vdots \\ * & * & \cdots & X_{1m}^2 + X_{2m}^2 + \cdots + X_{nm}^2 \end{pmatrix}.
 \end{aligned}$$

Here $*$ signifies an element we are not interested in. The sum of the diagonal elements is $\sum X_{ij}^2$.

The explained variation is the trace of $T^T T$, usually written as a percentage of the total variation.

In the next section, these notions will come to life as we look at graphical representations of real data subjected to PCA.

21.3 Extended example - meteorological data

Historical meteorological data is readily available online. Usually, there is a fair amount of work to be done in processing the data for use in your own programs. We will assume that there are scripts available to do this, and start from a matrix loaded into MATLAB, say. To be precise, we will look at daily measurements of maximal temperature and precipitation over a period of one year from a number of different places in Norway. All data used is freely available from `eklima.met.no`.

We will organize our data in this form:

Site	Rain Jan. 1	Rain Jan 2	...	Rain Dec. 31	Temp Jan. 1	...	Temp Dec. 31
Site 1	2mm	1.2 mm	...	0 mm	-3 °C	...	4°C
Site 2							
⋮							

With n sites (and not a leap year), the matrix will be an $n \times 730$ -matrix. Since precipitation in millimeters and temperature in degrees Celsius are incomparable, we

will both adjust to mean zero and variance one columns.

To get interesting results, we choose the following sites (in alphabetical order):

- **Bergen** - by far the most interesting place in Norway. Located near the coast at 60 degrees north.
- **Fagernes** - located inland, at 61 degrees north. 365 meters above sea level.
- **Karasjok** - located inland, at 69 degrees north. 135 meters above sea level.
- **Kongsberg** - located inland, at 60 degrees north. 162 meters above sea level
- **Kristiansand (Kjevik)** - located near the coast, at 58 degrees north.
- **Lindesnes** - located at the coast, at the southernmost tip of Norway. 58 degrees north.
- **Oslo (Blindern)** - located inland, by a fjord. 60 degrees north.
- **Røros** - located inland, at 600 meters elevation, 62 degrees north.
- **Sortland** - located in the Vesterålen archipelago, at 68 degrees north.
- **Vardø** - located at the coast at 70 degrees north.

The chosen data is from 2013. In the plots, a site will be identified with the first two letters (“Be” for Bergen, for instance). With these ten sites, we will have a 10×730 -matrix of measurements. The goal of performing PCA on this data is to see if there are structures not immediately obvious from the data. What sites will be close together? Can we see information about elevation, latitude or geographic proximity from these measurements? Other information that we haven’t thought about before?

To see if we have captured much of the interesting information, we can consider the percentage of the variance that is explained by the first a principal components. For our data, $a = 2$ gives us that 57% is explained. We plot the first two principal components as axes in Figure 21.3.

In this figure we see that Sortland seems to be the most special place. We also see that the five sites inland are to the lower left, with Oslo (inland but by a fjord) closest to sites by the coast. Note that the axes have no direct physical interpretation since they correspond to latent variables.

Figure 21.3: First two principal components

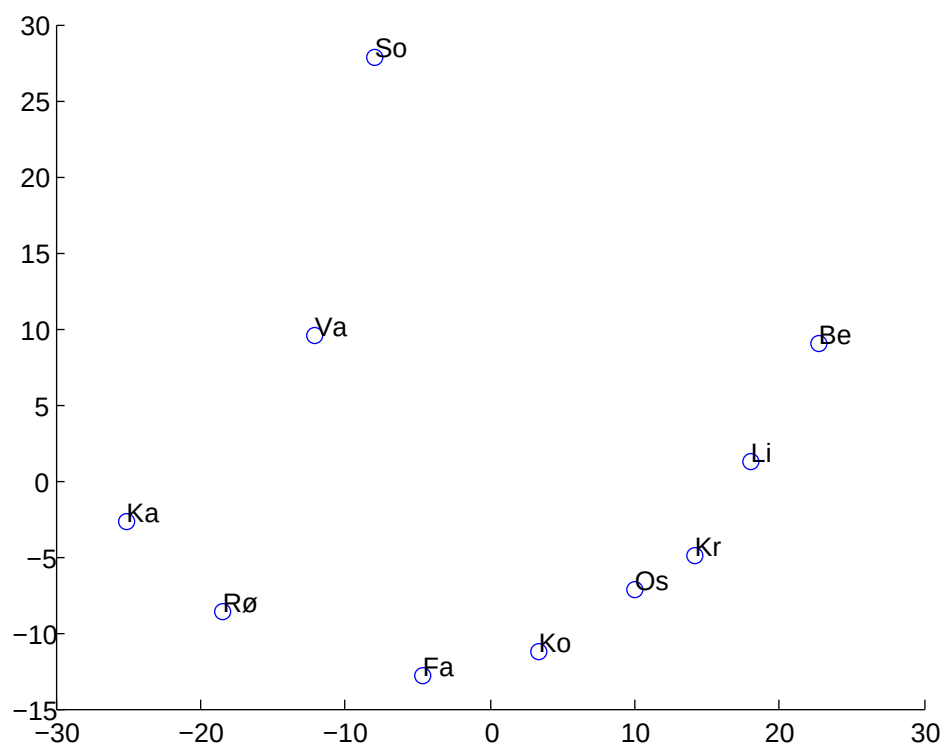
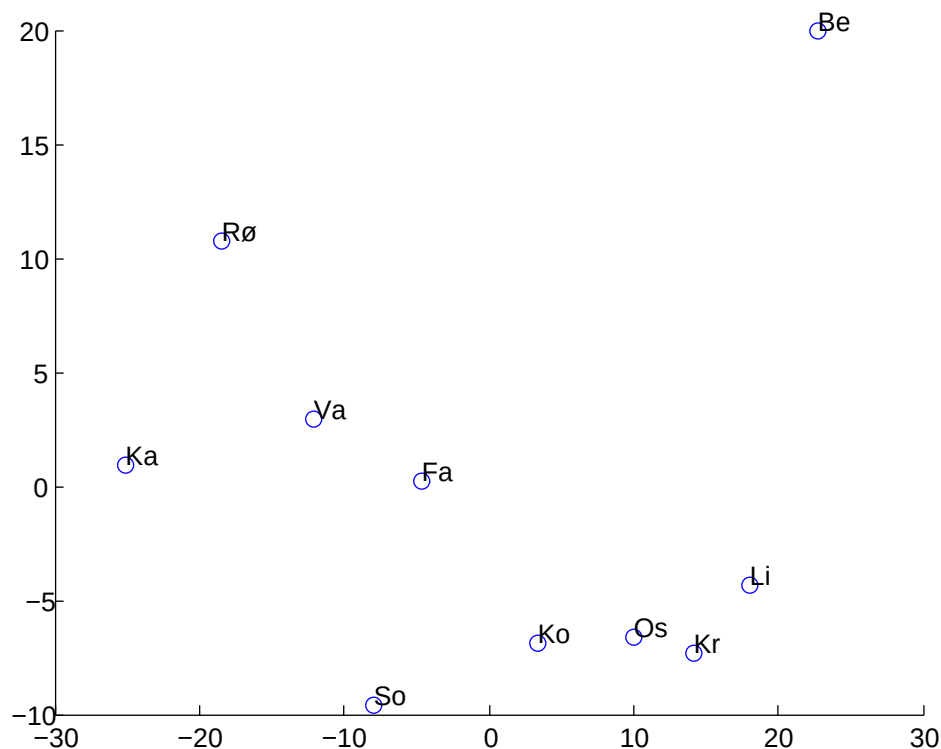


Figure 21.4: The first and the second principal components



In order to understand more of the variation, we can compute $a = 3$ principal components. This will explain 69% of the variance in our given data. It is now more difficult to plot the results, but one possibility is to make several two-dimensional plots complementing the one we have already seen. In Figure 21.4 we skip the second principal component, and plot the first versus the third. In Figure 21.5 we plot the second versus the third.

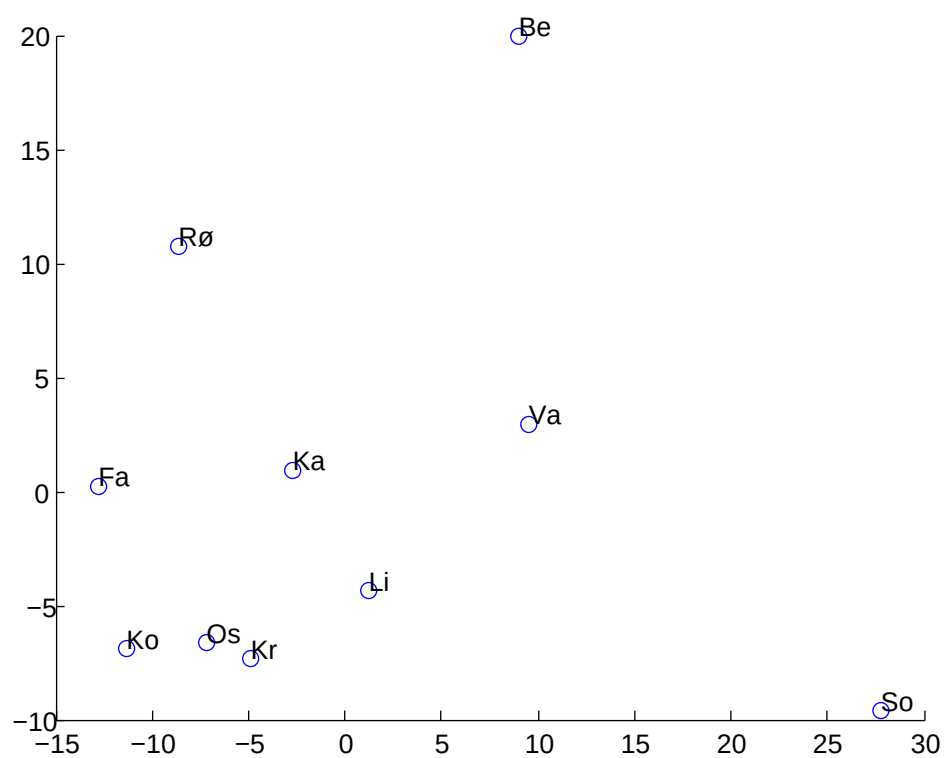
We note that Bergen now seems as separate from the others as Sortland, and that Oslo and Kristiansand are close together in all three plots. See for yourself what other information you can find. There is much more!

Comments

It is not clear how to interpret the results from PCA. Since the plots are given in terms of latent variables, we can seldom find meaningful explanations from them. PCA is a technique to describe rather than to explain data.

In data mining, there are very many techniques with different structures, different pros and cons. PCA is a relatively simple procedure, both theoretically and in

Figure 21.5: Second and third



implementation. Even so, it has had a profound impact in applications, for instance in the part of chemistry known as chemometrics.

As a final warning, if you look at meteorological data yourself, you will find that many sites will lack one or more of the fields we have used. It is very common in real data to have incomplete records. In this example, I have chosen sites with full records. Depending on the nature of the data, and the requirements of the methods used, there are various ways to deal with incomplete data. We will not pursue this here.

Index

- Addition, **40**
- adjacency matrix, 225
- Adleman, Leonard, 299
- anti-symmetric, 30

- Bayes' theorem, 243
- Bernoulli, 198
- bijective, **80**
- binary, **42**
- binary logarithm, 48
- binary number, 61
- binomial coefficient, **89**
- binomial distribution, 201
- Boolean, **11**

- cardinality, 31
- Cartesian product, **33**
- casting, 47
- Catalan number, 92
- characteristic equation, 119
- characteristic polynomial, 119
- codomain, 79
- coefficient of determination, 281
- complement, **26**
- composition, **80**
- conditional probability, 242
- congruence, **52**
- conjunction, **12**, 107
- connected component, 216
- continuous, 149
- continuous probability distribution, 250
- continuous stochastic variable, 250
- contradiction, 36
- converge, 69

- correlation, 246
- covariance, 247
- covariance matrix, 331
- cumulative probability, 177
- cycle, 218

- decrypt, 173
- deduction, 16, **108**
- del, 152
- diagonal matrix, 121
- diagonalization, 122
- difference, **25**
- differentiable, 151
- directed graph, 226
- disjoint, **24**
- disjunction, **12**, 107
- diverge, 69
- domain, 79
- double, 62

- edge, 216
- eigenvalue, 118
- eigenvector, 117
- element, **21**
- empirical variance, 193
- empty set, 22
- encrypt, 173
- equivalence, **15**, 30
- equivalence class, 134
- Eratosthenes' sieve, 309
- estimator, 260
- Euclidean algorithm, 84
- Eulerian cycle, 222
- exclusive or, **15**

- exist unique, 115
- existence quantifier, 106
- expectation, 195, 250
- exponent, **40**, 62
- factorial, , 87
- false, **11**
- float, 62
- floating points, 22, **59**
- for all, 106
- function, 30, **79**
- Gaussian distribution, 255
- generating function, 321
- generating series, 320
- gradient vector, 152
- graph, 149, 216
- Hamiltonian cycle, 222
- harmonic number, 68
- Hessian, 154
- hexadecimal numbers, 56
- Hierholzer's algorithm, 224
- homogeneous coordinates, 134, 136
- hypothesis test, 264
- if, 17
- image, 80
- implication, **14**
- inclusion, **22**
- independent, 197
- independent stochastic variables, 246
- induction, 68, **110**, 111
- injective, **80**
- int, 44
- integer, 44
- integer types, 44
- integers, **22**, **39**
- intersection, **23**
- interval, **60**
- Julia set, 187
- latent variable, 330
- limit, **69**
- linear order, *see* total order
- loadings, 332
- logical operators, **11**
- long**, 45
- loop, 216
- mantissa, 62
- maximum, 67, 90
- mean, 193, 194
- measure space, 235
- method, 87
- minimum, 90
- modulo, **52**
- Morgan, de, **26**, 28
- Multiplication, **41**
- multiplicative property of probability, 243
- nabla, 152
- NaN, 62
- natural logarithm, 57
- negation, **13**
- nested, 18
- NIPALS, 332
- node, 216
- normal distribution, 255
- Not a Number, 62
- operator order, 13
- outcome, 235
- PageRank, 130
- partial derivative, 151
- partial order, 31
- Pascal's triangle, 207
- path, 216
- PCA, 327
- permutation, 172
- power method, 127
- predicate, 106
- predicate logic, 106

- principal component, 332
- principal component analysis, 327
- probability, 169
- product, 33
- projective line, 134
- projective space, 136
- proposition, 105
- propositional logic, 105

- quadratic equation, 59, 66
- quantile, 260
- quotient, 51

- random walk, 200
- rational number, 59
- real number, 60
- real numbers, 22
- reflexive, 30
- regular, 221
- relation, 29
- remainder, 51
- Rivest, Ron, 299
- root, 219
- RSA, 299

- saddle point, 156
- scaling, 135
- scientific notation, 61
- sequence, 68
- series, **72**
- set, **21**
- Shamir, Adi, 299
- short**, 45
- shortest path, 228
- sieve, 309
- sigma-notation, $\sum$, 72
- signed zero, 62
- simple graph, 216
- slope, 133
- standard deviation, 194, 196
- standardized data matrix, 331
- stationary point, 152

- stochastic variable, 195, 235
- subnormal numbers, 62
- surjective, **80**, 107
- symmetric, 30

- tautology, 16
- there exists, 106
- total, 31
- total order, **31**
- transitive, 30
- travelling salesman, 226
- tree, 218
- true, **11**
- truth table, **12**
- two's complement, 50, 56

- union, **24**
- unique, 115
- universal quantifier, 106

- variance, 193, 196, 250
- Venn diagram, **22**

- weighted graph, 228

Appendix A

Solutions to exercises

A.1 Hints

2.6.2

(d) Use Example 2.5.3.

9.5.1

(d) Don't expand the characteristic polynomial.

12.5.6

(a) In ordinary text, whitespace characters are characterized by having ascii values ≤ 32 .

14.5.1 The simplest way to draw a circle is to choose a suitable parameter range, e.g. $t = 0 : 2 * \pi / n : 2 * \pi$ and compute $x = r * \cos(t)$, $y = r * \sin(t)$.

14.5.2

(c) The total probability is 1.

14.5.12 To compute $\binom{n}{k}$ you need an array of type `long[n-k+1][k+1]` in Java, or similarly in MATLAB using the type `int64`.

A.2 Answers

9.5.1

(a) $\lambda = 0$, $[-1, 1]$ and $\lambda = 4$, $[1, 1]$.

11.4.1 We give the general form of the gradient, and the value in the given point.

(a) $\nabla f = [4x^3 - 2y \cos(xy), -2x \cos(xy)]$, $\nabla f(2, \pi) = [32 - 2\pi, -4]$

(b) $\nabla f = [e^{x-y}, -e^{x-y}]$, $\nabla f(3, 3) = [1, -1]$

(c) $\nabla f = [2xy^2, 2x^2y]$, $\nabla f(1, -1) = [2, -2]$

11.4.2 We give the linear expression expressed around the point.

(a) $1 + 2(x - 1) - 2(y - 1)$

(b) $(x - 1) - (y + 1)$

(c) $2(x - 3) + (y - 2) + 8$ which is exactly equal to h .

11.4.3 We give the second order expression expressed around the point.

(a) $\ln 5 + \frac{1}{5}(x - 2) + \frac{1}{5}(y - 3) - \frac{1}{50}(x - 2)^2 - \frac{1}{25}(x - 2)(y - 3) - \frac{1}{50}(y - 3)^2$

(b) $-72 + 172(x + 2) + 80(y + 1) - 166(x + 2)^2 - 184(x + 2)(y + 1) - 8(y + 1)^2$

(c) $5(x - 1) + 5(y + 1) + 2(x - 1)^2 - (x - 1)(y + 1) - 3(y + 1)^2$

11.4.4 We give the points and their classification

(a) No stationary points, $\partial f / \partial x$ is always positive.

(b) Four stationary points $(\pm 2, \pm 3)$. $(2, 3)$ is a minimum, $(-2, -3)$ is a maximum, $(-2, 3)$ and $(2, -3)$ are saddle points.

(c) $(0, 0)$, cannot be classified by our methods since the determinant of the Hessian is zero.

11.4.5 Only the equation of the line is given.

(a) $y = 1.833 - 0.833x$.

(b) $y = 2$ (the horizontal line at height 2)

(c) $y = x + 2.5$

12.5.1 *die*

12.5.5 mathematics is more fun than programming

12.5.6

(a) 18280 spaces in Macbeth.

12.5.7

(b) pizza

(c) Five letters is too few to do meaningful statistical analysis. In addition, since the least common letter in Macbeth appears twice, we actually get the highest χ^2 -value for the correct decryption.

14.5.2

(a) $E(X) = 4$ and $\text{Var}(X) = 1.79^2$

(b) $P(X = 1) = 0.0576$, $P(X = 4) = 0.2182$

(c) 0.9309

14.5.3

(a) $X \sim \text{bin}(n, p) = \text{bin}\left(14, \frac{1}{6}\right)$.

(b) $E(X) = 2.33$, $\text{Var}(X) = 1.39^2$

(c) 0.125

(d) 0.704

14.5.4

(a) $E(X) = 1.5$, $\text{Var}(X) = 0.73^2$.

(b) $P(X = x)$ for $x = 0, 1, 2, 3$ is $\frac{5}{70}$, $\frac{30}{70}$, $\frac{30}{70}$, $\frac{5}{70}$.

14.5.5

(a) $X \sim \text{hypergeom}(N, M, n) = \text{hypergeom}(105, 42, 8)$

(b) $E(X) = 3.2$

(c) $P(X = 4) = 0.239$

(d) $P(X \leq 7) = 0.9996$

14.5.6 $E(X) = 3.55$, $\text{Var}(x) = 2.52^2$

14.5.7

(a) 0.234

(b) 0.736

14.5.8

(a) 0.7

(b) 0.00745

15.5.1

(a) No (unless the empty graph is considered a tree).

(b) No.

(c) Yes.

(d) Yes

15.5.2

(a) Complete graph on four vertices.

(b) A cycle with six vertices.

(c) A graph with two components, both of them triangles.

15.5.3 Twice the number of edges, or twice the number of vertices minus two.

15.5.4 Top left: $ABCDEBFCEFA$. The others are not Eulerian.

15.5.5 Top left: $ABGDFECA$. Top right: $ADBEGHFCA$ Bottom left: $ABIJHGFDECA$

16.6.1 $E(X) = 3.55$, $\text{Var}(X) = 2.52^2$

16.6.3

(a) $\rho(X, Y) = -0.0224$.

16.6.4

(a) 0

16.6.5

(a) $P(A) = 0.2$, $P(B) = 0.6$, $P(C) = 0.575$

(b) $P(A \cap B) = 0.12$, $P(A \cup C) = 0.625$ and $P(B \cap C) = 0.325$.

(c) $P(A|B) = 0.2$, $P(B|A) = 0.6$ and $P(A|C) = 0.261$.

(d) 0.145

16.6.6

(a) 3

(b) $3/4$

(c) 0.194^2

17.6.2 $E(X) = 0$ and $\text{Var}(X) = 1$.

17.6.3

(a) 0.8849

(b) 0.3821

(c) 0.5125

17.6.4

(a) 0.7257

(b) 0.4404

(c) 0.3132

17.6.5 We use the formula $\bar{X} \pm u_{\alpha/2} \frac{\sigma}{\sqrt{n}}$.

(a) 108.67 ± 3.4

(b) 178 ± 2.01

(c) -38 ± 2.22

17.6.6 62

17.6.7

(a) μ is the true speed of the vehicle, $\mu_0 = 80$ is the speed limit. $H_0 : \mu = 80$, $H_1 : \mu > 80$

(b) Convicting an innocent; claiming that someone who is really not speeding is speeding.

(c) Reject H_0 if $X > k = \mu_0 + u_\alpha \cdot \sigma = 91.63$.

(d) We can not reject H_0 , there is insufficient evidence to claim that the vehicle is speeding.

(e) Variance is divided by three, so the new k will be $k = \mu_0 + u_\alpha \cdot \sigma / \sqrt{3} = 86.7$. We will now reject H_0 , there is sufficient evidence to believe that vehicle is speeding.

17.6.8

(a) Reject H_0 .

(b) Do not reject H_0 .

(c) Reject H_0 .

(d) Reject H_0 .

17.6.9 Part c. In part a) we are 95% certain that H_1 is true. Then obviously we are also 90% certain.

17.6.10

(a) Since $k = -36.8$ we will reject H_0 .

(b) Since $|\bar{X} - \mu_0| = 2$ and $u_{\alpha/2} \frac{\sigma}{\sqrt{n}} = 2.15$, we will not reject H_0 .

17.6.11 Since the α is split into two parts in b), but the whole α is on the left in a), there is a small region where we will reject H_0 in the one-sided test, but not in the two-sided.

17.6.13

(a) $\hat{\mu} = 6.2$, $S = 0.237$.

(b) $t_{0.025, 5} = 2.571$, $6.2 \pm 0.248 = [5.95, 6.45]$.

(c) $6.2 \pm 0.19 = [6.01, 6.39]$. More is known.

17.6.14

(a) $Y \sim N(100, 7 - 75^2)$

(b) 0.90

(c) 0.80

17.6.15

(a) $N(150, 12.25^2)$.

(b) 0.11.

(c) 0.66.

17.6.16 $0.3115 \pm 0.064 = [0.247, 0.376]$

17.6.17

(a) 1

(b) 0.85

(c) 1

17.6.18

(a) $\text{hypergeom}(1000000, 550000, 100)$

(b) $\text{bin}(100, 0.55)$

(c) $N(55, 4.97^2)$

(d) $N(0.5, 0.7^2)$

18.3.1 In each part, we give y_{lin} , the equation for the line, y_{quad} , the equation for the second degree curve, r_{lin}^2 and r_{quad}^2 , the respective coefficients of determination.

(a) $y_{lin} = x + 2.5$, $y_{quad} = x^2 - x + 3$, $r_{lin}^2 = 0.4$, $r_{quad}^2 = 0.6$.

(b) $y_{lin} = -0.11x + 1.82$, $y_{quad} = 1.06x^2 - 11x - 0.31$, $r_{lin}^2 = 0.0076$, $r_{quad}^2 = 0.999$.

(c) $y_{lin} = 1.42x + 0.55$, $y_{quad} = -0.14x^2 + 2.68x - 1.77$, $r_{lin}^2 = 0.955$, $r_{quad}^2 = 0.972$.

18.3.2 0.9975

18.3.3

(a) $y = 10.85 - 3.3 \cos\left(\frac{2\pi t}{24}\right) - 1.78 \sin\left(\frac{2\pi t}{24}\right)$.

(b) $r^2 = 0.87$.

(c) Maximum at 14.6° at $t = 13.9$ (a few minutes before two o'clock), minimum 7.1° at $t = 1.9$ (a few minutes before two at night). These are approximate values.

(d) Cosine: because the temperature starts below the average value.

Sine: because the temperature decreases initially (compute y' to see this).

18.3.4 Don't reject H_0 .

18.3.5

(a)

H_0 : The observations follow the expected distribution

The alternative hypothesis is

H_1 : The observations don't follow the expected distribution

Species 1: 33.25

Species 2: 13.3

(b) Species 3: 19.95

Species 4: 39.9

Species 5: 6.65

Species 6: 19.95

(c) Don't reject H_0 .

18.3.7

(a) 51.2, 38.4, 9.6, 0.8

(b) 1.4

(c) There is not enough evidence to say that the model is false.

19.5.1

(a) 1000

(b) 7

(c) 1

19.5.2

(a) 3

(b) 43074

19.5.3 12503, 12511, 12517, 12527, 12539, 12541, 12547, 12553, 12569, 12577, 12583, 12589

19.5.4 Short answers only.

(a) Since $n = 5 \cdot 7$, $\phi = 4 \cdot 6 = 24$, which has no common divisor with $e = 5$.

(b) $d = 5$ as well.

(c) Coded: 18, 14, 20, 19, 7. Encrypted: 23, 14, 20, 24, 7

(d) 'back'.

19.5.6

24509 24517 24527 24533 24547 24551 24571 24593

19.5.7 Short answers only.

(a) Coded: $03040003 = 3040003$. Encrypted: 33398349

(b) The largest integer less than $\sqrt{n}$ is 10766. There are 1311 primes in the list, from 2 to 10753.

(c) $p = 9601$ and $q = 12073$.

(d) $\phi = 115891200$ has no common factor with $e = 133$. $d = 27883597$.

(e) The decrypted message is

99999902|14050504|4990818|99180417|21040399|199904|8060719

This translates as “Coffee is served at eight”.

20.3.1

(a) $|x| < 1$

(b) $|x| < 1/2$

(c) $|x| < 3$

20.3.2 The function expressions are given.

(a)

$$4/(1-x)$$

(b)

$$\frac{3}{(1-x)^2} - \frac{1}{1-x}$$

(c)

$$\frac{4}{(1-x)^2} + \frac{6}{1-x}$$

(d)

$$\frac{3}{(1-x)^2} - \frac{7}{1-x}$$

$$\mathbf{20.3.4} \quad \frac{2}{(1-2x)^2} = \sum_{m=0}^{\infty} (m+1)2^{m+1}x^m.$$

$$\mathbf{20.3.5} \quad |x| < 1/2$$

20.3.6

$$\sum_{n=0}^{\infty} 3^n x^n = \frac{1}{1-3x}$$

20.3.7 Short answers only.

(a)

$$\frac{1}{(1-x)^3} = 1 + 3x + 6x^2 + 10x^3 + 15x^4 + \dots$$

(b)

$$\begin{aligned} A + B + C &= 2 \\ 3A + 2B + C &= 3 \\ 6A + 3B + C &= 7 \end{aligned}$$

(c) $A = 3, B = -5, C = 4.$

A.3 Solutions

1.1.5 The or-statement is “It is Thursday or the sun is shining”.

The statements in the first row: “It isn’t Thursday” and “The sun isn’t shining”. It is false that “It is Thursday or the sun is shining”.

The statements in the second row: “It isn’t Thursday” and “The sun is shining”. It is true that “It is Thursday or the sun is shining”.

The statements in the third row: “It is Thursday” and “The sun isn’t shining”. It is true that “It is Thursday or the sun is shining”.

The statements in the fourth row: “It is Thursday” and “The sun is shining”. It is true that “It is Thursday or the sun is shining”.

1.1.8 The tables can be found as follows:

x	y	z	$x \vee y$	$(x \vee y) \vee z$	$y \vee z$	$x \vee (y \vee z)$
0	0	0	0	0	0	0
0	0	1	0	1	1	1
0	1	0	1	1	1	1
0	1	1	1	1	1	1
1	0	0	1	1	0	1
1	0	1	1	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

They are equal because both statements can be thought of as saying that at least one of the three is true. A simple example is assigning to the three variables the truth-values of “It is cold”, “It is raining” and “It is windy”, respectively. Then the whole statement is “It is cold, rainy or windy” no matter how the parentheses are placed.

1.2.2 The truth table is found as follows.

x	y	$\neg x$	$\neg x \vee y$
0	0	1	1
0	1	1	1
1	0	0	0
1	1	0	1

1.2.5 The truth table is found as follows.

x	y	$x \rightarrow y$	$y \rightarrow x$	$(x \rightarrow y) \wedge (y \rightarrow x)$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	1	1	1

1.2.8 The truth table is found as follows.

x	y	$x \leftrightarrow y$	$\neg(x \leftrightarrow y)$
0	0	1	0
0	1	0	1
1	0	0	1
1	1	1	0

1.2.11 We fill in the truth tables from left to right.

(a)

x	y	$y \vee x$	$x \leftrightarrow (y \vee x)$
0	0	0	1
0	1	1	0
1	0	1	1
1	1	1	1

(b)

x	y	z	$y \rightarrow z$	$x \leftrightarrow (y \rightarrow z)$
0	0	0	1	0
0	0	1	1	0
0	1	0	0	1
0	1	1	1	0
1	0	0	1	1
1	0	1	1	1
1	1	0	0	0
1	1	1	1	1

(c)

x	y	$\neg x$	$\neg y$	$\neg x \rightarrow \neg y$	$y \rightarrow x$	$(\neg x \rightarrow \neg y) \leftrightarrow (y \rightarrow x)$
0	0	1	1	1	1	1
0	1	1	0	0	0	1
1	0	0	1	1	1	1
1	1	0	0	1	1	1

1.3.3 Since 13 is both larger than 10 and smaller than 15, the middle block is executed. For the other two blocks, only one of the parts is true, so the code isn't executed. If we use 10 instead, nothing is executed. This is because $10 < 10$ is false.

1.4.1 We fill in the truth tables from left to right.

(a)

x	y	z	$x \wedge y$	$(x \wedge y) \wedge z$	$y \wedge z$	$x \wedge (y \wedge z)$
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	1	0
1	0	0	0	0	0	0
1	0	1	0	0	0	0
1	1	0	1	0	0	0
1	1	1	1	1	1	1

(b)

x	y	z	$x \wedge y$	$(x \wedge y) \vee z$	$y \vee z$	$x \wedge (y \vee z)$
0	0	0	0	0	0	0
0	0	1	0	1	1	0
0	1	0	0	0	1	0
0	1	1	0	1	1	0
1	0	0	0	0	0	0
1	0	1	0	1	1	1
1	1	0	1	1	1	1
1	1	1	1	1	1	1

(c)

x	y	z	$x \vee y$	$(x \vee y) \wedge z$	$y \wedge z$	$x \vee (y \wedge z)$
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	1	0	0	0
0	1	1	1	1	1	1
1	0	0	1	0	0	1
1	0	1	1	1	0	1
1	1	0	1	0	0	1
1	1	1	1	1	1	1

1.4.2 We fill in the truth tables from left to right.

(a)

x	y	$x \rightarrow y$	$(x \rightarrow y) \wedge x$	$((x \rightarrow y) \wedge x) \rightarrow y$
0	0	1	0	1
0	1	1	0	1
1	0	0	0	1
1	1	1	1	1

(b)

x	y	$\neg x$	$\neg y$	$x \rightarrow y$	$(x \rightarrow y) \wedge \neg y$	$((x \rightarrow y) \wedge \neg y) \rightarrow \neg x$
0	0	1	1	1	1	1
0	1	1	0	1	0	1
1	0	0	1	0	0	1
1	1	0	0	1	0	1

(c)

x	y	z	$x \rightarrow y$	$y \rightarrow z$	$(x \rightarrow y) \wedge (y \rightarrow z)$	$x \rightarrow z$	$((x \rightarrow y) \wedge (y \rightarrow z)) \rightarrow (x \rightarrow z)$
0	0	0	1	1	1	1	1
0	0	1	1	1	1	1	1
0	1	0	1	0	0	1	1
0	1	1	1	1	1	1	1
1	0	0	0	1	0	0	1
1	0	1	0	1	0	1	1
1	1	0	1	0	0	0	1
1	1	1	1	1	1	1	1

1.4.3 In each case, we write the whole code. Note that the first two parts simply have very minor changes, whereas the third part needs an additional nested layer.

(a) $\vee$ can be used directly.

```
boolean[] truthValues = {false, true};
for (boolean x : truthValues){
    for (boolean y : truthValues){
        System.out.printf("%b or %b is %b\n", x, y, x || y);
    }
}
```

(b) Truth table for implication, $\rightarrow$, using $x \rightarrow y = \neg x \vee y$:

```
boolean[] truthValues = {false, true};
for (boolean x : truthValues){
    for (boolean y : truthValues){
        System.out.printf("%b implies %b is %b\n", x, y, !x || y);
    }
}
```

(c) We use that $z \rightarrow (x \vee y) = \neg z \vee x \vee y$

```
boolean[] truthValues = {false, true};
for (boolean x : truthValues){
    for (boolean y : truthValues){
        for (boolean z : truthValues){
            System.out.printf("%b implies %b or %b is %b\n",
                z, x, y, !z || x || y);
        }
    }
}
```

2.1.4 The intersection is

$$A \cap B = \{3, 9\}$$

2.1.7 The union is

$$A \cup B = \{-4, 3, 9, 13, 14, 15\}$$

2.1.12 The two differences are

$$A \setminus B = \{-4\} \text{ and } B \setminus A = \{13, 14, 15\}$$

2.2.3 We prove that $\overline{A \cap B} \subset \overline{A} \cup \overline{B}$ first. So assume that $x \in \overline{A \cap B}$. Since $A \cap B$ is the set of elements that are in both A and B , x must be either outside A or outside

B . That is, $x \in \overline{A}$ or $x \in \overline{B}$. By definition, we then have $x \in \overline{A \cap B}$.

We then prove that $\overline{A \cap B} \subset \overline{A} \cup \overline{B}$. Assume $x \in \overline{A \cap B}$. Then x is outside A or outside B , so x can't be in both A and B . Since $A \cap B$ is the set of elements that are in both A and B , x must be outside this set. So $x \in \overline{A} \cup \overline{B}$.

We have proved that all the elements in $\overline{A \cap B}$ are in $\overline{A} \cup \overline{B}$, and that all the elements in $\overline{A} \cup \overline{B}$ are in $\overline{A \cap B}$. Therefore the two sets are equal.

In the Venn diagram in Figure 2.8, the complement of A is coloured blue, the complement of B is coloured yellow. The intersection then becomes green. So $\overline{A \cap B}$ is everything that is coloured, either because it is outside A (blue), outside B (yellow) or outside both (green). But this is the same as the complement of $A \cap B$, since $A \cap B$ is the only part that isn't coloured.

2.3.2 We produce the truth table:

x	y	$\neg x$	$\neg y$	$x \wedge y$	$\neg(x \wedge y)$	$(\neg x) \vee (\neg y)$
0	0	1	1	0	1	1
0	1	1	0	0	1	1
1	0	0	1	0	1	1
1	1	0	0	1	0	0

Since the last two columns are equal, the two expressions are equivalent.

2.4.5 For the properties of this exercise, we can distinguish between odd and even numbers. Let us check the three conditions.

- i) $x \sim x$ since x is even or odd.
- ii) If $x \sim y$, then either both are even or both are odd. But this means the same as $y \sim x$.
- iii) If $x \sim y$ and $y \sim z$, then if y is even, both x and z have to be even, so $x \sim z$, or all three must be odd, so $x \sim z$ again.

Note that these properties are so obvious that it is almost difficult to state them.

2.4.8 We check the four properties.

- i) $x \leq x$ is always true for real numbers.
- iii) If $x \leq y$ and $y \leq z$ then $x \leq y \leq z$, so x is related to z .

- iv) If $x \leq y$ and $y \leq x$ then $x \leq y \leq x$. Since the first and third parts are equal, the second must also be equal to them. So $x = y$.
- v) Given x and y , we can have $x \leq y$. If this holds, we are done. If it doesn't, then $x > y$, so in particular $y \leq x$.

2.6.1 We check the three properties, and that the extra condition for being total is wrong.

- i) A set is contained in itself, so $X \subset X$ is always true.
- iii) If $X \subset Y$ and $Y \subset Z$, then all elements of X are in Y and all elements in Y are in Z . But then all elements in X are in Z , so $X \subset Z$.
- iv) If $X \subset Y$ and $Y \subset X$, then all elements in X are in Y , and all elements in Y are in X . Thus the two sets have exactly the same elements, and are equal.
- v) Let x and y be different elements in A , and consider the two sets $X = \{x\}$ and $Y = \{y\}$. Then X is not a subset of Y , and Y is not a subset of X .

2.6.2

(a) We use elements from the start of the alphabet. If $n = 2$ the elements of P are

$$\emptyset, \{a\}, \{b\}, \{a, b\} = A$$

If $n = 3$ the elements of P are

$$\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\} = A$$

(b) 100011

(c) $\{c, d, f\}$

(d) For each element of P , there is a subset of A , and thus a sequence of zeroes and ones. This can be reversed, so that the elements of P correspond exactly to the lists of length n of zeroes and ones. In Example 2.5.3, we saw that there are 2^n such lists, therefore $|P| = 2^n$.

2.6.3 If $A \cap B \cap C \neq \emptyset$, the Venn diagram looks like Figure A.1.

If $A \cap B \cap C = \emptyset$, the Venn diagram looks like Figure A.2.

2.6.4 The Venn diagram is in Figure A.3. To prove the statement, we need to argue that every element $x \in A \cap C$ is also in $B \cap C$. x is in C because $x \in A \cap C$. x is in A for the same reason, and since $A \subset B$, every element in A is also in B . Thus x is in both B and C , so $x \in B \cap C$.

Figure A.1: The triple intersection is nonempty

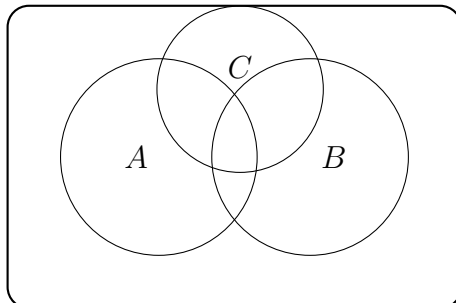
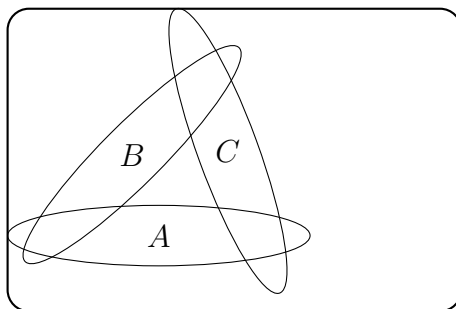


Figure A.2: The triple intersection is empty



2.6.5

(a) For each element of A there is one pair for each element of B . So there are n choices for an element in A , m choices for an element in B , and altogether nm elements in $A \times B$.

(b) Any element in A' is also an element of A , and every element of B' is also an element of B . Therefore each pair $(a', b') \in A' \times B'$ is also a pair in $A \times B$. So $A' \times B' \subset A \times B$.

(c) The pairs in $\mathbb{Z} \times \mathbb{Z}$ are the pairs (n, m) of points where both coordinates are integers. As a subset of $\mathbb{R}^2$, we can think about this as the “integer grid”.

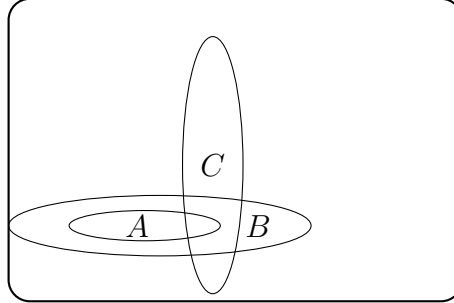
(d) Formally,

$$A \times (B \times C) = \text{the set of pairs } (a, \text{pair of } (b, c)).$$

But this is the same as a triple (a, b, c) . Similarly,

$$(A \times B) \times C = \text{the set of pairs } (\text{pair of } (a, b), c).$$

which again is the same as the set of triples.

Figure A.3: $A \cap C \subset B \cap C$ **2.6.6**

(a) For $x \in A \rightarrow x \in B$ to be true, we must either have that $x \in A$ is false, meaning that $x \in \bar{A}$, or both $x \in A$ and $x \in B$ must be true. In the latter case, $x \in A \cap B$. Thus x is in $\bar{A}$ or in $A \cap B$, so $x \in (A \cap B) \cup \bar{A}$.

(b) The only possibility for $x \in A \rightarrow x \in B$ to be false, is that $x \in A \cap \bar{B}$ ($x \in A$ true and $x \in B$ false). So $x \in A \rightarrow x \in B$ is a tautology if $A \cap \bar{B} = \emptyset$. But this means that $A \subset B$.

(c) If $x \in A \leftrightarrow x \in B$ is a tautology, using Exercise 1.2.5, is the same as saying that $x \in A \rightarrow x \in B$ and $x \in B \rightarrow x \in A$ are both tautologies. Therefore, $A \subset B$ and $B \subset A$, and $A = B$.

2.6.7 We answer this exercise by giving the entire programme. Note that the structure for finding the boolean test variables is the same as in the text, it is only the way they are used that is different.

```
//Initialization
int [] universe = {0,1,2,3,4,5,6,7,8,9,10};
int [] setA = {0,1,2,3,4,5};
int [] setB = {2,3,5,7};

//Intersection: we need both in A and in B
for (int number : universe){
    boolean testA = false;
    boolean testB = false;
    for (int numberA : setA){
        if (number == numberA){
            testA = true;

```

```

    }
}
for ( int numberB : setB){
    if (number == numberB){
        testB = true;
    }
}
if (testA && testB){
    System.out.printf("%d is in the union\n", number);
}
}

//Complement of A:
for (int number : universe){
    boolean testA = false;
    for ( int numberA : setA){
        if (number == numberA){
            testA = true;
        }
    }
    if (!testA){
        System.out.printf("%d is in the complement of A\n", number);
    }
}

```

2.6.8 We answer this exercise by giving the entire programme, with a note to what is added to the text of the exercise.

```

//Initialization from the text
char[] setA = {'a','b','c','d','e','f'};
char[][] relation ={{'a','a'},{'b','b'},{'c','c'},{'d','d'},
    {'e','e'},{'f','f'},{'a','d'},{'a','f'},{'c','d'},
    {'c','e'},{'c','f'},{'d','f'}};
//reflexivity from the text
int reflexivePairs = 0;
for (char c : setA){
    char[] cPair = new char[2];
    cPair[0] = c;
    cPair[1] = c;
    for (char[] pair : relation){
        if (pair[0] == cPair[0] && pair[1]==cPair[1]){

```

```

        reflexivePairs++;
    }
}
}
if(reflexivePairs == setA.length){
    System.out.println("The relation is reflexive");
}
//reflexivity ends

//antiSymmetry from the text
boolean isAntiSymmetric = true;
for (char[] pair : relation){
    for (char[] newPair : relation){
        if (newPair[0] == pair[1] && newPair[1] == pair[0] &&
            pair[0] != pair[1]){
            isAntiSymmetric = false;
        }
    }
}
if (isAntiSymmetric){
    System.out.println("The relation is antisymmetric");
}
//antiSymmetry ends

//transitivity - new stuff
boolean isTransitive = true;
for (char[] pair1 : relation ){
    for (char[] pair2 : relation){
        if (pair1[1]==pair2[0]){
            //check if pair1[0],pair2[1] is in the relation
            char[] testPair = new char[2];
            testPair[0] = pair1[0];
            testPair[1] = pair2[1];
            boolean test = false;
            for (char[] pair : relation){
                if(testPair[0] == pair[0] && testPair[1]==pair[1]){
                    test = true;
                }
            }
            if (!test){
                isTransitive = false;
            }
        }
    }
}

```

```

        }
    }
}
if (isTransitive){
    System.out.println("The relation is transitive");
}
//transitivity ends

//if partial order:
if(isAntiSymmetric && isTransitive && reflexivePairs == setA.length){
    System.out.println("All three conditions are fulfilled.");
    System.out.println("The relation is a partial order.");
}

```

2.6.9 We can check the first three points by finding truth tables. Or we can reason directly about the statements. It is possible to answer all three points in either way, so let us answer a) and b) directly, and make a truth table for c). For the set-theoretic questions, we use the set A to correspond to $x \in A$ or the boolean x , and similarly B for y .

(a) There is an “or” statement in the middle, so it is true if the left part is true. The left part is $x \wedge y$, which is true if both are true. Therefore there is at least one combination that makes the expression true, and it is not a contradiction.

(b) There is an “and”-statement in the middle, so both the right and the left must be true if the whole expression is true. If the left part is true, x must be true. If the right part is true, $\neg x$ must be true, so x must be false. Therefore there isn’t any possible combination leading to the value **true**, and the expression is a contradiction.

(c) Here is a truth table:

x	y	$\neg x$	$\neg y$	$x \rightarrow y$	$(x \rightarrow y) \wedge \neg y$	$((x \rightarrow y) \wedge \neg y) \rightarrow \neg x$	$\neg(((x \rightarrow y) \wedge \neg y) \rightarrow \neg x)$
0	0	1	1	1	1	1	0
0	1	1	0	1	0	1	0
1	0	0	1	0	0	1	0
1	1	0	0	1	0	1	0

Since the truth table show only 0, the expression is a contradiction.

(d) If A corresponds to x , $\overline{A}$ corresponds to $\neg x$. $\wedge$ translates to $\cap$, so the set corresponding to $x \wedge \neg x$ is

$$A \cap \overline{A} = \{ \text{elements in } A \text{ and outside } A \} = \emptyset$$

since no element can be both in and outside a given set.

(e) A contradiction is always false. The empty set has no elements, so $x \in \emptyset$ is always false. Thus the empty set corresponds to contradictions.

(f) We simply write the set

$$(A \cap B) \cup (\overline{A} \cap \overline{B})$$

In particular, this contains the intersection of A and B , and so can't be the empty set in general.

(g) We write the set as

$$(A \cap B) \cap (\overline{A} \cap \overline{B})$$

This set is contained in $A \cap \overline{A}$, which we saw in d) is the empty set. Thus this set is also empty.

2.6.10 By combining the code snippets in each part, you will get a complete programme for this problem.

(a) The length must be $2^3 = 8$. If there were 12 variables, we would need $2^{12} = 4096$. This is because we have two choices for each variable.

```
boolean [] truthTable = new boolean [8];
int index = 0; //keep track of index in truthTable
```

```
(b) boolean [] truthValues = {false , true};
for (boolean x : truthValues){
    for (boolean y : truthValues){
        for (boolean z : truthValues){
            truthTable[index] = x && (!y || z);
            index++;
        }
    }
}
```

```
(c) int numberOfTrue = 0;
for (boolean tableValue : truthTable){
    if (tableValue){
        numberOfTrue ++;
    }
}
System.out.println("Number of true");
System.out.println(numberOfTrue);
System.out.println("Number of false");
System.out.println(8-numberOfTrue);
```

```

(d) if (numberOfTrue == 0){
    System.out.println("The expression is a contradiction");
}
if (numberOfTrue == 8){
    System.out.println("The expression is a tautology");
}

```

(e) To check for $(x \wedge z) \wedge (y \wedge \neg z)$, we only need to change the innermost line in the loop to

```
truthTable[index] =(x && z) && (y && !z);
```

Similarly, for $(x \vee z) \vee (y \vee \neg z)$, we need

```
truthTable[index] =(x || z) || (y || !z);
```

In the first case, we get a contradiction, in the second a tautology.

3.0.1 The positive integers are

$$\{n \in \mathbb{Z} \mid n > 0\}$$

The negative integers are

$$\{n \in \mathbb{Z} \mid n < 0\}$$

The non-positive integers are

$$\{n \in \mathbb{Z} \mid n \leq 0\}$$

3.1.1 Multiply the single digit number with the ones digit. Thinking of the answer as tens plus ones, record the ones in the ones-place of the answer, and carry the tens. For each step after this one, move one place to the left in the multiple digit number and in the answer. At each step, multiply the digit with the current digit in the multiple digit number, and add whatever's been carried. Record the ones in the current digit of the answer, and carry the tens. Repeat until the end of the multiple digit number, possibly taking into account a final carried digit.

3.2.1

$$110110_2 = 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 32 + 16 + 4 + 2 = 54.$$

3.2.2

+	0	1	2
0	0	1	2
1	1	2	10
2	2	10	11

and

·	0	1	2
0	0	0	0
1	0	1	2
2	0	2	11

3.2.4

$$\begin{array}{cccccccc}
 & 1 & & 1 & & 1 & & 1 & & 1 & & & & & 1 \\
 & & 1 & & 1 & & 0 & & 1 & & 1 & & 0 & & 0 & & 1 \\
 + & & & & & & 1 & & 0 & & 1 & & 1 & & 0 & & 1 \\
 \hline
 & 1 & & 0 & & 0 & & 0 & & 0 & & 0 & & 1 & & 1 & & 0
 \end{array}$$

3.2.6 The addition, if we forget the row with zeroes only, is $1101 + 110100$. Taking note of what is carried gives

$$\begin{array}{ccccccc}
 & 1 & & 1 & & 1 & & 1 \\
 & & & & & 1 & & 1 & & 0 & & 1 \\
 + & 1 & & 1 & & 0 & & 1 & & 0 & & 0 \\
 \hline
 & 1 & & 0 & & 0 & & 0 & & 0 & & 0 & & 1
 \end{array}$$

3.2.7

$$\begin{array}{cccccccc}
 1 & 0 & 1 & 1 & \cdot & 1 & 0 & 1 & 0 \\
 \hline
 & & & & & 1 & 0 & 1 & 0 \\
 & & & & & 1 & 0 & 1 & 0 \\
 & & & & 0 & 0 & 0 & 0 & 0 \\
 & & 1 & 0 & 1 & 0 & & & \\
 \hline
 & 1 & 1 & 0 & 1 & 1 & 1 & 1 & 0
 \end{array}$$

3.2.10

$$\begin{aligned}
 3027 &= 1513 \cdot 2 + 1 \\
 1513 &= 756 \cdot 2 + 1 \\
 756 &= 378 \cdot 2 + 0 \\
 378 &= 189 \cdot 2 + 0 \\
 189 &= 94 \cdot 2 + 1 \\
 94 &= 47 \cdot 2 + 0 \\
 47 &= 23 \cdot 2 + 1 \\
 23 &= 11 \cdot 2 + 1 \\
 11 &= 5 \cdot 2 + 1 \\
 5 &= 2 \cdot 2 + 1 \\
 2 &= 1 \cdot 2 + 0 \\
 1 &= 0 \cdot 2 + 1
 \end{aligned}$$

Thus

$$3027 = 101111010011_2$$

3.3.2 Here is the list of multiples of two:

$$1, 2, 4, 8, 16, 32, 64, 128$$

We stop since $128 > 71$. There were seven multiplications, so our approximation is that $\log_2(71) \approx 7$.

3.3.3 $-4 = 1100$ and $3 = 0011$. Then $-4 + 3$

$$\begin{array}{r} 1 \ 1 \ 0 \ 0 \\ + \ 0 \ 0 \ 1 \ 1 \\ \hline 1 \ 1 \ 1 \ 1 \end{array}$$

Note that nothing was carried.

3.4.1 We run the two possibilities we have (using $\pm 7/3$), first with negative numerator:

```
System.out.printf("-7/3 = %d\n", (-7)/3); //quotient
System.out.printf("-7 %% 3 = %d\n", (-7) % 3); //remainder
returns
```

```
-7/3 = -2
-7 % 3 = -1
```

Then with negative denominator:

```
System.out.printf("7/-3 = %d\n", 7/(-3)); //quotient
System.out.printf("7 %% -3 = %d\n", 7 % (-3)); //remainder
returns
```

```
7/-3 = -2
7 % -3 = 1
```

In both cases, the result of dividing yields the quotient rounded towards 0, and the remainder is given such that

$$\text{numerator} = \text{quotient} \cdot \text{denominator} + \text{remainder}$$

3.4.4

- (a) The remainders are 1 and 3, so these numbers are not congruent modulo 6.
- (b) The remainders are 4 in both cases, so these are congruent modulo 6. Note that Java would produce different values with the `%`-operation.
- (c) The remainders are 3 in both cases, so they are congruent modulo 6.
- (d) Since the positive numbers are congruent, so are the negative ones.

3.4.6 This is really quite similar to Exercise 3.4.1, except for formatting. Running

(a) $7 = 111_2$ and $8 = 1000_2$. When computing $111 + 1000$ nothing is carried, and we end up with $1111_2 = 8 + 4 + 2 + 1 = 15$. When multiplying, again nothing is carried, and we end up with $111000_2 = 32 + 16 + 8 = 56$.

(b) $45 = 101101_2$ and $5 = 101_2$. Summing, we get $110010_2 = 32 + 16 + 2 = 50$. Multiplying yields

$$\begin{array}{r}
 1 \ 0 \ 1 \ . \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \\
 \hline
 1 \ 0 \ 1 \ 1 \ 0 \ 1 \\
 0 \ 0 \ 0 \ 0 \ 0 \ 0 \\
 1 \ 0 \ 1 \ 1 \ 0 \ 1 \\
 \hline
 1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 1
 \end{array}$$

The answer is $11100001_2 = 128 + 64 + 32 + 1 = 225$.

(c) $256 = 100000000_2$ and $13 = 1101_2$. Adding doesn't need any carrying, $100000000_2 + 1101_2 = 100001101_2 = 256 + 8 + 4 + 1 = 269$. Multiplying is also easy, we end up with

$$100000000 \cdot 1101 = 110100000000$$

Converting back to decimal gives

$$110100000000_2 = 2048 + 1024 + 256 = 3328$$

3.6.4

(a) $4AB2 = 4 \cdot 16^3 + 10 \cdot 16^2 + 11 \cdot 16 + 2 = 19122$

(b)

$$\begin{array}{r}
 1 \\
 1 \ A \\
 + \ B \ 2 \ E \\
 \hline
 B \ 4 \ 8
 \end{array}$$

The difficult part is that $A + E = 18_{16}$.

(c)

$$\begin{array}{r}
 1 \ A \ . \ B \ 2 \ E \\
 \hline
 6 \ F \ C \ C \\
 B \ 2 \ E \ 0 \\
 \hline
 1 \ 2 \ 2 \ A \ C
 \end{array}$$

(d)

$$\begin{aligned}
 15224 &= 951 \cdot 16 + 8 \\
 951 &= 59 \cdot 16 + 7 \\
 59 &= 3 \cdot 16 + 11 \\
 3 &= 0 \cdot 16 + 3
 \end{aligned}$$

The conclusion is that $15224_{10} = 3B78_{16}$.

(e) The table is as follows:

binary	hexadecimal
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	A
1011	B
1100	C
1101	D
1110	E
1111	F

We then see that

$$4E24C = 01001110001001001100$$

and

$$10011100110 = 4E6.$$

Note that the first 0 in the binary representation of $4E24C$ could be omitted, and that an extra 0 has to be put in to get $4 = 0100$ when going in the other direction.

3.6.5 We start with an example, with $a = 5 = 0101_2$ and $b = 2 = 0010_2$. Then we also need $-b = -2 = 1110_2$. Then

$$\begin{array}{rcccc}
 & & -1 & & & & 1 & 1 \\
 & 0 & 1 & 0 & 1 & & 0 & 1 & 0 & 1 \\
 - & 0 & 0 & 1 & 0 & & + & 1 & 1 & 1 & 0 \\
 \hline
 & 0 & 0 & 1 & 1 & & & 0 & 0 & 1 & 1
 \end{array}$$

To prove that this always works is somewhat difficult. One way is to change focus, and regard all the numbers as being positive! Formally, we do this by adding 16 to the negative numbers. This means that for instance 1001, which in two's complement is -7 , is considered as $9 = -7 + 16$, which is the “ordinary” interpretation of the binary number 1001. Restricting to four bits means that we work modulo 16.

Then if $a > b$ (when both are considered as positive numbers), subtracting $a - b$ works as it usually does for positive numbers. Also, taking negatives is the same as subtracting from 16, so that $-b = 16 - b$ (the computation can be performed with five bits, and the first bit can be discarded). Therefore, $a + (-b)$ is equal to the ordinary sum of the two positive numbers $a + (16 - b)$, considered modulo 16, and this gives the same answer as the natural subtraction $a - b$.

If $a < b$ we can prolong to five bits, exchange a by $a + 16$, do the same, and finally discard the first bit again. Modulo 16, this gives the same result.

3.6.6 In each case, predicting the answer really means to perform the operation for ordinary numbers, reduce modulo 16, and, if the remainder is greater than 7, subtract 16. The x s represent bits that we don't need to compute, as they are discarded.

(a)

$$\begin{array}{rcccccccc}
 0 & 0 & 1 & 0 & \cdot & 0 & 0 & 1 & 1 \\
 \hline
 & & & & & 0 & 0 & 0 & 0 \\
 & & & & & x & 0 & 1 & 1 \\
 & & & & & x & x & 0 & 0 \\
 & & & & & x & x & x & 0 \\
 \hline
 & & & & & x & x & x & 0 & 1 & 1 & 0
 \end{array}$$

(b)

$$\begin{array}{rcccccccc}
 1 & 1 & 1 & 0 & \cdot & 1 & 1 & 0 & 1 \\
 \hline
 & & & & & 0 & 0 & 0 & 0 \\
 & & & & & x & 1 & 0 & 1 \\
 & & & & & x & x & 0 & 1 \\
 & & & & & x & x & x & 1 \\
 \hline
 & & & & & x & x & x & 0 & 1 & 1 & 0
 \end{array}$$

(c)

$$\begin{array}{rcccccccc}
 0 & 0 & 1 & 0 & \cdot & 0 & 1 & 0 & 0 \\
 \hline
 & & & & & 0 & 0 & 0 & 0 \\
 & & & & & x & 1 & 0 & 0 \\
 & & & & & x & x & 0 & 0 \\
 & & & & & x & x & x & 0 \\
 \hline
 & & & & & x & x & x & 1 & 0 & 0 & 0
 \end{array}$$

(d)

$$\begin{array}{cccccccc}
 0 & 0 & 1 & 0 & \cdot & 0 & 1 & 0 & 1 \\
 \hline
 & & & & & 0 & 0 & 0 & 0 \\
 & & & & x & 1 & 0 & 1 & \\
 & & & x & x & 0 & 0 & & \\
 & & x & x & x & 0 & & & \\
 \hline
 & x & x & x & 1 & 0 & 1 & 0 &
 \end{array}$$

(e)

$$\begin{array}{cccccccc}
 1 & 1 & 0 & 1 & \cdot & 0 & 0 & 1 & 0 \\
 \hline
 & & & & & 0 & 0 & 1 & 0 \\
 & & & & x & 0 & 0 & 0 & \\
 & & & x & x & 1 & 0 & & \\
 & & x & x & x & 0 & & & \\
 \hline
 & x & x & x & 1 & 0 & 1 & 0 &
 \end{array}$$

(f)

$$\begin{array}{cccccccc}
 1 & 0 & 1 & 0 & \cdot & 0 & 1 & 1 & 1 \\
 \hline
 & & & & & 0 & 0 & 0 & 0 \\
 & & & & x & 1 & 1 & 1 & \\
 & & & x & x & 0 & 0 & & \\
 & & x & x & x & 1 & & & \\
 \hline
 & x & x & x & 0 & 1 & 1 & 0 &
 \end{array}$$

3.6.7 We argue for each point using the properties of the inverse function.

(a)

$$\ln(1) = \ln(e^0) = 0, \quad \ln(e) = \ln(e^1) = 1.$$

(b)

$$ab = e^{\ln a} e^{\ln b} = e^{\ln(a) + \ln(b)} \quad \text{and} \quad ab = e^{\ln(ab)}$$

Since e^x is injective, $\ln(ab) = \ln(a) + \ln(b)$.

(c)

$$a^b = (e^{\ln a})^b = e^{b \ln a} \quad \text{and} \quad a^b = e^{\ln(a^b)}$$

Since e^x is injective, $\ln(a^b) = b \ln(a)$.

(d) e^x is continuous, so $e^{\lim_{x \rightarrow a} x} = \lim_{x \rightarrow a} e^x$.

$$e^{\lim_{x \rightarrow 0^+} \ln(x)} = \lim_{x \rightarrow 0^+} e^{\ln x} = \lim_{x \rightarrow 0^+} x = 0 = e^{\lim_{x \rightarrow -\infty} x}$$

Again, since e^x is injective, this means that

$$\lim_{x \rightarrow 0^+} \ln(x) = \lim_{x \rightarrow -\infty} x = -\infty$$

(e)

$$e^{\lim_{x \rightarrow \infty} \ln(x)} = \lim_{x \rightarrow \infty} e^{\ln x} = \lim_{x \rightarrow \infty} x = \infty = e^{\lim_{x \rightarrow \infty} x}$$

We again conclude by injectivity:

$$\lim_{x \rightarrow \infty} \ln(x) = \lim_{x \rightarrow +\infty} x = \infty$$

3.6.8

(a) In other places than the four-bit, 4 has 0. Now $0 \vee 0 = 0$ and $1 \vee 0 = 1$, so in either case, nothing is changed. In the four-bit, 4 has 1. Since $1 \vee 1 = 1$ and $0 \vee 1 = 1$, in either case the four-bit is set.

(b) -5 is the number that has a 0 in the four-bit, and 1 everywhere else. In places other than the four-bit, using $0 \wedge 1 = 0$ and $1 \wedge 1 = 1$, we see that nothing is changed. In the four-bit, $0 \wedge 0 = 0$ and $1 \wedge 0 = 0$, so in either case the flag is turned off.

(c) 0 has 0 in all bits. $1 \oplus 0 = 1$ and $0 \oplus 0 = 0$, so nothing happens.

4 has 0 in all places different from the four-bit, so nothing happens there. In the four-bit, $0 \oplus 1 = 1$ and $1 \oplus 1 = 0$, so the value in this place is switched from on to off or from off to on.

-1 has 1 in all bits, so as in the previous step, we see that all bits are changed. Thus $a \wedge -1 = \sim a$.

(d) After the first step, `intX` is changed in all bits where `intY` has a 1. After the second step, `intY` gets the value computed from `intX` by changing all the bits where `intY` had a one *again*. Thus `intY` now has the original value of `intX`. Another way to think about `intX` after the first step, is to say that it comes from `intY` by changing all the bits where `intX` has a one. Since `intY` after step two is the original `intX`, after the third step, the changes in `intY` to `intX` in the first step are undone, and `intX` gets the original value of `intY`.

To conclude: The two variables change their values!

4.1.2

(a) $(-1, \pi]$.

(b) $[\sqrt{7}, \infty)$.

(c) $(-\infty, -3)$

4.1.3

$$101.01011 = 4 + 1 + \frac{1}{4} + \frac{1}{16} + \frac{1}{32} = 5.34375$$

Here I cheated and used the calculator.

To go the other way,

$$3.75 = 2 + 1 + \frac{1}{2} + \frac{1}{4} = 11.11_2$$

4.3.1 Using the calculator, we find that the largest is

$$(2 - 2^{-52}) \cdot 2^{1023} = \text{Math ERROR}$$

Oops! The number is too large for my calculator. What now?

Trying Java with the same formula, as in

```
double maxDouble = (2-Math.pow(2, -52)) * Math.pow(2,1023);
System.out.println(maxDouble);
```

returns

1.7976931348623157E308

So the maximal double is $\approx 1.8 \cdot 10^{308}$.

Note that expanding the parantheses would lead to an error:

```
double maxDouble = 2 * Math.pow(2,1023) -
    Math.pow(2, -52) * Math.pow(2,1023);
System.out.println(maxDouble);
```

yields **Infinity** since $2 \cdot 2^{1023} = 2^{1024}$ is too large for a double.

Similarly, we find the smallest double as

```
double minDouble = Math.pow(2, -1074);
System.out.println(minDouble);
```

returns

4.9E-324

so the minimal double is $\approx 4.9 \cdot 10^{-324}$.

4.3.4 Each of the expressions $0/0$, $1/0$ and $-1/0$ for integers lead to compilation errors. Running

```
double rootOfNegative = Math.sqrt(-1);
System.out.println(rootOfNegative);
```


returns *NaN*.

4.4.3 `long converted = (long) 124.234f;`

4.5.1 The output from running the code is

```
5.0E35
Infinity
```

Changing to doubles, we get $4.99 \cdots 10^{35}$ from both computations. The problem is that bc is too large for a `float`, so that infinity is returned when this computation comes first. To get the same problem for `double`, we need to input larger numbers. Trial and error, and remembering the result from Exercise 4.3.1, we get the same type of result from running

```
double a=0.25e-12;
double b=4e12;
double c=5e305;
System.out.println((a*b)*c);
System.out.println(a*(b*c));
```

4.5.2 Here is code for the programme, initialized with the first set of coefficients from the table. It would be better to print a more informative statement.

```
float a = 1.0f;
float b = 2.0f;
float c = -3.0f;
float x_0 = (float) (-b + Math.sqrt(b*b - 4*a*c))/(2*a);
float x_1 = (float) (-b - Math.sqrt(b*b - 4*a*c))/(2*a);
System.out.println(x_0);
System.out.println(x_1);
//test. This should return 0
System.out.println(a*x_0*x_0 + b*x_0+c);
System.out.println(a*x_1*x_1 + b*x_1+c);
```

This returns

```
1.0
-3.0
0.0
0.0
```

In the next set of values for a, b, c , we get

```
-1.0
-1.0
0.0
0.0
```

The third:

```
NaN
NaN
NaN
NaN
```

Looking at the computation, we see that we try to compute the square root of a negative number, and this returns *NaN* as in Exercise 4.3.4.

The last:

```
20000.0
0.0
1.0
1.0
```

Here the two solutions give 1 instead of 0 when plugged into the original equation. The reason is that there are errors coming from rounding. If all the variables are changed into `doubles` (and we delete the casting to `float`), the output will be

```
19999.999949999998
5.000000055588316E-5
-5.9604644775390625E-8
-8.617663249665952E-9
```

We see that the approximate solution 20000 is actually a bit too large, and 0 a bit too small. Also, the test doesn't return 0, but numbers quite close to 0, which is to be expected because of rounding errors.

4.5.3 Here is the code for writing the points to the terminal window:

```
for (double x = -7.0; x < 7.0; x += 0.01) {
    double y = Math.sin(x);
    System.out.printf("Point (%f, %f)\n", x, y);
}
```

To find the easiest formulation for the range of an integer variable, note that if we multiply with a hundred, the increment is changed to +1. So we can write

```

for(int i = -700; i < 700; i++){
    double x = ((double) i)/100;
    double y = Math.sin(x);
    System.out.printf("Point (%f, %f)\n ", x, y);
}

```

Note that we also need to compute x , and we need to make sure that $i/100$ is not considered integer division.

4.5.4 There are many ways to solve this exercise, so if your solution doesn't resemble this one, don't worry.

```

int power = 1;
int powerOf2 = 2;
int oneLower = 1;
while (powerOf2 > oneLower){
    System.out.println(powerOf2);
    powerOf2 *= 2;
    oneLower *= 2;
    power++;
}
System.out.printf("Longs from power %d\n", power);
long powerOf2L = 2*(long) oneLower;
long oneLowerL = (long) oneLower;
while (powerOf2L > oneLowerL){
    System.out.println(powerOf2L);
    powerOf2L *= 2;
    oneLowerL *= 2;
    power++;
}
System.out.printf("Floats from power %d\n", power);
float powerOf2f = 2* (float) oneLowerL;
float oneLowerf = (float) oneLowerL;
Float infinity = new Float(Float.POSITIVE_INFINITY);
while (powerOf2f < infinity){
    System.out.println(powerOf2f);
    powerOf2f *= 2;
    oneLowerf *= 2;
    power++;
}
System.out.printf("Floats from power %d\n", power);
double powerOf2Double = 2* (double) oneLowerf;

```

```

double infinityDouble = (double) infinity;
while (powerOf2Double < infinityDouble){
    System.out.println(powerOf2Double);
    powerOf2Double *= 2;
    power++;
}
System.out.printf("Final power is %d\n", power);

```

Changing all instances of 2 into 3s actually works, as a test it stops at 3^{647} .

4.5.5 This code produces the tables. It is not the simplest construction for the problem, and it might be better to format the output differently.

```

Float minusInfinity = new Float(Float.NEGATIVE_INFINITY);
Float plusInfinity = new Float(Float.POSITIVE_INFINITY);
Float nan = new Float(Float.NaN);
Float zero = new Float(0.0f);
Float minusZero = new Float(-0.0f);

Float[] specialFloats = new Float[5];
specialFloats[0] = minusInfinity;
specialFloats[1] = plusInfinity;
specialFloats[2] = nan;
specialFloats[3] = zero;
specialFloats[4] = minusZero;

//multiplication table:
for (Float special : specialFloats){
    for (Float special2 : specialFloats){
        System.out.printf("%f * %f = %f\n", special, special2,
            special * special2);
    }
}

//addition table:
for (Float special : specialFloats){
    for (Float special2 : specialFloats){
        System.out.printf("%f + %f = %f\n", special, special2,
            special + special2);
    }
}

```

5.1.8 We need to make sure we understand what “the hundredth term” refers to.

To be precise, we assume that the first term is a_0 and the term we are interested in is a_{99} .

In the first, the answer is 100.

In the second, $a_n = -2.2 + n \cdot 2.4$ so that $a_{99} = -2.2 + 99 \cdot 2.4 = 235.4$.

In the third, $a_n = 1 + n \cdot (-4)$ so that $a_{99} = -395$.

5.1.9 s = the first number, so $s = 2$. t = the number we need to add at each step, so $t = 8 - 2 = 6$.

5.1.10

$$2.3 \quad 0.6 \quad -1.1 \quad -2.8 \quad -4.5$$

5.1.13 In the first, $r = 1$ and $k = 2$, so $a_{99} = 2^{99}$.

In the second, $r = 1$ and $k = -1$, so $a_{99} = (-1)^{99} = -1$.

In the third, $r = 2$ and $k = 1/3$, so $a_{99} = \frac{2}{3^{99}}$.

5.1.14 r = the first term, so $r = 4$. k is what we multiply by, so $k = 12/4 = 3$.

5.1.15

$$\begin{aligned} 0.5 \\ 3 \cdot 0.5 &= 1.5 \\ 3 \cdot 1.5 &= 4.5 \\ 3 \cdot 4.5 &= 13.5 \\ 3 \cdot 13.5 &= 40.5 \end{aligned}$$

5.2.3

$$\begin{aligned} \sum_{i=0}^4 2i - 2 &= -2 + 0 + 2 + 4 + 6 = 10 \\ \sum_{n=1}^3 n^3 &= 1 + 8 + 27 = 36 \\ \sum_{n=1}^{10} n &= 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 + 10 = 55 \end{aligned}$$

5.2.7 Here $s = 8$, $t = 3$, and the last term is $71 = 8 + 21 \cdot 3$, so $n = 21$. Therefore the sum is

$$\frac{(n+1)(2s+nt)}{2} = \frac{22 \cdot (16+63)}{2} = 869$$

5.2.9 Here $r = 2$ and $k = 3$. The last term is $13122 = 2 \cdot 3^8$, so $n = 8$. Therefore, the sum is

$$\frac{r - rk^{n+1}}{1 - k} = \frac{2 - 2 \cdot 3^9}{-2} = 19682.$$

5.2.11 Here $r = 17$ and $k = 1/7$, which is less than 1, so the series is convergent. The sum is therefore

$$\frac{r}{1 - k} = \frac{17}{1 - \frac{1}{7}} = \frac{119}{6}$$

5.3.1 We have

$$\begin{aligned} a_0 &= \frac{2 \cdot 0 - 1}{4} = -\frac{1}{4} \\ a_1 &= \frac{2 \cdot 1 - 1}{4} = \frac{1}{4} \\ a_2 &= \frac{2 \cdot 2 - 1}{4} = \frac{3}{4} \\ a_3 &= \frac{2 \cdot 3 - 1}{4} = \frac{5}{4} \\ a_4 &= \frac{2 \cdot 4 - 1}{4} = \frac{7}{4} \end{aligned}$$

5.3.2 Comparing with the previous exercise, we see that $a_n = (2n - 1)$ for $n \geq 0$.

5.3.3

(a)

1	$\geq$	1
$\frac{1}{2}$	$\geq$	$\frac{1}{2}$
$\frac{1}{3}$	$\geq$	$\frac{1}{4}$
$\frac{1}{4}$	$\geq$	$\frac{1}{4}$
$\frac{1}{5}$	$\geq$	$\frac{1}{8}$
$\frac{1}{6}$	$\geq$	$\frac{1}{8}$
$\frac{1}{7}$	$\geq$	$\frac{1}{8}$
$\frac{1}{8}$	$\geq$	$\frac{1}{8}$
$\frac{1}{9}$	$\geq$	$\frac{1}{16}$
$\frac{1}{10}$	$\geq$	$\frac{1}{16}$
$\frac{1}{11}$	$\geq$	$\frac{1}{16}$
$\frac{1}{12}$	$\geq$	$\frac{1}{16}$
$\frac{1}{13}$	$\geq$	$\frac{1}{16}$
$\frac{1}{14}$	$\geq$	$\frac{1}{16}$
$\frac{1}{15}$	$\geq$	$\frac{1}{16}$
$\frac{1}{16}$	$\geq$	$\frac{1}{16}$

If we sum the right hand sides, we get

$$1 + \frac{1}{2} + 2 \cdot \frac{1}{4} + 4 \cdot \frac{1}{8} + 8 \cdot \frac{1}{16} = 1 + \frac{1}{2} + \frac{1}{2} + \frac{1}{2} + \frac{1}{2} = 3$$

(b) $1/n$ rounds down to $1/2^m$ if $2^{m-1} < n \leq 2^m$. In terms of the binary logarithm, this means that $m - 1 < \log_2(n) \leq m$.

(c) The number of terms rounding down to $1/2^m$ can be found by counting the numbers in the set $\{2^{m-1} + 1, 2^{m-1} + 2, \dots, 2^m = 2^{m-1} + 2^{m-1}\}$. We see that there are exactly 2^{m-1} numbers in this set.

(d) From c), we see that the sum of the terms rounding down to $1/2^m$ is $2^{m-1} \cdot 1/2^m = 1/2$. Therefore we get the 1 in the beginning, plus one half for each $i = 1, 2, 3, \dots, m$, so the answer for the lower bound is $1 + \frac{m}{2}$.

(e) Since our sum is greater than a sum with partial sums $1 + \frac{m}{2}$, and since this sequence goes to infinity, the sum we're interested in must also go to infinity.

5.3.4 We say that $a = (a_0, a_1, \dots)$ diverges to $-\infty$ if for any real number R there exists an integer N such that

$$a_n < R \text{ for all } n > N.$$

5.3.5 The formula in the text of the exercise yields

$$\sum_{i=m+1}^n rk^i = \sum_{i=0}^n rk^i - \sum_{i=0}^m rk^i$$

Both terms on the right are given by the formula in Theorem 5.2.8: The formula in the text of the exercise yields

$$\sum_{i=m+1}^n rk^i = \sum_{i=0}^n rk^i - \sum_{i=0}^m rk^i = \frac{r - rk^{n+1}}{1 - k} - \frac{r - rk^{m+1}}{1 - k}$$

Since the denominators are the same, summing the fractions is straight-forward, and we get

$$\sum_{i=m+1}^n rk^i = \frac{r - rk^{n+1} - (r - rk^{m+1})}{1 - k} = \frac{rk^{m+1} - rk^{n+1}}{1 - k}$$

Another way to compute this is to realize that

$$\sum_{i=m+1}^n rk^i = \sum_{i=0}^{n-m-1} rk^{m+1}k^i$$

This is a geometric series with first term rk^{m+1} and quotient k . By appealing to Theorem 5.2.8 again, we get

$$\sum_{i=0}^{n-m-1} rk^{m+1}k^i = \frac{rk^{m+1} - rk^{m+1}k^{n-m}}{1-k} = \frac{rk^{m+1} - rk^{n+1}}{1-k}$$

Again, we get the same result.

5.3.6 We can write

$$\sum_{i=m}^{\infty} rk^i = \sum_{i=0}^{\infty} rk^i - \sum_{i=0}^{m-1} rk^i$$

Using Theorem 5.2.10 for the infinite series and Theorem 5.2.8 for the finite series yields

$$\sum_{i=m}^{\infty} rk^i = \sum_{i=0}^{\infty} rk^i - \sum_{i=0}^{m-1} rk^i = \frac{r}{1-k} - \frac{r - rk^m}{1-k}$$

Simplifying, we get

$$\sum_{i=m}^{\infty} rk^i = \frac{r}{1-k} - \frac{r - rk^m}{1-k} = \frac{r - (r - rk^m)}{1-k} = \frac{rk^m}{1-k}$$

We can compute the sum by appealing to Theorem 5.2.10, by writing

$$\sum_{i=m}^{\infty} rk^i = \sum_{i=0}^{\infty} rk^m k^i = \frac{rk^m}{1-k}$$

We get the same result twice.

6.1.10 By definition, the image $B' \subset B$ is the set of bs such that there exists an element $a \in A$ with $f(a) = b$. So for any $b \in B'$, $b = f(a) = g(a)$ for some $a \in A$, and g is surjective.

If in addition f is injective, then $f(a) = f(a')$ implies $a = a'$. Then, if $g(a) = g(a')$ we have $f(a) = g(a) = g(a') = f(a')$, so $a = a'$ and g is also injective. Being both injective and surjective, g is bijective.

6.1.13 Since $f(0) = \tan 0 = 0$ and $f(\pi) = \tan \pi = 0$, f is not injective. But f is continuous and defined on the open interval $(-\frac{\pi}{2}, \frac{\pi}{2})$, and

$$\lim_{x \rightarrow -\frac{\pi}{2}^+} \tan x = -\infty, \quad \lim_{x \rightarrow \frac{\pi}{2}^-} \tan x = \infty$$

By the intermediate value theorem, every real number is therefore in the image of f , and f is surjective.

6.1.14 We first show that f is injective. So assume that $f(a) = f(a')$. Then $a + 2 = a' + 2$ and $a = a'$, so f is indeed injective.

To show that f is surjective, we need to show that any real number b is equal to $f(a)$ for some a . But solving $b = f(a) = a + 2$ shows that $a = b - 2$ is a solution.

Since f is both injective and surjective, it is bijective.

6.1.18 For $k = 2$ we get this table:

$$\begin{aligned} f(0) &= 0 \cdot 2 = 0 \bmod 5 \\ f(1) &= 1 \cdot 2 = 2 \bmod 5 \\ f(2) &= 2 \cdot 2 = 4 \bmod 5 \\ f(3) &= 3 \cdot 2 = 1 \bmod 5 \\ f(4) &= 4 \cdot 2 = 3 \bmod 5 \end{aligned}$$

Since each number from 0 to 4 appears exactly once in the table of values, the function is bijective.

6.2.2 For $n = 17$ the answer is -288522240 . But obviously all factorials are positive. The only possible explanation is overflow, we have passed the maximum `int`.

6.2.4 We get 0. The only possible explanation is overflow, we have passed the maximum `int`. Note that we saw this already for $17!$ in Exercise 6.2.2, and now we have tried to compute $34!$. In fact, $34!$ returns zero even before we divide it by anything.

6.2.6 There isn't anything to say about this!

6.3.1 Note that there are many possible examples for the last three questions. Here the smallest possibilities are presented.

(a) Assume $f(a_1) = f(a_2)$. We need to show that $a_1 = a_2$.

Since $f(a_1) = f(a_2)$, $g(f(a_1)) = g(f(a_2))$. Since $g \circ f$ is injective, this means that $a_1 = a_2$.

(b) For any $c \in C$, we need to find $b \in B$ such that $g(b) = c$. Since $g \circ f$ is surjective, there is an $a \in A$ such that $g(f(a)) = c$. But then, if we set $b = f(a)$, we have $g(b) = g(f(a)) = c$ as needed.

(c) There are two problems here. Assume first that f is bijective. Since $g \circ f$ is bijective, it is surjective, so by b), g is surjective. We need to show that g is injective. If $g(b_1) = g(b_2)$, we need to show that $b_1 = b_2$. Since f is bijective, it is surjective, and there are elements $a_1, a_2 \in A$ such that $f(a_1) = b_1$, $f(a_2) = b_2$. Since

$g \circ f$ is bijective, it is injective. Therefore, since $g(f(a_1)) = g(b_1) = g(b_2) = g(f(a_2))$, $a_1 = a_2$. But then $b_1 = f(a_1) = f(a_2) = b_2$ as needed.

Assume next that g is bijective. Since $g \circ f$ is bijective, it is injective, so by a), f is injective. We need to show that f is surjective. Let $b \in B$ be any element. We need to show that there is an element $a \in A$ such that $f(a) = b$. Since $g \circ f$ is bijective, it is surjective, and there is an element $a \in A$ such that $g(f(a)) = g(b)$. Since g is bijective, it is injective, and therefore $f(a) = b$ as needed.

(d) Let the sets be $A = \{a\}$, $B = \{b_1, b_2\}$ and $C = \{c\}$. Define $f(a) = b_1$ and $g(b_1) = g(b_2) = c$. Then $g \circ f$ is the function defined by $g(f(a)) = c$, so it is injective. But g is not injective.

(e) Let the sets be $A = \{a\}$, $B = \{b_1, b_2\}$ and $C = \{c\}$. Define $f(a) = b_1$ and $g(b_1) = g(b_2) = c$. Then $g \circ f$ is the function defined by $g(f(a)) = c$, so it is injective. But g is not injective.

(f) Again, using the same example, our conditions are fulfilled by the reasons already explained.

6.3.2 If f is injective, all the values are different. Since there are as many different values as there are elements in A , there are also as many different values as there are elements in B . Thus f is surjective. If we assume that f is surjective, we again get that there are as many different values as there are elements in B , hence as there are elements in A . Thus f is injective. Finally, f is bijective if and only if it is both injective and surjective. But we have already checked that injective and surjective mean the same thing. So if f is injective, it is also surjective, hence bijective. If f is surjective, it is also injective, hence bijective. If f is bijective it is (always) both injective and surjective.

6.3.3 The prescription defines a function, since there is a pair of elements in $C \times D$ for each pair in $A \times B$.

(a) If $f \times g(a, b) = f \times g(a', b')$, we need to check that $(a, b) = (a', b')'$. By definition,

$$f \times g(a, b) = (f(a), g(b)) = (f(a'), g(b')) = f \times g(a', b')$$

Hence $f(a) = f(a')$ and $g(b) = g(b')$. Since these functions are injective, $a = a'$ and $b = b'$. Thus $(a, b) = (a', b')'$ as needed.

(b) Let (c, d) be any pair in $C \times D$. We need to find (a, b) such that $f \times g(a, b) = (c, d)$.

Since f is surjective, there is an element $a \in A$ such that $f(a) = c$. Similarly, since g is surjective, there is an element $b \in B$ such that $g(b) = d$. But then $f \times g(a, b) = (f(a), g(b)) = (c, d)$ as needed.

(c) This is the simultaneous statement of a) and b).

6.3.4 We can use the previously defined method `computeBinomialCoeff`. It is a fact that the Catalan numbers are integers, so we will use the `int` type.

```
public static int computeCatalan(int n){
    return computeBinomialCoeff(2*n,n)/(n+1);
}
```

The first few values are 1, 1, 2, 5, 14, 42.

6.3.5 Here is a possible programme. Note that the `break` statement prevents repetitions to be counted; each value is counted only once.

```
public class FunctionFromRemainders{
    public static void main(String [] args){
        int n = 7;
        int m = 8;
        int k = 3;
        int [] setA = new int [n];
        int [] setB = new int [m];
        int [] functionf = new int [n];
        int numberOfValues = 0;

        for (int i = 0; i < n; i++){
            setA[i] = i;
        }
        for (int i = 0; i < m; i++){
            setB[i] = i;
        }
        for (int i = 0; i < n; i++){
            functionf[i] = (i*k) % m;
        }

        for (int b : setB){
            for (int i = 0; i < n; i++){
                if (b == functionf[i]){
                    numberOfValues ++;
                    break;
                }
            }
        }
        if (numberOfValues == n){
```

```

        System.out.println(" Injective!" );
    }
    if (numberOfValues == m){
        System.out.println(" Surjective!" );
    }
}
}

```

7.1.1 1000 iterations give 3.139555466911023.

2000 iterations give 3.140579505911964.

10000 iterations give 3.141391477611321.

To compare with the actual value, we call *Math.PI* to get 3.141592653589793. We see that even with ten thousand iterations, this method isn't very good.

7.2.3 Here is a loop that does the trick:

```

for (double i = 0.5; i > 0.000001; i /= 2){
    System.out.println(i);
}

```

7.2.7 We need a new integer variable, say `numberOfIterations`, that is increased by one each time the loop is executed. Here is code, including both the original loop, and the new lines:

```

double x = 1.0;
int numberOfIterations = 0; // new
while (Math.sin(x)/x < 0.99999){
    x /= 10;
    numberOfIterations ++; //new
}

```

It turns out that only three iterations are needed.

7.2.9 Here is code to get the maximum number of times the procedure is performed.

```

int max = 1;
for (int i = 1; i < 1000; i++){
    int n = i;
    int timesPerformed = 0;
    while (n != 1){
        timesPerformed ++;
        if (n%2 == 0){ // n is even
            n /= 2;

```

```

        } else { // n is odd
            n = 3*n+1;
        }
    }
    if (max < timesPerformed){
        max = timesPerformed;
    }
}
System.out.println(max);

```

The answer turns out to be 178, which happens for $i = 871$.

7.3.2 We add up terms until the sum passes 8. Note that by initializing and increasing n as in this code, the value is correct when the loop finishes.

```

double answer = 0.0;
int n = -1;
while (answer < 8){
    n++;
    answer += 1/(1 + Math.sqrt(n));
}
System.out.println(n);

```

Running this code returns $n = 31$

7.4.1 We need to cast each of the characters 'a', 'b', ..., 'z' to `int` and sum the results. This code performs these operations, keeping a running total.

```

int sumOfChars = 0;
for (char c = 'a'; c <= 'z'; c++){
    sumOfChars += (int) c;
}

```

The answer is 2847. Note that if we drop the explicit cast (`int`), Java casts implicitly for us.

7.4.2 An easy solution is to compute using `long`s, and compare with the largest `int`, also represented as a `long`-. Here is code using this approach.

```

long maxInt = Integer.MAX_VALUE;
long runningTotal = 0;
int i = 0; //summation index
while (runningTotal < maxInt){
    i++;
    runningTotal += i*i;
}

```

```

}
System.out.println(i);

```

The answer turns out to be 1861.

7.4.3 There are many ways to solve this exercise. We will take an approach that hopefully keeps the programming understandable. Since the negative cubes are the negatives of the positive cubes, we can make an array of positive cubes, and combine cubes with plus or minus. To make sure that we have enough cubes, we could make the array long enough to hold all cubes less than the number we're interested in. This would work if we knew the cubes were positive! Since the cubes can be negative as well, we increase the length of the array by including the cubes of all numbers whose *square* is smaller than the number we're interested in. This is a crude solution, and the following code performs a lot of unnecessary work. A closer analysis of the problem would yield significantly reduced running time.

```

//int n = 3370318;
int n = 2716057;
int numberOfCubes = (int) Math.sqrt(n);
// numberOfCubes is more than large enough
int[] cubes = new int[numberOfCubes];
for (int i = 1; i <= numberOfCubes; i++){
    cubes[i-1] = i*i*i;
}

for (int i : cubes){
    for (int j : cubes){
        if (i + j == n){
            System.out.printf("%d + %d = %d\n", i, j, n);
        }
        if (i-j == n){
            System.out.printf("%d - %d = %d\n", i, j, n);
        }
    }
}

```

In the end, this gives us that

$$\begin{array}{rcl}
 3370318 & = & 1685159 + 1685159 = 119^3 + 119^3 \\
 2716057 & = & 862801408 - 860085351 = 952^3 + (-951)^3
 \end{array}$$

7.4.4 The code is relatively straight-forward. Note that $(-1)^{n+1}$ is 1 if n is odd, -1 if it is even, so we check for parity using the test `i % 2 == 0`.

```

int n = 100;
double runningSum = 0.0;
for (int i = 1; i < n; i++){
    if (i % 2 == 0){
        runningSum -= 1.0/i;
    } else{
        runningSum += 1.0/i;
    }
}
System.out.println(runningSum);

```

Calling `Math.log(2)` gives 0.6931471805599453. You will need a large number n to get close to this number.

7.4.5 We sum the series, up to a chosen number of terms n , and multiply by 6, to get the approximation for π^2 . This code does this for us:

```

int n = 100;
double runningSum = 0.0;
for (int i = 1; i < n; i++){
    runningSum += 1.0/(i*i);
}
runningSum *= 6;
System.out.println(runningSum);

```

Calling `Math.PI*Math.PI` yields 9.869604401089358. Again, we need many terms to get a good approximation.

8.1.4 We can choose p as “I’m wet”, q as “I’m dirty” and r as “I put on new clothes”.

8.2.9 For each $b \in B$ there is an $a \in A$ such that $f(a) = b$. This is the definition of f being surjective.

8.2.10

(a) $\forall x P(x)$ is true if all the statements $P(x)$ are true. So we need to check something for every x . Similarly, $\exists x P(x)$ is false if all the statements $P(x)$ are false, so again we need to check something for every x .

(b) $\forall x P(x)$ is false if any of the statements $P(x)$ is false. So it is enough to find a single x such that $P(x)$ is false. Similarly, $\exists x P(x)$ is true if any of the statements $P(x)$ is true, so again we need to find a single x such that $P(x)$ is true.

(c) To formulate de Morgan's laws, we combine negation and $\forall, \exists$ in the same way that we combined negation and $\wedge, \vee$ earlier:

$$\neg \forall x P(x) = \exists x \neg P(x)$$

$$\neg \exists x P(x) = \forall x \neg P(x)$$

8.3.5 Modus ponens:

$$\frac{\forall x P(x) \rightarrow Q(x) \quad P(6)}{Q(6)}$$

This now states that since 6 is even, $6 + 1 = 7$ is odd.

Modus tollens:

$$\frac{\forall x P(x) \rightarrow Q(x) \quad \neg Q(3)}{\neg P(3)}$$

This states that since $3 + 1 = 4$ is not odd, 3 is not even.

8.4.5 We start with $n = 7$. At the basis step, $3^7 = 2187$ while $7! = 5040$. So the inequality is correct for $n = 7$.

For the induction step, we assume that $3^n < n!$ for some $n \geq 7$, and we want to prove that $3^{n+1} < (n+1)!$. Then

$$3^{n+1} = 3 \cdot 3^n < 3 \cdot n!$$

since we have assumed that $3^n < n!$. Furthermore, since $3 < 7 < n + 1$,

$$3^{n+1} < 3 \cdot n! < (n+1)n! = (n+1)!$$

We have proved that the inequality holds for all $n \geq 7$ by induction.

8.4.7 Part a) first. In the basis step, $a_0 = 1$ and $a_1 = \sqrt{a_0 + 1} = \sqrt{2}$ which is greater than $1 = a_0$. For the induction step, assume that $a_n > a_{n-1}$ for some $n \geq 1$. We want to prove that $a_{n+1} > a_n$. Then

$$a_{n+1} = \sqrt{a_n + 1} > \sqrt{a_{n-1} + 1} = a_n$$

We have proved that the sequence is increasing by induction.

For part b), $a_0 = 1 < 2$, so the basis step is true. For the induction step, assume that $a_n < 2$. We need to prove that $a_{n+1} < 2$. But

$$a_{n+1} = \sqrt{a_n + 1} < \sqrt{2 + 1} = \sqrt{3} < 2$$

We have proved that the sequence is bounded by 2 by induction.

8.6.1

(a) The expression can be read as “there exists a unique positive integer which is an even prime”. This is true, since 2 is prime and even, and all larger even numbers are divisible by 2, hence not prime.

(b) The statement can be read as “for each $b \in B$ there exists a unique $a \in A$ with $f(a) = b$ ”. That there exists such an a means that the function is surjective. Since there is only one possibility for a , different elements in A are sent to different values. Thus the function is also injective. Since it is both injective and surjective, the outcome is that this expression means that the function is bijective.

(c) There exist uniquely means that there is one, but not more than one, x for which $P(x)$ is true. This is the same as the definition of exclusive or.

9.5.1 In each case, the eigenvectors are found from the characteristic equation.

(a) $\lambda_1 = 0$ and $\lambda_2 = 4$. Gauss-Jordan-elimination of

$$\begin{pmatrix} 2-0 & 2 \\ 2 & 2-0 \end{pmatrix}$$

yields

$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix}$$

The eigenvectors are therefore $[x, y]^T = y[-1, 1]^T$ using y as a parameter. Similarly, Gauss-Jordan-elimination of

$$\begin{pmatrix} 2-4 & 2 \\ 2 & 2-4 \end{pmatrix}$$

yields

$$\begin{pmatrix} 1 & -1 \\ 0 & 0 \end{pmatrix}$$

The eigenvectors are therefore $[x, y]^T = y[1, 1]^T$ using y as a parameter.

11.4.4

(a) Computing the partial derivatives, we see that $f_x(x, y) = 6x^2 + y^2 + 12$ is always greater than 12. Since it can never be zero, there are no stationary points.

(b) The gradient is $\nabla g = [x^2 - 4, y^2 - 9]$. The simultaneous equations $x^2 - 4 = 0$ and $y^2 - 9 = 0$ give us four solutions, $x = \pm 2$ and $y = \pm 3$. The determinant of the Hessian is

$$\begin{vmatrix} f_{xx}(a, b) & f_{yx}(a, b) \\ f_{xy}(a, b) & f_{yy}(a, b) \end{vmatrix} = \begin{vmatrix} 2x & 0 \\ 0 & 2y \end{vmatrix} = 4xy$$

$(2, -3)$ and $(-2, 3)$ give a negative determinant -24 , so these are saddle points. $(2, 3)$ and $(-2, -3)$ give a positive determinant 24 . For $(2, 3)$, $f_{xx} = 2x = 4 > 0$, so this is a minimum. For $(-2, -3)$, $f_{xx} = 2x = -4 < 0$, so this is a maximum.

(c) The gradient is $\nabla h = [6x + y^2, 2xy]$. The equation $2xy = 0$ means that either $x = 0$ or $y = 0$. If $x = 0$, $6x + y^2 = y^2 = 0$ gives $y = 0$ also. If $y = 0$, $6x + y^2 = 6x = 0$ gives $x = 0$. We see that $(0, 0)$ is the only stationary point. The determinant of the Hessian is

$$\begin{vmatrix} f_{xx}(a, b) & f_{yx}(a, b) \\ f_{xy}(a, b) & f_{yy}(a, b) \end{vmatrix} = \begin{vmatrix} 6 & 2y \\ 2y & 2x \end{vmatrix} = 12x - 4y^2$$

If we evaluate this at our stationary point, we get $12 \cdot 0 - 4 \cdot 0^2 = 0$. Therefore, our methods do not apply, and we are not able to classify this stationary point.

11.4.5

(a) **Linear algebra approach:** The matrices are

$$M = \begin{pmatrix} X_1 & 1 \\ X_2 & 1 \\ X_3 & 1 \\ X_4 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \\ 3 & 1 \\ 6 & 1 \end{pmatrix} \quad \text{and} \quad \vec{Y} = \begin{pmatrix} Y_1 \\ Y_2 \\ Y_3 \\ Y_4 \end{pmatrix} = \begin{pmatrix} 2 \\ 1 \\ -1 \\ -3 \end{pmatrix}$$

We compute

$$M^T M = \begin{pmatrix} 0^2 + 1^2 + 3^2 + 6^2 & 0 + 1 + 3 + 6 \\ 0 + 1 + 3 + 6 & 1^2 + 1^2 + 1^2 + 1^2 \end{pmatrix} = \begin{pmatrix} 46 & 10 \\ 10 & 4 \end{pmatrix}$$

and

$$M^T \vec{Y} = \begin{pmatrix} 0 \cdot 2 + 1 \cdot 1 + 3 \cdot (-1) + 6 \cdot (-3) \\ 1 \cdot 2 + 1 \cdot 1 + 1 \cdot (-1) + 1 \cdot (-3) \end{pmatrix} = \begin{pmatrix} -20 \\ -1 \end{pmatrix}$$

Gauss-elimination of the matrix

$$\begin{pmatrix} 46 & 10 & -20 \\ 10 & 4 & -1 \end{pmatrix} \quad \text{yields} \quad \begin{pmatrix} 1 & 0 & -5/6 \\ 0 & 1 & 11/6 \end{pmatrix}$$

Therefore the best line approximating these points is $y = \frac{11}{6} - \frac{5x}{6} \approx 1.833 - 0.833x$.

Partial derivatives approach: We compute in succession:

$$\begin{aligned}\bar{X} &= \frac{1}{4}(0 + 1 + 3 + 6) = 2.5 \\ \bar{Y} &= \frac{1}{4}(2 + 1 - 1 - 3) = -0.25 \\ \sum (X_i - \bar{X})^2 &= (-2.5)^2 + (1 - 2.5)^2 + (3 - 2.5)^2 + (6 - 2.5)^2 = 21 \\ \sum Y_i(X_i - \bar{X}) &= 2(-2.5) + (1 - 2.5) - (3 - 2.5) - 3(6 - 2.5) = -17.5 \\ A &= \frac{-17.5}{21} = -0.833 \\ B &= -0.25 - (-0.833) \cdot 2.5 = 1.833\end{aligned}$$

We get again $y = 1.833 - 0.833x$.

(b) Linear algebra approach: The matrices are

$$M = \begin{pmatrix} X_1 & 1 \\ X_2 & 1 \\ X_3 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \\ 3 & 1 \end{pmatrix} \quad \text{and} \quad \vec{Y} = \begin{pmatrix} Y_1 \\ Y_2 \\ Y_3 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix}$$

We compute

$$M^T M = \begin{pmatrix} 2^2 + 1^2 + 3^2 & 2 + 1 + 3 \\ 2 + 1 + 3 & 1^2 + 1^2 + 1^2 \end{pmatrix} = \begin{pmatrix} 14 & 6 \\ 6 & 3 \end{pmatrix}$$

and

$$M^T \vec{Y} = \begin{pmatrix} 2 \cdot 2 + 1 \cdot 2 + 3 \cdot 2 \\ 1 \cdot 2 + 1 \cdot 2 + 1 \cdot 2 \end{pmatrix} = \begin{pmatrix} 12 \\ 6 \end{pmatrix}$$

Gauss-elimination of the matrix

$$\begin{pmatrix} 14 & 6 & 12 \\ 6 & 3 & 6 \end{pmatrix} \quad \text{yields} \quad \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 2 \end{pmatrix}$$

Therefore, the best line approximating these points is $y = 2$, a horizontal line.

Partial derivatives approach: We compute in succession:

$$\begin{aligned}\bar{X} &= \frac{1}{3}(2 + 1 + 3) = 2 \\ \bar{Y} &= \frac{1}{3}(2 + 2 + 2) = 2 \\ \sum (X_i - \bar{X})^2 &= (2 - 2)^2 + (1 - 2)^2 + (3 - 2)^2 = 2 \\ \sum Y_i(X_i - \bar{X}) &= 2(2 - 2) + 2(1 - 2) + 2(3 - 2) = 0 \\ A &= \frac{0}{2} = 0 \\ B &= 2 - 0 \cdot 2 = 2\end{aligned}$$

We get again $y = 2$.

(c) Linear algebra approach: The matrices are

$$M = \begin{pmatrix} X_1 & 1 \\ X_2 & 1 \\ X_3 & 1 \\ X_4 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 1 \\ 1 & 1 \\ 1 & 1 \\ 2 & 1 \end{pmatrix} \quad \text{and} \quad \vec{Y} = \begin{pmatrix} Y_1 \\ Y_2 \\ Y_3 \\ Y_4 \end{pmatrix} = \begin{pmatrix} 3 \\ 2 \\ 4 \\ 5 \end{pmatrix}$$

We compute

$$M^T M = \begin{pmatrix} 0^2 + 1^2 + 1^2 + 2^2 & 0 + 1 + 1 + 2 \\ 0 + 1 + 2 + 2 & 1^2 + 1^2 + 1^2 + 1^2 \end{pmatrix} = \begin{pmatrix} 6 & 4 \\ 4 & 4 \end{pmatrix}$$

and

$$M^T \vec{Y} = \begin{pmatrix} 0 \cdot 3 + 1 \cdot 2 + 1 \cdot 4 + 2 \cdot 5 \\ 1 \cdot 3 + 1 \cdot 2 + 1 \cdot 4 + 1 \cdot 5 \end{pmatrix} = \begin{pmatrix} 16 \\ 14 \end{pmatrix}$$

Gauss-elimination of the matrix

$$\begin{pmatrix} 6 & 4 & 16 \\ 4 & 4 & 14 \end{pmatrix} \quad \text{yields} \quad \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 5/2 \end{pmatrix}$$

Therefore the best line approximating these points is $y = x + 2.5$.

Partial derivatives approach: We compute in succession:

$$\begin{aligned} \bar{X} &= \frac{1}{4}(0 + 1 + 1 + 2) = 1 \\ \bar{Y} &= \frac{1}{4}(3 + 2 + 4 + 5) = 7/2 \\ \sum (X_i - \bar{X})^2 &= (-1)^2 + (1 - 1)^2 + (1 - 1)^2 + (2 - 1)^2 = 2 \\ \sum Y_i (X_i - \bar{X}) &= 3(-1) + 2(1 - 1) + 4(1 - 1) + 5(2 - 1) = 2 \\ A &= \frac{2}{2} = 1 \\ B &= 7/2 - 1 \cdot 1 = 5/2 \end{aligned}$$

We get again $y = x + 2.5$.

```
12.5.2 double[] frekvensen = FrequencyOfString.frequencyOfString(kryptert);
int maximumIndex = 0;
double maximum = frekvensen[0];
for (int i = 1; i < 26; i++) {
    if (frekvensen[i] > maximum) {
        maximum = frekvensen[i];
        maximumIndex = i;
    }
}
int decryptionKey = 0;
```

```

if (maximumIndex < 4) {
    decryptionKey = 4-maximumIndex;
}
else {
    decryptionKey = 4-maximumIndex+26;
}
String svaret = CaesarCipher.encryptCaesar(decryptionKey, kryptert);
System.out.println(svaret);

```

12.5.3 First consider $n = 1$. There is exactly one function from a set of one element, say $\{a\}$ to itself, namely the function sending a to itself. This is a bijection, so the number of permutations is one, which is the same as $1!$.

Assume now that the statement is true for any set with n elements, *i.e.* there are $n!$ permutations. Note that this also means that there are exactly $n!$ bijections between any two sets with n elements. We want to prove that for a set with $n + 1$ elements, there are $(n + 1)!$ permutations. Note that $(n + 1)! = (n + 1)n!$.

Let the set be $\{a_1, a_1, \dots, a_{n+1}\}$, and let f be a permutation. For some index i , $f(a_{n+1}) = a_i$, and there is no other element that is mapped to a_i since f is a bijection. There are $n + 1$ choices for i . The first elements $\{a_1, a_2, \dots, a_n\}$ are sent (bijectively) to $\{a_1, a_2, \dots, a_{n+1}\} \setminus \{a_i\}$ by f . By the induction hypothesis, there are $n!$ choices for the first part. In total, there are $(n + 1)n! = (n + 1)!$ choices involved in f , and we have proved the statement.

12.5.5 mathematics is more fun than programming

12.5.6 You will probably need to have an ascii table available to solve this exercise.

14.5.1 MATLAB code for the solution. Note that it is not necessary to store more than the ending point of each walk.

```

n = 1000; % number of steps in each walk
N = 200; % number of walks
ends = zeros(2,N); % to hold ending points of the walks
for i = 1:N
    x = 0;
    y = 0;
    for j = 2:n
        t = rand;
        if t < 0.5
            x = x + 1;

```

```

        else
            x = x - 1;
        end
        t = rand;
        if t < 0.5
            y = y + 1;
        else
            y = y - 1;
        end
    end
    ends(1,i) = x;
    ends(2,i) = y;
end
scatter(ends(1,:), ends(2,:)) % plot the ending points
hold on
t = 0:2*pi/100:2*pi;
xCircle = sqrt(2*n-2)*cos(t);
yCircle = sqrt(2*n-2)*sin(t);
plot(xCircle, yCircle) % shows average distance from (0,0)
axis equal

```

14.5.2

(a) $E(X) = np = 20 \cdot 0.2 = 4$ and $\text{Var}(X) = np(1-p) = 20 \cdot 0.2 \cdot 0.8 = 3.2 = 1.79^2$

(b) $P(X = 1) = \binom{20}{1} 0.2^1 0.8^{19} = 0.0576$, $P(X = 4) = \binom{20}{4} 0.2^4 0.8^{16} = 0.2182$

(c) $P(X \geq 2) = 1 - P(X = 0) - P(X = 1) = 1 - 0.0115 - 0.0576 = 0.9309$

14.5.3

(a) $X \sim \text{bin}(n, p) = \text{bin}\left(14, \frac{1}{6}\right)$.

(b) $E(X) = np = 14/6 \approx 2.33$, $\text{Var}(X) = np(1-p) = 14 \cdot \frac{1}{6} \cdot \frac{5}{6} \approx 1.94 = 1.39^2$

(c) $P(X = 4) = \binom{14}{4} \left(\frac{1}{6}\right)^4 \left(\frac{5}{6}\right)^{10} \approx 0.125$

(d) $P(X \geq 2) = 1 - P(X < 2) = 1 - P(X = 0) - P(X = 1) \approx 0.704$

14.5.4

(a) According to the formulae,

$$\begin{aligned} E(X) &= n \frac{M}{N} = 4 \cdot \frac{3}{8} = 1.5 \\ \text{Var}(X) &= n \frac{M}{N} \left(1 - \frac{M}{N}\right) \frac{N-n}{N-1} \\ &= 4 \cdot \frac{3}{8} \left(1 - \frac{3}{8}\right) \frac{4}{7} \\ &= \frac{240}{448} = 0.5357 \\ &= 0.73^2 \end{aligned}$$

(b) Using the formula

$$P(X = x) = \frac{\binom{M}{x} \binom{N-M}{n-x}}{\binom{N}{n}}$$

for $x = 0, 1, 2, 3$ yields $\frac{5}{70}, \frac{30}{70}, \frac{30}{70}, \frac{5}{70}$. If you use the formula for $x = 4$, the binomial coefficient $\binom{3}{4}$ will appear. Depending on how you try to calculate this, you will get either some error, or the result 0. That the answer is zero is easy to understand: If we pick 4 elements, and there are only 3 special elements, we cannot pick 4 special elements.

14.5.5

(a) $X \sim \text{hypergeom}(N, M, n) = \text{hypergeom}(105, 42, 8)$

(b) $E(X) = nM/N = 8 \cdot 42/105 \approx 3.2$

(c) $P(X = 4) = \frac{\binom{42}{4} \binom{63}{4}}{\binom{105}{8}} \approx 0.239$

(d) $P(X \leq 7) = 1 - P(X = 8) = 1 - \frac{\binom{42}{8} \binom{63}{0}}{\binom{105}{8}} \approx 0.9996$

14.5.6

$$E(X) = \sum xP(X = x) = 0 \cdot 0.2 + 2 \cdot 0.2 + 3 \cdot 0.1 + 4 \cdot 0.2 + 6 \cdot 0.05 + 7 \cdot 0.25 = 3.55$$

$$\begin{aligned} \text{Var}(X) &= \sum (x - E(X))^2 P(X = x) \\ &= (0 - 3.55)^2 \cdot 0.2 + (2 - 3.55)^2 \cdot 0.2 + (3 - 3.55)^2 \cdot 0.1 \\ &\quad + (4 - 3.55)^2 \cdot 0.2 + (6 - 3.55)^2 \cdot 0.05 + (7 - 3.55)^2 \cdot 0.25 \\ &= 6.3475 = 2.52^2 \end{aligned}$$

14.5.7 Since 20 seconds is one third of a minute, we have that $X \sim \text{Po}(\lambda t) = \text{Po}(4/3)$.

(a) $P(X = 2) = \frac{(4/3)^2}{2!} e^{-4/3} = 0.234$

(b) $P(X \geq 1) = 1 - P(X = 0) = 1 - e^{-4/3} = 0.736$

14.5.8

(a) $\lambda = 0.7$ if we use units corps times years.

(b) In this case, λt is $0.7 \cdot 14 \cdot 0.5 = 4.9$, so the answer is $P(X = 0) = e^{-4.9} = 0.00745$

14.5.9 function [expected] = GuessExpectation(X, trials)

%GuessExpectation returns the mean after the experiment X is conducted

%trial number of times

% X should be a function handle, like X = @() BinomialDistribution(100,0.13)

% trials should be a positive integer

expected = 0;

for i=1:trials

 expected = expected + X()/trials;

end

end

14.5.10

(a) function [variance] = GuessVariance(X, trials, mu)

%GuessVariance estimates the variance of the stochastic variable X with

%expectation mu as the empirical variance of performing the given number of

%trials.

% X should be a function handle, like X = @() BinomialDistribution(100,0.13)

% mu should be a previously computed expectation for X

% trials should be a positive integer

variance = 0;

for i=1:trials

 variance = variance + (X() - mu)^2;

end

variance = variance/(trials -1);

end

(b) function [variance] = GuessVariance(X, trials, varargin)

%GuessVariance estimates the variance of the stochastic variable X with

%expectation mu as the empirical variance of performing the given number of


```

%trials.
%  X  should be a function handle, like X = @() BinomialDistribution(100,0.13)
%  mu should be a previously computed expectation for X
%  trials should be a positive integer
if nargin == 2
    mu = GuessExpectation(X, trials);
else
    mu = varargin{1};
end
variance = 0;

for i=1:trials

    variance = variance + (X() - mu)^2;
end

variance = variance/(trials -1);
end

```

14.5.11 function [numberOfSuccesses] = HypergeometricDistribution(N, M, n)

```

numberOfSuccesses = 0;
for i = 1:n
    t = rand(1);
    if (t < M/N)
        % drawn from the special set
        numberOfSuccesses = numberOfSuccesses + 1;
        M = M-1;
        N = N-1;
    else
        % drawn from the rest
        N = N-1;
    end
end
end
end

```

14.5.12 Here is a possible solution in Java:

```

public static long computeBinCoeff(int n, int k){
    long [][] lesserNumbers = new long [n-k+1][k+1];
    for (int i = 0; i < n-k+1;i++){
        lesserNumbers[i][0] = 1; // Starting ones
    }
}

```

```

for (int j = 0; j < k+1; j++){
    lesserNumbers[0][j] = 1; // Starting ones
}
for (int i = 1; i < n-k+1; i++){
    for (int j = 1; j < k+1; j++){
        // the recursion
        lesserNumbers[i][j] = lesserNumbers[i-1][j] \
            + lesserNumbers[i][j-1];
    }
}
return lesserNumbers[n-k][k];
}

```

For a MATLAB version, this works

```

function [bincoeff] = BinomialCoefficient(n,k)
LowerNumbers = ones(n-k+1,k+1);
for i = 2:n-k+1
    for j = 2:k+1
        LowerNumbers(i,j) = LowerNumbers(i-1,j) + LowerNumbers(i,j-1);
    end
end
bincoeff = LowerNumbers(n-k+1,k+1);
end

```

15.4.6 Take care to represent non-existing edges carefully, for instance by using **Inf**.

15.5.1

(c) Yes, for instance if G has one component that is a square with one diagonal, and one component that is a tree (any tree will do).

(d) Yes, for instance if G has two components that are triangles.

16.6.1

$$E(X) = \sum xP(X = x) = 0 \cdot 0.2 + 2 \cdot 0.2 + 3 \cdot 0.1 + 4 \cdot 0.2 + 6 \cdot 0.05 + 7 \cdot 0.25 = 3.55$$

$$\begin{aligned}
 \text{Var}(X) &= \sum (x - E(X))^2 P(X = x) \\
 &= (0 - 3.55)^2 \cdot 0.2 + (2 - 3.55)^2 \cdot 0.2 + (3 - 3.55)^2 \cdot 0.1 \\
 &\quad + (4 - 3.55)^2 \cdot 0.2 + (6 - 3.55)^2 \cdot 0.05 + (7 - 3.55)^2 \cdot 0.25 \\
 &= 6.3475 = 2.52^2
 \end{aligned}$$

16.6.2 Since

$$A = (A \cap B) \cup (A \cap \overline{B}) \quad \text{a disjoint union}$$

we see that

$$P(A) = P(A \cap B) + P(A \cap \overline{B}).$$

Using the multiplicative property from Equation (16.1) for both terms yield

$$P(A) = P(A|B)P(B) + P(A|\overline{B})P(\overline{B}).$$

16.6.3

(a) We compute that

$$\begin{aligned}\mu_X &= 6.17 \\ \mu_Y &= 1.28 \\ \sigma_X^2 &= 0.5525^2 \\ \sigma_Y^2 &= 0.4502^2 \\ Cov(X, Y) &= -0.0056 \\ \rho(X, Y) &= -0.0224\end{aligned}$$

Since the correlation coefficient is nonzero, the variables are dependent.

(b) For independent variables, each joint probability must be the product of the marginal probabilities. It is enough to find one counter-example in the table to conclude that the variables are not independent. For instance,

$$P(X = 6 \wedge Y = 2) = 0.1148 \neq 0.6658 \cdot 0.2824 = P(X = 6)P(Y = 2)$$

16.6.4

(a) We compute that

$$\begin{aligned}\mu_X &= 2 \\ \mu_Y &= 1.3 \\ \sigma_X^2 &= 0.6325^2 \\ \sigma_Y^2 &= 0.6403^2 \\ Cov(X, Y) &= 0 \\ \rho(X, Y) &= 0\end{aligned}$$

Since the correlation coefficient is zero, it is possible that the variables are independent, but we can't know for sure.

(b) Since each probability in the joint distribution table is the product of the marginal distributions (*e.g.* in position $x = 2, y = 1$ $P(X = 2 \wedge Y = 1) = 0.3 = 0.6 \cdot 0.5 = P(X = 2)P(Y = 1)$), the stochastic variables are independent.

16.6.5 The information in the text translates to formulas as follows: $|A \cap C| = 0.75 \cdot 40 = 30$, $P(A|B) = P(A|\bar{B})$ and $|B \cap C| = 65$.

(a) $P(A) = 40/200 = 0.2$, $P(B) = 120/200 = 0.6$ and $P(C) = 115/200 = 0.575$.

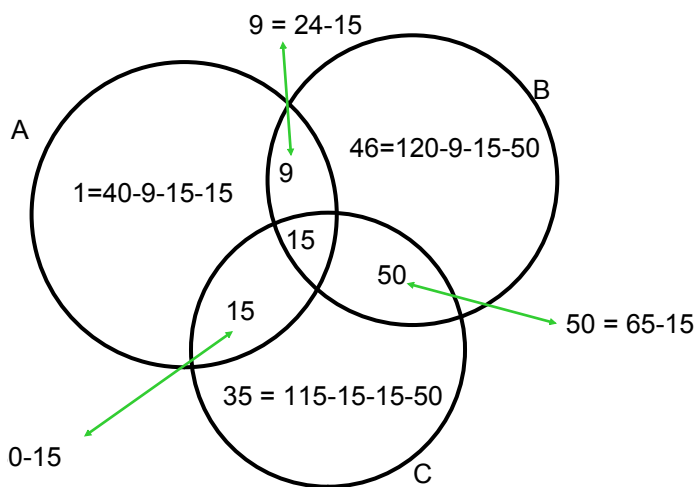
(b) $P(A) = P(A|B)P(B) + P(A|\bar{B})P(\bar{B})$. Since $P(A|B) = P(A|\bar{B})$, we get $P(A) = P(A|B)(P(B) + P(\bar{B})) = P(A|B)$, so A and B are independent. But then $P(A \cap B) = P(A)P(B) = 0.2 \cdot 0.6 = 0.12$.

$$P(A \cup C) = P(A) + P(C) - P(A \cap C) = 0.2 + 0.575 - \frac{30}{200} = 0.625.$$

$$P(B \cap C) = \frac{65}{200} = 0.325$$

(c) $P(A|B) = P(A \cap B)/P(B) = 0.12/0.6 = 0.2$, $P(B|A) = P(B \cap A)/P(A) = 0.12/0.2 = 0.6$ and $P(A|C) = P(A \cap C)/P(C) = 0.15/0.575 = 0.261$

(d) This is $P(\overline{A \cup B \cup C})$. It is easiest to compute this by labelling all parts of the Venn diagram with the number of people, and see how many are left outside of



the total of 200 people.

The number of people is $200 - 1 - 9 - 15 - 15 - 50 - 46 - 35 = 29$, so the answer is $29/200 = 0.145$.

16.6.6

(a)

$$1 = \int_{-\infty}^{\infty} f(x) dx = \int_0^1 cx^2 dx = \frac{cx^3}{3} \Big|_0^1 = \frac{c}{3}$$

Therefore $c = 3$.

(b)

$$E(X) = \int_{-\infty}^{\infty} x f(x) dx = \int_0^1 3x^3 dx = \left. \frac{3x^4}{4} \right|_0^1 = \frac{3}{4}$$

(c)

$$\text{Var}(X) = \int_{-\infty}^{\infty} x^2 f(x) dx - E(X)^2 = \int_0^1 3x^4 dx - \frac{3^2}{4^2} = \left. \frac{3x^5}{5} \right|_0^1 - \frac{9}{16} = \frac{3}{80} = 0.194^2$$

17.6.1 We use that $Z = \sigma X + \mu$ and $X = (Z - \mu)/\sigma$.

(a)

$$\begin{aligned} P(Z \geq \mu + x) &= P(\sigma X + \mu \geq \mu + x) \\ &= P(X \geq x/\sigma) \\ &= P(X \leq -x/\sigma) \\ &= P(\sigma X + \mu \leq \mu - x) \\ &= P(Z \leq \mu - x) \end{aligned}$$

(b)

$$\begin{aligned} P(Z \leq \mu) &= P(\sigma X + \mu \leq \mu) \\ &= P(X \leq 0/\sigma) = P(X \leq 0) \\ &= 0.5 \end{aligned}$$

(c)

$$\begin{aligned} P(\mu - x \leq Z \leq \mu + x) &= P(\mu - x \leq \sigma X + \mu \leq \mu + x) \\ &= P\left(\frac{-x}{\sigma} \leq X \leq \frac{x}{\sigma}\right) \\ &= 1 - 2P(X \leq -x/\sigma) \\ &= 1 - 2P(\sigma X + \mu \leq \mu - x) \\ &= 1 - 2P(Z \leq \mu - x) \end{aligned}$$

17.6.2

$$\begin{aligned} E(X) &= E\left(\frac{Z - \mu}{\sigma}\right) = \frac{E(Z) - \mu}{\sigma} = \frac{\mu - \mu}{\sigma} = 0 \\ \text{Var}(X) &= \text{Var}\left(\frac{Z - \mu}{\sigma}\right) = \frac{\text{Var}(Z - \mu)}{\sigma^2} = \frac{\text{Var}(Z)}{\sigma^2} = \frac{\sigma^2}{\sigma^2} = 1 \end{aligned}$$

17.6.3

(a) `0.5+0.5*erf(1.2/sqrt(2))` returns 0.8849.

(b) $P(X > 0.3) = 1 - P(X \leq 0.3)$. `1- (0.5+0.5*erf(0.3/sqrt(2)))` i returns 0.3821.

(c) Using MATLAB, and that $P(a < X \leq b) = P(X \leq b) - P(X \leq a)$, `0.5+0.5*erf(1.5/sqrt(2))` returns 0.5125

17.6.4

(a) `0.5+0.5*erf(1.2/(2*sqrt(2)))` returns 0.7257.

(b) $P(X > 0.3) = 1 - P(X \leq 0.3)$. `1-(0.5+0.5*erf(0.3/(2*np.sqrt(2))))` returns 0.4404.

(c) Since $P(-0.2 < X \leq 1.5) = P(X \leq 1.5) - P(X \leq -0.2)$, we can compute `0.5+0.5*erf(1.5/(2*sqrt(2)))` minus `0.5+0.5*erf(-0.2/(2*sqrt(2)))`. The answer is 0.3132.

17.6.6 Since the margin is $u_{\alpha/2} \frac{\sigma}{\sqrt{n}} = 0.75$, we can solve for n and find

$$n = \left(u_{\alpha/2} \frac{\sigma}{0.75} \right)^2 = \left(1.96 \cdot \frac{3}{0.75} \right)^2 \approx 61.47$$

Since n must be an integer, we must round up to $n = 62$.

17.6.14

(a) $\mu = np = 250 \cdot 0.4 = 100$, $\sigma^2 = np(1-p) = 250 \cdot 0.4 \cdot 0.6 = 60 = 7.75^2$. Therefore the corresponding normal approximation is $Y \sim N(\mu, \sigma^2) = N(100, 7.75^2)$.

(b) $0.5 + 0.5 \cdot \text{erf}((110 - 100)/(7.75\sqrt{2})) = 0.90$

(c) $0.5 + 0.5 \cdot \text{erf}((110 - 100)/(7.75\sqrt{2})) - (0.5 + 0.5 \cdot \text{erf}((90 - 100)/(7.75\sqrt{2}))) = 0.80$. Note that this could be computed from the previous point using symmetry: $P(Y > 110) = P(Y < 90)$ and $P(Y > 110) = 1 - P(Y \leq 110) = 0.10$, so $P(90 < Y < 110) = 1 - 2 \cdot 0.10 = 0.80$.

17.6.15

(a) Since $\mu = \lambda t = 150$ and $\sigma^2 = \lambda t = 12.25^2$, $Y \sim N(150, 12.25^2)$.

(b) $P(Y < 135) = 0.5 + 0.5 \cdot \text{erf}\left(\frac{135-150}{12.25\sqrt{2}}\right) = 0.11$.

(c) $P(Y > 145) = 1 - \left(0.5 + 0.5 \cdot \text{erf}\left(\frac{145-150}{12.25\sqrt{2}}\right)\right) = 0.66$.

17.6.16 Since $\mu = np$, the estimate that is naturally in the middle of the confidence interval is $\hat{p} = X/n = 62.3/200 = 0.3115$. The normal approximation for X is $N(\mu, \sigma^2) = N(np, np(1-p))$, so we can approximate $\hat{p} = X/n$ by $N(p, p(1-p)/n)$. The best value we can use for p is 0.3115, so we get $N(0.3115, 0.3115 \cdot (1 - 0.3115)/200) = N(0.3115, 0.033^2)$. Therefore the approximate 95% confidence interval is

$$0.3115 \pm 1.96 \cdot 0.033 = 0.3115 \pm 0.064 = [0.247, 0.376]$$

17.6.17

(a) Since X only takes integer values, the two inequalities $X \leq 11$ and $X \geq 12$ cover all possible results for X . Therefore the sum of the probabilities is 1.

(b) By computing np and $np(1-p)$ we see that $Y \sim N(12.5, 2.5^2)$. Therefore

$$P(Y \leq 11) + P(Y \geq 12) = 0.5 + 0.5 \cdot \operatorname{erf}\left(\frac{11 - 12.5}{2.5\sqrt{2}}\right) + 1 - \left(0.5 + 0.5 \cdot \operatorname{erf}\left(\frac{12 - 12.5}{2.5\sqrt{2}}\right)\right) = 0.85$$

(c)

$$P(Y \leq 11.5) + P(Y \geq 11.5) = 0.5 + 0.5 \cdot \operatorname{erf}\left(\frac{11.5 - 12.5}{2.5\sqrt{2}}\right) + 1 - \left(0.5 + 0.5 \cdot \operatorname{erf}\left(\frac{11.5 - 12.5}{2.5\sqrt{2}}\right)\right) = 1$$

17.6.18

(a) Since each voter is chosen only once, this is a typical hypergeometric situation. The total number of voters must be $N =$ one million, the number of voters in favour must be $M = 550000$ and the number we draw is $n = 100$. Therefore $X \sim \text{hypergeom}(1000000, 550000, 100)$.

(b) Since the number of voters questioned in the poll is so small compared to the total number of voters, the probability does not change much after we have drawn a few voters. Therefore it is natural to consider a binomial distribution. The probability for choosing someone in favour of the given question must be $p = M/N = 0.55$ and the number drawn is $n = 100$. Therefore $Y \sim \text{bin}(100, 0.55)$.

(c) The expectation is $np = 55$ and the variance is $\sigma^2 = np(1-p) = 100 \cdot 0.55 \cdot (1 - 0.55) = 24.75 = 4.97^2$. Therefore $Z \sim N(55, 4.97^2)$.

(d) With this number, $p = 0.005$. The expectation would be 0.5 and the variance $100 \cdot 0.05 \cdot (1 - 0.05) = 0.7^2$. We see that the standard deviation is larger than the expectation, so in particular there will be a significant probability that the normal approximation is smaller than zero (which is impossible for the true distribution). This explains that it is necessary to perform more measurements to accurately estimate a rare phenomenon than a common phenomenon.

18.3.1 MATLAB-kode for å generere verdiene, og plotte det fornuftig. Bare de to første linjene må endres fra eksempel til eksempel. Merk at det i senere oppgaver vil komme mer fornuftige måter å strukturere denne koden på.

```
x = [ 2;3;5;7]; % verdiene i c)
y = [ 3.4;4.3;8.6;10]; % verdiene i c)
Mlin = zeros(length(x), 2);
Mkvad = zeros(length(x), 3);

% De neste to linjene viser hvordan vi kan bruke MATLABs tabellfunksjoner.
Mlin(:,1) = x;
Mlin(:,2) = 1;
Mkvad(:,1) = x.^2;
Mkvad(:,2) = x;
Mkvad(:,3) = 1;

Clin = transpose(Mlin)*Mlin;
Dlin = transpose(Mlin)*y;
plin = Clin\Dlin % løser ligningssystemet

Ckvad = transpose(Mkvad)*Mkvad;
Dkvad = transpose(Mkvad)*y;
pkvad = Ckvad\Dkvad % løser ligningssystemet

% Coefficient of determination r2:
ystrek = mean(y);
sy2 = 0;
sselin = 0;
ssekvad = 0;
for i = 1:length(x)
    sy2 = sy2 + (y(i) - ystrek)^2;
    sselin = sselin + (y(i) - (plin(1)*x(i)+plin(2)))^2;
    ssekvad = ssekvad + (y(i) - (pkvad(1)*x(i)^2+pkvad(2)*x(i)+pkvad(3)))^2;
end
r2lin = (sy2-sselin)/sy2
r2kvad = (sy2-ssekvad)/sy2
scatter(x,y)
hold on
xplot = [min(x)-1:0.1:max(x)+1];
```



```

ylinplot = plin(1)*xplot + plin(2);
ykvadplot = pkvad(1)*xplot.^2+pkvad(2)*xplot+pkvad(3);
plot(xplot,ylinplot, xplot, ykvadplot)

```

18.3.2 We use the data and results of the computations from Example 18.1.7, namely X, Y, ω and p in the first lines of code.

```

X = [125;154;163;185;261];
Y = [323;261;252;260;433];
omega = 365;
p = [450.8;197.3;-26.6];
for i = 1:length(X)
    z(i) = p(1) + p(2)*cos(2*pi*X(i)/omega) + p(3)*sin(2*pi*X(i)/omega);
end
sy2 = 0;
sse = 0;
for i = 1:length(X)
    sy2 = sy2 + (Y(i) - mean(Y))^2;
    sse = sse + (Y(i) - z(i))^2;
end
r2Sinusoidal = (sy2-sse)/sy2

```

The code returns 0.9975. This is close to one, and the model is a very good one! A score this high will in practice seldom appear outside astronomy.

18.3.4

H_0 : X and Y are independent

H_1 : X and Y are not independent

Under H_0 , the expected values are found by multiplying the marginal probabilities to get a 2×4 matrix, and dividing by the sum of the elements of the matrix to get a probability distribution. This results in the table of expected values

	a	b	c	d	Y
p	11.1789	14.2842	16.7684	16.7684	59
q	6.8211	8.7158	10.2316	10.2316	36
X	18	23	27	27	95

We can now compute $Q = \sum_{i,j} \frac{(obs_{ij} - exp_{ij})^2}{exp_{ij}}$. Note that there are eight terms in this sum. The result is $Q = 6.57 < 7.81$. Therefore we don't have sufficient evidence to reject H_0 . The observations do not allow us to conclude that X and Y are dependent, so we stick to the hypothesis that they are independent.

18.3.5**(a)** H_0 : The observations follow the expected distribution

The alternative hypothesis is

 H_1 : The observations don't follow the expected distribution**(b)** The total number of birds is 133. Therefore we get the expected numbers by multiplying the percentages with this number (or with 1.33):

Species 1: 33.25

Species 2: 13.3

Species 3: 19.95

Species 4: 39.9

Species 5: 6.65

Species 6: 19.95

(c) We then compute $Q = \sum \frac{(\text{observed}(i) - \text{expected}(i))^2}{\text{expected}(i)}$. Note that there are six terms in this sum. The result is that $Q = 1.91 < 11.07$, so there is not sufficient evidence to reject H_0 . The observations do not indicate any significant change in the distribution of these species of birds.**18.3.7****(a)** The expected number of observations of i , for i from 0 to $n = 3$, is computed by

$$100 \cdot P(X = i) = 100 \cdot \binom{n}{i} p^i (1-p)^{(n-i)}$$

This produces the numbers 51.2, 38.4, 9.6, 0.8

(b)

$$Q = \frac{(55 - 51.2)^2}{51.2} + \frac{(37 - 38.4)^2}{38.4} + \frac{(8 - 9.6)^2}{9.6} + \frac{(0 - 0.8)^2}{0.8} = 1.4$$

(c) The null hypothesis is H_0 : The distribution is $\text{bin}(3, 0.2)$

The alternative hypothesis is

 H_1 : It is a different distributionA table gives us $z_{0.05,3} = 7.815$. Since $Q = 1.4 < 7.815$ we conclude that there is not sufficient evidence to reject H_0 , the data does not tell us that the distribution is different from the proposed binomial distribution.

18.3.8 Code will be given for each task, even though the structure is quite similar in each case.

```
(a) function [p] = LinearApproximation(X,Y)
%LinearApproximation computes the least-squares best linear regression
%line to the points whose X and Y coordinates are given.
%X and Y should be two column vectors of the same length. p is the column
%vector [A;B] such that the best linear approximation is  $y = Ax+B$ .
M = ones(length(X),2);
for i=1:length(X)
    M(i,1)=X(i);
end
p = (M'*M)\(M'*Y); % \ solves a linear system of equations
end

(b) function [p] = QuadraticApproximation(X,Y)
% QuadraticApproximation computes the least-squares best quadratic regression
%curve to the points whose X and Y coordinates are given.
%X and Y should be two column vectors of the same length. p is the column
%vector [A;B;C] such that the best quadratic approximation is  $y = Ax^2+Bx+C$ .
M = ones(length(X),3);
for i=1:length(X)
    M(i,1) = X(i)^2;
    M(i,2) = X(i);
end
p = (M'*M)\(M'*Y); % \ solves a linear system of equations
end

(c) function [p] = CubicApproximation(X,Y)
% CubicApproximation computes the least-squares best cubic regression curve
%to the points whose X and Y coordinates are given.
%X and Y should be two column vectors of the same length. p is the column
%vector [A;B;C] such that the best cubic approximation is  $y = Ax^3+Bx^2+Cx+D$ .
M = ones(length(X),4);
for i=1:length(X)
    M(i,1) = X(i)^3;
    M(i,2) = X(i)^2;
    M(i,3) = X(i);
end
p = (M'*M)\(M'*Y); % \ solves a linear system of equations
end
```

```
(d) function [p] = SinusoidalApproximation(X,Y,omega)
% SinusoidalApproximation computes the least-squares best sinusoidal regression
% curve to the points whose X and Y coordinates and
% whose frequency omega are given.
%X and Y should be two column vectors of the same length. p is the column
% vector [A0;A1;B1] such that the best sinusoidal approximation is
%  $y = A_0 + A_1 \cos(2\pi x/\omega) + B_1 \sin(2\pi x/\omega)$ .
M = ones(length(X),3);
for i=1:length(X)
    M(i,2) = cos(2*pi*X(i)/omega);
    M(i,3) = sin(2*pi*X(i)/omega);
end
p = (M'*M)\(M'*Y); % \ solves a linear system of equations
end
```

```
(e) function [p] = LinearApproximation2D(X,Y,Z)
% LinearApproximation2D computes the least-squares best linear regression
% plane to the points whose X, Y and Z coordinates are given.
%X, Y and Z should be three column vectors of the same length. p is the column
% vector [A;B;C] such that the best linear approximation plane is  $z = Ax + By + C$ .
M = ones(length(X),3);
for i=1:length(X)
    M(i,1) = X(i);
    M(i,2) = Y(i);
end
p = (M'*M)\(M'*Z); % \ solves a linear system of equations
end
```

19.5.1

(a)

$$\begin{aligned} 19000 &= 1 \cdot 17000 + 2000 \\ 17000 &= 8 \cdot 2000 + 1000 \\ 2000 &= 2 \cdot 1000 + 0 \end{aligned}$$

Therefore the greatest common divisor is 1000.

(b)

$$\begin{aligned} 2415 &= 1 \cdot 1624 + 791 \\ 1624 &= 2 \cdot 791 + 42 \\ 791 &= 18 \cdot 42 + 35 \\ 42 &= 1 \cdot 35 + 7 \\ 35 &= 5 \cdot 7 + 0 \end{aligned}$$

Therefore the greatest common divisor is 7.

(c)

$$\begin{aligned}
 3524578 &= 1 \cdot 2178309 + 1346269 \\
 2178309 &= 1 \cdot 1346269 + 832040 \\
 1346269 &= 1 \cdot 832040 + 514229 \\
 832040 &= 1 \cdot 514229 + 317811 \\
 514229 &= 1 \cdot 317811 + 196418 \\
 317811 &= 1 \cdot 196418 + 121393 \\
 196418 &= 1 \cdot 121393 + 75025 \\
 121393 &= 1 \cdot 75025 + 46368 \\
 75025 &= 1 \cdot 46368 + 28657 \\
 46368 &= 1 \cdot 28657 + 17711 \\
 28657 &= 1 \cdot 17711 + 10946 \\
 17711 &= 1 \cdot 10946 + 6765 \\
 10946 &= 1 \cdot 6765 + 4181 \\
 6765 &= 1 \cdot 4181 + 2584 \\
 4181 &= 1 \cdot 2584 + 1597 \\
 2584 &= 1 \cdot 1597 + 987 \\
 1597 &= 1 \cdot 987 + 610 \\
 987 &= 1 \cdot 610 + 377 \\
 610 &= 1 \cdot 377 + 233 \\
 377 &= 1 \cdot 233 + 144 \\
 233 &= 1 \cdot 144 + 89 \\
 144 &= 1 \cdot 89 + 55 \\
 89 &= 1 \cdot 55 + 34 \\
 55 &= 1 \cdot 34 + 21 \\
 34 &= 1 \cdot 21 + 13 \\
 21 &= 1 \cdot 13 + 8 \\
 13 &= 1 \cdot 8 + 5 \\
 8 &= 1 \cdot 5 + 3 \\
 5 &= 1 \cdot 3 + 2 \\
 3 &= 1 \cdot 2 + 1 \\
 2 &= 2 \cdot 1 + 0
 \end{aligned}$$

Therefore the greatest common divisor is 1.

19.5.2

(a) Since $35 = 2 \cdot 12 + 11$ and $12 = 1 \cdot 11 + 1$, we get by backtracking that $1 = 12 - 11 = 12 - (35 - 2 \cdot 12) = 3 \cdot 12 - 35$. Therefore the multiplicative inverse of 12 is 3.

(b)

$$\begin{aligned}
65537 &= 59 \cdot 1097 + 814 \\
1097 &= 1 \cdot 814 + 283 \\
814 &= 2 \cdot 283 + 248 \\
283 &= 1 \cdot 248 + 35 \\
248 &= 7 \cdot 35 + 3 \\
35 &= 11 \cdot 3 + 2 \\
3 &= 1 \cdot 2 + 1
\end{aligned}$$

Backtracking then gives

$$\begin{aligned}
1 &= 3 - 2 \\
&= 3 - (35 - 11 \cdot 3) = 12 \cdot 3 - 35 \\
&= 12 \cdot (248 - 7 \cdot 35) - 35 = 12 \cdot 248 - 85 \cdot 35 \\
&= 12 \cdot 248 - 85 \cdot (283 - 248) = 97 \cdot 248 - 85 \cdot 283 \\
&= 97 \cdot (814 - 2 \cdot 283) - 85 \cdot 283 = 97 \cdot 814 - 279 \cdot 283 \\
&= 97 \cdot 814 - 279 \cdot (1097 - 814) = 376 \cdot 814 - 279 \cdot 1097 \\
&= 376 \cdot (65537 - 59 \cdot 1097) - 279 \cdot 1097 = 376 \cdot 65537 - 22463 \cdot 1097
\end{aligned}$$

Therefore the inverse is $-22463 = 43074 \bmod 65537$.

19.5.3 Since $\sqrt{12600} < 113$, we first need the list of primes less than 112. In addition to the primes less than 100 listed in the text, the four numbers 101, 103, 107, 109 are primes. For each prime in this list, we remove the multiples of it in the given range. The remaining twelve numbers are primes: 12503, 12511, 12517, 12527, 12539, 12541, 12547, 12553, 12569, 12577, 12583, 12589

19.5.4

(a) Since $n = 5 \cdot 7$, $\phi = 4 \cdot 6 = 24$, which has no common divisor with $e = 5$.

(b) $24 = 4 \cdot 5 + 4$ and $5 = 1 \cdot 4 + 1$. Therefore $1 = 5 - 4 = 5 - (24 - 4 \cdot 5) = 5 \cdot 5 - 24$. So the inverse $s = 5$.

(c) Coded: 18, 14, 20, 19, 7. Encrypted: $18^5 = 1889568 = 23 \bmod 35$, and similarly for the four others give 23, 14, 20, 24, 7 as the answer.

(d) Decrypt $1^5 = 1 \bmod 35$, so the first letter is *b*. Similarly for the other letters we get the list of decrypted integers 1, 0, 2, 10 which decoded spells the word *back*.

20.3.3 For $n = 0$ this is the geometric series that we started the chapter with, since $\binom{0+k}{k} = 1$.

Assume the formula holds for a given n , and consider $n + 1$. We want to prove that

$$\sum_{k=0}^{\infty} \binom{n+1+k}{k} x^k = \frac{1}{(1-x)^{n+2}}$$

Starting from the right hand side, we see that

$$\frac{d}{dx} \frac{1}{(1-x)^{n+1}} = \frac{d}{dx} (1-x)^{-n-1} = \frac{n+1}{(1-x)^{n+2}}$$

Note that we get rid of the minus because of the chain rule. The result is $n+1$ times the right hand side of the formula we want. Starting from the right hand side, we see that

$$\frac{d}{dx} \sum_{k=0}^{\infty} \binom{n+k}{k} x^k = \sum_{k=0}^{\infty} k \binom{n+k}{k} x^{k-1} = \sum_{m=0}^{\infty} (m+1) \binom{n+m+1}{m+1} x^m$$

Here we use the substitution $m = k - 1$. If the two expressions we have computed are to be equal ($\square$), we need that (after dividing by $n+1$ and renaming the index as k)

$$\binom{n+1+k}{k} x^k = \frac{k+1}{n+1} \binom{n+k+1}{k+1} x^k$$

This is true, as can be seen by expanding the binomial coefficient and using $(k+1)! = (k+1)k!$ and similarly for n :

$$\frac{k+1}{n+1} \binom{n+k+1}{k+1} = \frac{k+1}{n+1} \frac{(n+k+1)!}{(k+1)!n!} = \frac{(n+k+1)!}{k!(n+1)!} = \binom{n+1+k}{k}.$$

20.3.4 Differentiating:

$$\begin{aligned} \frac{\frac{d}{dx} \frac{1}{1-2x}}{\frac{-1}{(1-2x)^2} \cdot (-2)} &= \frac{\frac{d}{dx} \sum_{n=0}^{\infty} 2^n x^n}{\sum_{n=0}^{\infty} n 2^n x^{n-1}} = \sum_{m=0}^{\infty} (m+1) 2^{m+1} x^m. \end{aligned}$$

Here we used the substitution $m = n - 1$. Since the starting equation is valid for $|x| < 1/2$, the same is true for the differentiated equation.

20.3.5 Ratio test:

$$\lim_{n \rightarrow \infty} \left| \frac{2^{n+1} x^{n+1}}{2^n x^n} \right| = \lim_{n \rightarrow \infty} |2x| = |2x|$$

When this limit is less than 1, the series converges. This means that $|x| < 1/2$.

We can look at $u = 2x$ and compare with $1/(1-u)$, whose series converges for $|u| < 1$. But then our series must converge for $|2x| < 1$, and we get the same result as before.

20.3.6 Comparing with Example 20.2.6, we proceed as follows. The coefficients are 3^n . If we substitute $u = 3x$, we get a series we already know, so that

$$\sum_{n=0}^{\infty} 3^n x^n = \sum_{n=0}^{\infty} u^n = \frac{1}{1-u} = \frac{1}{1-3x}$$

The equation is valid for $|u| < 1$, that is $|3x| < 1$ or $|x| < 1/3$.

20.3.8 In the displayed equation, the coefficient of x^n is

$$F(n) - F(n-1) - F(n-2)$$

Beacuse of the recursion, this is zero as long as all indices are ≥ 0 , which means that $n \geq 2$. The coefficient for x is $F(1) - F(0) = 1$, and the constant coefficient is $F(0) = 0$. Therefore

$$g(x) - xg(x) - x^2g(x) = x \quad \text{or} \quad g(x) = \frac{x}{1 - x - x^2}$$